

Embedded AI

Table of Contents Draft

Embedded Systems & AI: Constraints, Inference, and Integration

Version 0.1 — Working Draft **Type:** Course Textbook — 15-week semester **Target Reader:** Final-year undergraduate / entry-level graduate, ECE or CS **Prerequisites:** C/C++, digital logic and microcontroller basics, signals and real-time concepts **ML assumed:** None

Thesis

This book argues that AI deployment on embedded hardware is fundamentally a constrained optimization problem — and that the designer who cannot reason about both sides of that constraint (what the model needs, what the hardware can give) will make integration decisions that fail in production. The field has produced excellent books on embedded systems and excellent books on machine learning. It has not produced a book that treats the intersection as a design discipline.

Terminal Capability

A student who completes this course can examine a real embedded design problem, assess whether AI integration is appropriate, select an inference and deployment strategy, evaluate whether a different model would improve that strategy, and justify every decision against the hardware constraints of power, memory, latency, and reliability.

Three-Act Learning Arc

Arc statement: This book takes the student from *understanding what embedded constraints mean* to *evaluating AI integration decisions in constrained systems* by first establishing the collision between what AI needs and what hardware can give, then building the diagnostic tools for inference-time evaluation, then extending into model-aware design where the model itself becomes a design variable.

Act One — The Collision (Chapters 1–3) Establish the problem. The student who finishes Act One knows why AI and embedded systems are in tension, has functional ML literacy at the level needed to reason about inference, and can describe the design space where integration decisions live. The inciting question Act One raises: *given a model and a hardware target, how do we know if deployment is feasible?*

Act Two — Inference-First Evaluation (Chapters 4–10) Build the diagnostic toolkit. Each chapter adds one constraint dimension — memory, compute, power, hardware, communication, real-time — and teaches the student to evaluate AI integration against that dimension. The student who finishes Act Two can audit a proposed AI deployment and identify every constraint failure. The transition question: *we know whether this model can run here — but is this the right model?*

Act Three — Model-Aware Design (Chapters 11–14) Extend the toolkit. The model is no longer fixed — it is a design variable. The student learns to select, compress, and optimize models for deployment, and to make integration decisions across the full hardware-model-system stack.

Chapter-by-Chapter TOC

PART I — THE EMBEDDED-AI PROBLEM

Chapter 1 — When AI Meets Constrained Hardware

The student learns to describe the embedded-AI intersection as a design problem, not a technology trend.

Learning Outcomes:

- LO1.1 [Understand] Explain the defining characteristics of an embedded system that make AI integration non-trivial
- LO1.2 [Analyze] Distinguish between AI applications that are server-appropriate and those that require on-device deployment
- LO1.3 [Evaluate] Assess a proposed AI embedded application and identify the constraint categories that will determine feasibility

Opening: A real failure case — a deployed product where AI integration caused a latency, power, or reliability failure in production. The student should not know how to diagnose it yet. That becomes the hook for the entire course.

Core Content:

- What "embedded" means as a constraint environment vs. a device category
- Why AI moved to the edge: privacy, latency, connectivity, cost
- The four constraint categories that govern every decision in this book: memory, compute, power, real-time
- The design space: where embedded constraints and AI requirements collide
- Course roadmap: what the student will be able to do by the end

Worked Example: A smart doorbell camera — from server-side AI to on-device inference. What had to change and why.

Bridge to Chapter 2: The student now knows *why* the problem exists. Chapter 2 makes the constraint categories precise enough to use as diagnostic tools.

Chapter 2 — Embedded Constraints as Design Variables

The student learns to quantify embedded system constraints in terms that make AI integration decisions computable.

Learning Outcomes:

- LO2.1 [Apply] Specify the memory, compute, power, and timing constraints of a given embedded target
- LO2.2 [Analyze] Translate datasheet specifications into design constraints relevant to AI deployment
- LO2.3 [Evaluate] Determine whether a proposed deployment target is feasible before selecting a model

Note for students arriving with embedded background: This chapter reframes what you already know. The content is not new — the framing is. Constraint quantification is the language the rest of this book uses to make decisions.

Opening: Two microcontrollers, same price point, same application. One can run an AI model. One cannot. Why — and how would you know before designing the system?

Core Content:

- Memory hierarchy in embedded systems: flash, SRAM, external RAM — and what each means for model storage and inference
- Compute budget: clock speed, instruction throughput, and what they translate to in inference operations
- Power budget: average vs. peak, duty cycle, battery life math for AI inference
- Real-time constraints: hard vs. soft deadlines and what AI inference latency means for each
- Reading hardware constraint specifications for AI suitability

Worked Example: Constraint profile comparison — STM32F4 vs. STM32H7 vs. Raspberry Pi 4. Three different constraint envelopes; three different AI deployment possibilities.

Bridge to Chapter 3: Constraints are now precise. What remains is understanding what AI actually *demands* from those constraints — which requires knowing what AI is.

Chapter 3 — Machine Learning for Embedded Engineers: A Functional Foundation

The student builds enough ML literacy to reason about inference requirements without needing to train models.

Learning Outcomes:

- LO3.1 [Understand] Explain supervised learning, inference, and the trained model as artifacts with specific resource profiles
- LO3.2 [Apply] Identify the computational operations a neural network performs during inference
- LO3.3 [Analyze] Map neural network architecture choices to memory and compute requirements
- LO3.4 [Evaluate] Assess which class of ML model (CNN, RNN, decision tree, etc.) is appropriate for a given embedded sensing task

Critical design note: This chapter teaches ML from the perspective of what inference *costs*, not how training works. Training is introduced only to the degree needed to understand what a trained model is. A student who finishes this chapter cannot train a model — but can reason about what any trained model will demand from constrained hardware.

Opening: What is inside a `.tflite` file? Open one. The student sees weights, graph structure, input/output tensors. This is the artifact they will deploy. Understanding it is the chapter's purpose.

Core Content:

- Supervised learning: inputs, labels, training, and the trained model as a fixed artifact
- Neural network anatomy: layers, weights, activations — as memory and compute objects
- The inference pass: what happens when input data enters the model (matrix multiply, activation, repeat)
- Common architectures for embedded sensing: CNNs for image, RNNs/LSTMs for sequence, MobileNet family, decision trees
- Model size, parameter count, and FLOP count — and why they matter differently
- The ML-to-embedded translation table: ML terms mapped to embedded constraint terms

Worked Example: MobileNetV2 on a constrained target — walk through parameter count, memory footprint, and inference FLOP count before writing a single line of code.

Bridge to Chapter 4: The student now knows what AI demands. Part Two teaches how to evaluate whether those demands can be met.

PART II — INFERENCE-FIRST EVALUATION

Chapter 4 — Inference Mechanics: What a Deployed Model Actually Does

The student learns to trace the inference execution path on constrained hardware and identify where failures occur.

Learning Outcomes:

- LO4.1 [Apply] Trace the inference execution path of a deployed model on a target microcontroller
- LO4.2 [Analyze] Identify the computational bottleneck in a given model-hardware pairing
- LO4.3 [Evaluate] Determine whether a proposed model-hardware configuration can meet a specified inference latency requirement

Opening: Take the model from Chapter 3. Deploy it. Watch it fail — wrong latency, memory overflow, or incorrect output. The student has all the vocabulary to diagnose this now. This chapter gives them the tools.

Core Content:

- The inference execution pipeline on bare-metal and RTOS-based systems
- Operator-level execution: how convolution, matrix multiply, and activation functions run on CPU vs. specialized hardware
- Memory allocation during inference: static vs. dynamic, and why dynamic allocation fails on embedded targets
- Latency profiling: how to measure and interpret inference time on target hardware
- Numerical precision: float32 vs. float16 vs. int8 and what the tradeoffs are

Worked Example: Profile inference of a keyword spotting model on an Arduino Nano 33 BLE Sense. Measure latency, identify the bottleneck operation, propose one mitigation.

Bridge to Chapter 5: Inference execution is understood. The first constraint dimension — memory — is examined in full.

Chapter 5 — Memory: Fitting Models into Constrained Spaces

The student learns to analyze and reduce the memory footprint of AI models for embedded deployment.

Learning Outcomes:

- LO5.1 [Apply] Calculate the flash and SRAM requirements of a given model for a specified target
- LO5.2 [Analyze] Identify which components of a model consume the most memory and why
- LO5.3 [Evaluate] Determine whether a model fits a target's memory envelope and specify what must change if it does not

Core Content:

- Flash vs. SRAM in AI deployment: where weights live, where activations live, and why both matter
- Weight memory: $\text{parameter count} \times \text{precision} = \text{storage requirement}$
- Activation memory: the hidden footprint — how intermediate tensors consume SRAM during inference
- Memory layout strategies: in-place computation, buffer reuse, scratch pads
- Static memory allocation for inference: why malloc is not your friend
- Memory profiling tools: how to measure what a deployed model actually uses

Worked Example: A model that "fits" by parameter count fails at runtime due to activation memory. Diagnose, then apply buffer reuse to fix it.

Bridge to Chapter 6: Memory is characterized. The next constraint dimension — compute and latency — is examined.

Chapter 6 — Compute: Latency, Throughput, and Real-Time Guarantees

The student learns to evaluate whether a model can meet latency requirements on a given hardware target.

Learning Outcomes:

- LO6.1 [Apply] Estimate the inference latency of a model on a specified hardware target using FLOP count and throughput specifications
- LO6.2 [Analyze] Identify the operations that dominate inference compute time and explain why
- LO6.3 [Evaluate] Determine whether a proposed model-hardware-deadline combination is feasible and specify the required changes if it is not

Core Content:

- FLOPs, MACs, and what they actually mean for inference time on real hardware
- Compute-bound vs. memory-bound inference: which bottleneck you have changes the fix
- Clock speed, pipeline efficiency, and SIMD — what the processor specification tells you about AI throughput
- Worst-case execution time (WCET) analysis for AI inference
- The latency-accuracy tradeoff: smaller models run faster but produce worse outputs

Worked Example: Evaluate three model candidates for a gesture recognition system with a 50ms hard deadline. Use FLOP count and hardware throughput to predict feasibility before deployment.

Bridge to Chapter 7: Compute is characterized. The energy cost of that compute is examined next.

Chapter 7 — Power and Energy: The Efficiency Imperative

The student learns to evaluate the energy cost of AI inference and design for battery-powered or energy-harvesting embedded targets.

Learning Outcomes:

- LO7.1 [Apply] Estimate the energy cost of a single inference event on a specified hardware target
- LO7.2 [Analyze] Determine whether a proposed AI deployment strategy is compatible with a given power budget
- LO7.3 [Evaluate] Compare inference scheduling strategies on energy cost and select the optimal strategy for a specified application

Core Content:

- Power vs. energy: the distinction that matters for embedded AI design
- Dynamic and static power in AI inference workloads
- Duty cycling and inference scheduling: when to run the model, when to sleep
- Energy harvesting constraints: what "always-on" AI requires from an energy source
- Event-driven AI: using low-power sensors to trigger inference rather than polling
- Power profiling: how to measure inference energy consumption on target hardware

Worked Example: Design an inference scheduling strategy for a wildlife monitoring camera trap running on two AA batteries for six months. Justify the duty cycle decision with energy math.

Bridge to Chapter 8: Energy is characterized. Hardware accelerators — the answer to compute and energy constraints simultaneously — are examined next.

Chapter 8 — Hardware for AI: Accelerators, DSPs, and FPGAs

The student learns to evaluate hardware acceleration options and select the appropriate acceleration strategy for a constrained AI deployment.

Learning Outcomes:

- LO8.1 [Understand] Explain how neural processing units (NPU), DSPs, and FPGAs accelerate AI inference differently
- LO8.2 [Analyze] Compare the memory, compute, power, and flexibility tradeoffs of different acceleration architectures
- LO8.3 [Evaluate] Select an appropriate hardware acceleration strategy for a given application's constraint profile

Core Content:

- Why CPUs are inefficient for neural network inference: the mismatch explained
- DSPs for AI: fixed-point arithmetic, SIMD, and signal processing pipelines
- NPUs and microNPUs: what they are, what they accelerate well, and what they don't
- FPGAs for embedded AI: reconfigurability vs. power vs. development cost

- Integrated AI hardware: STM32N6, Kendryte K210, Coral Edge TPU — real devices, real tradeoffs
- When NOT to use a hardware accelerator: the cases where a well-optimized CPU model wins

Worked Example: Same model, three hardware targets: Cortex-M4 CPU, Cortex-M55 + Ethos-U55 NPU, Xilinx PYNQ FPGA. Compare latency, power, and development effort.

Bridge to Chapter 9: Local hardware is characterized. The decision of whether to process locally or involve the network is examined next.

Chapter 9 — Communication and the Edge-Cloud Spectrum

The student learns to evaluate the tradeoffs of on-device, edge, and cloud AI processing and design hybrid deployment strategies.

Learning Outcomes:

- LO9.1 [Analyze] Quantify the latency, bandwidth, power, and privacy costs of offloading inference to the cloud
- LO9.2 [Evaluate] Determine the appropriate processing location (device, edge, cloud) for a given application's requirements
- LO9.3 [Apply] Design a hybrid inference strategy that distributes computation across processing tiers

Core Content:

- The edge-cloud spectrum: on-device, near-edge, far-edge, cloud
- Communication cost as a constraint: bandwidth, latency, and energy for data transmission
- Privacy and regulatory constraints on data transmission (GDPR, medical data, industrial data)
- Latency requirements that preclude cloud inference: what "real-time" actually demands
- Split inference: running the early layers on device, the later layers on a server
- Federated learning basics: training at the edge without transmitting raw data (conceptual only)

Worked Example: A predictive maintenance system for industrial motors. Evaluate three deployment strategies — fully on-device, split inference, cloud — against latency, cost, and privacy requirements.

Bridge to Chapter 10: The physical and network constraints are characterized. The hardest constraint category — real-time guarantees with AI — is examined last in Act Two.

Chapter 10 — Real-Time AI: Reliability, Safety, and Deadlines

The student learns to evaluate whether AI inference can be integrated into real-time embedded systems without violating timing guarantees or safety requirements.

Learning Outcomes:

- LO10.1 [Analyze] Identify the properties of AI inference that make WCET analysis difficult and explain why determinism matters for real-time systems
- LO10.2 [Evaluate] Assess whether a proposed AI component can be safely integrated into a hard real-time system
- LO10.3 [Apply] Apply design patterns that reduce the real-time risk of AI inference in safety-critical applications

Opening: A hard case: an autonomous vehicle that fails to meet a braking deadline because the inference system ran long. Why did WCET analysis miss it? What should the designer have done differently?

Core Content:

- Hard vs. soft real-time requirements and what they demand from AI inference
- Non-determinism in neural network inference: why the same model can have variable execution time
- WCET analysis for AI: what works, what doesn't, and what the open problems are
- Safety-critical AI: functional safety standards (IEC 61508, ISO 26262) and what they require of AI components
- Design patterns for real-time-safe AI: bounded execution models, rejection thresholds, confidence gating
- Failure mode analysis for AI in embedded systems

Worked Example: Integrate a neural network-based anomaly detector into a hard real-time industrial control loop. Identify the failure modes, apply a confidence gating pattern, verify the timing guarantee.

Bridge to Chapter 11: The student can now evaluate AI integration against all major constraint dimensions. Part Three asks: what if the model itself is the wrong choice?

PART III — MODEL-AWARE DESIGN

Chapter 11 — Selecting Models for Constrained Deployment

The student learns to evaluate and compare model architectures for embedded deployment, selecting on constraint profile rather than benchmark accuracy alone.

Learning Outcomes:

- LO11.1 [Analyze] Compare model architectures on memory, compute, power, and accuracy tradeoffs for a specified deployment target
- LO11.2 [Evaluate] Select a model architecture appropriate for a constrained embedded target and justify the selection against the constraint profile
- LO11.3 [Apply] Use deployment-aware benchmarks (latency on target hardware, not accuracy on ImageNet) to evaluate model candidates

The transition this chapter makes: Until now, the model was given. From here, it is chosen. This is the shift from inference-first to model-aware thinking.

Core Content:

- Why accuracy benchmarks (ImageNet top-1, COCO mAP) are the wrong metric for embedded selection
- Deployment-aware metrics: latency on target, memory footprint, energy per inference, accuracy under quantization
- Efficient architecture families: MobileNet, EfficientNet-Lite, SqueezeNet, YOLO-nano — and what made them efficient
- Task-specific model selection: keyword spotting, image classification, anomaly detection, time-series forecasting
- The accuracy-efficiency Pareto frontier: how to read it and use it for selection
- Neural Architecture Search (NAS) for embedded targets: what it is and when it matters

Worked Example: Select a model for a wearable fall detection system. Evaluate five candidates on accuracy, latency (on target hardware), memory, and power. Make and justify the selection.

Bridge to Chapter 12: A model is selected. It doesn't fit perfectly. Chapter 12 teaches how to make it fit.

Chapter 12 — Optimizing Models: Quantization, Pruning, and Distillation

The student learns to apply model compression techniques and evaluate the tradeoffs of each for embedded deployment.

Learning Outcomes:

- LO12.1 [Apply] Apply post-training quantization to a model and measure the accuracy-efficiency tradeoff
- LO12.2 [Analyze] Compare quantization, pruning, and knowledge distillation on their effects on model size, latency, power, and accuracy
- LO12.3 [Evaluate] Select and apply the appropriate compression technique(s) for a specified model-hardware-accuracy constraint set

Core Content:

- Quantization: float32 → int8, what changes, what breaks, and how to measure the damage
- Post-training quantization vs. quantization-aware training: when each is appropriate
- Pruning: structured vs. unstructured, and why structured pruning is the embedded-relevant version
- Knowledge distillation: training a small model to mimic a large model (conceptual + practical)
- Combining techniques: quantize then prune, distill then quantize — ordering matters
- The accuracy floor: how to know when further compression will break the application

Worked Example: Take a model that barely fails the memory constraint from Chapter 5's target. Apply post-training quantization. Measure accuracy loss. Apply structured pruning. Re-measure. Determine whether the application requirement is still met.

Bridge to Chapter 13: Models are optimized. The toolchains that take them from trained artifact to deployed firmware are examined next.

Chapter 13 — TinyML Toolchains and Deployment Pipelines

The student learns to navigate the end-to-end toolchain from trained model to deployed firmware and identify where decisions in the pipeline affect the final system.

Learning Outcomes:

- LO13.1 [Apply] Convert a trained model to a deployment format (TFLite, ONNX) and deploy it to a target microcontroller using a standard toolchain
- LO13.2 [Analyze] Identify the decisions made by the toolchain that affect inference accuracy, latency, and memory usage
- LO13.3 [Evaluate] Select an appropriate toolchain for a specified hardware target and deployment requirement

Core Content:

- The deployment pipeline: train → convert → optimize → compile → deploy → verify

- TensorFlow Lite for Microcontrollers: the dominant path and its constraints
- Edge Impulse: end-to-end toolchain for embedded ML — what it handles, what it hides
- ONNX and vendor-specific compilers: when the generic path doesn't work
- Toolchain-induced accuracy loss: what the converter does that the designer needs to understand
- Testing and validation after deployment: why the model that passed accuracy tests may fail on device

Worked Example: Full pipeline walkthrough — take a keyword spotting model trained in TensorFlow, convert to TFLite, optimize with post-training quantization, deploy to Arduino Nano 33 BLE Sense, verify inference accuracy on device.

Bridge to Chapter 14: Every tool is in hand. Chapter 14 uses them all.

Chapter 14 — Integration Case Studies: Full Design Decisions

The student applies the complete decision framework — constraint evaluation, inference analysis, model selection, and optimization — to three full embedded AI design problems.

Learning Outcomes:

- LO14.1 [Evaluate] Apply the complete AI integration evaluation framework to a novel embedded design problem
- LO14.2 [Create] Produce a structured integration decision document that specifies the model, hardware, deployment strategy, and constraint justification for a given application
- LO14.3 [Evaluate] Critique a proposed AI embedded system design and identify the decisions most likely to fail in production

Three case studies — each at Act Three synthesis level:

Case A — Industrial Anomaly Detection A vibration sensor system on manufacturing equipment. Hard latency requirement (10ms), battery-powered (18-month life), no cloud connectivity allowed. Full decision chain: sensor → model selection → quantization → hardware → scheduling strategy.

Case B — Medical Wearable: Cardiac Arrhythmia Detection Wearable ECG monitor with AI-based arrhythmia classification. Safety-critical (misclassification has clinical consequences), battery-constrained, regulatory requirements (FDA SaMD pathway). Introduces the safety and reliability considerations from Chapter 10 at full complexity.

Case C — Agricultural Edge Sensor: Crop Disease Detection Low-cost, solar-powered image classification system for disease detection in the field. Intermittent connectivity, energy

harvesting, low unit cost constraint. Tests the edge-cloud decision from Chapter 9 and the hardware selection from Chapter 8.

Final Chapter Note: Each case study ends with an explicit list of decisions that were made, the alternatives that were rejected, and the conditions under which a rejected alternative would have been the right choice. This is the evaluation skill the book was built to develop.

Appendices

Appendix A — Embedded Systems Quick Reference Constraint quantification templates, datasheet reading guide for AI suitability, memory and power calculation worksheets. For students who need to refresh their embedded foundations.

Appendix B — Machine Learning Quick Reference Core ML concepts, common architectures, vocabulary table. For students who want a reference without re-reading Chapter 3.

Appendix C — Hardware Reference: AI-Capable Microcontroller and SoC Comparison Comparative table of embedded AI targets: Arduino Nano 33, STM32 family, ESP32-S3, Raspberry Pi family, Coral Edge TPU, STM32N6. Memory, compute, power, accelerator, toolchain support. Updated as companion content online.

Appendix D — Toolchain Setup Guides Step-by-step setup for TensorFlow Lite for Microcontrollers, Edge Impulse, and STM32Cube.AI.

Course Calendar: 15-Week Semester Mapping

Course: Embedded Systems & AI: Constraints, Inference, and Integration

Level: Final-year undergraduate / Entry-level graduate (ECE/CS)

Credit Hours: 3 credits

Prerequisites: C/C++, digital logic and microcontroller basics, signals and real-time concepts

ML Assumed: None (taught from inference perspective)

Course Structure Overview

Three-Act Learning Arc

Weeks 1–3 (Act I): The Collision

Students understand why embedded AI is a constrained optimization problem and gain functional ML literacy.

Weeks 4–7, 9–11 (Act II): Inference-First Evaluation

Students build the diagnostic toolkit to evaluate AI integration against memory, compute, power, hardware, communication, and real-time constraints.

Weeks 12–15 (Act III): Model-Aware Design

Students extend evaluation into model selection, optimization, and full-system integration.

Weekly Schedule

Week 1: When AI Meets Constrained Hardware

Chapter: 1

Learning Outcomes:

- LO1.1: Explain the defining characteristics of an embedded system that make AI integration non-trivial
- LO1.2: Distinguish between AI applications that are server-appropriate and those that require on-device deployment

- LO1.3: Assess a proposed AI embedded application and identify the constraint categories that will determine feasibility

Topics:

- What "embedded" means as a constraint environment
- Why AI moved to the edge (latency, privacy, connectivity, cost)
- The four constraint categories: memory, compute, power, real-time
- Opening failure case: smart doorbell battery drain

Activities:

- **Lecture (150 min):** Case study walkthrough, constraint taxonomy, design space introduction
- **Lab (0 min):** No lab this week (setup week)
- **Discussion:** Students share examples of embedded systems they've encountered. Classify each as constrained vs. elastic.

Assignments:

- **Reading:** Chapter 1, Appendix A (constraint template)
- **Homework 1 (due Week 2):** Analyze a consumer IoT device (Fitbit, Ring doorbell, smart thermostat). Identify its constraints (memory, compute, power, connectivity). 2 pages.

Assessment: Homework 1 (formative, not graded—provides baseline)

Week 2: Embedded Constraints as Design Variables

Chapter: 2

Learning Outcomes:

- LO2.1: Specify the memory, compute, power, and timing constraints of a given embedded target
- LO2.2: Translate datasheet specifications into design constraints relevant to AI deployment
- LO2.3: Determine whether a proposed deployment target is feasible before selecting a model

Topics:

- Memory hierarchy: flash, SRAM, external RAM
- Compute budget: clock speed, throughput, and inference operations
- Power budget: duty cycle, battery life math

- Real-time constraints: hard vs. soft deadlines
- Reading datasheets for AI suitability

Activities:

- **Lecture (150 min):** Constraint quantification, worked examples (nRF52840, ESP32-S3, STM32L4R5, Raspberry Pi Zero 2 W)
- **Lab (120 min):** Datasheet scavenger hunt—students extract constraints from 3 microcontroller datasheets and compare viability for a keyword spotting application

Assignments:

- **Reading:** Chapter 2, Appendix A.2–A.4 (datasheet guide, calculation worksheets)
- **Homework 2 (due Week 3):** Constraint profile comparison—STM32F4 vs. STM32H7 vs. Raspberry Pi 4. Fill out constraint template for each. Which can run a 500K-parameter CNN? Justify with calculations. 3 pages.

Assessment: Homework 2 (10% of grade)

Week 3: Machine Learning for Embedded Engineers

Chapter: 3

Learning Outcomes:

- LO3.1: Explain supervised learning, inference, and the trained model as artifacts with specific resource profiles
- LO3.2: Identify the computational operations a neural network performs during inference
- LO3.3: Map neural network architecture choices to memory and compute requirements
- LO3.4: Assess which class of ML model is appropriate for a given embedded sensing task

Topics:

- Supervised learning: training vs. inference
- Neural network anatomy: layers, weights, activations
- The inference pass: forward propagation
- CNNs, RNNs, MobileNets, decision trees
- ML-to-embedded translation table

Activities:

- **Lecture (150 min):** ML literacy from inference perspective, architecture survey

- **Lab (120 min):** Open a `.tflite` file, examine its structure using TensorFlow Lite's analyzer tool. Calculate parameter count, memory footprint, FLOP count for MobileNetV2.

Assignments:

- **Reading:** Chapter 3, Appendix B (ML quick reference)
- **Homework 3 (due Week 4):** Model resource profiling—given three models (MobileNetV2-0.35, LSTM, decision tree), calculate memory and compute requirements. Which fits on an nRF52840? 2 pages + calculations.

Assessment: Homework 3 (10% of grade)

Week 4: Inference Mechanics

Chapter: 4

Learning Outcomes:

- LO4.1: Trace the inference execution path of a deployed model on a target microcontroller
- LO4.2: Identify the computational bottleneck in a given model-hardware pairing
- LO4.3: Determine whether a proposed model-hardware configuration can meet a specified inference latency requirement

Topics:

- Inference execution pipeline (bare-metal vs. RTOS)
- Operator-level execution: convolution, matrix multiply, activations
- Memory allocation during inference (static vs. dynamic)
- Latency profiling on target hardware
- Numerical precision: float32 vs. int8

Activities:

- **Lecture (150 min):** Inference mechanics, profiling techniques, worked example (keyword spotting on Arduino Nano 33 BLE)
- **Lab (120 min):** Deploy a pretrained TFLite model to Arduino Nano 33 BLE Sense. Measure latency using cycle counters. Profile layer-wise timing.

Assignments:

- **Reading:** Chapter 4, Appendix D.1 (TFLite Micro setup)

- **Project Milestone 1 (due Week 5):** Select an embedded AI application for semester project. Write 1-page proposal: application description, constraints, success criteria.

Assessment: Project Milestone 1 (5% of grade)

Week 5: Memory Constraints

Chapter: 5

Learning Outcomes:

- LO5.1: Calculate the flash and SRAM requirements of a given model for a specified target
- LO5.2: Identify which components of a model consume the most memory and why
- LO5.3: Determine whether a model fits a target's memory envelope and specify what must change if it does not

Topics:

- Flash vs. SRAM: where weights live, where activations live
- Weight memory: parameter count $\times$ precision
- Activation memory: the hidden footprint
- Memory layout strategies: in-place computation, buffer reuse
- Static memory allocation for inference

Activities:

- **Lecture (150 min):** Memory constraint analysis, worked example (model that barely fails memory constraint, fixed via optimization)
- **Lab (120 min):** Memory profiling—measure actual SRAM usage during inference on Arduino. Identify activation memory vs. weight memory. Compare to theoretical calculations.

Assignments:

- **Reading:** Chapter 5
- **Homework 4 (due Week 6):** Memory constraint evaluation—given a model and target (STM32H7), calculate flash and SRAM requirements. Model exceeds SRAM by 50 KB. Propose three solutions with tradeoffs. 3 pages.

Assessment: Homework 4 (10% of grade)

Week 6: Compute and Latency

Chapter: 6

Learning Outcomes:

- LO6.1: Estimate the inference latency of a model on a specified hardware target using FLOP count and throughput specifications
- LO6.2: Identify the operations that dominate inference compute time and explain why
- LO6.3: Determine whether a proposed model-hardware-deadline combination is feasible and specify the required changes if it is not

Topics:

- FLOPs, MACs, and actual inference time
- Compute-bound vs. memory-bound inference
- Clock speed, SIMD, and throughput
- Worst-case execution time (WCET) analysis
- The latency-accuracy tradeoff

Activities:

- **Lecture (150 min):** Compute constraint analysis, WCET challenges
- **Lab (120 min):** Latency profiling—run inference on multiple hardware targets (Arduino, STM32). Measure mean, 95th percentile, max latency. Compare to theoretical predictions.

Assignments:

- **Reading:** Chapter 6
- **Quiz 1 (in class, Week 7):** Covers Chapters 1–6 (constraints, ML basics, inference mechanics, memory, compute). 30 minutes.

Assessment: Quiz 1 (10% of grade)

Week 7: Power and Energy

Chapter: 7

Learning Outcomes:

- LO7.1: Estimate the energy cost of a single inference event on a specified hardware target
- LO7.2: Determine whether a proposed AI deployment strategy is compatible with a given power budget

- LO7.3: Compare inference scheduling strategies on energy cost and select the optimal strategy for a specified application

Topics:

- Power vs. energy: the distinction that matters
- Dynamic and static power in AI workloads
- Duty cycling and inference scheduling
- Energy harvesting constraints
- Event-driven AI: low-power sensors trigger inference

Activities:

- **Lecture (150 min):** Power constraint analysis, duty cycle optimization, worked example (wildlife camera on AA batteries)
- **Lab (120 min):** Power profiling—measure current draw during inference using a power profiler or multimeter. Calculate battery life for different duty cycles.

Assignments:

- **Reading:** Chapter 7
- **Project Milestone 2 (due Week 8):** For your semester project, quantify all four constraints (memory, compute, power, latency). Fill out the constraint template from Appendix A. 2 pages.

Assessment: Project Milestone 2 (5% of grade)

Week 8: Midterm Exam / Project Check-In

No new chapter—consolidation week

Activities:

- **Midterm Exam (150 min):** Covers Chapters 1–7 (all constraint evaluation topics). Mix of calculations, short answer, and constraint analysis problems.
- **Project office hours (120 min):** Students meet with instructor to discuss project progress and get feedback on Milestone 2.

Assignments:

- **Reading:** Review Chapters 1–7, prepare for midterm

Assessment: Midterm Exam (20% of grade)

Week 9: Hardware Accelerators

Chapter: 8

Learning Outcomes:

- LO8.1: Explain how NPUs, DSPs, and FPGAs accelerate AI inference differently
- LO8.2: Compare the memory, compute, power, and flexibility tradeoffs of different acceleration architectures
- LO8.3: Select an appropriate hardware acceleration strategy for a given application's constraint profile

Topics:

- Why CPUs are inefficient for neural networks
- DSPs for AI: fixed-point arithmetic, SIMD
- NPUs and microNPUs: what they accelerate well
- FPGAs for embedded AI
- Integrated AI hardware: real devices, real tradeoffs
- When NOT to use a hardware accelerator

Activities:

- **Lecture (150 min):** Hardware acceleration options, worked example (same model, three targets)
- **Lab (120 min):** Hardware comparison—run the same model on CPU-only (Arduino) vs. NPU-accelerated (if available, e.g., Coral Dev Board). Compare latency and power.

Assignments:

- **Reading:** Chapter 8, Appendix C (hardware reference)
- **Homework 5 (due Week 10):** Hardware selection—given an application with constraints (latency < 50 ms, power < 500 mW, cost < \$15), evaluate three hardware options. Which meets all constraints? 3 pages.

Assessment: Homework 5 (10% of grade)

Week 10: Communication and the Edge-Cloud Spectrum

Chapter: 9

Learning Outcomes:

- LO9.1: Quantify the latency, bandwidth, power, and privacy costs of offloading inference to the cloud
- LO9.2: Determine the appropriate processing location (device, edge, cloud) for a given application's requirements
- LO9.3: Design a hybrid inference strategy that distributes computation across processing tiers

Topics:

- The edge-cloud spectrum: on-device, near-edge, far-edge, cloud
- Communication cost as a constraint: latency, bandwidth, energy, money
- Privacy and regulatory constraints on data transmission
- Split inference: early layers on-device, later layers on server
- Federated learning basics (conceptual only)

Activities:

- **Lecture (150 min):** Edge-cloud tradeoffs, worked example (predictive maintenance with three deployment strategies)
- **Lab (120 min):** Split inference simulation—partition a model, run early layers on Arduino, send activations to laptop (simulating edge server), run later layers. Measure end-to-end latency.

Assignments:

- **Reading:** Chapter 9
- **Project Milestone 3 (due Week 11):** For your project, specify processing architecture (on-device, edge, cloud, hybrid). Justify with latency/bandwidth/power/privacy analysis. 2 pages.

Assessment: Project Milestone 3 (5% of grade)

Week 11: Real-Time AI

Chapter: 10

Learning Outcomes:

- LO10.1: Identify the properties of AI inference that make WCET analysis difficult and explain why determinism matters for real-time systems
- LO10.2: Assess whether a proposed AI component can be safely integrated into a hard real-time system
- LO10.3: Apply design patterns that reduce the real-time risk of AI inference in safety-critical applications

Topics:

- Hard vs. soft real-time requirements
- Non-determinism in neural network inference
- WCET analysis for AI: what works, what doesn't
- Safety-critical AI: IEC 61508, ISO 26262
- Design patterns for real-time-safe AI: AI as advisory, bounded execution, confidence gating

Activities:

- **Lecture (150 min):** Real-time constraints, safety standards, worked example (integrating AI into hard real-time control loop)
- **Discussion (120 min):** Case study—autonomous vehicle perception failure (Uber 2018). What went wrong? How could real-time design patterns have prevented it?

Assignments:

- **Reading:** Chapter 10
- **Quiz 2 (in class, Week 12):** Covers Chapters 8–10 (hardware, communication, real-time). 30 minutes.

Assessment: Quiz 2 (10% of grade)

Week 12: Selecting Models for Constrained Deployment

Chapter: 11

Learning Outcomes:

- LO11.1: Compare model architectures on memory, compute, power, and accuracy tradeoffs for a specified deployment target
- LO11.2: Select a model architecture appropriate for a constrained embedded target and justify the selection against the constraint profile
- LO11.3: Use deployment-aware benchmarks (latency on target hardware, not accuracy on ImageNet) to evaluate model candidates

Topics:

- Why ImageNet accuracy is the wrong metric
- Deployment-aware metrics: latency, memory, energy, accuracy on your dataset
- Efficient architecture families: MobileNet, EfficientNet-Lite, SqueezeNet
- Task-specific model selection
- The accuracy-efficiency Pareto frontier

- Neural Architecture Search (NAS) for embedded targets

Activities:

- **Lecture (150 min):** Model selection framework, worked example (wearable fall detection—5 candidates evaluated)
- **Lab (120 min):** Model selection exercise—students evaluate 3 models for a given application. Measure latency on target. Plot Pareto frontier.

Assignments:

- **Reading:** Chapter 11
- **Project Milestone 4 (due Week 13):** For your project, select a model. Justify selection based on measured latency, memory, and accuracy on your dataset. Compare to at least 2 alternatives. 3 pages.

Assessment: Project Milestone 4 (10% of grade)

Week 13: Optimizing Models

Chapter: 12

Learning Outcomes:

- LO12.1: Apply post-training quantization to a model and measure the accuracy-efficiency tradeoff
- LO12.2: Compare quantization, pruning, and knowledge distillation on their effects on model size, latency, power, and accuracy
- LO12.3: Select and apply the appropriate compression technique(s) for a specified model-hardware-accuracy constraint set

Topics:

- Quantization: float32 → int8, what changes, what breaks
- Post-training quantization vs. quantization-aware training
- Pruning: structured vs. unstructured
- Knowledge distillation: training a small model to mimic a large model
- Combining techniques: quantize, prune, distill—ordering matters
- The accuracy floor: when compression breaks the application

Activities:

- **Lecture (150 min):** Optimization techniques, worked example (compressing a model that barely fails constraints)

- **Lab (120 min):** Apply post-training quantization to your project model. Measure accuracy before/after. If accuracy loss > 5%, apply iterative optimization.

Assignments:

- **Reading:** Chapter 12, Appendix B.5 (quantization reference)
- **Homework 6 (due Week 14):** Optimization challenge—given a model that exceeds flash by 100 KB and latency by 70 ms, apply pruning and quantization to meet constraints. Report accuracy loss. 3 pages.

Assessment: Homework 6 (10% of grade)

Week 14: Toolchains and Deployment Pipelines

Chapter: 13

Learning Outcomes:

- LO13.1: Convert a trained model to a deployment format (TFLite, ONNX) and deploy it to a target microcontroller using a standard toolchain
- LO13.2: Identify the decisions made by the toolchain that affect inference accuracy, latency, and memory usage
- LO13.3: Select an appropriate toolchain for a specified hardware target and deployment requirement

Topics:

- The deployment pipeline: train → convert → optimize → compile → deploy → verify
- TensorFlow Lite for Microcontrollers
- Edge Impulse: end-to-end toolchain
- ONNX and vendor-specific compilers (STM32Cube.AI)
- Toolchain-induced accuracy loss
- Testing and validation after deployment

Activities:

- **Lecture (150 min):** Toolchain overview, worked example (Keras to Arduino Nano 33 BLE full pipeline)
- **Lab (120 min):** Full deployment—students convert their project model, deploy to target hardware, verify accuracy and latency match predictions.

Assignments:

- **Reading:** Chapter 13, Appendix D (toolchain setup guides)

- **Final Project Report (due Week 15):** Complete integration decision document for your project. See rubric below.

Assessment: Final Project (30% of grade)

Week 15: Integration Case Studies / Final Presentations

Chapter: 14

Learning Outcomes:

- LO14.1: Apply the complete AI integration evaluation framework to a novel embedded design problem
- LO14.2: Produce a structured integration decision document that specifies the model, hardware, deployment strategy, and constraint justification for a given application
- LO14.3: Critique a proposed AI embedded system design and identify the decisions most likely to fail in production

Topics:

- Case A: Industrial anomaly detection (vibration monitoring)
- Case B: Medical wearable (cardiac arrhythmia detection)
- Case C: Agricultural edge sensor (crop disease detection)
- Synthesis: what the case studies teach

Activities:

- **Final Project Presentations (150 min):** Each student/team presents their project (10–15 min per presentation). Format: problem statement, constraints, hardware selection, model selection, optimization applied, deployment results, alternatives rejected and why.
- **Peer evaluation (120 min):** Students critique peer projects using the evaluation framework from Chapter 14.

Assignments:

- **Final Project Report (due end of Week 15):** 10–15 page report documenting the full integration decision for the semester project.

Assessment:

- Final Project Presentation (10% of grade)
- Final Project Report (remaining portion of 30% final project grade)

Assessment Breakdown

Component	Weight	Description
Homework 2	10%	Constraint profile comparison
Homework 3	10%	Model resource profiling
Homework 4	10%	Memory constraint evaluation
Homework 5	10%	Hardware selection
Homework 6	10%	Optimization challenge
Quiz 1 (Week 7)	10%	Chapters 1–6
Quiz 2 (Week 12)	10%	Chapters 8–10
Midterm Exam (Week 8)	20%	Chapters 1–7
Semester Project	30%	Broken down below

Total **120%** **Best 100% counts**

Semester Project Components (30% total)

Milestone	Weight	Due Week
Milestone 1: Proposal	5%	Week 5
Milestone 2: Constraint quantification	5%	Week 8
Milestone 3: Processing architecture	5%	Week 11
Milestone 4: Model selection	10%	Week 13
Final Presentation	10%	Week 15
Final Report	65%	Week 15

Note: Total assessment adds to 120%. The best 100% counts (allows students to drop lowest homework or quiz).

Final Project Report Rubric

Total: 100 points

1. Problem Statement and Requirements (10 points)

- Clear application description
- Functional requirements specified
- Constraints quantified (memory, compute, power, latency)
- Success criteria defined

2. Hardware Selection (15 points)

- At least 3 candidates evaluated
- Constraint satisfaction analysis for each
- Selection justified with calculations
- Alternatives rejected with rationale

3. Model Selection (15 points)

- At least 3 models evaluated
- Deployment-aware metrics measured (latency, memory, accuracy on target)
- Selection justified against constraints
- Alternatives rejected with rationale

4. Optimization (15 points)

- Optimization techniques applied (quantization, pruning, etc.)
- Accuracy-efficiency tradeoff documented
- Before/after measurements provided
- Accuracy floor identified

5. Deployment and Validation (20 points)

- Model successfully deployed to target hardware
- Inference latency measured and verified
- Accuracy validated on target
- Memory usage profiled
- Power consumption measured (if battery-powered)

6. Integration Decision Document (15 points)

- Structured documentation of all decisions
- Constraint justification for each choice
- List of alternatives rejected and why
- Conditions under which rejected alternatives would be correct

7. Presentation Quality (10 points)

- Clear, well-organized slides
 - Effective communication of technical content
 - Time management (within 10–15 min limit)
 - Answers to peer/instructor questions
-

Lab Equipment Requirements

Per Student (or per team of 2)

- Arduino Nano 33 BLE Sense (or equivalent: STM32 Nucleo, ESP32 DevKit)
- USB cable
- Laptop with TensorFlow, Arduino IDE, or STM32CubeIDE installed

Per Lab Section (shared)

- 2× STM32 Nucleo boards (H7 series preferred)
- 2× Raspberry Pi Zero 2 W (for comparison)
- 1× Coral Dev Board or Edge TPU USB accelerator (optional, for Week 9)
- 2× Power profilers (e.g., Nordic PPK II, Keysight N6705C, or multimeters with logging)
- Oscilloscope (for advanced power profiling, optional)

Software (free)

- TensorFlow 2.14+
 - Arduino IDE or PlatformIO
 - STM32CubeIDE (free, requires ST account)
 - Edge Impulse CLI (free tier)
 - Python 3.8+ with Jupyter
-

Additional Resources for Instructors

Lecture Slides

Provided as companion material (not included in this document). Each chapter has 50–80 slides covering:

- Key concepts
- Worked examples

- Calculation walkthroughs
- Code snippets

Lab Handouts

Provided as companion material. Each lab includes:

- Objectives
- Step-by-step procedures
- Expected results
- Troubleshooting guide

Sample Exam Questions

Provided as companion material. Question bank includes:

- 50 multiple-choice questions
- 30 short-answer questions
- 20 calculation problems
- 10 design problems

Grading Scripts

Python scripts for auto-grading constraint calculations and model profiling assignments.

Adapting This Course

For 10-Week Quarter System

Compress to 10 weeks by:

- Combining Weeks 1–2 (intro + constraints) into one week
- Skipping Week 8 midterm (replace with take-home exam)
- Combining Weeks 12–13 (model selection + optimization) into one week
- Reducing project milestones to 2 (proposal + final)

For Graduate-Level Course

Add depth:

- **Week 3:** Add training fundamentals (backpropagation, gradient descent)
- **Week 10:** Add federated learning implementation (not just conceptual)

- **Week 11:** Add formal WCET analysis tools (aiT, SWEET)
- **Week 13:** Add neural architecture search lab (students run NAS)
- **Final Project:** Require quantitative evaluation against published benchmarks (MLPerf Tiny)

For Purely Online Course

Adapt labs:

- Use TinkerCAD or Wokwi for Arduino simulation (no physical hardware required)
- Use Google Colab for model training and profiling
- Ship hardware kits to students (Arduino Nano 33 BLE Sense + sensors)
- Record lab demonstrations as video walkthroughs

Learning Outcomes Alignment with ABET Criteria

This course addresses the following ABET ECE/CS outcomes:

1. An ability to identify, formulate, and solve complex engineering problems

→ Semester project requires students to formulate constraint optimization problem and solve it through hardware/model selection.

2. An ability to apply engineering design to produce solutions that meet specified needs with consideration of public health, safety, and welfare

→ Chapter 10 (real-time AI) and Case Study B (medical wearable) address safety-critical design.

3. An ability to acquire and apply new knowledge as needed, using appropriate learning strategies

→ Students learn ML from scratch (no ML prerequisite) and apply it to embedded systems integration.

4. An ability to function effectively on a team

→ Lab activities and semester project can be team-based (optional, instructor's choice).

5. An ability to develop and conduct appropriate experimentation, analyze and interpret data

→ Labs require students to measure latency, memory, power on actual hardware and analyze results.

This course calendar provides a complete 15-week roadmap for teaching embedded AI integration using the textbook. Instructors can adapt timing, labs, and assessments to their institution's needs while maintaining the three-act learning arc.

Chapter 1: When AI Meets Constrained Hardware

On November 12, 2019, a smart doorbell manufactured by a well-known home security company began draining its battery in forty-eight hours instead of the advertised six months. The doorbell had shipped with a firmware update three weeks earlier—an update that added person detection using an on-device neural network. The hardware hadn't changed. The battery capacity hadn't changed. The update added AI, and the product failed in production.

The failure wasn't catastrophic. No one was injured. No homes were burgled because a doorbell died. But twelve thousand units were returned in the first month, support tickets flooded the company's queue, and the feature was rolled back by December. The engineering team had tested the AI model in isolation. They had verified its accuracy. They had confirmed it could run on the device's processor. What they hadn't done was measure how much power the model consumed during continuous operation, compare that consumption against the device's sleep/wake duty cycle, and project battery life under realistic deployment conditions.

This is not a story about bad engineering. It's a story about integration without evaluation—about treating AI as software that happens to run on hardware, rather than as a design decision with resource consequences that must be quantified before deployment. The doorbell team knew how to design embedded systems. They knew how to integrate sensors, manage power budgets, and optimize firmware. But when AI entered the system, they treated the model as a black box: something that either "worked" or "didn't work," rather than something with a measurable cost in memory, compute cycles, power draw, and latency.

This chapter establishes the problem that defines this entire book: artificial intelligence and embedded systems exist in tension, and that tension produces design constraints that cannot be ignored. By the end of this chapter, you will be able to describe why AI deployment on constrained hardware is not a technology trend but a design problem, distinguish between applications where on-device AI is necessary versus optional, and identify the four constraint categories—memory, compute, power, and real-time performance—that will determine whether any proposed AI integration is feasible.

You are here because you need to make AI work on hardware that doesn't want to run it.

What "Embedded" Means as a Constraint Environment

An embedded system is not defined by what it is. It is defined by what it cannot do.

A laptop can swap memory to disk when RAM runs out. A cloud server can autoscale compute resources when demand spikes. A desktop GPU can draw three hundred watts from a wall socket without consequence. An embedded system cannot do any of these things. It has the

memory it shipped with. It has the processor it was designed around. It has a power budget determined by a battery, an energy harvester, or a constrained supply rail. When you exceed those limits, the system doesn't slow down gracefully—it fails.

This is the first principle of embedded design: resources are fixed, and exceeding them is not recoverable. There is no elasticity. There is no "try again with more RAM." You design within the envelope or you don't deploy.

The word "embedded" is often misused as a synonym for "small" or "low-cost" or "IoT device." These correlations exist, but they are not the definition. A Raspberry Pi 4 is not constrained in the same way an Arduino Nano 33 is constrained, even though both are "embedded" in casual usage. A Tesla's autopilot computer is embedded, but it has more compute power than most laptops. The defining characteristic of an embedded system is not size or cost—it is that the system has been purpose-built for a specific application, and its resources have been allocated with no overhead for generality.

This matters because AI models are general-purpose artifacts. A neural network trained to detect faces doesn't "know" it's running on a battery-powered camera. It doesn't care whether the processor has a floating-point unit or whether SRAM is shared with the communication stack. The model's resource requirements are determined by its architecture—by the number of layers, the size of the weight matrices, the precision of the arithmetic—and those requirements are fixed when the model is trained. The embedded system, meanwhile, has its own fixed resource budget determined by cost, power, thermal, and application constraints.

Integration is the design problem that emerges when you try to fit a fixed-resource-demand artifact into a fixed-resource-supply environment.

Consider a wearable health monitor—a wristband that tracks heart rate, activity, and sleep. The device runs on a coin cell battery that must last at least fourteen days between charges. It has 256 KB of SRAM and 1 MB of flash memory. Its microcontroller runs at 64 MHz with no hardware floating-point unit. These constraints are not negotiable. They were set by industrial design (the device must be small and light), cost targets (coin cells are cheap, lithium-polymer packs are not), and user experience requirements (fourteen-day battery life is the competitive threshold).

Now suppose you want to add on-device heart arrhythmia detection using a recurrent neural network. The model has been trained and achieves 94% accuracy on a validation dataset. It requires 180 KB of memory for weights, 60 KB for intermediate activations during inference, and performs 22 million multiply-accumulate operations per inference pass. The model runs. But does it fit?

The memory budget is consumed: 180 KB for weights, 60 KB for activations, which leaves 16 KB for the rest of the firmware stack—sensor drivers, Bluetooth communication, user interface, and power management. That's tight, but possible with careful optimization. The compute budget is more problematic. At 64 MHz with no FPU, the processor can perform roughly 64

million integer operations per second—but floating-point operations require software emulation, which degrades throughput by a factor of ten or more. The 22 million multiply-accumulate operations become 220 million equivalent integer operations, which translates to roughly 3.4 seconds per inference pass. The model must run every thirty seconds to catch arrhythmia events, which means the processor is active for 3.4 seconds out of every 30-second window—an 11% duty cycle.

That duty cycle determines power consumption. If the processor draws 15 mA when active and 5 μ A when sleeping, the average current draw is 1.7 mA. A 220 mAh coin cell can sustain that for roughly 130 hours—five days, not fourteen. The integration fails the battery life constraint.

This is not a hypothetical. Deployed wearables have failed this way. The failure mode is not that the model crashes or produces incorrect results. The failure mode is that the device works exactly as designed—and runs out of power too quickly to be useful.

Embedded constraints are not bugs. They are the design space.

Why AI Moved to the Edge

If embedded systems are constrained and AI models are resource-intensive, the obvious solution is to avoid running AI on embedded systems. Send the data to a server. Let the cloud handle inference. Return the result over a network connection. This is the architecture that dominated the first wave of commercial AI applications: Alexa sends your voice to Amazon's servers, Google Photos runs recognition in the cloud, and autonomous vehicles upload sensor data for post-processing when connectivity allows.

But that architecture has failed for an entire class of applications—and it's those applications that motivate this book.

The failure modes are fourfold: latency, privacy, connectivity, and cost. Each one represents a constraint that cannot be relaxed by better server hardware, faster networks, or more efficient cloud infrastructure. They are fundamental to the application, not the implementation.

Latency becomes a hard constraint when the time between sensing and action is measured in milliseconds. Consider an industrial robotic arm performing quality inspection on a production line moving at two meters per second. A visual defect detection model must classify each part before it moves out of the inspection zone—a window of roughly 100 milliseconds. If inference happens in the cloud, the round-trip latency includes sensor readout, network transmission, server queueing, inference execution, and result return. Even on a fast local network, that round-trip can exceed 50 milliseconds, leaving only 50 milliseconds for the physical inspection apparatus to respond. On a congested network or over a cellular backhaul, the latency balloons to hundreds of milliseconds, and the inspection window closes before the result returns. The application requires on-device inference not because the cloud is slow, but because the speed of light is finite and the application cannot wait for a photon to travel to a datacenter and back.

Privacy becomes a hard constraint when the data itself cannot leave the device. Medical wearables that monitor cardiac rhythms or detect seizures handle protected health information governed by HIPAA in the United States and GDPR in Europe. Even if the user consents to cloud processing, the regulatory and liability risks of transmitting raw physiological data over a network often make cloud inference non-viable. The same constraint applies to home security cameras, workplace monitoring systems, and any application where the data has intrinsic sensitivity. On-device inference keeps the data local. The model runs where the sensor is, and only the classification result—not the raw input—leaves the device.

Connectivity becomes a hard constraint when the network is intermittent, expensive, or absent. Agricultural sensors deployed in remote fields may have cellular connectivity only when a vehicle passes by. Underwater inspection drones operate with no real-time uplink. Disaster response robots enter environments where infrastructure has failed. These applications cannot assume network availability. The model must run locally because the alternative is not "slower inference"—it is "no inference."

Cost becomes a hard constraint when network transmission dominates the operational budget. A smart parking sensor that sends a few bytes per hour—occupancy state, timestamp—can run for years on a coin cell. If that same sensor had to stream video to a cloud server for vehicle detection, the cellular data costs and power consumption would render the deployment economically infeasible. Even when connectivity exists, bandwidth has a price, and for low-margin, high-volume deployments, that price can exceed the cost of the hardware. On-device inference shifts compute cost from operational expense (recurring network charges) to capital expense (one-time hardware cost).

These four constraints—latency, privacy, connectivity, and cost—define the boundary where cloud AI stops working and edge AI becomes necessary. But necessity is not the same as feasibility. An application may require on-device inference and still fail to achieve it because the embedded hardware cannot meet the model's resource demands. That gap—between what the application requires and what the hardware can deliver—is the design space this book teaches you to navigate.

The Four Constraint Categories

Every AI integration decision on embedded hardware is governed by four resource constraints: memory, compute, power, and real-time performance. These are not independent. They interact, trade off against each other, and create a multidimensional design space where satisfying one constraint can violate another. Understanding how to quantify each constraint and how they couple is the foundational skill of embedded AI design.

Memory determines what you can store. A trained neural network is a large data structure—millions of weight parameters, layer configurations, and intermediate activation buffers. All of it must fit into the device's available memory. Embedded processors typically have two memory regions: flash (non-volatile storage for code and constant data) and SRAM (volatile

storage for runtime variables and computation). Flash is abundant but slow and read-only during execution. SRAM is fast and writable but scarce—often measured in kilobytes, not megabytes. A model's weights can live in flash, but its activations must live in SRAM during inference. If the activation memory exceeds available SRAM, the model cannot run, regardless of how much flash you have.

Compute determines how fast you can run. Inference is a sequence of mathematical operations—matrix multiplications, convolutions, activation functions—that must execute in a specific order. The time required to complete one inference pass is determined by the number of operations, the processor's throughput, and whether specialized hardware (like a floating-point unit or neural network accelerator) is available. Compute constraints manifest as latency: how long does it take to process one input and produce one output? For some applications, latency is soft—a delay of a few hundred milliseconds is acceptable. For others, latency is hard—exceeding the deadline means the result is useless.

Power determines how long you can run. Every compute operation consumes energy. Every memory access consumes energy. Energy comes from a battery, an energy harvester, or a power supply with a limited budget. If your model's average power consumption exceeds the available power budget, the device's battery life shrinks below acceptable thresholds, the thermal envelope is violated, or the power supply cannot sustain the load. Power constraints are often the binding constraint for battery-powered devices. A model that fits in memory and meets latency requirements can still fail if it drains the battery too quickly.

Real-time performance determines whether you can meet application deadlines. Real-time is not the same as "fast." A real-time system is one where correctness depends not just on the result but on when the result is delivered. A soft real-time system prefers to meet deadlines but can tolerate occasional misses. A hard real-time system must meet deadlines with provable guarantees—missing a deadline is a system failure. AI inference introduces variable latency: execution time can depend on input data, cache state, memory contention, and processor scheduling. For hard real-time applications, worst-case execution time must be bounded and proven, which is difficult when the model includes dynamic operations like attention mechanisms or conditional branches.

These four constraints do not exist in isolation. Reducing memory usage by quantizing weights from 32-bit floats to 8-bit integers reduces memory footprint and speeds up inference (fewer bytes to move), but may degrade model accuracy. Offloading inference to a hardware accelerator reduces latency and power consumption but introduces integration complexity and may increase cost. Running inference at a lower duty cycle (less frequently) reduces average power but increases latency between sensing and action. Every design decision in embedded AI is a trade-off between these constraints, and the designer who cannot quantify the trade-offs cannot make informed decisions.

The Design Space: Where Embedded Constraints and AI Requirements Collide

The design space for embedded AI is defined by the intersection of two surfaces: what the model demands and what the hardware can supply. When those surfaces overlap, deployment is feasible. When they don't, the designer must modify one or both.

Consider a keyword spotting application—a voice-activated interface that listens for a wake word like "Hey, device" and triggers a response. The model is a convolutional neural network trained on audio spectrograms. It has 42,000 parameters, requires 180 KB of memory (model weights plus activation buffers), and performs 1.8 million operations per inference pass. Inference must complete within 100 milliseconds to feel responsive, and the device runs on a 1000 mAh lithium-ion battery that must last at least one week with continuous listening.

Now consider two hardware targets.

Target A is a low-power ARM Cortex-M4 microcontroller running at 80 MHz with 256 KB of SRAM and a hardware floating-point unit. It draws 20 mA when active and 10 μ A when sleeping. Flash storage is 1 MB. Cost is \$3 per unit.

Target B is an ARM Cortex-M0+ microcontroller running at 48 MHz with 32 KB of SRAM and no FPU. It draws 8 mA when active and 2 μ A when sleeping. Flash storage is 256 KB. Cost is \$0.80 per unit.

Does the model fit on Target A? The memory constraint is satisfied: 180 KB required, 256 KB available. The compute constraint requires calculation. With an FPU, the processor can sustain roughly 80 million floating-point operations per second (FLOPS). The model requires 1.8 million operations, so inference time is approximately 22 milliseconds—well within the 100 ms latency budget. Power consumption depends on duty cycle. If the model runs continuously (inference every 100 ms), the processor is active 22% of the time. Average current draw is 4.4 mA, which gives a battery life of roughly 227 hours, or nine days. The model fits—barely. The battery life is tighter than the one-week target, but close enough to be acceptable with optimization.

Does the model fit on Target B? The memory constraint fails immediately: 180 KB required, 32 KB available. Even if you could compress the model to fit, the compute constraint likely fails as well. Without an FPU, floating-point operations are emulated in software, reducing throughput by a factor of ten or more. The 1.8 million operations would take 375 milliseconds or longer, violating the 100 ms latency requirement. Target B cannot run this model as-is.

But Target B is one-quarter the cost of Target A. For a consumer product with tight margins and high production volume, that cost difference is decisive. The designer now faces a choice: reject Target B and accept higher unit cost, or modify the model to fit Target B's constraints. Modifying the model might mean quantizing weights to 8-bit integers (reducing memory and enabling integer-only inference), pruning layers to reduce operation count, or switching to a smaller architecture like a depthwise separable convolution network. Each modification changes the model's accuracy, and if accuracy drops below the application's threshold, the model becomes useless regardless of cost savings.

This is the design space. You have a model with fixed resource demands and a hardware target with fixed resource supply. You have an application with minimum accuracy, latency, and battery life requirements. Your job is to find a configuration where all constraints are satisfied simultaneously—or prove that no such configuration exists and change the requirements.

The Structure of This Course

This book teaches you to navigate that design space by building the evaluation tools in deliberate sequence.

Part I (Chapters 1–3) establishes the problem. You now know why embedded AI is a constrained optimization problem. Chapter 2 quantifies each constraint category precisely—how to read datasheets, how to calculate memory footprints, how to translate processor specs into inference throughput. Chapter 3 builds your ML literacy at the level needed to reason about inference without needing to train models—what happens during a forward pass, how architectures map to resource demands, and which model classes are appropriate for which embedded sensing tasks.

Part II (Chapters 4–10) builds the diagnostic toolkit. Each chapter adds one constraint dimension and teaches you to evaluate AI integration against that dimension. By the end of Part II, you can audit a proposed deployment and identify every constraint failure before writing a line of code.

Part III (Chapters 11–14) extends the toolkit into model-aware design. The model is no longer fixed—it becomes a design variable. You learn to select architectures for deployment constraints, compress models through quantization and pruning, and navigate the toolchains that convert trained models into firmware. The final chapter synthesizes everything through three full case studies—industrial anomaly detection, medical wearable design, and agricultural edge sensing—where you apply the complete decision framework to real deployment problems.

The skill you will have at the end is this: given a sensing task, an application requirement, and a hardware target, you can determine whether AI integration is feasible, select a model and deployment strategy, justify every decision against resource constraints, and document the trade-offs you made and the alternatives you rejected.

That skill does not make you a machine learning researcher. It does not make you an embedded systems architect. It makes you a systems integrator who can reason across both domains—and that is the role the industry needs and currently lacks.

What You Cannot Learn from This Book Alone

This book teaches evaluation and integration, not invention. It assumes you can obtain a trained model—from a colleague, a model zoo, a vendor library, or a pre-trained architecture. It does not teach you how to train models from scratch, how to collect and label training datasets, or

how to design novel neural network architectures. Those are machine learning skills, and they are taught in machine learning courses.

It also assumes you have working knowledge of embedded systems fundamentals—digital logic, microcontroller programming in C or C++, basic real-time concepts, and familiarity with datasheets. If you lack that foundation, Appendix A provides a quick reference, but it is not a substitute for a proper embedded systems course.

What this book does teach—what no other book currently teaches—is how to evaluate the collision between ML models and embedded constraints, how to make integration decisions that survive deployment, and how to document those decisions so that future engineers (including future you) understand why the system is designed the way it is.

The doorbell camera that opened this chapter failed because the integration decision was not evaluated. The model worked. The hardware worked. But the system—the model running on that hardware under realistic operating conditions—did not work. The failure was not a software bug. It was a design error: a decision made without quantifying its resource consequences.

This book teaches you not to make that error.

You start by learning to quantify constraints—which is what Chapter 2 does next.

Chapter 2: Embedded Constraints as Design Variables

You already know what a microcontroller datasheet looks like. You've read power consumption tables, decoded memory maps, and calculated whether a peripheral can meet a timing requirement. If you've designed embedded systems before, this chapter will feel familiar—but the questions you ask of the data will be different.

When you evaluated hardware for a sensor acquisition system, you cared whether the ADC could sample at 10 kHz, whether the UART buffer could handle peak data rates, and whether sleep mode current was low enough for a year of battery life. Those constraints still matter. But when AI enters the system, you add new questions: Can this processor multiply a 128×128 matrix in under 50 milliseconds? Can this memory hierarchy store 400,000 floating-point weights and still leave room for activation buffers? Can this device run inference every 100 milliseconds for a week on a coin cell?

The specifications haven't changed. The microcontroller's SRAM capacity, clock speed, and power consumption are the same whether you're running a PID controller or a neural network. What has changed is how you translate those specifications into design constraints. This chapter teaches that translation—how to take the hardware you already understand and ask the new questions that AI integration demands.

By the end of this chapter, you will be able to specify the memory, compute, power, and timing constraints of any embedded target in terms relevant to AI deployment. You will be able to read a datasheet and determine whether a proposed hardware platform can support a given model before you write a single line of code. And you will be able to compare two targets and explain precisely why one can run an AI model and the other cannot.

The skill is not new datasheet literacy. The skill is knowing what to look for when the application includes inference.

Memory: What Fits and Where It Lives

A neural network is a data structure. It has weights, biases, layer configurations, and intermediate buffers that must exist somewhere in the device's memory. The first constraint question for any AI deployment is: does it fit?

But "fit" is not a single yes-or-no question. Embedded memory is partitioned into regions with different characteristics, and where data lives determines how it can be used. A model that "fits" in total memory may still fail if its runtime activations exceed the available fast memory, or if its weights must be paged from external storage during inference.

Consider the nRF52840, a popular Bluetooth Low Energy microcontroller used in wearables and IoT devices. The datasheet lists 1 MB of flash and 256 KB of RAM. At first glance, that sounds generous for embedded AI—enough to store a moderately sized model and run inference. But the memory specification requires interpretation.

Flash is non-volatile storage. It retains data when power is removed, which makes it suitable for storing the firmware binary, including the model's trained weights. Flash is read-only during execution—you cannot write to it from running code without invoking special erase and write cycles that are slow and have limited endurance. Flash is also slower to access than RAM, typically requiring wait states when reading at high clock speeds. For AI deployment, flash is where the model weights live. A model with 300,000 parameters stored as 32-bit floats requires 1.2 MB—which exceeds the nRF52840's 1 MB flash budget before you account for the rest of the firmware.

RAM (SRAM on most microcontrollers) is volatile, fast, and writable. It holds the variables, stack, heap, and buffers that change during program execution. For AI inference, RAM is where activations live—the intermediate results computed as data flows through the network. When you run inference on an image classification network, each layer produces an output tensor that becomes the input to the next layer. Those tensors must exist in memory simultaneously, at least for the duration of the inference pass. The size of the largest intermediate tensor, plus any scratch buffers required by the inference engine, determines the minimum RAM requirement.

The nRF52840's 256 KB of RAM is shared between the application firmware, the Bluetooth stack (which can consume 20–40 KB depending on configuration), the system heap, and the inference activations. If your model's activation memory footprint is 180 KB, and the Bluetooth

stack consumes 30 KB, you have 46 KB remaining for everything else. That's workable for a simple application, but tight. If the model's activations require 200 KB, the deployment fails the RAM constraint even though the model "fits" in total memory.

External memory complicates the picture. Some microcontrollers support external SRAM or DRAM via parallel or serial interfaces. The ESP32-S3, for example, includes integrated support for external PSRAM (pseudo-static RAM) up to 32 MB. This dramatically increases available memory, but at a cost: external memory is slower to access than on-chip SRAM, and accessing it consumes more power because data must traverse the external bus. If your model's activations live in external PSRAM, every layer's computation involves reading inputs from external memory and writing outputs back to external memory, which increases both latency and energy consumption.

The memory constraint question, then, is not "does the model fit?" but "does the model fit in the right kind of memory, and does the partitioning leave enough headroom for the rest of the system?"

To answer that question, you need two numbers from the model: the size of the weights (which determines flash usage) and the size of the activation buffers (which determines RAM usage). You need three numbers from the hardware: flash capacity, on-chip RAM capacity, and external memory capacity if available. And you need one number from the system: how much of that memory is already allocated to other functions.

The calculation is straightforward. For the weights:

Flash required = (number of parameters) × (bytes per parameter)

A model with 500,000 parameters stored as 32-bit floats requires 2 MB. Stored as 8-bit integers (quantized), it requires 500 KB. If the target has 1 MB of flash and 200 KB is already consumed by the application firmware, you have 800 KB available. The float32 model doesn't fit. The int8 model does.

For the activations, the calculation depends on the network architecture. For a feedforward network, you need memory for the largest layer's output tensor, plus any scratch buffers. For a convolutional network, you need memory for the current layer's output and sometimes the previous layer's output if they're used together. The exact requirement depends on the inference engine's memory allocation strategy—whether it reuses buffers or allocates fresh memory for each layer.

Many TinyML frameworks provide a memory profiler that reports activation memory usage for a given model. TensorFlow Lite for Microcontrollers, for example, includes a function that returns the required tensor arena size—the working memory needed for inference. If that number exceeds available RAM, the model doesn't deploy.

Here's what that looks like in practice. Suppose you're evaluating the ESP32-S3 for a keyword spotting application. The datasheet lists 512 KB of on-chip SRAM and support for up to 32 MB

of external PSRAM. The model has 80,000 parameters (320 KB as float32, 80 KB as int8) and requires 240 KB of activation memory during inference.

The weights fit comfortably in flash—the ESP32-S3 has 8 MB of flash, and even the float32 version uses only 320 KB. The activations are the binding constraint. With 512 KB of on-chip SRAM, the 240 KB activation requirement leaves 272 KB for the rest of the system. But if 150 KB is already consumed by the RTOS, WiFi stack, and application logic, you have 122 KB of headroom—tight, but feasible. If the system also needs large buffers for audio processing (the keyword spotting model runs on audio spectrograms), those buffers push SRAM usage over the limit, and activations spill into external PSRAM.

The design choice is now explicit: accept the latency and power cost of external PSRAM access, or reduce the model's activation footprint by using a smaller architecture, quantizing activations, or applying layer fusion techniques that reduce intermediate tensor sizes.

This is what "memory as a design variable" means. The constraint is not a hard wall—it's a trade-off surface where you balance model size, inference speed, power consumption, and system complexity.

Compute: Throughput, Latency, and What the Processor Can Actually Do

A microcontroller's clock speed tells you how many cycles per second the processor executes. It does not tell you how many neural network operations per second it can perform. That translation—from MHz to multiply-accumulates per second—is the second constraint you must quantify.

The gap between clock speed and useful throughput exists because different operations take different numbers of cycles, and not all cycles are compute cycles. Memory access consumes cycles. Pipeline stalls consume cycles. Interrupt handling consumes cycles. The effective compute throughput—the rate at which you can execute the multiply-accumulate operations that dominate neural network inference—depends on the processor's instruction set, its pipeline architecture, whether it has dedicated hardware for floating-point or integer arithmetic, and how efficiently the inference code maps to those resources.

Consider three representative targets: the STM32F411 (ARM Cortex-M4 at 100 MHz), the STM32H7A3 (ARM Cortex-M7 at 280 MHz), and the Raspberry Pi Zero 2 W (ARM Cortex-A53 quad-core at 1 GHz). All three are "embedded" in the sense that they're used in resource-constrained applications, but their compute capabilities differ by orders of magnitude.

The STM32F411 is a mid-range microcontroller. Its Cortex-M4 core includes a single-precision floating-point unit (FPU) that can execute one floating-point multiply-accumulate (MAC) per cycle when data is in registers. At 100 MHz, that's a theoretical peak of 100 million MACs per second (100 MMAC/s). But that's peak throughput, achieved only when the pipeline is fully

utilized and all data is in cache or registers. In practice, you lose cycles to memory access, instruction fetch, and pipeline bubbles. A realistic sustained throughput for inference is 60–70 MMAC/s for well-optimized code.

The Cortex-M4 also supports single instruction, multiple data (SIMD) operations via the ARM DSP extensions, which allow it to perform multiple 8-bit or 16-bit integer operations in parallel. For quantized neural networks that use 8-bit integer arithmetic, SIMD can double or quadruple throughput compared to scalar operations. But using SIMD requires careful code optimization—it's not automatic.

Now consider the STM32H7A3. Its Cortex-M7 core runs at 280 MHz and includes a double-precision FPU, deeper pipelines, and more aggressive prefetching and caching. Theoretical peak throughput is 280 MMAC/s for single-precision floats. Sustained throughput is typically 180–220 MMAC/s, depending on memory access patterns and cache efficiency. The H7 family also includes larger caches (16 KB L1 instruction, 16 KB L1 data, 512 KB L2) compared to the F4's smaller caches (4 KB instruction, no data cache), which reduces the performance penalty when working with large data structures like neural network weights.

The Raspberry Pi Zero 2 W is an entirely different class. It's a Linux-capable application processor with four Cortex-A53 cores, each capable of out-of-order execution, NEON SIMD extensions for vector operations, and clock speeds up to 1 GHz. The Cortex-A53 can issue multiple instructions per cycle, and NEON can execute 16 8-bit MACs per cycle per core. Theoretical peak throughput is over 60 billion operations per second (GOPS) for 8-bit integer inference. Sustained throughput is lower due to memory bandwidth limits and OS overhead, but still an order of magnitude higher than the STM32H7.

These differences matter because they determine how long inference takes. Suppose you have a convolutional neural network that requires 50 million MACs per inference pass. On the STM32F411 at 60 MMAC/s sustained, inference takes roughly 830 milliseconds. On the STM32H7 at 200 MMAC/s, it takes 250 milliseconds. On the Raspberry Pi Zero 2 W, with optimized NEON code, it might take 20–30 milliseconds.

If your application has a hard latency requirement of 100 milliseconds, the STM32F411 fails immediately. The STM32H7 fails if you cannot achieve 200 MMAC/s sustained (which you might not if memory accesses dominate). The Raspberry Pi meets the requirement comfortably—but at a cost in power, complexity, and boot time.

Here's the calculation you need to perform for any target. First, determine the model's operation count—the total number of MACs or floating-point operations (FLOPs) required per inference. Many ML frameworks report this during model compilation. If not, you can estimate it from the architecture: for a fully connected layer, the operation count is (input size) × (output size). For a convolutional layer, it's (output height) × (output width) × (output channels) × (kernel height) × (kernel width) × (input channels).

Second, estimate the processor's sustained throughput for the relevant operation type (float32 MACs, int8 MACs, etc.). This requires either profiling or consulting published benchmarks. ARM provides CoreMark scores and EEMBC benchmarks for their cores, which give rough guidance. Vendor application notes sometimes include ML inference benchmarks for their specific chips.

Third, calculate latency:

Inference latency (seconds) = (total operations) / (sustained throughput)

For example, a model with 120 million int8 MACs running on an ESP32-S3 (claimed 400 GOPS for 8-bit inference with acceleration) gives a theoretical latency of 0.3 seconds. But that assumes perfect utilization of the accelerator, which requires the model to be compiled specifically for the ESP32's AI extensions and all activations to fit in the optimized data path. In practice, you multiply the theoretical latency by a utilization factor (often 0.5–0.7) to account for real-world inefficiencies.

The compute constraint is satisfied if the calculated latency is less than the application's latency budget. If not, you have three options: choose a faster processor, reduce the model's operation count, or offload inference to a hardware accelerator.

Power: Average, Peak, and the Battery Life Equation

A neural network doesn't just consume time—it consumes energy. For battery-powered devices, energy is the binding constraint more often than memory or compute. A model that fits in memory and meets latency requirements can still fail if it drains the battery too quickly.

Power consumption in embedded systems is not a single number. It's a profile that varies with operating mode, clock speed, peripheral activity, and duty cycle. A microcontroller running inference draws one current when the processor is active at full clock speed, a different current when peripherals are active but the processor is idle, and a third current when the device is in deep sleep. Total energy consumption is the integral of that profile over time.

For AI deployment, the power profile has three regimes: inference (processor active, high clock, high power), idle (processor sleeping, peripherals off, low power), and peripheral activity (sensors active, processor sleeping or running at reduced clock). The duty cycle—the fraction of time spent in each regime—determines average power, and average power determines battery life.

Let's quantify this for a wearable fitness tracker using the nRF52840. The application runs a neural network to classify user activity (walking, running, sitting, cycling) from accelerometer data. Inference runs every 5 seconds. Between inference passes, the device sleeps.

The datasheet provides power specifications for different operating modes:

- Active (CPU running at 64 MHz, flash access, radio off): 5.3 mA

- Idle (CPU in sleep, RAM retention, RTC running): 2.8 μ A
- System off (RAM not retained, wake on reset): 0.4 μ A

Inference takes 80 milliseconds (measured on target, not theoretical). During those 80 ms, the processor is active and draws 5.3 mA. For the remaining 4920 ms of the 5-second cycle, the device is in idle mode and draws 2.8 μ A.

Average current is the time-weighted sum of each mode's current:

$$I_{avg} = (I_{active} \times t_{active} + I_{idle} \times t_{idle}) / (t_{active} + t_{idle})$$

For this profile:

$$\begin{aligned} I_{avg} &= (5.3 \text{ mA} \times 0.08 \text{ s} + 0.0028 \text{ mA} \times 4.92 \text{ s}) / 5 \text{ s} \\ I_{avg} &= (0.424 \text{ mA}\cdot\text{s} + 0.0138 \text{ mA}\cdot\text{s}) / 5 \text{ s} \\ I_{avg} &= 0.088 \text{ mA} \end{aligned}$$

A 220 mAh coin cell battery can sustain 0.088 mA for roughly 2500 hours, or 104 days. That exceeds the two-week target comfortably.

Now suppose the inference latency doubles to 160 milliseconds because you added another layer to improve accuracy. The active time per cycle increases from 80 ms to 160 ms:

$$\begin{aligned} I_{avg} &= (5.3 \text{ mA} \times 0.16 \text{ s} + 0.0028 \text{ mA} \times 4.84 \text{ s}) / 5 \text{ s} \\ I_{avg} &= 0.172 \text{ mA} \end{aligned}$$

Battery life drops to 1279 hours, or 53 days—still acceptable.

But suppose the model now requires inference every second instead of every 5 seconds to catch short-duration activities. With 160 ms inference time per 1-second cycle:

$$\begin{aligned} I_{avg} &= (5.3 \text{ mA} \times 0.16 \text{ s} + 0.0028 \text{ mA} \times 0.84 \text{ s}) / 1 \text{ s} \\ I_{avg} &= 0.850 \text{ mA} \end{aligned}$$

Battery life drops to 259 hours, or 11 days—below the two-week threshold. The power constraint is violated.

This is not a hypothetical calculation. Deployed wearables fail this way. The failure is not sudden—the device works correctly, it just runs out of power faster than users expect. The constraint violation manifests as a user experience problem: more frequent charging, which erodes the product's value proposition.

The power constraint is harder to satisfy than the memory or compute constraints because it couples to both. Reducing inference latency (compute constraint) by running at higher clock speed increases active current. Reducing memory usage by keeping activations in external RAM increases power because external memory accesses cost more energy than on-chip SRAM accesses. Every optimization that improves one constraint can degrade another.

Power also has a peak constraint, not just an average constraint. Even if average power is acceptable, peak power can violate supply limits. When a processor runs inference, current draw spikes. If that spike exceeds what the power supply or battery can deliver, the voltage sags, the processor brown-outs, or the system resets. This is especially problematic for coin cell batteries, which have high internal resistance and cannot sustain high current draws even though their total capacity is adequate.

For example, a CR2032 coin cell has a nominal capacity of 220 mAh, but its maximum continuous discharge current is typically 3 mA. If your inference routine draws 10 mA, the battery voltage collapses under load, and the system crashes. The solution is to use a larger battery, add a capacitor to supply peak current, or reduce peak power by slowing the clock or spreading the computation over time.

The power constraint equation you need is:

$$\text{Battery life (hours)} = \text{Battery capacity (mAh)} / \text{Average current (mA)}$$

But to get average current, you must profile the actual power consumption of inference on the target hardware, measure the duty cycle, and account for all system activity—not just the inference itself, but sensor power, communication overhead, and any background tasks.

Real-Time Constraints: Hard Deadlines and Worst-Case Execution Time

Latency is not the same as real-time performance. A system with 100 ms average latency is not the same as a system with a 100 ms worst-case latency guarantee. Real-time constraints care about the tail of the distribution, not the mean.

A real-time system is one where correctness depends on timing. A soft real-time system prefers to meet deadlines but tolerates occasional misses. A hard real-time system must meet deadlines with provable certainty—missing a deadline is a system failure, not a performance degradation.

Most embedded AI applications are soft real-time. A smart thermostat that classifies occupancy to adjust temperature can tolerate a delayed inference—the consequence is a slightly slower response, not a safety failure. But some applications are hard real-time. An industrial motor controller that uses AI to detect bearing faults must respond within a fixed time window or the fault progresses to catastrophic failure. A medical device that uses AI to detect cardiac arrhythmia must trigger an alert within a bounded time or the therapeutic window closes.

Neural network inference introduces variability that makes hard real-time guarantees difficult. Execution time can depend on:

- Input data (some activations may short-circuit, some may trigger expensive branches)

- Cache state (whether weights and activations are cached or must be fetched from main memory)
- Memory contention (other tasks or DMA transfers competing for bus bandwidth)
- Interrupt latency (if inference runs at normal priority and can be preempted)

For soft real-time applications, you measure average latency and ensure it's comfortably below the deadline with margin for variance. For hard real-time applications, you must bound the worst-case execution time (WCET) and prove that even the worst case meets the deadline.

WCET analysis for neural networks is an active research area with no settled practice. Traditional WCET analysis tools (like aiT or SWEET) model the processor's pipeline, cache, and memory system and compute the longest possible path through the code. But neural network inference code is large, complex, and often includes dynamic memory allocation and library calls that are difficult to analyze statically.

The pragmatic approach for hard real-time embedded AI is to constrain the problem:

1. Use a fixed-point or integer-only network with no dynamic control flow. Every layer executes the same operations regardless of input data.
2. Disable interrupts during inference or run inference at highest priority so it cannot be preempted.
3. Lock critical data (weights, activations) into cache or tightly coupled memory (TCM) to eliminate cache misses.
4. Measure WCET empirically by profiling inference on pathological inputs and adding safety margin (typically 20–50% over measured maximum).
5. Use a scheduling framework that enforces execution time budgets and kills tasks that overrun.

For example, suppose you have a vision-based inspection system that must classify parts on a conveyor belt moving at 1 meter per second. The inspection window is 200 ms. Inference must complete within that window, or the part leaves the field of view and the result is useless.

You profile inference on 10,000 test images and measure latency. The distribution shows a mean of 120 ms, a 95th percentile of 145 ms, and a maximum observed latency of 160 ms. Is 160 ms the WCET? Not necessarily—you may not have tested the input that triggers the worst case. You add 30% margin: $160 \text{ ms} \times 1.3 = 208 \text{ ms}$, which exceeds the 200 ms deadline.

The deployment fails the real-time constraint. You have three options: reduce the model's operation count to lower latency, increase the processor's clock speed, or redesign the inspection system to give inference more time (slow the conveyor, trigger earlier, etc.).

Real-time constraints are unforgiving. Average performance doesn't matter. Only the worst case matters. And if you cannot bound the worst case, you cannot deploy in a hard real-time application.

Reading Datasheets for AI Suitability

You have a model. You have an application with memory, compute, power, and real-time requirements. You need to select hardware. The datasheet is your source of truth—but datasheets are not written with AI deployment in mind, and extracting the relevant constraints requires knowing where to look and how to interpret marketing claims.

Start with memory specifications. Look for flash capacity, SRAM capacity, and external memory support. Flash capacity must accommodate the firmware plus model weights. SRAM capacity must accommodate the application heap, stack, RTOS overhead (if applicable), and inference activation buffers. If the datasheet lists "total memory" without breaking down flash vs. RAM, it's hiding something—usually that RAM is scarce.

Watch for memory region details. Some microcontrollers partition SRAM into tightly coupled memory (TCM), which is fast and deterministic, and standard SRAM, which may have wait states or arbitration delays. TCM is ideal for activations. If TCM is listed separately, note its size—it's often much smaller than total SRAM.

Next, examine compute specifications. Clock speed is the starting point, but not sufficient. Look for the processor core type (Cortex-M0, M4, M7, A53, etc.) and check whether it includes an FPU, DSP extensions, or SIMD support. A Cortex-M4 with FPU can run float32 inference efficiently. A Cortex-M0+ without FPU cannot—it will emulate floating-point in software, which is ten times slower.

If the datasheet claims "AI acceleration" or "neural network support," dig deeper. What does that mean? Some vendors include dedicated MAC units or vector processors that accelerate matrix operations. Others simply mean the chip is fast enough to run inference in software. Look for benchmark numbers: GOPS (giga-operations per second), CoreMark scores, or published inference latency for standard models like MobileNet or ResNet. If no benchmarks are provided, the "AI" claim is marketing.

Power specifications require the most careful interpretation. Datasheets list current consumption for different operating modes, but the definitions vary by vendor. "Active current" might mean CPU running at maximum clock with all peripherals enabled, or it might mean CPU running with flash access but peripherals off. "Sleep current" might include RAM retention or it might mean full power-down.

Find the power vs. clock speed curve. Most microcontrollers can run at reduced clock to save power. If your inference doesn't need maximum performance, running at a lower clock reduces both active current and the energy per inference. But some chips have stepped power profiles—reducing clock by 50% doesn't reduce power by 50% because static leakage and peripheral power remain constant.

Look for real-world power numbers, not just active and sleep. Application notes often include measured power for specific use cases. A note titled "Battery Life Estimation for Bluetooth LE

Sensors" might include the data you need to estimate inference power if the sensor acquisition profile is similar to your application.

Finally, check for errata. Every silicon chip has bugs. Some errata affect power consumption (e.g., "SRAM retention current higher than specified"), some affect peripherals, some affect timing. Read the errata document before committing to a platform. A bug that degrades sleep current by 10 μ A can destroy battery life projections for ultra-low-power designs.

Here's the workflow:

1. Identify flash and SRAM capacity. Compare against model weight size and activation memory requirement.
2. Identify processor core and clock speed. Look up published benchmarks or estimate throughput from architecture specs.
3. Calculate inference latency from model operation count and processor throughput.
4. Find active and sleep current. Estimate average power from duty cycle.
5. Calculate battery life from average power and battery capacity.
6. Check real-time capabilities: cache, TCM, interrupt latency, worst-case timing.

If any constraint fails, the target is not viable. If all constraints pass with margin, the target is a candidate. If constraints pass with no margin, revisit the assumptions—measurements on real hardware often differ from datasheet projections by 20–30%.

Comparing Targets: A Worked Example

You are designing a smart agriculture sensor that monitors crop health using a camera and on-device image classification. The system must:

- Capture a 96×96 RGB image every 10 minutes
- Run inference to classify healthy/diseased/pest-damaged
- Report results over LoRaWAN once per hour
- Operate for 6 months on a 5000 mAh battery pack (solar charging is not viable due to tree canopy shading)

You have three candidate platforms.

Option A: ESP32-S3

- Dual-core Xtensa LX7 at 240 MHz
- 512 KB SRAM, 8 MB flash
- AI acceleration (vector instructions, up to 400 GOPS claimed)
- WiFi and Bluetooth (not needed but included)
- Active current: 40 mA (CPU + AI accelerator)
- Deep sleep: 10 μ A
- Cost: \$4.50 per unit

Option B: STM32L4R5

- ARM Cortex-M4 at 120 MHz with FPU
- 640 KB SRAM, 2 MB flash
- No AI acceleration
- Ultra-low-power design
- Active current: $100 \mu\text{A}/\text{MHz} = 12 \text{ mA}$ at 120 MHz
- Deep sleep: $0.4 \mu\text{A}$
- Cost: \$6.00 per unit

Option C: Raspberry Pi Zero 2 W

- Quad-core ARM Cortex-A53 at 1 GHz
- 512 MB LPDDR2 RAM
- Linux-capable, extensive software ecosystem
- Active current: $\sim 150 \text{ mA}$ (idle), $\sim 350 \text{ mA}$ (CPU load)
- Sleep modes: not designed for ultra-low-power
- Cost: \$15.00 per unit

You have a MobileNetV2-based classifier with 300,000 parameters (1.2 MB as float32, 300 KB as int8) and requires 45 million MACs per inference. Profiled activation memory is 200 KB.

Memory analysis:

All three platforms have sufficient flash for the quantized model (300 KB). All have sufficient RAM for activations (200 KB). Memory is not the binding constraint.

Compute analysis:

ESP32-S3: With AI acceleration, 45 million MACs at 400 GOPS claimed throughput gives 112 ms theoretical, likely 200–300 ms in practice depending on how well the model maps to the accelerator.

STM32L4R5: Cortex-M4 with FPU at 120 MHz sustains roughly 70 MMAC/s for float32. Inference takes $45\text{M} / 70\text{M} = 643 \text{ ms}$. For int8 with SIMD, throughput doubles to $\sim 140 \text{ MMAC/s}$, giving 320 ms.

Raspberry Pi Zero 2 W: With NEON SIMD and four cores, sustained throughput for int8 inference can exceed 5 GOPS. Inference takes under 10 ms.

All three meet the compute constraint comfortably—inference once every 10 minutes allows latency up to several seconds without violating the application timing.

Power analysis:

This is where the platforms diverge.

The duty cycle is: inference every 10 minutes for ~300 ms (ESP32 or STM32), idle the rest of the time. LoRaWAN transmission is once per hour, negligible contribution to average power.

ESP32-S3:

Active for 0.3 s per 600 s cycle = 0.05% duty cycle.

$I_{avg} = (40 \text{ mA} \times 0.3 \text{ s} + 0.01 \text{ mA} \times 599.7 \text{ s}) / 600 \text{ s} = 0.03 \text{ mA}$.

Battery life = 5000 mAh / 0.03 mA = 166,667 hours = 19 years.

Wait. That can't be right. The sleep current is 10 μ A, not 0.01 mA—that's the same thing. Let me recalculate.

$I_{avg} = (40 \text{ mA} \times 0.3 \text{ s} + 0.01 \text{ mA} \times 599.7 \text{ s}) / 600 \text{ s} = (12 + 6) / 600 = 0.03 \text{ mA}$.

Actually, that is right. The ultra-low sleep current dominates the calculation when inference duty cycle is so low. This is why low-power embedded systems spend 99%+ of their time asleep.

STM32L4R5:

Active for 0.32 s per 600 s cycle (slightly longer inference).

$I_{avg} = (12 \text{ mA} \times 0.32 \text{ s} + 0.0004 \text{ mA} \times 599.68 \text{ s}) / 600 \text{ s} = 0.0064 \text{ mA} + 0.0004 \text{ mA} = 0.0068 \text{ mA}$.

Battery life = 5000 mAh / 0.0068 mA = 735,000 hours = 84 years.

The STM32L4's lower active current and even lower sleep current give it a multi-decade battery life. This is unrealistic—battery self-discharge and LoRaWAN overhead will dominate.

Raspberry Pi Zero 2 W:

The Pi doesn't have a true deep sleep mode. Best case, it idles at ~150 mA with reduced clock.

$I_{avg} \approx 150 \text{ mA}$ (continuous).

Battery life = 5000 mAh / 150 mA = 33 hours.

The Raspberry Pi fails the power constraint catastrophically. Even if you could reduce idle power to 50 mA (aggressive optimization), battery life is only 100 hours—four days, not six months.

Conclusion:

Option A (ESP32-S3) and Option B (STM32L4R5) both meet all constraints. The STM32 has better battery life and lower cost on paper, but the ESP32's AI accelerator gives faster inference and more headroom for future model updates. The Raspberry Pi is disqualified by power consumption.

In practice, you'd select based on secondary factors: development ecosystem, LoRaWAN library support, camera interface availability, and whether you trust the claimed AI acceleration of the ESP32 (which requires profiling on real hardware).

This is the constraint-driven evaluation this chapter teaches. The numbers are not abstract—they determine which hardware can deploy the model and which cannot. When you

move to Chapter 3, you'll learn how to derive those model numbers—the operation count, the memory footprint—from the neural network architecture itself.

But first, you must be able to read a datasheet and extract constraints. The model is coming. The hardware is here. The question is: can they fit together?

Chapter 3: Machine Learning for Embedded Engineers: A Functional Foundation

Open a `.tflite` file in a hex editor. You'll see a binary blob—structured data, not executable code. Somewhere in those bytes are floating-point numbers arranged in matrices, integers describing tensor shapes, and metadata specifying how data flows through a computational graph. This file is the artifact you will deploy to embedded hardware. It was produced by a training process you may never perform yourself, using datasets you may never see, on servers with resources your target device cannot imagine. But once training completes, that process is irrelevant to your work. The `.tflite` file is what remains—a fixed data structure with known size, known operations, and known resource demands.

This chapter teaches you to reason about that artifact. Not how to create it, but how to understand what it costs to execute. By the end of this chapter, you will know what happens during inference, how neural network architectures map to memory and compute requirements, which model classes are appropriate for which embedded sensing tasks, and how to translate machine learning terminology into the constraint language you learned in Chapter 2.

You will not learn how to train a model. You will not learn gradient descent, backpropagation, or loss functions. Those are the concerns of machine learning engineers. Your concern is: given a trained model, can it run on constrained hardware, and if so, how much will it cost?

This is ML literacy for integration, not for invention. It is enough.

Supervised Learning: The Process That Produces the Artifact

A neural network is a function approximator. You give it inputs, it produces outputs, and the relationship between inputs and outputs is determined by millions of parameters learned from data. The learning process is called training. The result of training is the model—the fixed set of parameters that define the function.

Supervised learning is the most common training paradigm for embedded AI applications. It works like this: you collect a dataset of input-output pairs where the outputs are labeled by humans or known from ground truth. For image classification, the inputs are images and the outputs are category labels ("cat," "dog," "car"). For keyword spotting, the inputs are audio spectrograms and the outputs are word labels ("yes," "no," "unknown"). For anomaly detection, the inputs are sensor readings and the outputs are binary labels ("normal," "fault").

The training algorithm adjusts the model's parameters to minimize the difference between the model's predictions and the true labels. This happens iteratively: the model makes predictions, compares them to the labels, computes an error, and updates the parameters to reduce that error. After thousands or millions of iterations over the dataset, the model's predictions become accurate on inputs it has seen. The real test is whether it generalizes—whether it produces accurate predictions on new inputs it has never encountered.

When training finishes, the model is frozen. The parameters no longer change. The model becomes a lookup table in the functional sense: given any input from the distribution it was trained on, it produces a prediction based on the learned parameters. Those parameters are what you store in flash memory. Those parameters are what determine the model's size.

Training happens once, typically on a GPU server with gigabytes of RAM and hours or days of compute time. Inference—running the trained model on new inputs—happens repeatedly, on your embedded device, with the resources you quantified in Chapter 2. Training is not your problem. Inference is.

This distinction is critical because the embedded systems literature sometimes conflates the two. A paper that claims "deep learning on microcontrollers" usually means inference on microcontrollers, not training. Training a neural network on a device with 256 KB of RAM is impractical for all but the smallest models and datasets. The models you deploy were trained elsewhere. Your job is to make them run efficiently in inference mode.

Neural Network Anatomy: Layers, Weights, and Activations

A neural network is a directed graph where each node is a computation and each edge carries data. In the most common architecture—the feedforward network—data flows from input to output through a sequence of layers. Each layer performs a transformation on its input and passes the result to the next layer. The final layer produces the model's output.

Each layer has two kinds of data: weights (also called parameters) and activations. Weights are fixed after training. Activations are computed during inference and depend on the input data.

Weights are the learned parameters that define the layer's transformation. For a fully connected layer, the weights are a matrix. For a convolutional layer, the weights are a set of small filters (kernels). The total number of weights in the network determines its memory footprint in flash. A

model with 500,000 weights stored as 32-bit floats occupies 2 MB. Stored as 8-bit integers, it occupies 500 KB.

Activations are the intermediate results computed as data flows through the network. When you run inference on an image, the first layer computes activations from the image pixels, the second layer computes activations from the first layer's output, and so on until the final layer produces the classification result. Activations live in RAM during inference. The size of the activation buffers determines the minimum RAM requirement.

Consider a simple feedforward network with three layers:

- Input layer: 784 neurons (28×28 grayscale image)
- Hidden layer: 128 neurons
- Output layer: 10 neurons (classification into 10 classes)

The hidden layer has weights connecting each input neuron to each hidden neuron: $784 \times 128 = 100,352$ weights. As 32-bit floats, that's 401 KB. The hidden layer also has 128 biases (one per neuron), adding another 512 bytes. Total weight memory for this layer: ~401 KB.

The output layer has $128 \times 10 = 1,280$ weights plus 10 biases. Total weight memory: ~5 KB.

Total model size: ~406 KB for weights. This fits comfortably in 1 MB of flash.

But what about activations? During inference, you compute the hidden layer's 128 activations and store them in RAM. As 32-bit floats, that's 512 bytes. Then you compute the output layer's 10 activations (40 bytes). The peak activation memory is the sum of the largest layer's output plus any scratch buffers needed for computation. For this simple network, that's under 1 KB—negligible compared to typical SRAM budgets.

This scales poorly. A convolutional network for image classification might have dozens of layers, each producing activation tensors with thousands or millions of elements. The activation memory requirement can exceed the weight memory requirement by an order of magnitude.

The Inference Pass: What Happens When Input Enters the Model

Inference is a forward pass through the network. You load an input (an image, an audio spectrogram, a sensor reading), feed it to the first layer, compute that layer's output, feed the output to the next layer, and repeat until you reach the final layer. The final layer's output is the model's prediction.

Each layer performs two operations: a linear transformation followed by a nonlinear activation function.

The linear transformation is a matrix multiplication (for fully connected layers) or a convolution (for convolutional layers). This is where the weights are used. For a fully connected layer, the transformation is:

$$\mathbf{y} = \mathbf{W} \times \mathbf{x} + \mathbf{b}$$

where $\mathbf{x}$ is the input vector, $\mathbf{W}$ is the weight matrix, $\mathbf{b}$ is the bias vector, and $\mathbf{y}$ is the output. The operation count is the number of multiply-accumulate (MAC) operations: (input size) $\times$ (output size). For a layer with 784 inputs and 128 outputs, that's 100,352 MACs.

The nonlinear activation function is applied element-wise to the output of the linear transformation. Common activation functions include ReLU (rectified linear unit), sigmoid, and tanh. ReLU is the simplest: it replaces negative values with zero. Mathematically:

$$\text{ReLU}(\mathbf{y}) = \max(0, \mathbf{y})$$

ReLU is fast—it's a single comparison and conditional assignment per element. Sigmoid and tanh involve exponentials, which are more expensive on embedded processors without hardware support.

After the activation function, the result becomes the input to the next layer. The process repeats until all layers have executed.

Here's the inference algorithm for a three-layer network:

1. Load input $\mathbf{x}$ (e.g., 784-element vector from a 28×28 image)
2. Compute hidden layer: $\mathbf{h} = \text{ReLU}(\mathbf{W}_1 \times \mathbf{x} + \mathbf{b}_1)$ (128 elements)
3. Compute output layer: $\mathbf{y} = \text{softmax}(\mathbf{W}_2 \times \mathbf{h} + \mathbf{b}_2)$ (10 elements)
4. Return $\mathbf{y}$ as the predicted class probabilities

The total operation count is $(784 \times 128) + (128 \times 10) = 101,632$ MACs, plus the cost of activation functions. On a processor sustaining 60 MMAC/s, this takes roughly 1.7 milliseconds. On a processor sustaining 10 MMAC/s (a slower Cortex-M4 without FPU, using integer arithmetic), it takes 10 milliseconds.

This is the inference cost: the number of operations that must execute, multiplied by the time per operation. The model's architecture determines the operation count. The processor's throughput determines the time per operation. Together, they determine latency.

Convolutional Neural Networks: Why They Dominate Embedded Vision

Fully connected layers are simple but inefficient for image data. A 224×224 RGB image has 150,528 pixels. A fully connected layer connecting that input to 128 outputs requires 19 million weights. That's 76 MB as float32—too large for most embedded targets.

Convolutional layers solve this by exploiting spatial locality. Instead of connecting every input to every output, a convolutional layer applies a small filter (e.g., 3×3 or 5×5) that slides across the input, computing a weighted sum at each position. The same filter weights are reused across the entire image, which dramatically reduces the parameter count.

A 3×3 convolutional filter applied to a single input channel has 9 weights. If the layer has 32 output channels (32 different filters), the total weight count is $9 \times 32 = 288$ weights—orders of magnitude fewer than a fully connected layer.

Convolution also preserves spatial structure. A fully connected layer treats the image as a flat vector, discarding information about which pixels are neighbors. A convolutional layer maintains the 2D structure, allowing it to detect local patterns like edges, textures, and shapes.

The operation count for a convolutional layer is:

(output height) × (output width) × (output channels) × (kernel height) × (kernel width) × (input channels)

For a 224×224 input image with 3 channels (RGB), a 3×3 convolution producing 32 output channels:

$224 \times 224 \times 32 \times 3 \times 3 \times 3 = 43.6$ million MACs

That's expensive—but still far cheaper than the equivalent fully connected layer.

Convolutional networks (CNNs) for image classification typically consist of:

- Several convolutional layers to extract features
- Pooling layers to reduce spatial resolution (e.g., max pooling, which takes the maximum value in each 2×2 region)
- A few fully connected layers at the end to combine features and produce class predictions

CNNs dominate embedded vision tasks—face detection, object recognition, defect classification—because they achieve high accuracy with manageable parameter counts. A well-designed CNN for 96×96 image classification might have 200,000–500,000 parameters, fitting comfortably in 1–2 MB of flash when quantized.

Recurrent Neural Networks: Why They're Hard to Deploy

Recurrent neural networks (RNNs) are designed for sequence data—time series, audio, text. Unlike feedforward networks, which process each input independently, RNNs maintain a hidden state that carries information from previous time steps.

The hidden state makes RNNs powerful for tasks like speech recognition and anomaly detection in sensor streams. But it also makes them difficult to deploy on constrained hardware because the hidden state must be stored in RAM between time steps, and the state size scales with the model's capacity.

A simple RNN layer with 128 hidden units requires storing 128 activations between time steps. A more powerful variant called LSTM (long short-term memory) requires four times as much state—512 values for a 128-unit LSTM layer. For a model processing audio at 16 kHz with 10 ms frames, that state is updated 100 times per second, and every update involves reading and writing the hidden state to RAM.

RNNs also have high operation counts. Each time step performs a matrix multiplication involving the input and the hidden state. For a 128-unit LSTM processing 40-dimensional audio features, the operation count is roughly 260,000 MACs per time step. Over a 1-second audio window (100 frames), that's 26 million MACs—comparable to a small CNN.

The combination of state memory requirements and high operation counts makes RNNs the most challenging model class for embedded deployment. When possible, embedded audio applications prefer alternative architectures like 1D convolutional networks or depthwise separable convolutions, which achieve similar accuracy with lower cost.

MobileNets and Depthwise Separable Convolutions: Efficient Architectures for Embedded

Standard convolutional layers are expensive because they combine spatial filtering and channel mixing in a single operation. A 3×3 convolution with 32 input channels and 64 output channels performs $3 \times 3 \times 32 \times 64 = 18,432$ multiplications per output pixel.

Depthwise separable convolution splits this into two cheaper operations: a depthwise convolution that filters each input channel independently, followed by a pointwise convolution (1×1) that mixes channels.

For the same example:

- Depthwise: $3 \times 3 \times 32 = 288$ multiplications per output pixel
- Pointwise: $1 \times 1 \times 32 \times 64 = 2,048$ multiplications per output pixel
- Total: 2,336 multiplications per output pixel

That's 8 times fewer operations than standard convolution, with minimal accuracy loss for many tasks.

MobileNet is a family of CNN architectures built entirely from depthwise separable convolutions. MobileNetV1 was the first widely-used efficient architecture for embedded vision. MobileNetV2 added inverted residual blocks and linear bottlenecks, improving accuracy while maintaining low cost. MobileNetV3 used neural architecture search to optimize for specific hardware targets.

A typical MobileNetV2 model for 224×224 image classification has:

- 3.5 million parameters (~14 MB as float32, 3.5 MB as int8)
- 300 million MACs per inference
- Top-5 accuracy of 90% on ImageNet

For embedded deployment, MobileNets are often scaled down. A 96×96 input resolution with 0.35× width multiplier (reducing the number of channels in each layer) gives:

- 400,000 parameters (~1.6 MB as float32, 400 KB as int8)
- 25 million MACs per inference
- Application-specific accuracy (depends on the task and dataset)

This scaled-down version fits on an STM32H7 (2 MB flash, 1 MB RAM) and runs inference in under 200 ms. It's the sweet spot for embedded vision: accurate enough for real-world tasks, cheap enough to deploy on microcontrollers.

Decision Trees and Random Forests: When Neural Networks Are Overkill

Not every embedded AI application needs a neural network. For structured data—sensor readings with known features like temperature, pressure, vibration frequency—decision trees and random forests often achieve equivalent accuracy with far lower cost.

A decision tree is a series of if-then rules. Each node tests a feature against a threshold and branches left or right. The leaves contain class labels or regression values. Inference is a single path from root to leaf, requiring one comparison per tree level.

A decision tree with 10 levels and 1,023 nodes (a complete binary tree) requires 1,023 threshold values stored in flash—4 KB as float32. Inference performs 10 comparisons—negligible compute cost, even on a Cortex-M0. No activation buffers are needed. No floating-point arithmetic is required if thresholds are quantized to integers.

Random forests are ensembles of decision trees. Each tree votes on the output, and the majority vote determines the final prediction. A random forest with 50 trees, each 10 levels deep, requires 200 KB of flash (50 trees × 4 KB) and performs 500 comparisons per inference. That's still cheaper than most neural networks.

For applications like anomaly detection in industrial sensors, predictive maintenance, or binary classification tasks with engineered features, decision trees should be the default choice. Only when accuracy gains from deep learning justify the cost should you move to neural networks.

The decision tree vs. neural network trade-off is this: decision trees are interpretable, fast, and small, but they require good feature engineering. Neural networks can learn features from raw data (pixels, audio samples) but cost more in memory and compute. If your data is already structured and you know which features matter, use a decision tree. If your data is raw and high-dimensional, use a neural network.

Model Size, Parameter Count, and FLOP Count: The Three Metrics That Determine Cost

Every model has three resource metrics you must quantify before deployment:

Parameter count is the total number of weights and biases in the model. This determines the model's size in flash. A model with 1 million parameters stored as float32 requires 4 MB. Stored as int8, it requires 1 MB. Parameter count is architecture-dependent—deeper networks and wider layers have more parameters.

FLOP count (floating-point operations) is the number of arithmetic operations required per inference. This determines compute cost and latency. FLOP count includes multiplications, additions, and sometimes activations, depending on how it's measured. For embedded deployment, MAC count (multiply-accumulate operations) is often used instead because it maps directly to processor throughput.

Activation memory is the size of the intermediate tensors during inference. This determines the minimum RAM requirement. Activation memory depends on the largest layer's output size and whether the inference engine reuses buffers or allocates new memory for each layer.

These three metrics are not independent. Reducing parameter count (e.g., by pruning layers) also reduces FLOP count and usually reduces activation memory. But the relationship is not always linear. A model with 50% fewer parameters might have only 30% fewer FLOPs if the pruned layers were small and the remaining layers are large.

To get these metrics for a trained model, use the framework's profiling tools:

- TensorFlow: `tf.profiler` or `tflite.Analyzer`
- PyTorch: `torchinfo.summary()` or `thop.profile()`
- ONNX: `onnx.helper.printable_graph()`

The output will list each layer's parameter count, output shape, and operation count. Sum them to get the total model cost.

For example, TensorFlow Lite's analyzer might report:

Model size: 1.2 MB
Total parameters: 315,000
Total MACs: 42 million
Activation memory: 180 KB

Now you have the numbers you need. Go back to Chapter 2's constraint analysis:

- Does 1.2 MB fit in flash? (Check flash capacity)
- Does 180 KB fit in RAM? (Check SRAM capacity minus system overhead)
- Can the processor execute 42 million MACs within the latency budget? (Check throughput)

If all three constraints are satisfied, the model is deployable. If any constraint fails, you must either select different hardware or modify the model.

The ML-to-Embedded Translation Table

Machine learning terminology and embedded systems terminology describe the same underlying resources in different languages. This table translates between them:

ML Term	Embedded Equivalent	Example
Model size	Flash memory requirement	1.2 MB model → needs ≥1.2 MB free flash
Parameter count	Weight memory footprint	300k params × 4 bytes = 1.2 MB
Activation memory	RAM requirement for inference	180 KB activations → needs ≥180 KB free SRAM
FLOP count / MAC count	Compute operations per inference	42M MACs @ 60 MMAC/s = 700 ms latency
Inference latency	Execution time per forward pass	Time from input loaded to output ready
Batch size	Number of inputs processed together	Always 1 on embedded (no batching)
Quantization	Reducing numeric precision	float32 → int8 saves 75% memory
Pruning	Removing weights/layers	Reduces param count and FLOPs

Input shape	Sensor data dimensions	96×96×3 image = 27,648 input values
Output shape	Prediction dimensions	10 classes = 10-element output vector

When a machine learning engineer says "the model has high memory footprint," they mean it has many parameters and large activations. When you say "it doesn't fit in RAM," you mean the activation memory exceeds available SRAM. You're describing the same problem in different terms.

Learning this translation allows you to communicate across domains. When you collaborate with ML engineers to select or optimize a model, you can specify constraints in their language ("we need activation memory under 150 KB") and they can propose solutions in theirs ("try MobileNetV2 with 0.35 width multiplier").

Which Model Class for Which Task

Not every model architecture is appropriate for every embedded sensing task. The model class you choose depends on the input data structure, the application's accuracy requirements, and the available hardware resources.

For image classification (camera-based sensing, visual inspection, object detection):

Use convolutional neural networks. Start with MobileNetV2 or MobileNetV3 as the baseline. Scale down the input resolution and width multiplier until you meet memory and compute constraints. If accuracy is still insufficient, try EfficientNet-Lite, which uses neural architecture search to optimize for efficiency.

For keyword spotting and audio classification (wake word detection, sound event recognition):

Use 1D convolutional networks on audio spectrograms or depthwise separable convolutions on raw waveforms. Avoid RNNs unless you have sufficient RAM for hidden state and sufficient compute for recurrent operations. Models like DS-CNN (depthwise separable CNN) achieve 95%+ accuracy on keyword spotting with under 100 KB of weights.

For time series anomaly detection (vibration monitoring, predictive maintenance, health monitoring):

Start with decision trees or random forests if you can engineer good features (statistical moments, frequency-domain features, autocorrelation). If raw sensor streams are required, use 1D CNNs or autoencoders. Autoencoders learn a compressed representation of normal behavior and flag deviations as anomalies.

For structured tabular data (sensor fusion, multi-modal classification with engineered features):

Use decision trees, random forests, or gradient-boosted trees. Neural networks rarely outperform tree-based models on small structured datasets, and trees are far cheaper to deploy.

For gesture recognition and motion classification (accelerometer-based interfaces, activity tracking):

Use 1D CNNs or LSTMs if you have enough RAM. For ultra-low-power devices, use decision trees on hand-crafted features like peak acceleration, zero-crossing rate, and signal energy.

The pattern is this: use the simplest model that meets accuracy requirements. Decision trees are simpler than neural networks. Feedforward networks are simpler than recurrent networks. Small CNNs are simpler than large CNNs. Every increment in model complexity costs memory, compute, and power. Only increase complexity when the accuracy gain justifies the cost.

What You Still Cannot Do

You can now read a model's specification and determine its resource cost. You can identify which model class is appropriate for a given sensing task. You can translate between ML terminology and embedded constraints. But you cannot yet answer the question: "Can this model run on this hardware?"

That question requires integrating the model cost (which you now understand) with the hardware constraints (which Chapter 2 taught you to quantify). It requires knowing how to measure actual inference latency, not just theoretical operation count. It requires understanding how quantization, pruning, and hardware accelerators change the cost profile. And it requires knowing when the theoretical analysis is insufficient and you must profile on real hardware.

Those skills are built in Part II. Chapter 4 begins by teaching you to measure and optimize inference mechanics—the actual execution of the model on the target processor. But before you can measure, you must understand what you're measuring. That understanding is what this chapter provided.

The `.tflite` file is no longer a binary blob. It is a data structure with known size, known operations, and known memory requirements. It is a design artifact that you can evaluate against hardware constraints before you deploy it.

That evaluation is what Chapter 4 teaches next.

Chapter 4: Inference Mechanics: What a Deployed Model Actually Does

You have a trained model. You have selected hardware. The model's parameter count fits in flash, its activation memory fits in RAM, and the theoretical inference latency calculated from

FLOP count and processor throughput is within budget. You compile the model for the target, flash the firmware, and run your first test.

The inference takes three times longer than predicted. Or the output is garbage—numerical overflow or NaN values where predictions should be. Or the system crashes halfway through the first inference pass with a memory fault. The model that worked perfectly in Python on your laptop fails on the embedded target in ways you didn't anticipate.

This is not a rare failure mode. It is the default outcome when you deploy without understanding what inference actually does on constrained hardware. The theoretical analysis from Chapters 2 and 3 told you whether deployment was feasible in principle. This chapter teaches you to trace what happens in practice—how the model executes, where it fails, and how to diagnose the gap between prediction and reality.

By the end of this chapter, you will be able to trace the inference execution path on a microcontroller, identify the computational bottleneck in a model-hardware pairing, measure actual latency on target hardware, and determine whether numerical precision issues are corrupting your results. You will understand why the same model that runs smoothly on a desktop GPU requires careful optimization to run at all on an embedded processor.

The skill is not debugging in the general sense. The skill is inference-specific diagnosis: recognizing when latency violations come from cache misses versus memory bandwidth limits versus numerical precision mismatches, and knowing which fix applies to which failure.

The Inference Execution Pipeline on Bare-Metal and RTOS Systems

When you call an inference function on a microcontroller, a sequence of operations executes in a specific order. The execution pipeline determines how long inference takes, how much memory it consumes, and where failures occur. Understanding this pipeline is the prerequisite for optimization.

On a bare-metal system—no operating system, just your firmware running directly on the processor—the inference pipeline is deterministic. You allocate memory once during initialization, load the model weights from flash, and execute inference in a loop triggered by sensor data or a timer. The processor runs at a fixed clock speed with no competing processes. If inference takes 200 milliseconds on the first pass, it takes 200 milliseconds on every pass, barring cache effects or thermal throttling.

On an RTOS (real-time operating system) like FreeRTOS or Zephyr, inference runs as a task competing with other tasks for processor time. The scheduler preempts inference when higher-priority tasks need to run, the timer interrupt fires, or communication events arrive. The 200-millisecond inference might spread across 250 milliseconds of wall-clock time due to scheduling delays, or it might be interrupted entirely if a critical task preempts it.

Most embedded AI deployments use bare-metal execution for latency-critical paths and RTOS execution for systems with multiple concurrent functions. A keyword spotting system might run inference in a bare-metal loop to guarantee sub-100ms response. A wearable health monitor might run inference as a low-priority RTOS task that yields to communication and sensor acquisition.

The inference pipeline itself has four stages, regardless of whether the system is bare-metal or RTOS-based:

Stage 1: Input preparation. Sensor data arrives in raw format—ADC samples, pixel values, digital sensor readings. The data must be preprocessed into the format the model expects: normalized to the range $[0, 1]$ or $[-1, 1]$, reshaped into the correct tensor dimensions, and converted to the numeric type the model uses (float32, int8, etc.). If preprocessing includes FFT (for audio features) or image scaling, this stage can dominate total latency.

Stage 2: Inference execution. The model's computational graph executes layer by layer. Each layer loads its weights from flash (or from a cached copy in SRAM), reads its input activations, performs the layer's operation (convolution, matrix multiply, activation function), and writes the output activations to the next layer's input buffer. This is where the FLOPs are computed. This is where most of the time is spent.

Stage 3: Output interpretation. The model produces raw outputs—logits for classification, bounding boxes for detection, regression values for prediction. These must be post-processed into actionable results: applying softmax to convert logits to probabilities, applying non-maximum suppression to merge overlapping bounding boxes, or thresholding regression outputs against application-specific bounds.

Stage 4: Result delivery. The processed output is stored, transmitted, or used to trigger an action. For a keyword spotter, this might be setting a GPIO pin high. For a predictive maintenance system, it might be writing a fault code to flash or sending a LoRaWAN packet. This stage is usually negligible compared to inference itself, but network transmission or flash writes can add latency if not properly managed.

When you measure inference latency, you must decide which stages to include. The model's inference time is Stage 2 only—the time spent executing the computational graph. Application latency is Stage 1 through Stage 4—the time from sensor data arriving to action being taken. The difference can be substantial. A model with 50ms inference time might have 150ms application latency if input preprocessing includes a 256-point FFT and output delivery includes a network transmission.

Most profiling tools report model inference time (Stage 2). If your application has a hard latency requirement, you must profile the full pipeline on target hardware to verify that all stages combined meet the deadline.

Operator-Level Execution: How Operations Run on CPU vs. Specialized Hardware

A neural network is a sequence of operations: matrix multiplications, convolutions, activations, pooling. On a desktop GPU, these operations map to thousands of parallel execution units. On a microcontroller, they map to a single CPU core executing instructions sequentially, possibly assisted by SIMD extensions or a hardware accelerator.

The performance of each operation depends on how well it maps to the available hardware. Some operations are compute-bound—their execution time is determined by how many arithmetic operations must execute. Other operations are memory-bound—their execution time is determined by how fast data can move between memory and the processor.

Consider matrix multiplication: $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, where $\mathbf{A}$ is $M \times K$ and $\mathbf{B}$ is $K \times N$. The output $\mathbf{C}$ is $M \times N$. The operation count is $M \times N \times K$ multiply-accumulates. For a 128×128 matrix multiplied by a 128×10 matrix, that's 163,840 MACs.

On a Cortex-M4 with FPU running at 100 MHz, each MAC takes one cycle when operands are in registers. Theoretical time is 1.6 milliseconds. But operands are not in registers—they're in SRAM or flash. Loading a 128×128 matrix from SRAM requires 16,384 reads. If each read takes 2 cycles due to SRAM access latency, that's 32,768 cycles of memory overhead. The operation becomes memory-bound, and actual time increases to 3–5 milliseconds depending on cache efficiency.

Now consider the same operation on a Cortex-M7 with 16 KB of L1 data cache. If the 128×128 matrix (64 KB as float32) doesn't fit in cache, it's fetched from SRAM repeatedly. But if the algorithm is structured to multiply in 64×64 blocks, each block fits in cache and is reused across multiple output elements. Cache-aware blocking reduces memory accesses by an order of magnitude, and execution time drops to 1.8 milliseconds—close to the theoretical limit.

This is why theoretical FLOP-based latency predictions fail. Memory bandwidth, cache behavior, and data layout can change execution time by 3–10× for the same operation count.

Convolution is even more memory-intensive. A 2D convolution with a 3×3 kernel requires loading 9 input pixels for each output pixel. For a 96×96 input with 32 channels convolved to produce 32 output channels, that's $96 \times 96 \times 32 \times 9 \times 32 = 84$ million memory accesses. If inputs are in external PSRAM instead of on-chip SRAM, each access costs 10–20 cycles instead of 1–2 cycles, and latency increases by an order of magnitude.

Activation functions—ReLU, sigmoid, tanh—are compute-light but can be memory-bound if not fused with the preceding operation. A ReLU applies element-wise to an activation tensor: if $x < 0$, output 0; else output x . For a 4096-element tensor, that's 4096 comparisons and 4096 conditional writes. On a pipelined processor, the conditional can cause branch mispredictions,

which flush the pipeline and waste cycles. A well-optimized implementation uses SIMD to process 4 or 8 elements per instruction, eliminating branches entirely.

Hardware accelerators change the execution model. A neural processing unit (NPU) includes dedicated matrix multiply engines that execute hundreds of MACs per cycle. An operation that takes 100,000 cycles on a CPU might take 1,000 cycles on an NPU. But the accelerator is only faster if the operation maps to its supported primitives. A standard 3×3 convolution maps well. A depthwise convolution might map poorly if the accelerator doesn't support channel-wise operations. Attention mechanisms or dynamic control flow (if-then logic inside the network) usually don't map at all, falling back to CPU execution.

When profiling a model, you must identify which operations execute where. Most inference frameworks include a layer-wise profiling mode that reports execution time per operation. If 80% of inference time is spent in one convolutional layer, that's your bottleneck. Optimize that layer—by reducing its size, quantizing it, or offloading it to an accelerator—and total latency drops proportionally.

Memory Allocation During Inference: Static vs. Dynamic, and Why Dynamic Fails

Memory allocation strategies determine whether inference succeeds or crashes. On embedded systems, dynamic memory allocation during inference is a failure mode, not a feature.

Dynamic allocation—calling `malloc()` or `new` to allocate memory at runtime—is standard practice in desktop software. You request memory when you need it, use it, and free it when you're done. The operating system manages a heap and reclaims memory from terminated processes. If allocation fails, the system swaps to disk or terminates low-priority processes to free space.

Embedded systems have no such safety net. The heap is a fixed region of SRAM. When you allocate memory, it comes from that region. When you free memory, it returns to the heap, but fragmentation can prevent future allocations even when total free memory is sufficient. If allocation fails, there is no recovery—no swap space, no other processes to terminate. The system crashes or returns a null pointer, and inference fails.

Worse, memory allocation itself consumes time. A `malloc()` call searches the heap for a suitable block, which can take hundreds of microseconds. If you allocate memory during inference, that overhead adds to latency. If you allocate repeatedly inside a loop (say, allocating a temporary buffer for each convolutional layer), the overhead accumulates to milliseconds.

For embedded AI, the correct allocation strategy is static pre-allocation. All memory required for inference is allocated once during system initialization, before the first inference runs. This includes:

- Model weights (loaded from flash into SRAM if needed, or accessed directly from flash)
- Activation buffers for every layer's intermediate results
- Scratch buffers for temporary computations (e.g., im2col transformations for convolution)
- Input and output tensors

The total memory requirement is the sum of these buffers. You allocate a contiguous block of SRAM—called the tensor arena in TensorFlow Lite Micro terminology—and the inference engine manages it internally, reusing buffers where possible.

Buffer reuse is critical. A naive implementation allocates separate memory for each layer's output, which means peak memory usage is the sum of all activation tensors. A buffer-reuse implementation recognizes that once a layer's output has been consumed by the next layer, its buffer can be reused for a later layer's output. Peak memory usage becomes the size of the largest layer's output plus any buffers that must coexist.

For example, consider a three-layer network:

- Layer 1 output: 4096 floats (16 KB)
- Layer 2 output: 2048 floats (8 KB)
- Layer 3 output: 10 floats (40 bytes)

Without buffer reuse, you need $16 + 8 + 0.04 = 24$ KB. With buffer reuse, Layer 1's output buffer is reused for Layer 2's output (Layer 2 produces fewer elements, so it fits), and both are reused for Layer 3. Peak memory is 16 KB, a 33% reduction.

Static allocation also enables compile-time memory checking. If your model requires 180 KB of activation memory and your target has 200 KB of SRAM, you know at compile time that deployment is feasible (with 20 KB margin for the rest of the system). If the model requires 250 KB, you know at compile time that deployment fails, and you don't waste time debugging runtime crashes.

The workflow for static allocation is:

1. Profile the model on desktop to determine total activation memory requirement.
2. Allocate a static buffer of that size in your embedded firmware.
3. Initialize the inference engine with a pointer to that buffer.
4. Run inference. The engine uses the buffer internally and never calls `malloc()`.

If you see `malloc()` calls in your inference code path, something is misconfigured. Trace them, identify the source, and replace them with pre-allocated buffers.

Latency Profiling: How to Measure and Interpret Inference Time on Target Hardware

Theoretical latency—FLOP count divided by processor throughput—is a starting estimate. Actual latency is measured on target hardware under realistic operating conditions.

The most basic profiling method is cycle counting. Most ARM Cortex-M cores include a Data Watchpoint and Trace (DWT) unit with a cycle counter. You enable the counter, read it before inference starts, read it after inference completes, and compute the difference. The result is the number of CPU cycles consumed by inference.

Here's the code pattern:

```
// Enable DWT cycle counter (once during initialization)
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

// Measure inference
uint32_t start = DWT->CYCCNT;
run_inference(); // Your inference function
uint32_t end = DWT->CYCCNT;
uint32_t cycles = end - start;

// Convert to time
float latency_ms = (cycles / (float)CPU_FREQ_HZ) * 1000.0f;
```

For a processor running at 100 MHz, 10 million cycles equals 100 milliseconds. If you measure 15 million cycles, inference takes 150 milliseconds.

This tells you total latency but not where the time is spent. For operator-level profiling, you instrument each layer:

```
uint32_t layer1_start = DWT->CYCCNT;
run_layer1();
uint32_t layer1_end = DWT->CYCCNT;

uint32_t layer2_start = DWT->CYCCNT;
run_layer2();
uint32_t layer2_end = DWT->CYCCNT;

printf("Layer 1: %lu cycles\n", layer1_end - layer1_start);
printf("Layer 2: %lu cycles\n", layer2_end - layer2_start);
```

Most embedded ML frameworks include built-in profiling. TensorFlow Lite Micro has a profiler that reports per-operator timing. You enable it at compile time, run inference, and read the profiling output over UART or semihosting. The output looks like:

```
Operator,      Count, Time (ms)
CONV_2D,        1,   45.2
DEPTHWISE_CONV_2D, 13,  82.3
FULLY_CONNECTED,  1,   3.1
SOFTMAX,        1,   0.8
Total inference time: 131.4 ms
```

This immediately tells you where optimization effort should focus. If depthwise convolutions consume 63% of total time, that's your bottleneck.

Profiling must account for variance. Run inference 100 times and record the distribution of latencies. If all runs take 130–132 ms, latency is stable. If runs vary from 120 to 180 ms, you have non-determinism—possibly from cache effects, interrupt handling, or thermal throttling. For soft real-time applications, use the mean or 95th percentile latency. For hard real-time applications, use the maximum observed latency plus margin.

Energy profiling requires hardware measurement. A current sense amplifier on the device's power rail measures instantaneous current draw. You log current over time, correlate it with inference start/stop timestamps, and integrate to get energy per inference:

$$\text{Energy (Joules)} = \int V(t) \times I(t) \, dt$$

For a 3.3V device drawing 40 mA during 150 ms of inference:

$$\text{Energy} = 3.3\text{V} \times 0.040\text{A} \times 0.150\text{s} = 0.0198 \text{ J} = 19.8 \text{ mJ}$$

If inference runs every 10 seconds, average power is $19.8 \text{ mJ} / 10 \text{ s} = 1.98 \text{ mW}$. A 1000 mAh battery at 3.3V stores 11,880 Joules. Battery life is $11,880 \text{ J} / 1.98 \text{ mW} = 6 \text{ million seconds} = 69 \text{ days}$.

Profiling tools like SEGGER SystemView, Percepio Tracealyzer, and ARM Development Studio provide graphical timelines showing when each function executes, how long it takes, and how tasks interact on RTOS systems. These are invaluable for diagnosing scheduling delays, interrupt latency, and resource contention.

Numerical Precision: float32 vs. float16 vs. int8 and What the Tradeoffs Are

Neural networks are trained using 32-bit floating-point arithmetic. Each weight, each activation, and each intermediate result is a float32 value. This gives high precision but consumes memory and compute resources.

Embedded deployment often uses reduced precision: float16 (16-bit floating-point), bfloat16 (brain floating-point, a variant of float16), or int8 (8-bit integer). Each reduction halves memory usage and can double or quadruple throughput on processors with SIMD or specialized hardware.

But reducing precision introduces numerical errors. The errors accumulate through the network's layers, and if they grow too large, the model's predictions degrade.

float32 is the baseline. Each value ranges from $\pm 1.4 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$ with roughly 7 decimal digits of precision. Operations are exact within floating-point rounding error, which is negligible for neural network inference. The cost is 4 bytes per value and relatively slow execution on processors without FPUs.

float16 halves memory to 2 bytes per value. Range is $\pm 6.5 \times 10^4$ with roughly 3 decimal digits of precision. Many trained networks tolerate float16 conversion with less than 1% accuracy loss. The problem is that most microcontrollers don't support native float16 arithmetic—they emulate it in software, which is slower than float32 on an FPU. float16 is common on GPUs and some mobile processors but rare on microcontrollers.

int8 reduces memory to 1 byte per value. Integers range from -128 to 127. Neural network weights and activations don't naturally fit this range, so they must be quantized: scaled and shifted to map the float32 range onto [-128, 127]. Quantization introduces rounding error, and if not done carefully, accuracy degrades by 5–20%.

Post-training quantization (PTQ) is the simplest approach: you take a trained float32 model, measure the range of weights and activations over a representative dataset, compute scaling factors, and convert everything to int8. The inference engine multiplies int8 values, accumulates results in int32 to avoid overflow, and rescales back to int8 after each operation.

Quantization-aware training (QAT) simulates quantization during training, allowing the model to learn weights that remain accurate even after quantization. QAT typically achieves better accuracy than PTQ but requires retraining the model, which you may not control.

The compute advantage of int8 is substantial. On processors with SIMD extensions, you can pack four int8 values into a 32-bit register and perform four operations in parallel. On specialized accelerators like ARM Ethos-U55, int8 MAC throughput can be 64× higher than float32 throughput on the same chip.

The tradeoff is accuracy. For image classification, int8 quantization typically costs 1–3% accuracy. For regression tasks with small dynamic range (e.g., sensor anomaly detection), it might cost 10% or more. For some models, particularly those with batch normalization or large

dynamic range in activations, int8 quantization can destroy accuracy entirely if not tuned carefully.

The decision workflow is:

1. Deploy the model in float32 and measure latency and memory.
2. If latency or memory exceeds budget, quantize to int8.
3. Measure accuracy loss. If loss is acceptable (<5% for most applications), deploy quantized.
4. If loss is unacceptable, try QAT, adjust activation ranges, or use mixed precision (some layers int8, some float32).

Never assume quantization is free. Always measure accuracy before and after, and always profile latency and memory to confirm the expected speedup materializes.

A Worked Example: Profiling Keyword Spotting on Arduino Nano 33 BLE Sense

You have a keyword spotting model trained to detect "yes," "no," and "unknown" from 1-second audio clips. The model is a depthwise separable CNN with 80,000 parameters. Theoretical analysis predicts:

- Model size: 320 KB (float32)
- Activation memory: 45 KB
- Operation count: 22 million MACs
- Target: Arduino Nano 33 BLE Sense (nRF52840, 64 MHz Cortex-M4 with FPU, 256 KB RAM, 1 MB flash)

Theoretical latency at 60 MMAC/s sustained: $22\text{M} / 60\text{M} = 367$ milliseconds.

You deploy the model using TensorFlow Lite Micro, compile, flash, and run. The first inference takes 1,200 milliseconds. That's 3× slower than predicted.

Diagnosis Step 1: Check precision.

You examine the model file. It's float32. The Cortex-M4 has an FPU, so float32 should be efficient. But you check the compiled code and discover that the TFLite Micro kernel is using a reference C implementation, not the optimized CMSIS-NN kernels that leverage the FPU and DSP extensions. The reference implementation is slower by 3–5×.

Fix: Reconfigure the build to use CMSIS-NN optimized kernels. Recompile and re-flash.

New latency: 420 milliseconds. Much better—close to the 367 ms prediction. But still over budget if your application has a 100 ms deadline.

Diagnosis Step 2: Identify bottleneck.

You enable TFLite Micro's profiling output and run inference again. The profiler reports:

DEPTHWISE_CONV_2D (layer 3): 180 ms
DEPTHWISE_CONV_2D (layer 5): 120 ms
CONV_2D (layer 1): 60 ms
Other layers: 60 ms
Total: 420 ms

Two depthwise convolutions consume 71% of total time. These layers operate on 32×32 and 24×24 feature maps with 64 channels each.

Diagnosis Step 3: Check memory layout.

You inspect the activation buffers. Layers 3 and 5's inputs are in external PSRAM because total activations exceed the 256 KB on-chip SRAM budget. External memory access costs 10× more than on-chip SRAM access, which explains why these layers are so slow.

Fix: Reduce the model's activation memory by decreasing feature map sizes. You retrain the model with 16×16 maximum feature maps instead of 32×32. Activation memory drops to 28 KB, fitting entirely in on-chip SRAM.

New latency: 140 milliseconds.

Diagnosis Step 4: Quantize.

140 ms is still above the 100 ms target. You apply post-training int8 quantization. The model size drops to 80 KB, activation memory stays at 28 KB, and int8 SIMD operations execute 4× faster than float32 scalar operations on the Cortex-M4.

New latency: 45 milliseconds.

Accuracy before quantization: 94.2%

Accuracy after quantization: 92.8%

The 1.4% accuracy loss is acceptable for the application. The model now meets all constraints: 80 KB flash, 28 KB RAM, 45 ms latency.

This is the diagnostic process. Theoretical analysis predicted 367 ms. Deployment measured 1,200 ms. Profiling identified the bottleneck (slow kernels, then external memory access). Optimization (CMSIS-NN, model shrinking, quantization) brought latency to 45 ms—well below target.

You could not have found this without measurement. The theoretical prediction was directionally correct but quantitatively wrong by 25×. Only profiling on target hardware revealed the real bottlenecks.

When Inference Fails: Common Failure Modes and How to Diagnose Them

Inference failures on embedded hardware fall into four categories: crashes, incorrect output, excessive latency, and excessive power consumption. Each has characteristic symptoms and diagnostic paths.

Crash: Memory fault or hardfault exception.

The system halts during inference, and the debugger shows a HardFault or MemManage exception. This usually means the code tried to access invalid memory—a null pointer dereference, a stack overflow, or an out-of-bounds array access.

Check activation memory allocation. If the tensor arena is too small, the inference engine may write past the end of the buffer, corrupting the stack or heap. Increase the arena size and re-test. If the crash persists, check for stack overflow—inference may be using more stack than allocated, especially if the model includes recursive operations or deep call chains.

Incorrect output: NaN, Inf, or garbage predictions.

The model runs without crashing but produces invalid results—not-a-number (NaN), infinity (Inf), or predictions that don't match expected accuracy.

Check numerical precision. If you quantized the model, verify that the quantization parameters (scale and zero-point for each layer) are correct. A mismatch between training and inference quantization ranges can produce overflow, underflow, or saturation. Re-run post-training quantization with a larger calibration dataset.

Check input preprocessing. If the model expects inputs normalized to [0,1] but you're feeding raw ADC values in the range [0,4095], the model will produce garbage. Verify that input scaling, mean subtraction, and tensor layout (NHWC vs. NCHW) match the training configuration.

Excessive latency: Inference takes 10× longer than predicted.

The model runs correctly but too slowly. Check which kernels are executing. If the framework is using reference C implementations instead of optimized SIMD or hardware-accelerated kernels, latency balloons. Reconfigure the build to enable optimized implementations.

Check memory access patterns. If activations are in external memory, latency increases by an order of magnitude. Move activations to on-chip SRAM by reducing model size or increasing SRAM allocation.

Check clock configuration. If the processor is running at 10 MHz instead of 100 MHz due to misconfigured clocks, all compute operations take 10× longer. Verify the clock tree configuration in your startup code.

Excessive power consumption: Battery drains faster than calculated.

The model runs correctly, but battery life is half of what the duty cycle calculation predicted. Measure actual current draw during inference with a current meter. If measured current is 2× higher than the datasheet active current, the model is accessing peripherals or memory in an inefficient way.

Check for busy-waiting. If the code polls a flag or spins in a loop waiting for inference to complete, the processor stays active instead of sleeping, and power consumption increases. Replace polling with interrupt-driven or DMA-driven execution.

Check for unintended peripheral activation. If the code leaves LEDs, sensors, or communication modules powered during inference, they contribute to active current. Disable all non-essential peripherals before inference and re-enable them afterward.

These diagnostic paths are not exhaustive, but they cover 90% of deployment failures. The key is systematic measurement: profile latency, measure memory usage, log output values, and compare against predictions. The gap between prediction and measurement tells you where the problem is.

What Comes Next: From Mechanics to Constraints

You can now trace inference execution, measure latency and memory usage, and identify bottlenecks. But you still haven't systematically evaluated each constraint dimension. This chapter taught you how inference works. The next six chapters teach you how to evaluate whether it works well enough—memory (Chapter 5), compute (Chapter 6), power (Chapter 7), hardware acceleration (Chapter 8), communication (Chapter 9), and real-time guarantees (Chapter 10).

Each chapter adds one diagnostic tool to your toolkit. By the end of Part II, you will be able to audit any proposed embedded AI deployment and identify every constraint violation before deployment. That audit is what separates successful integrations from expensive failures.

But first, the most visible constraint: memory. Chapter 5 examines why models that "fit" in total memory still fail at runtime, and what you can do about it.

Chapter 5: Memory: Fitting Models into Constrained Spaces

The datasheet says 256 KB of RAM. Your model's weights occupy 180 KB. Simple arithmetic: $256 - 180 = 76$ KB of headroom. The model fits.

You compile, flash, and run. The system crashes 40 milliseconds into the first inference with a memory fault. The debugger shows the heap allocator failed, malloc returned null, and the inference engine tried to write to address 0x00000000.

The model's weights were never the problem. Weights live in flash. What crashed was activation memory—the intermediate tensors created during inference, which live in RAM. Your model's activation footprint is 210 KB. Add 30 KB for the Bluetooth stack, 8 KB for the system heap, 4 KB for the call stack, and you need 252 KB before the model even starts. The model that "fit" by parameter count doesn't fit at runtime.

This failure mode is common because activation memory is invisible until you measure it. Datasheets specify total RAM. Model summaries report parameter count and weights size. Neither tells you how much RAM inference will consume while executing. That number—activation memory—is the hidden constraint that determines whether a model deploys successfully or crashes the first time it runs.

This chapter teaches you to calculate activation memory before deployment, identify which components of a model consume the most memory and why, apply memory layout strategies to reduce footprint, and determine whether a model fits within a target's memory envelope with enough margin for the rest of the system. By the end of this chapter, you will never deploy a model without first accounting for every byte of RAM it requires.

Flash vs. SRAM in AI Deployment: Where Weights Live, Where Activations Live

Memory in embedded systems is partitioned into regions with different properties. The most critical distinction for AI deployment is flash versus SRAM.

Flash is non-volatile storage. Data written to flash persists when power is removed. Flash is abundant—microcontrollers commonly have 512 KB to 2 MB of flash. But flash is read-only during normal operation. You cannot write to flash from running code without invoking special erase-and-program cycles that take milliseconds and degrade the flash over time (limited to ~10,000–100,000 erase cycles).

For AI deployment, flash stores:

- The firmware binary (your application code, the inference engine, libraries)
- The model's weights and biases (trained parameters that don't change during inference)
- Constant lookup tables (e.g., quantization parameters, activation function tables)

SRAM is volatile storage. Data in SRAM is lost when power is removed. SRAM is fast—reads and writes complete in 1–2 CPU cycles. SRAM is writable—you can modify it freely without wear concerns. But SRAM is scarce—microcontrollers commonly have 64 KB to 512 KB of SRAM, and high-end parts top out at 1–2 MB.

For AI deployment, SRAM stores:

- The application's heap and stack
- Global and static variables
- RTOS task control blocks and message queues (if using an RTOS)
- The model's activation tensors during inference
- Scratch buffers for temporary computations

The memory constraint is not "does the model fit in total memory?" but "does each component fit in the right type of memory?"

Consider a model with 500,000 parameters (2 MB as float32, 500 KB as int8) deployed on an STM32L4 with 640 KB of SRAM and 2 MB of flash. The int8 weights fit in flash with room to spare. But if the model's activations require 400 KB of SRAM, and the application already uses 200 KB for the Bluetooth stack and buffers, total SRAM usage is 600 KB—within the 640 KB limit, but with only 40 KB of margin. If the system also needs a 64 KB DMA buffer for sensor data, SRAM usage exceeds capacity, and deployment fails.

Some microcontrollers support execute-in-place (XIP) from flash, allowing the processor to fetch instructions and read-only data directly from flash without copying to SRAM. This is efficient for weights—the inference engine reads weights from flash as needed during each layer's computation. But activations cannot live in flash because they change during inference. Activations must be in writable memory, which means SRAM or external DRAM.

External memory (PSRAM, SDRAM, or eMMC) expands total memory capacity but at a cost. External memory is slower than on-chip SRAM—access latency increases from 1–2 cycles to 10–50 cycles depending on the interface. External memory consumes more power because data traverses the external bus. And external memory increases BOM cost and board complexity.

If your model's activations exceed on-chip SRAM, you have four options:

1. Reduce the model's activation footprint (shrink layers, reduce channels, use depthwise separable convolutions)
2. Add external PSRAM and accept the latency and power penalty
3. Stream activations to flash between layers (extremely slow, only viable for ultra-low-latency-tolerant applications)
4. Reject the target and select hardware with more SRAM

The decision depends on whether the latency and power cost of external memory is acceptable for the application. For a battery-powered wearable with a 100 ms latency budget, external

memory might violate both power and latency constraints. For a mains-powered industrial sensor with a 1-second latency budget, external memory might be perfectly acceptable.

Weight Memory: Parameter Count × Precision = Storage Requirement

The size of a model's weights is straightforward to calculate. Count the parameters, multiply by bytes per parameter, and you have the flash requirement.

For a fully connected layer with `input_size` inputs and `output_size` outputs: **Parameters = (input_size × output_size) + output_size**

The first term is the weight matrix. The second term is the bias vector (one bias per output).

For a 2D convolutional layer with `input_channels`, `output_channels`, and a `kernel_height × kernel_width` filter: **Parameters = (kernel_height × kernel_width × input_channels × output_channels) + output_channels**

For a depthwise separable convolution (depthwise + pointwise): **Depthwise params = (kernel_height × kernel_width × channels) + channels**

Pointwise params = (channels × output_channels) + output_channels

Total = depthwise + pointwise

Most ML frameworks report total parameter count when you compile the model. TensorFlow's `model.summary()` lists parameters per layer. PyTorch's `torchinfo.summary()` does the same. If you don't have access to the framework, you can calculate manually from the architecture specification.

Precision determines bytes per parameter:

- float32: 4 bytes
- float16: 2 bytes
- int8: 1 byte

A model with 1 million parameters occupies:

- 4 MB as float32
- 2 MB as float16
- 1 MB as int8

Quantization from float32 to int8 reduces weight memory by 75%. For a device with 1 MB of flash and 200 KB already consumed by firmware, a 1.2 MB float32 model doesn't fit, but the 300 KB int8 version does.

Weight memory is fixed after training. During inference, weights are read-only. They never change. This makes weight memory the easiest constraint to satisfy—if the weights don't fit in flash, you quantize them or prune the model until they do.

The one complication is that some inference engines copy weights from flash to SRAM for faster access. TensorFlow Lite Micro, for example, can load weights into the tensor arena if configured to do so. This speeds up inference (SRAM access is faster than flash access) but consumes SRAM. If your SRAM budget is tight, configure the engine to access weights directly from flash and accept the latency penalty.

Activation Memory: The Hidden Footprint

Activation memory is the RAM consumed by intermediate tensors during inference. When you run a forward pass through the network, each layer produces an output tensor that becomes the input to the next layer. Those tensors exist in memory while the layer executes. The total size of those tensors—plus any scratch buffers the inference engine uses internally—is the activation memory footprint.

Unlike weight memory, activation memory is not obvious from the model architecture. You cannot calculate it by counting parameters. You must trace the dataflow through the network and account for every intermediate tensor that exists simultaneously.

Consider a simple convolutional network:

- Input: $96 \times 96 \times 3$ image (27,648 elements, 110 KB as float32)
- Conv1: $96 \times 96 \times 32$ output (294,912 elements, 1.15 MB as float32)
- Pool1: $48 \times 48 \times 32$ output (73,728 elements, 288 KB as float32)
- Conv2: $48 \times 48 \times 64$ output (147,456 elements, 576 KB as float32)
- Pool2: $24 \times 24 \times 64$ output (36,864 elements, 144 KB as float32)
- Flatten and FC: 10 outputs (10 elements, 40 bytes as float32)

If the inference engine allocates separate buffers for each layer's output, peak memory usage is the sum of all activation tensors: $110 + 1,150 + 288 + 576 + 144 + 0.04 = 2,268$ KB. That's 2.2 MB of SRAM, which exceeds the capacity of most microcontrollers.

But the engine doesn't need to keep all tensors in memory simultaneously. Once Conv1's output has been consumed by Pool1, Conv1's buffer can be reused for Conv2's input. Once Pool1's output has been consumed by Conv2, Pool1's buffer can be reused for Pool2's input. This is called buffer reuse or in-place computation.

With buffer reuse, peak memory usage is determined by the largest tensor that must exist at any point during inference. For this network, Conv1's 1.15 MB output is the peak. The engine allocates 1.15 MB once and reuses it for all subsequent layers (which produce smaller outputs). Peak activation memory drops from 2.2 MB to 1.15 MB—a 48% reduction.

But 1.15 MB is still too large for most embedded targets. The solution is to reduce layer output sizes. If the input resolution drops from 96×96 to 48×48, Conv1's output becomes 48×48×32 = 73,728 elements = 288 KB as float32. If the model is quantized to int8, that drops to 72 KB. Suddenly, the model fits in the SRAM budget.

The activation memory requirement depends on:

1. The largest intermediate tensor in the network (determined by layer output dimensions and channel count)
2. The number of tensors that must coexist (determined by the network's dataflow—feedforward networks reuse buffers better than networks with skip connections or branches)
3. Scratch buffers required by specific operations (e.g., im2col transformations for convolution can temporarily double memory usage)

Most inference frameworks include a memory profiler that reports activation memory. TensorFlow Lite's Benchmark Tool outputs "tensor arena size." TensorFlow Lite Micro's interpreter initialization fails if the tensor arena is too small, and it reports the required size in the error message. Use these tools to measure activation memory before deployment.

Memory Layout Strategies: In-Place Computation, Buffer Reuse, Scratch Pads

Reducing activation memory requires understanding how the inference engine allocates and reuses buffers. There are three primary strategies: in-place computation, buffer reuse, and scratch pads.

In-place computation means that a layer writes its output to the same buffer that held its input. This is possible for operations where the output has the same dimensions as the input—element-wise operations like ReLU, batch normalization, or addition of residual connections.

For example, ReLU applies element-wise: `output[i] = max(0, input[i])`. The engine reads `input[i]`, computes the result, and writes it back to `input[i]`, overwriting the original value. No additional buffer is needed. Activation memory for ReLU is zero.

In-place computation fails when the operation changes the tensor's dimensions. A convolutional layer that reduces spatial resolution (stride > 1) or increases channel count produces a different-sized output, so it cannot overwrite the input buffer.

Buffer reuse means the engine recycles buffers across layers. After a layer consumes its input, the engine marks that input buffer as free and allocates it to a later layer's output. This is automatic in most inference engines—you allocate a large tensor arena, and the engine manages it internally.

The efficiency of buffer reuse depends on the network's structure. Feedforward networks (sequential layers with no branches) reuse buffers maximally because each layer's output is only used once. Networks with skip connections or concatenation operations (e.g., ResNet, DenseNet) cannot reuse buffers as aggressively because multiple layers' outputs must coexist.

For example, a ResNet block computes `output = conv(input) + input`, which means the input tensor must remain in memory while the convolution executes, then it's added to the convolution's output. The engine cannot reuse the input buffer until after the addition, which increases peak memory.

Scratch pads are temporary buffers used by specific operations that need intermediate storage. The most common example is the im2col transformation for convolution.

Convolution with a 3×3 kernel requires reading 9 input pixels for each output pixel. Naive implementations re-read those pixels from memory on every MAC operation. The im2col (image-to-column) transformation rearranges the input into a temporary matrix where each row contains the 9 pixels for one output position. The convolution then becomes a matrix multiply, which is faster on many processors.

But im2col requires a scratch buffer as large as $(\text{output_height} \times \text{output_width} \times \text{kernel_height} \times \text{kernel_width} \times \text{input_channels})$. For a 96×96 input with 32 channels and a 3×3 kernel, the scratch buffer is $96 \times 96 \times 3 \times 3 \times 32 = 2.65$ million elements = 10.6 MB as float32. That's impractical on embedded hardware.

Embedded-optimized inference engines either avoid im2col entirely (using direct convolution with careful register blocking) or use partial im2col that processes the image in tiles small enough to fit in cache. The scratch buffer size becomes $(\text{tile_size} \times \text{kernel_size} \times \text{channels})$, which is orders of magnitude smaller.

You control memory layout by configuring the inference engine at compile time. TensorFlow Lite Micro requires you to allocate a static tensor arena and specify its size:

```
constexpr int kTensorArenaSize = 200 * 1024; // 200 KB
alignas(16) uint8_t tensor_arena[kTensorArenaSize];
```

```
tflite::MicroInterpreter interpreter(model, resolver, tensor_arena, kTensorArenaSize);
```

If the arena is too small, initialization fails with an error message indicating the required size. You increase the arena size, recompile, and re-test. If the arena fits but the system later crashes, you've exceeded total SRAM capacity—the arena plus the rest of the firmware's heap and stack exceed available RAM.

The workflow is:

1. Start with the smallest arena size that initialization accepts (reported by the framework).

2. Add 20% margin for runtime variation.
3. Verify that total SRAM usage (arena + stack + heap + firmware globals) is below 80% of capacity.
4. If total usage exceeds capacity, reduce the model's activation footprint or add external memory.

Static Memory Allocation for Inference: Why malloc() Is Not Your Friend

Dynamic memory allocation during inference is a design error on embedded systems. The heap is unreliable, fragmentation is unavoidable, and allocation failures are unrecoverable. The correct pattern is static pre-allocation of all memory required for inference.

This means:

- The tensor arena is a static array allocated at compile time.
- Model weights are either stored in flash and accessed read-only, or copied to a static array in SRAM during initialization.
- Input and output buffers are static arrays.
- Scratch buffers, if needed, are static arrays.

Nothing is allocated with `malloc()` or `new` during inference. All memory is accounted for at compile time and allocated before the first inference runs.

The advantage is predictability. You know at compile time whether the model fits. You know that inference will never fail due to heap exhaustion. You know that memory usage is constant across inference passes, which simplifies power profiling and worst-case execution time analysis.

The disadvantage is inflexibility. If you want to run a different model, you must recompile the firmware with a different arena size. If you want to support multiple models simultaneously, you must allocate separate arenas for each, even if they never run at the same time.

For most embedded AI applications, predictability outweighs flexibility. You deploy one model or a small set of models, and they run on fixed hardware for months or years. Static allocation is the right choice.

Some frameworks, like TensorFlow Lite Micro, enforce static allocation by default—there is no dynamic allocation code in the inference engine. Others, like PyTorch Mobile or ONNX Runtime, assume dynamic allocation and must be configured explicitly to use pre-allocated buffers. Check your framework's documentation and verify that no `malloc()` calls occur during inference.

Memory Profiling Tools: How to Measure What a Deployed Model Actually Uses

Theoretical memory calculations tell you the minimum required. Profiling tells you what the model actually consumes on target hardware, including overhead from the inference engine, alignment padding, and runtime allocation.

The simplest profiling method is to fill RAM with a known pattern, run inference, and see how much of the pattern was overwritten.

```
// Fill unused RAM with 0xAA
extern uint8_t _end; // End of .bss section (linker-provided symbol)
extern uint8_t _estack; // Top of stack
memset(&_end, 0xAA, &_estack - &_end);

// Run inference
run_inference();

// Count how many bytes were overwritten
uint8_t* ptr = &_end;
size_t used = 0;
while (ptr < &_estack && *ptr != 0xAA) {
    ptr++;
    used++;
}

printf("RAM used: %zu bytes\n", used);
```

This gives a conservative estimate—it includes stack growth, heap allocations, and any other runtime memory usage, not just the tensor arena.

For finer granularity, use the framework's built-in profiler. TensorFlow Lite Micro's interpreter reports tensor arena usage after initialization:

```
size_t used = interpreter.arena_used_bytes();
printf("Tensor arena used: %zu bytes\n", used);
```

This tells you how much of the arena was actually allocated, which might be less than the total arena size if you over-provisioned.

For layer-wise memory profiling, some frameworks support memory tracing. You enable tracing, run inference, and the framework logs each allocation and deallocation. The output shows which layers allocate the largest tensors and when buffers are reused.

ARM's Arm NN framework includes a memory profiler that generates a timeline showing activation memory usage over the course of inference. Peaks in the timeline indicate which layers consume the most memory. If one convolutional layer allocates a 500 KB tensor while all others allocate under 100 KB, that layer is the bottleneck.

External tools like Percepio Tracealyzer or SEGGER SystemView visualize memory usage in real time on RTOS-based systems. They show heap allocations, stack usage, and task memory consumption as the system runs. If inference causes the heap to fragment or the stack to overflow, these tools catch it visually.

The goal of profiling is to answer three questions:

1. How much SRAM does inference consume at peak?
2. Which layer or operation causes the peak?
3. Is there margin for the rest of the system, or is SRAM at capacity?

If peak usage exceeds 80% of total SRAM, you're in the danger zone. Real systems have runtime variation—interrupt handlers push to the stack, communication buffers fill, sensor data arrives. If inference leaves no headroom, the system will crash under load even if it works in controlled testing.

A Worked Example: Diagnosing and Fixing an Activation Memory Failure

You are deploying a MobileNetV2-based image classifier to an STM32H743 (Cortex-M7 at 480 MHz, 1 MB SRAM, 2 MB flash). The model has 1.2 million parameters (int8 quantized, 1.2 MB in flash). The model summary reports:

- Input: 96×96×3 (27,648 elements)
- Largest layer output: 24×24×192 (110,592 elements)
- Total parameters: 1.2M (1.2 MB as int8)

You calculate activation memory conservatively: $110,592 \text{ elements} \times 1 \text{ byte (int8)} = 108 \text{ KB}$. You add 50% margin for scratch buffers: 162 KB. The STM32H743 has 1 MB of SRAM, so this seems safe.

You compile with a 200 KB tensor arena and deploy. Initialization succeeds. The first inference crashes with a memory fault.

Diagnosis Step 1: Check actual arena usage.

You enable TFLite Micro's diagnostic output. The interpreter reports:

Tensor arena used: 340 KB
Tensor arena size: 200 KB
ERROR: Insufficient arena size

The model requires 340 KB, not 162 KB. Your estimate was wrong.

Diagnosis Step 2: Identify the cause.

You examine the layer-wise memory profile. The bottleneck is not the $24 \times 24 \times 192$ tensor (108 KB). It's a depthwise convolution that uses an im2col scratch buffer of 180 KB. Your conservative estimate didn't account for operation-specific scratch memory.

Fix Attempt 1: Increase the arena.

You increase the tensor arena to 350 KB. Initialization succeeds. Inference runs but crashes 80 ms in. The debugger shows a stack overflow.

Diagnosis Step 3: Check total SRAM usage.

The firmware uses:

- Tensor arena: 350 KB
- Application heap: 80 KB
- RTOS kernel and task stacks: 120 KB
- Communication buffers: 200 KB
- Global variables: 50 KB **Total: 800 KB**

That leaves 224 KB of the 1 MB SRAM for the call stack. But the inference function is deeply recursive due to the framework's layer dispatching, and it overflows the stack during execution.

Fix Attempt 2: Reduce the model's activation footprint.

You cannot increase SRAM—the hardware is fixed. You must reduce memory consumption. You modify the model:

- Reduce input resolution from 96×96 to 80×80 .
- Reduce the maximum channel count from 192 to 128.
- Replace standard convolutions with depthwise separable convolutions (which have lower scratch buffer requirements).

You retrain the model with these changes. The new model has:

- 800,000 parameters (800 KB as int8)
- Largest layer output: $20 \times 20 \times 128$ (51,200 elements = 50 KB)

- Tensor arena required: 180 KB (measured)

New total SRAM usage:

- Tensor arena: 180 KB
- Application heap: 80 KB
- RTOS and stacks: 120 KB
- Communication buffers: 200 KB
- Globals: 50 KB **Total: 630 KB**

This leaves 394 KB of headroom, which is sufficient. You deploy and test. Inference runs successfully. Accuracy drops from 89.2% (original model) to 86.4% (reduced model), which is acceptable for the application.

This is the memory constraint workflow. The original model "fit" by parameter count (1.2 MB in 2 MB flash) but failed at runtime due to activation memory (340 KB required, only 200 KB allocated). Profiling revealed the actual requirement. Reducing the model's size brought activation memory within budget, at the cost of a 3% accuracy drop.

You could not have diagnosed this without measurement. The theoretical estimate was off by 2×. Only runtime profiling exposed the scratch buffer overhead.

What Happens When You Run Out of Memory

There are three ways a model fails the memory constraint: it doesn't fit in flash, it doesn't fit in SRAM, or it leaves insufficient margin for the rest of the system.

Doesn't fit in flash: The model's weights exceed available flash capacity. This is the easiest failure to diagnose and fix. Quantize the model from float32 to int8 (75% reduction in size), prune unnecessary layers, or select a smaller architecture. If quantization and pruning are insufficient, you've chosen the wrong model for the hardware. Select a smaller model or upgrade the hardware.

Doesn't fit in SRAM: The model's activations exceed available SRAM. This is harder to diagnose because activation memory is not visible in model summaries—you must profile it. Fix by reducing layer sizes, reducing channel counts, or using architectures with lower activation memory (e.g., MobileNet instead of ResNet). If the model is already minimal and still doesn't fit, add external PSRAM or upgrade to hardware with more SRAM.

Insufficient margin: The model fits, but total SRAM usage (model + firmware) exceeds 80–90% of capacity. This is the failure mode that shows up only under load. In testing, the system works. In production, when communication buffers fill, sensor data queues up, and interrupt handlers run frequently, the system crashes. Fix by reducing the model's memory footprint, reducing the application's memory usage (smaller buffers, fewer RTOS tasks), or upgrading to hardware with more SRAM.

The memory constraint is the first constraint you check and the easiest to violate. If the model doesn't fit in memory, no amount of compute optimization or power tuning will make it work. Memory is binary: it fits or it doesn't.

That's why this chapter comes first in Part II. Before you evaluate latency, power, or acceleration, you verify that the model fits in memory. If it doesn't, you fix it. Only then do you move to the next constraint.

Chapter 6 examines that next constraint: compute. The model fits in memory. Now, can it run fast enough?

Chapter 6: Compute: Latency, Throughput, and Real-Time Guarantees

The model fits in memory. Weights occupy 600 KB of flash, activations require 140 KB of SRAM, and total memory usage leaves 200 KB of headroom. The memory constraint is satisfied.

You run your first inference pass and time it: 1,850 milliseconds. Your application requirement is 50 milliseconds. The model that fits in memory cannot run fast enough to be useful.

This is the compute constraint. The model's operations must execute within the time budget the application allows. If inference takes too long, the result arrives after the decision window closes—the object has moved out of the camera's field of view, the anomaly has progressed to failure, or the user has given up waiting. Latency violations don't crash the system the way memory violations do. They make the system correct but useless.

This chapter teaches you to estimate inference latency from operation count and hardware throughput, identify which operations dominate compute time, determine whether a model-hardware-deadline combination is feasible, and apply the optimizations that reduce latency when the initial estimate fails. By the end of this chapter, you will be able to evaluate the compute constraint before deployment and specify precisely what must change if the model is too slow.

FLOPs, MACs, and What They Actually Mean for Inference Time

A neural network's compute requirement is quantified in two ways: FLOPs (floating-point operations) and MACs (multiply-accumulate operations). Both measure the number of arithmetic operations the model performs per inference, but they count differently.

A FLOP is one floating-point operation—an addition, subtraction, multiplication, or division. A fully connected layer performing $\mathbf{y} = \mathbf{W} \times \mathbf{x} + \mathbf{b}$ involves:

- $(\text{input_size} \times \text{output_size})$ multiplications
- $(\text{input_size} \times \text{output_size})$ additions for the matrix multiply
- (output_size) additions for the bias

Total FLOPs = $2 \times (\text{input_size} \times \text{output_size}) + \text{output_size}$.

A MAC is one multiply-accumulate operation: **result += a × b**. This is the fundamental operation in matrix multiplication and convolution. Processors with MAC units (most ARM Cortex-M and Cortex-A cores, DSPs, and NPUs) can execute a MAC in a single instruction. The FLOP count is roughly 2× the MAC count because each MAC includes both a multiply and an add.

For embedded AI, MAC count is the more useful metric because it maps directly to hardware capabilities. A processor that executes 100 million MACs per second (100 MMAC/s) will take roughly $(\text{total MACs} / 100\text{M})$ seconds to execute the model, ignoring memory access overhead.

Most ML frameworks report FLOPs. TensorFlow's model summary includes FLOP count. PyTorch's `thop` library computes FLOPs. To convert to MACs, divide by 2 for operations that are pure multiply-adds (convolution, matrix multiply). For mixed operations (activations, pooling), use the FLOP count directly—it's a conservative upper bound.

The relationship between operation count and latency is:

Latency (seconds) = (Total operations) / (Sustained throughput)

where sustained throughput accounts for memory access, cache efficiency, and pipeline utilization. Theoretical peak throughput (from the datasheet) is always higher than sustained throughput. The ratio between peak and sustained varies from 0.5× to 0.9× depending on the processor, the operation, and how well the code is optimized.

For example, a Cortex-M7 at 400 MHz with an FPU has a theoretical peak of 400 MMAC/s (one MAC per cycle). But real code achieves 250–350 MMAC/s sustained due to memory bottlenecks and instruction overhead. A model with 50 million MACs takes:

- Theoretical: $50\text{M} / 400\text{M} = 125$ milliseconds
- Actual: $50\text{M} / 300\text{M} = 167$ milliseconds

The 33% gap between theoretical and actual is why you must profile on target hardware. The theoretical estimate tells you whether deployment is plausible. Profiling tells you whether it actually works.

Compute-Bound vs. Memory-Bound Inference

An operation is compute-bound if its execution time is dominated by arithmetic operations. It's memory-bound if execution time is dominated by loading data from memory. The distinction matters because the fix is different.

A small matrix multiply ($64 \times 64 \times 64 \times 64$) on a Cortex-M7 with 64 KB of L1 cache is compute-bound. Both matrices fit in cache, so every multiply-add executes at peak throughput. Doubling the clock speed halves the execution time.

A large matrix multiply ($512 \times 512 \times 512 \times 512$) on the same Cortex-M7 is memory-bound. The matrices are 2 MB total, far exceeding the 64 KB cache. Each element is fetched from SRAM multiple times, and memory bandwidth limits throughput. Doubling the clock speed might reduce execution time by only 20% because the processor spends most of its time waiting for memory.

Convolution is almost always memory-bound on microcontrollers. A 3×3 convolution on a 96×96 image with 32 input channels and 64 output channels requires:

- **$96 \times 96 \times 64 \times 3 \times 3 \times 32 = 169$ million MACs**
- **$96 \times 96 \times 3 \times 3 \times 32 = 2.66$ million input element reads** (each input is reused across output channels, so actual reads are lower with good locality)

Even with perfect arithmetic throughput, if each input read costs 10 cycles, memory access time dominates.

How do you determine whether an operation is compute-bound or memory-bound? Measure arithmetic intensity:

Arithmetic intensity = (Operations) / (Bytes transferred)

High arithmetic intensity (>10 ops/byte) suggests compute-bound. Low arithmetic intensity (<1 op/byte) suggests memory-bound.

For a 64×64 matrix multiply:

- Operations: $64 \times 64 \times 64 = 262,144$ MACs
- Bytes transferred: $(64 \times 64 + 64 \times 64) \times 4$ bytes = 32 KB (assuming float32 and both matrices are in cache, read once)
- Arithmetic intensity: $262,144 / 32,768 = 8$ ops/byte

This is borderline. With good cache behavior, it's compute-bound. With poor cache behavior (cache thrashing), it becomes memory-bound.

For a 3×3 convolution on a 96×96 image:

- Operations: 169 million MACs
- Bytes transferred (worst case, no reuse): $96 \times 96 \times 3 \times 3 \times 32 \times 4$ bytes = 10.6 MB
- Arithmetic intensity: $169\text{M} / 10.6\text{M} = 16$ ops/byte

This looks compute-bound, but in practice, convolutional kernels reread input data from memory because the working set exceeds cache size. Effective arithmetic intensity drops to 1–3 ops/byte, making it memory-bound.

The optimization strategy differs:

- **For compute-bound operations:** Increase clock speed, use SIMD, offload to hardware accelerator.
- **For memory-bound operations:** Reduce data movement (tiling, buffer reuse), increase memory bandwidth (on-chip SRAM instead of external DRAM), compress data (quantization reduces bytes transferred).

Profiling tells you which bottleneck you have. If increasing clock speed from 100 MHz to 200 MHz doubles performance, you're compute-bound. If it increases performance by only 10%, you're memory-bound, and you need to fix memory access patterns, not arithmetic throughput.

Clock Speed, Pipeline Efficiency, and SIMD

A processor's clock speed—measured in MHz or GHz—tells you how many cycles per second it executes. But cycles are not operations. The number of operations per cycle depends on the instruction set, pipeline depth, and whether SIMD (single instruction, multiple data) extensions are available.

A Cortex-M0+ executes one instruction per cycle in ideal conditions. It has no pipeline forwarding, no branch prediction, and no SIMD. An add instruction takes 1 cycle if operands are in registers. A load from memory takes 2 cycles. A multiply takes 32 cycles (software emulation if no hardware multiplier is present). At 48 MHz, peak throughput is 48 million simple instructions per second, but MAC throughput is far lower.

A Cortex-M4 with DSP extensions executes one instruction per cycle with a 3-stage pipeline. It includes a single-cycle hardware multiplier and an FPU (floating-point unit). SIMD instructions (SIMD32 in ARM documentation) allow four 8-bit or two 16-bit operations in parallel. At 100 MHz, peak throughput is 100 MMAC/s for single-precision floats, or 400 million 8-bit MACs per second with SIMD.

A Cortex-M7 has a 6-stage superscalar pipeline, dual-issue execution (two instructions per cycle if they don't conflict), and a double-precision FPU. Cache and branch prediction reduce pipeline stalls. At 400 MHz, theoretical peak is 800 MMAC/s for single-precision floats (dual-issue MAC instructions), though sustained throughput is 400–600 MMAC/s due to instruction dependencies and memory access.

A Cortex-A53 (used in Raspberry Pi Zero 2 W) has an 8-stage pipeline, out-of-order execution, and ARM NEON SIMD extensions. NEON can execute 16 8-bit MACs per cycle per core. With 4 cores at 1 GHz, theoretical peak is 64 billion 8-bit MACs per second (64 GMAC/s). Sustained throughput is 30–50 GMAC/s depending on memory bandwidth and cache efficiency.

SIMD is the key to high throughput on microcontrollers. A Cortex-M4 executing scalar int8 MACs achieves 100 million MACs per second. With SIMD (packing four int8 values into a 32-bit register), the same processor achieves 400 million int8 MACs per second—a 4× improvement with no clock speed increase.

But SIMD requires explicit code optimization. Most inference frameworks include SIMD-optimized kernels for common operations (convolution, matrix multiply, activation functions). TensorFlow Lite Micro uses CMSIS-NN, ARM's optimized neural network library, which exploits SIMD on Cortex-M cores. If your inference code isn't using SIMD-optimized kernels, you're leaving a 4–8× performance improvement on the table.

To verify SIMD usage, disassemble the compiled binary and check for SIMD instructions:

- Cortex-M4: `SMLAD`, `SMLALD`, `USAD8` (SIMD multiply-accumulate)
- Cortex-A: `VMLA.I8`, `VMLA.I16` (NEON multiply-accumulate)

If you see only scalar instructions (`MUL`, `ADD`), the code is not using SIMD, and you need to enable optimized kernels in your build configuration.

Worst-Case Execution Time (WCET) Analysis for AI Inference

Real-time systems care about worst-case execution time (WCET), not average execution time. A system with 50 ms average latency and 500 ms worst-case latency fails if the deadline is 100 ms—the worst case violates the requirement.

WCET is the maximum time an operation can take under any input, any cache state, and any scheduling condition. For traditional embedded code (PID controllers, sensor drivers, communication protocols), WCET is analyzable using static analysis tools that model the processor's pipeline, cache, and memory system and compute the longest possible execution path.

For neural network inference, WCET analysis is difficult because:

1. **Path explosion:** A network with 20 layers has billions of possible execution paths when considering all input-dependent branches (though modern feedforward networks have few data-dependent branches).
2. **Variable memory latency:** Cache hit/miss patterns depend on input data and prior state. A cache miss can increase execution time by 10–100 cycles per memory access.
3. **Compiler optimizations:** Modern compilers reorder instructions, unroll loops, and inline functions in ways that make static analysis intractable.
4. **Framework complexity:** Inference engines (TensorFlow Lite Micro, PyTorch Mobile) are large codebases with dynamic dispatch and function pointers, making static path analysis infeasible.

The pragmatic approach to WCET for neural network inference is empirical: run the model on pathological inputs and measure the maximum observed latency, then add safety margin.

Step 1: Identify pathological inputs. For most feedforward networks with fixed-point or integer arithmetic, input data does not affect execution path—all inputs take the same path through the network. But for networks with dynamic layers (attention mechanisms, early exit branches, conditional execution), different inputs trigger different paths. Generate inputs that maximize the computational path (e.g., inputs that trigger all branches, avoid early exits, or produce the largest intermediate activations).

Step 2: Measure latency distribution. Run inference on thousands of test inputs and log latency for each. Plot the distribution. The maximum observed latency is your empirical WCET estimate.

Step 3: Add safety margin. The maximum observed latency is not guaranteed to be the true WCET—you may not have tested the input that triggers the worst case. Industry practice for safety-critical systems is to add 20–50% margin above the observed maximum. If measured maximum is 80 ms, WCET estimate is $80 \times 1.3 = 104$ ms.

Step 4: Verify on target hardware. WCET depends on the specific hardware configuration—clock speed, cache size, memory latency. Measure on the actual deployment target, not a development board with different specs.

For hard real-time applications (automotive, industrial control, medical devices), you may need provable WCET guarantees. This requires:

- Using a certified WCET analysis tool (e.g., AbsInt aiT, Rapita RVS) that supports your processor.
- Restricting the network to fixed-path architectures (no dynamic control flow).
- Disabling caches or using cache locking to make memory latency deterministic.
- Disabling interrupts during inference or running inference at highest priority.

These restrictions are expensive—fixed-path architectures limit model expressiveness, cache locking limits performance, and disabling interrupts increases system latency for other tasks. They're justified only when hard real-time guarantees are mandatory.

For soft real-time applications (most embedded AI deployments), empirical WCET with margin is sufficient. Test thoroughly, measure the 99.9th percentile latency, and add margin. If the application can tolerate occasional deadline misses, this is the right balance between rigor and practicality.

The Latency-Accuracy Tradeoff

Smaller models run faster but produce worse predictions. Larger models produce better predictions but run slower. The latency-accuracy tradeoff is the central design constraint for embedded AI.

Consider three variants of a gesture recognition model:

- **Model A:** 2 million parameters, 80 million MACs, 92% accuracy
- **Model B:** 800,000 parameters, 35 million MACs, 89% accuracy
- **Model C:** 300,000 parameters, 12 million MACs, 84% accuracy

On a Cortex-M4 at 100 MHz sustaining 60 MMAC/s:

- Model A: 1,333 ms latency
- Model B: 583 ms latency
- Model C: 200 ms latency

If your application has a 250 ms deadline, Model A fails, Model B fails, and Model C passes. But Model C's 84% accuracy may be below your application's requirement (say, 88% minimum). You're caught between latency and accuracy constraints.

The solution is to search the model architecture space for designs that lie on the Pareto frontier—models where no other model is both faster and more accurate. Model B is not on the Pareto frontier because Model C is faster (though less accurate) and Model A is more accurate (though slower). But if there exists a Model D with 500,000 parameters, 25 million MACs, and 90% accuracy, it Pareto-dominates Model C (same latency, better accuracy) and might be the optimal choice.

How do you find Pareto-optimal models? Two approaches:

Manual architecture search: Start with a well-known efficient architecture (MobileNetV2, EfficientNet-Lite) and scale it. Reduce input resolution, reduce channel count (width multiplier), reduce depth (number of layers). Measure latency and accuracy for each configuration. Plot them on a latency-accuracy graph and identify the Pareto frontier.

Neural Architecture Search (NAS): Use automated search algorithms to explore architecture space. NAS tools like Once-for-All (OFA), ProxylessNAS, or MnasNet search over architectures optimized for specific hardware constraints. You specify the target device and latency budget, and NAS proposes architectures that maximize accuracy subject to those constraints.

NAS is expensive—it requires GPU-hours to GPU-days to search the architecture space—but it often finds architectures that manual search misses. For high-volume products where even a 1% accuracy improvement justifies the search cost, NAS is worth it. For low-volume or one-off deployments, manual scaling is sufficient.

The latency-accuracy tradeoff is not negotiable. You cannot have both the best accuracy and the lowest latency. You must decide which constraint is binding—is latency a hard ceiling (the

model must run in under 100 ms), or is accuracy a hard floor (the model must achieve at least 90%)? The binding constraint determines which models are viable.

A Worked Example: Evaluating Three Model Candidates for a 50ms Deadline

You are designing a gesture recognition system for a wearable fitness tracker. The system uses a 3-axis accelerometer sampled at 100 Hz. Gestures are classified every 1 second (100 samples). The application requires:

- Latency: ≤ 50 ms from sample window complete to gesture output
- Accuracy: $\geq 88\%$ on a validation set of 20 gestures
- Power: < 5 mA average (the tracker runs on a coin cell)

You have three candidate models:

Model A: 1D CNN

- Architecture: 4 convolutional layers + 2 fully connected layers
- Parameters: 120,000 (480 KB as float32, 120 KB as int8)
- Operation count: 18 million MACs
- Measured accuracy: 91.2%

Model B: LSTM

- Architecture: 2 LSTM layers (64 units each) + 1 fully connected layer
- Parameters: 85,000 (340 KB as float32, 85 KB as int8)
- Operation count: 14 million MACs (per time step, 100 steps total = 1.4 billion MACs)
- Measured accuracy: 93.5%

Model C: Depthwise Separable CNN

- Architecture: 3 depthwise separable blocks + 1 fully connected layer
- Parameters: 45,000 (180 KB as float32, 45 KB as int8)
- Operation count: 8 million MACs
- Measured accuracy: 88.9%

Your target hardware is the nRF52840 (Cortex-M4 at 64 MHz with FPU). From prior profiling, you know the device achieves 50 MMAC/s sustained throughput for int8 inference with CMSIS-NN optimized kernels.

Latency analysis:

Model A: $18\text{M} / 50\text{M} = 360$ ms. **Fails the 50 ms deadline.**

Model B: $1.4\text{B} / 50\text{M} = 28,000\text{ ms} = 28\text{ seconds}$. **Catastrophic failure.** LSTMs are extremely expensive for long sequences on microcontrollers.

Model C: $8\text{M} / 50\text{M} = 160\text{ ms}$. **Fails the 50 ms deadline.**

None of the models meet the latency requirement at 64 MHz. But the nRF52840 can run at 64 MHz in low-power mode or temporarily boost to 128 MHz for inference bursts (at higher power cost).

At 128 MHz, sustained throughput doubles to $\sim 100\text{ MMAC/s}$:

Model A: $18\text{M} / 100\text{M} = 180\text{ ms}$. Still fails.

Model C: $8\text{M} / 100\text{M} = 80\text{ ms}$. Still fails.

You realize the deadline is too aggressive for any model on this hardware without further optimization. You revisit the requirement with the product team and negotiate: latency can be up to 100 ms if accuracy is at least 88%.

At 128 MHz with the 100 ms deadline:

Model C: 80 ms latency, 88.9% accuracy. **Passes both constraints.**

Power analysis:

Running at 128 MHz increases active current from 15 mA (64 MHz) to 28 mA (128 MHz). Inference takes 80 ms. The device sleeps for 920 ms between classifications.

Average current: $(28\text{ mA} \times 0.08\text{ s} + 0.002\text{ mA} \times 0.92\text{ s}) / 1\text{ s} = 2.24\text{ mA} + 0.002\text{ mA} = 2.24\text{ mA}$.

The 5 mA budget allows for 2.24 mA inference plus 2.76 mA for sensor acquisition and communication. This is tight but feasible.

Conclusion:

Model C is the only viable option. Models A and B fail the latency constraint even with clock boosting. Model B (LSTM) is disqualified entirely—its 28-second latency is unusable on this hardware. Model C meets the negotiated requirements: 80 ms latency (under 100 ms), 88.9% accuracy (above 88%), and 2.24 mA average power (under 5 mA).

If the product team cannot accept the relaxed 100 ms deadline, the hardware is insufficient. You would need to:

- Upgrade to a faster processor (e.g., STM32H7 at 480 MHz, which would achieve 400+ MMAC/s and bring Model C's latency to 20 ms), or
- Add a hardware accelerator (e.g., Cortex-M55 + Ethos-U55 NPU), or
- Further reduce the model (fewer layers, fewer channels, lower accuracy).

This is the compute constraint evaluation workflow. Theoretical analysis predicted feasibility. Profiling revealed the real bottleneck. Constraint relaxation (100 ms instead of 50 ms) made deployment viable. Without this analysis, you would have discovered the failure only after manufacturing thousands of units.

When You Cannot Meet the Latency Budget

If none of the available models meet the latency requirement, you have four options:

Option 1: Upgrade the hardware. Select a faster processor, add a hardware accelerator, or move to an application processor with NEON SIMD. This increases cost and power but solves the compute constraint definitively.

Option 2: Reduce the model's operation count. Shrink the architecture—fewer layers, fewer channels, lower input resolution. Measure the accuracy cost. If the accuracy floor is still met, deploy the smaller model.

Option 3: Relax the latency requirement. Negotiate with the product team. If the requirement was 50 ms because "that feels responsive," but 100 ms is actually acceptable, the constraint problem may disappear.

Option 4: Reject AI for this application. If no model meets both the accuracy and latency requirements on feasible hardware, AI is the wrong solution. Use a classical algorithm instead—a hand-tuned threshold-based classifier, a decision tree, or a finite state machine.

The compute constraint is unforgiving. Unlike memory, where you can swap to external storage at a performance cost, there is no workaround for insufficient compute. Either the processor can execute the operations in time, or it cannot. Optimization—SIMD, quantization, hardware acceleration—buys you 2–10× improvement. If you need 100× improvement, you've chosen the wrong hardware or the wrong model.

That's why this chapter follows memory in the diagnostic sequence. If the model fits in memory but cannot run fast enough, you know immediately that you need a faster processor, a smaller model, or a relaxed deadline. The power constraint (Chapter 7) examines what happens when the compute is fast enough but consumes too much energy.

Chapter 7: Power and Energy: The Efficiency Imperative

The model fits in memory. Inference completes in 45 milliseconds, well within the 100 millisecond deadline. You deploy to a battery-powered wildlife monitoring camera programmed

to run inference every 30 seconds and transmit results once per hour. The system is spec'd for six months of operation on four AA batteries.

Three weeks into field testing, the batteries die. The deployed units report normal operation—correct classifications, no crashes, no errors—they just run out of power 22 times faster than calculated.

You retrieve a unit and profile it. Measured current draw during inference: 85 mA. Expected current draw: 40 mA. The model runs correctly but consumes twice the predicted power. When you scale that to continuous operation—inference every 30 seconds, 24 hours a day—the battery that should last 4,380 hours lasts 200 hours instead.

This is the power constraint. A model can fit in memory, meet latency requirements, and still fail in deployment because it drains the battery too quickly. For battery-powered and energy-harvesting systems, power is often the binding constraint—the one that determines whether the product is viable or whether the deployment fails in the field.

This chapter teaches you to estimate the energy cost of inference, determine whether a deployment strategy is compatible with a power budget, design inference scheduling strategies that maximize battery life, and profile power consumption on target hardware. By the end of this chapter, you will be able to evaluate the power constraint as rigorously as you evaluate memory and compute.

Power vs. Energy: The Distinction That Matters

Power and energy are related but not interchangeable. Confusing them leads to incorrect calculations and failed deployments.

Power is the instantaneous rate of energy consumption, measured in watts (W) or milliwatts (mW). A device drawing 50 mW consumes energy at a rate of 50 millijoules per second.

Energy is the total amount of work done or consumed over time, measured in joules (J) or watt-hours (Wh). A device drawing 50 mW for 10 seconds consumes 0.5 joules of energy.

The relationship is:

$$\text{Energy (J)} = \text{Power (W)} \times \text{Time (s)}$$

For embedded systems running on batteries, what matters is total energy consumption, not instantaneous power—unless instantaneous power exceeds what the battery or power supply can deliver (the peak power constraint).

A battery stores a fixed amount of energy. A 1000 mAh lithium-ion cell at 3.7V stores:

$$\text{Energy} = 1000 \text{ mAh} \times 3.7 \text{ V} = 3.7 \text{ Wh} = 13,320 \text{ J}$$

If your device consumes energy at an average rate of 10 mW, battery life is:

$$\text{Battery life} = 13,320 \text{ J} / 0.01 \text{ W} = 1,332,000 \text{ seconds} = 370 \text{ hours} = 15.4 \text{ days}$$

But if your device consumes 10 mW on average by running at 200 mW for 5% of the time and sleeping at 0.5 mW for 95% of the time, the calculation is different:

$$\text{Average power} = (200 \text{ mW} \times 0.05) + (0.5 \text{ mW} \times 0.95) = 10 + 0.475 = 10.475 \text{ mW}$$

The battery life is nearly the same because average power is nearly the same. This is why duty cycling—running the processor at high power for short bursts, then sleeping—is fundamental to embedded AI power management.

The mistake embedded designers make is calculating power based on active current alone. A model that runs at 60 mA for 80 ms every 10 seconds does not consume 60 mA average. It consumes:

$$I_{\text{avg}} = (60 \text{ mA} \times 0.08 \text{ s} + 0.005 \text{ mA} \times 9.92 \text{ s}) / 10 \text{ s} = 0.48 \text{ mA} + 0.05 \text{ mA} = 0.53 \text{ mA}$$

That's 100× lower than the active current. The sleep current dominates average power when duty cycle is low.

The power constraint question is: given a battery capacity, an inference schedule, and measured active and sleep currents, how long does the battery last? If the answer is less than the application requirement, the power constraint fails.

Dynamic and Static Power in AI Inference Workloads

Power consumption in digital circuits has two components: dynamic power and static power.

Dynamic power is the energy consumed by switching transistors—charging and discharging capacitive loads as logic gates toggle between 0 and 1. Dynamic power is proportional to:

- Clock frequency (higher frequency = more transitions per second)
- Supply voltage squared (higher voltage = more energy per transition)
- Activity factor (fraction of transistors switching per cycle)

$$P_{\text{dynamic}} = \alpha \times C \times V^2 \times f$$

where α is the activity factor, C is the capacitance being switched, V is the supply voltage, and f is the clock frequency.

For AI inference, dynamic power dominates when the processor is active. Every multiply-accumulate operation charges and discharges transistors. Every memory access switches address and data lines. Clock frequency directly controls dynamic power—running at

200 MHz consumes roughly twice the dynamic power of running at 100 MHz, assuming voltage stays constant.

Static power (also called leakage power) is the energy consumed by transistors even when they're not switching—current that leaks through the gate oxide and junctions. Static power is nearly independent of clock frequency. It depends on:

- Supply voltage (higher voltage = more leakage)
- Temperature (higher temperature = more leakage, exponentially)
- Process technology (smaller transistors = more leakage per area)

For modern microcontrollers at room temperature, static power is 1–10% of dynamic power when the processor is active. But when the processor is asleep (clock gated, most transistors off), static power dominates. Sleep current is almost entirely leakage.

This matters for AI inference because active and sleep power scale differently. Reducing clock frequency reduces dynamic power linearly but doesn't affect static power. If your inference workload is short bursts (10 ms active, 990 ms sleep), reducing clock frequency from 200 MHz to 100 MHz halves the active power but barely changes average power because sleep power dominates.

The optimization strategy depends on duty cycle:

- **High duty cycle (>50% active time):** Reduce clock frequency, reduce voltage (DVFS—dynamic voltage and frequency scaling), or offload to a more efficient accelerator.
- **Low duty cycle (<5% active time):** Reduce sleep current (use deep sleep modes, disable peripherals, gate power rails), minimize active time (faster inference at higher clock, not slower inference at lower clock).

A common mistake is running inference at low clock speed to "save power." If inference takes 400 ms at 64 MHz but only 200 ms at 128 MHz, and sleep current is 5 μ A, which is better?

At 64 MHz:

- Active current: 20 mA for 400 ms
- Sleep current: 0.005 mA for 9,600 ms
- Average current: $(20 \times 0.4 + 0.005 \times 9.6) / 10 = 0.8 + 0.048 = 0.85$ mA

At 128 MHz:

- Active current: 35 mA for 200 ms
- Sleep current: 0.005 mA for 9,800 ms
- Average current: $(35 \times 0.2 + 0.005 \times 9.8) / 10 = 0.7 + 0.049 = 0.75$ mA

Running at 128 MHz (higher clock, higher active power, shorter active time) saves 12% average power compared to 64 MHz. This is "race to sleep"—finish inference as quickly as possible and return to low-power sleep mode.

The crossover point depends on the ratio of active current to sleep current and the duty cycle. For systems that spend >90% of the time asleep, race to sleep almost always wins. For systems that are continuously active, reducing clock frequency saves power.

Duty Cycling and Inference Scheduling

Duty cycling is the technique of alternating between active and sleep states to reduce average power consumption. For embedded AI, the duty cycle is determined by:

- How often inference must run (the inference interval)
- How long each inference takes (the inference latency)
- What sleep mode the system uses between inferences

A wildlife camera that runs inference once every 60 seconds, where inference takes 150 ms, has a duty cycle of:

$$\text{Duty cycle} = 0.15 \text{ s} / 60 \text{ s} = 0.25\%$$

The processor is active for 0.25% of the time and asleep for 99.75% of the time. If active current is 50 mA and sleep current is 10 μ A, average current is:

$$I_{\text{avg}} = (50 \text{ mA} \times 0.0025) + (0.01 \text{ mA} \times 0.9975) = 0.125 \text{ mA} + 0.01 \text{ mA} = 0.135 \text{ mA}$$

A 2000 mAh battery lasts $2000 / 0.135 = 14,800$ hours = 617 days.

But if the camera must run inference every 5 seconds instead of every 60 seconds (to catch faster-moving animals), duty cycle increases to 3%:

$$I_{\text{avg}} = (50 \text{ mA} \times 0.03) + (0.01 \text{ mA} \times 0.97) = 1.5 \text{ mA} + 0.01 \text{ mA} = 1.51 \text{ mA}$$

Battery life drops to 1324 hours = 55 days.

Increasing the inference rate by 12 $\times$ reduces battery life by 11 $\times$. This is the inference scheduling constraint: more frequent inference costs energy proportional to the increase in duty cycle.

The design question is: how often must inference run to meet the application requirement? There are three scheduling strategies:

Periodic inference: Run inference at fixed intervals (every N seconds). This is simple and predictable. It works when the application has no external trigger—anomaly detection on continuous sensor streams, environmental monitoring, health tracking.

Event-driven inference: Run inference only when a low-power sensor detects an event of interest. A motion detector wakes the system, then the camera runs inference. A voice activity detector (VAD) wakes the system, then the keyword spotter runs. This dramatically reduces duty cycle for applications where events are rare—a camera that triggers on motion might run inference 10 times per day instead of 1440 times per day.

Adaptive inference: Vary the inference interval based on context. Run inference frequently when activity is detected, infrequently when the environment is stable. A predictive maintenance system might run inference every 10 seconds when vibration levels are high, but only every 10 minutes when vibration is within normal range. This balances responsiveness and power consumption.

Event-driven inference is the most power-efficient when applicable. The energy cost of the trigger mechanism (motion detector, VAD) must be lower than the energy cost of continuous inference. A motion detector that consumes 50 μ A continuously saves energy if it avoids running inference that consumes 50 mA for 150 ms every 10 seconds—the motion detector uses 0.05 mA continuous, while periodic inference would use $(50 \times 0.015 + 0.01 \times 9.985) / 10 = 0.075 + 0.1 = 0.175$ mA average.

The workflow for duty cycle optimization is:

1. Determine the minimum inference interval that meets application requirements.
2. Measure inference latency on target hardware.
3. Calculate duty cycle and average current.
4. If average current exceeds the power budget, apply one of: reduce inference interval (if the application allows), reduce inference latency (faster model or faster processor), or use event-driven triggering.

Energy Harvesting Constraints

Energy harvesting systems—powered by solar cells, piezoelectric generators, thermoelectric generators, or RF harvesting—have different power constraints than battery-powered systems. Instead of a fixed energy budget that depletes over time, energy harvesters have a power budget that must be balanced in steady state.

An energy harvesting system is viable if:

Average power consumption $\leq$ Average power generation

A solar-powered sensor with a 1 cm² solar cell generates roughly 10 mW in full sunlight (100 mW/cm² irradiance $\times$ 10% conversion efficiency). If the sensor consumes 5 mW on average, the system is energy-positive and can run indefinitely. If the sensor consumes 15 mW on average, the system is energy-negative and will deplete any storage capacitor or battery backup.

But energy harvesting is intermittent. Solar power drops to near-zero at night. Vibration harvesting depends on machine operation. RF harvesting depends on proximity to a transmitter. The system must store enough energy during high-harvesting periods to survive low-harvesting periods.

For AI inference, this creates two constraints:

Peak power constraint: The instantaneous power draw during inference must not exceed the harvester's peak output. If the harvester produces 20 mW peak and inference draws 100 mW, the system cannot run inference directly from harvested power—it must accumulate energy in a storage capacitor, then discharge the capacitor during inference.

Average energy balance: Over a full harvesting cycle (e.g., 24 hours for solar), total energy consumed must not exceed total energy harvested. If the harvester generates 10 mWh per day and the system consumes 12 mWh per day, the system fails.

A worked example: A vibration-powered industrial sensor harvests 5 mW when the motor is running (16 hours per day) and 0 mW when the motor is off (8 hours per day). Total harvested energy per day:

$$E_{\text{harvested}} = 5 \text{ mW} \times 16 \text{ hours} = 80 \text{ mWh}$$

The sensor runs inference every 60 seconds. Inference takes 120 ms at 80 mA (3.3V supply). Energy per inference:

$$E_{\text{inference}} = 3.3 \text{ V} \times 0.08 \text{ A} \times 0.12 \text{ s} = 0.0317 \text{ J} = 0.0088 \text{ mWh}$$

Inferences per day: 1440. Total inference energy: $1440 \times 0.0088 = 12.7 \text{ mWh}$.

Sleep power: $3.3 \text{ V} \times 0.02 \text{ mA} = 0.066 \text{ mW}$. Sleep time per day: $24 \text{ hours} - (1440 \times 0.12 \text{ s}) / 3600 \text{ s} = 24 - 0.048 = 23.95 \text{ hours}$. Sleep energy: $0.066 \text{ mW} \times 23.95 \text{ hours} = 1.58 \text{ mWh}$.

Total energy consumed: $12.7 + 1.58 = 14.3 \text{ mWh}$.

The system consumes 14.3 mWh but harvests 80 mWh. Energy balance is positive by a factor of 5.6×. The system can run indefinitely with significant margin.

But if inference must run every 10 seconds instead of every 60 seconds, inferences per day increase to 8,640. Inference energy: $8,640 \times 0.0088 = 76 \text{ mWh}$. Sleep energy remains roughly the same. Total: 77.6 mWh.

The system now consumes 77.6 mWh and harvests 80 mWh. Energy balance is positive by only 3%. This is marginal—harvester output varies with temperature, vibration amplitude, and mechanical aging. A 5% drop in harvesting efficiency causes the system to fail.

The energy harvesting constraint is unforgiving. Unlike battery-powered systems where you can ship with a larger battery, energy harvesting systems have fixed input power. If the system

consumes more than it harvests, no amount of optimization will make it viable—you must reduce consumption, increase harvesting area, or reject the deployment.

Event-Driven AI: Using Low-Power Sensors to Trigger Inference

A camera running continuous inference at 10 fps draws power continuously. A camera that runs inference only when motion is detected draws power in bursts. The difference in battery life can be 100× for applications where events are rare.

Event-driven AI uses a low-power trigger mechanism to wake the system and run inference. The trigger is always-on, but its power consumption is orders of magnitude lower than the AI model.

Common trigger mechanisms:

Motion detectors: Passive infrared (PIR) sensors consume 50–200 μA and wake on motion. A wildlife camera with a PIR trigger runs inference only when an animal enters the field of view. If animals trigger the camera 20 times per day, the camera runs 20 inferences per day instead of 86,400 (one per second). Energy savings: 4,300×.

Voice activity detectors (VAD): Detect speech in audio without running a full keyword spotter. A VAD can be as simple as an energy threshold detector or as complex as a small neural network running on a low-power DSP. When the VAD detects speech, it wakes the main processor to run the keyword spotter. If speech occurs 5% of the time, energy savings are 20×.

Accelerometer interrupts: Configure an accelerometer to generate an interrupt when motion exceeds a threshold. A gesture recognition system runs inference only when the wearable detects movement, not continuously. If the user is stationary 80% of the time, energy savings are 5×.

Hierarchical inference: Run a tiny, low-power model continuously to detect events, then run a larger, more accurate model only when the first model triggers. A two-stage system might use a 10,000-parameter decision tree (1 ms inference, 2 mA) running every 100 ms to detect anomalies, then trigger a 500,000-parameter CNN (150 ms inference, 60 mA) only when the decision tree flags a potential anomaly. If anomalies occur 1% of the time, the two-stage system consumes 90% less power than running the CNN continuously.

The design workflow is:

1. Identify a low-power signal that correlates with the need for inference.
2. Measure the trigger mechanism's power consumption and false-positive rate.
3. Calculate average power with event-driven triggering: $(\text{trigger power} \times 100\%) + (\text{inference power} \times \text{event rate} \times \text{inference time})$.
4. Compare to continuous inference power.

5. If event-driven power is lower and the false-positive rate is acceptable, deploy event-driven inference.

The failure mode is high false-positive rates. If the motion detector triggers on wind, shadows, or small insects, the camera runs inference thousands of times per day instead of 20 times per day, and energy savings disappear. Trigger mechanisms must be tuned to the application's event statistics.

Power Profiling: How to Measure Inference Energy Consumption

Estimating power consumption from datasheets is unreliable. Active current depends on clock speed, memory access patterns, and peripheral usage. Sleep current depends on temperature, enabled peripherals, and retention modes. The only way to know actual power consumption is to measure it.

The standard method is a current sense resistor in series with the power supply. You insert a small resistor (0.1–1 Ω) between the battery and the device, measure the voltage drop across the resistor, and compute current using Ohm's law. The voltage drop must be small enough not to affect device operation—a 0.1 Ω resistor with 100 mA current drops only 10 mV, which is negligible.

For dynamic measurements, you log voltage over time using an oscilloscope or a data acquisition system. Modern power profilers (e.g., Nordic Power Profiler Kit II, Keysight N6705C) measure current at kHz to MHz sampling rates, allowing you to see individual inference bursts.

A typical power profile for inference looks like:

- Baseline: sleep current (5–50 μ A)
- Spike: sensor acquisition (10–50 ms at 5–20 mA)
- Spike: inference execution (50–500 ms at 30–200 mA)
- Return to baseline: sleep current

You integrate the area under the current curve to get total energy per inference:

$$\text{Energy (J)} = \int V(t) \times I(t) dt$$

For a 3.3V device:

$$\text{Energy (J)} = 3.3 V \times \int I(t) dt$$

If the current profile is a square wave (simplified assumption), the integral becomes:

$$\text{Energy} = V \times I_{\text{active}} \times t_{\text{active}}$$

For 80 mA active current, 150 ms active time:

$$\text{Energy} = 3.3 \text{ V} \times 0.08 \text{ A} \times 0.15 \text{ s} = 0.0396 \text{ J} = 39.6 \text{ mJ}$$

If inference runs every 30 seconds, average power is:

$$P_{\text{avg}} = 0.0396 \text{ J} / 30 \text{ s} = 1.32 \text{ mW}$$

A 2000 mAh battery at 3.3V stores 6600 mWh = 23,760 J. Battery life:

$$\text{Life} = 23,760 \text{ J} / 1.32 \text{ mW} = 18 \text{ million seconds} = 208 \text{ days}$$

But this assumes sleep current is zero, which is unrealistic. If sleep current is 10 μ A:

$$\text{Sleep power} = 3.3 \text{ V} \times 0.01 \text{ mA} = 0.033 \text{ mW}$$

Sleep time per cycle: 30 s - 0.15 s = 29.85 s. Sleep energy: 0.033 mW $\times$ 29.85 s = 0.985 mJ.

Total energy per cycle: 39.6 mJ (inference) + 0.985 mJ (sleep) = 40.6 mJ.

Average power: 40.6 mJ / 30 s = 1.35 mW.

Battery life: 23,760 J / 1.35 mW = 17.6 million seconds = 204 days.

The sleep energy contribution is small (2.4%) because duty cycle is low. But for higher duty cycles or higher sleep currents, sleep energy can dominate.

Profiling reveals surprises. Peripherals left enabled during sleep, unintended wake-ups, GPIO leakage currents—all contribute to higher-than-expected power consumption. The wildlife camera that drained batteries in three weeks likely had one of these issues: a peripheral left active, a firmware bug that prevented deep sleep entry, or GPIO pins floating and drawing leakage current.

The profiling workflow is:

1. Connect a current sense resistor and measurement equipment.
2. Run the full application (sensor acquisition, inference, communication, sleep) in a realistic duty cycle.
3. Log current over multiple inference cycles (at least 10).
4. Compute energy per cycle and average power.
5. Calculate battery life: (battery capacity in Wh) / (average power in W).
6. If battery life is below requirement, identify the power-hungry component and optimize it.

A Worked Example: Designing Inference Scheduling for Six-Month Battery Life

You are designing a remote agricultural sensor that monitors soil moisture and runs on two AA alkaline batteries. The system must last six months (4,380 hours) in the field. The sensor uses:

- STM32L4 microcontroller (1 μ A deep sleep, 12 mA active at 80 MHz)
- Soil moisture sensor (500 μ A when powered, 0 μ A when off)
- LoRaWAN radio (40 mA transmit, 15 mA receive, 1 μ A sleep)

You have a neural network for anomaly detection in moisture patterns:

- 100,000 parameters (int8, 100 KB in flash)
- 15 million MACs per inference
- Measured latency: 220 ms (profiled on target)
- Active current during inference: 12 mA (measured)

Two AA alkaline batteries provide 3V and 2850 mAh capacity. Total energy:

$$E_{\text{battery}} = 3 \text{ V} \times 2850 \text{ mAh} = 8550 \text{ mWh} = 30,780 \text{ J}$$

Target average power for six months:

$$P_{\text{avg}} = 30,780 \text{ J} / (4380 \text{ hours} \times 3600 \text{ s/hour}) = 30,780 \text{ J} / 15.77 \text{ million s} = 1.95 \text{ mW}$$

Your task: design an inference schedule that keeps average power below 1.95 mW.

Option 1: Periodic inference every 10 minutes

Energy per inference cycle (10 minutes = 600 s):

- Sensor acquisition: $3 \text{ V} \times 0.5 \text{ mA} \times 5 \text{ s} = 7.5 \text{ mJ}$
- Inference: $3 \text{ V} \times 12 \text{ mA} \times 0.22 \text{ s} = 7.92 \text{ mJ}$
- LoRa transmission: $3 \text{ V} \times 40 \text{ mA} \times 0.5 \text{ s} = 60 \text{ mJ}$ (once per hour, so 1/6 per cycle = 10 mJ per cycle)
- Sleep: $3 \text{ V} \times 0.001 \text{ mA} \times 594.28 \text{ s} = 1.78 \text{ mJ}$

Total per cycle: $7.5 + 7.92 + 10 + 1.78 = 27.2 \text{ mJ}$

Average power: $27.2 \text{ mJ} / 600 \text{ s} = 0.045 \text{ mW}$

This is far below the 1.95 mW budget. Battery life:

$$\text{Life} = 30,780 \text{ J} / 0.045 \text{ mW} = 684 \text{ million s} = 7,903 \text{ days} = 21.6 \text{ years}$$

The constraint is not tight. You can run inference much more frequently if needed.

Option 2: Periodic inference every 1 minute

Energy per cycle (60 s):

- Sensor: 7.5 mJ
- Inference: 7.92 mJ
- LoRa: 10 mJ (amortized per hour still 1/60 per cycle = 1 mJ per minute cycle... wait, let me recalculate)

If LoRa transmits once per hour and inference runs every minute, there are 60 inference cycles per LoRa transmission. LoRa energy per cycle:

$$E_{\text{LoRa_per_cycle}} = 60 \text{ mJ} / 60 = 1 \text{ mJ}$$

Energy per cycle:

- Sensor: 7.5 mJ
- Inference: 7.92 mJ
- LoRa (amortized): 1 mJ
- Sleep: $3 \text{ V} \times 0.001 \text{ mA} \times 51.78 \text{ s} = 0.155 \text{ mJ}$

Total: 16.6 mJ per minute

Average power: $16.6 \text{ mJ} / 60 \text{ s} = 0.277 \text{ mW}$

Battery life: $30,780 \text{ J} / 0.277 \text{ mW} = 111 \text{ million s} = 3.5 \text{ years}$

Still far below budget.

Option 3: Continuous inference (every 1 second)

This is likely unnecessary for soil moisture monitoring, but let's check the power budget:

Energy per cycle (1 s):

- Sensor: 7.5 mJ
- Inference: 7.92 mJ
- LoRa (amortized): $60 \text{ mJ} / 3600 = 0.0167 \text{ mJ}$
- Sleep: $3 \text{ V} \times 0.001 \text{ mA} \times 0.28 \text{ s} = 0.00084 \text{ mJ}$

Total: 15.44 mJ per second

Average power: 15.44 mW

Battery life: $30,780 \text{ J} / 15.44 \text{ mW} = 1,994,000 \text{ s} = 554 \text{ hours} = 23 \text{ days}$

This violates the six-month requirement. Continuous inference is too aggressive.

Conclusion:

Inference every 10 minutes (Option 1) or even every 1 minute (Option 2) satisfies the power budget with massive margin. The binding constraint is not inference frequency but communication frequency—LoRa transmission consumes 60 mJ per event, which dominates the energy budget if transmissions are frequent.

If you need to maximize battery life, reduce LoRa transmission frequency to once every 6 hours instead of once per hour. This reduces average power from 0.277 mW (1-minute inference, 1-hour LoRa) to:

Energy per 6-hour period:

- Sensor: $7.5 \text{ mJ} \times 360 \text{ readings} = 2,700 \text{ mJ}$
- Inference: $7.92 \text{ mJ} \times 360 \text{ inferences} = 2,851 \text{ mJ}$
- LoRa: $60 \text{ mJ} \times 1 \text{ transmission} = 60 \text{ mJ}$
- Sleep: $3 \text{ V} \times 0.001 \text{ mA} \times (6 \times 3600 - 360 \times 5.22) \text{ s} = 3 \text{ V} \times 0.001 \text{ mA} \times 19,720 \text{ s} = 59.2 \text{ mJ}$

Total: 5,670 mJ per 6 hours = 21,600 s

Average power: $5,670 \text{ mJ} / 21,600 \text{ s} = 0.263 \text{ mW}$

Battery life: $30,780 \text{ J} / 0.263 \text{ mW} = 117 \text{ million s} = 3.7 \text{ years}$

The system can run for nearly four years on two AA batteries with inference every minute and LoRa transmission every six hours. The power constraint is not the bottleneck—communication frequency is.

This is the power constraint workflow. You start with a target battery life, calculate the allowable average power, design an inference schedule, compute actual power consumption, and verify that the schedule meets the budget. If it doesn't, you reduce inference frequency, reduce transmission frequency, or optimize the model to reduce inference energy.

When You Cannot Meet the Power Budget

If the power budget cannot be satisfied with any reasonable inference schedule, you have four options:

Option 1: Use a larger battery or energy harvesting. This increases cost and size but solves the power constraint definitively.

Option 2: Reduce inference latency. Faster inference reduces active time, which reduces energy per inference. This often means upgrading to a faster processor or adding a hardware accelerator.

Option 3: Use event-driven triggering. Replace periodic inference with motion, threshold, or pattern-based triggers. This reduces inference frequency by 10–100× for sparse-event applications.

Option 4: Offload inference to the cloud. If communication cost (latency, energy, privacy) is acceptable, run inference on a server and transmit only sensor data. This trades inference energy for communication energy.

The power constraint is different from memory and compute constraints. Memory and compute are fixed by hardware—you cannot create more SRAM or more MACs/second without changing the chip. But power is variable—you can always reduce average power by reducing duty cycle, even if it means the system runs inference less frequently.

The question is whether the reduced inference frequency still meets the application requirement. If the answer is yes, the power constraint is solvable. If the answer is no, you've chosen the wrong hardware, the wrong model, or the wrong architecture for the application.

Chapter 8 examines one solution to simultaneous memory, compute, and power constraints: hardware accelerators that execute AI operations faster and more efficiently than general-purpose CPUs.

Chapter 8: Hardware for AI: Accelerators, DSPs, and FPGAs

The model runs correctly on a Cortex-M4 at 100 MHz, but inference takes 850 milliseconds and your deadline is 100 milliseconds. You've already quantized to int8, pruned 30% of the weights, and optimized the layer sequence. The model cannot get smaller without accuracy falling below the application's threshold. The CPU cannot get faster without replacing the entire hardware platform.

Your colleague suggests adding a neural processing unit (NPU)—a specialized accelerator that executes matrix multiplications 10–50× faster than the CPU. You evaluate an STM32N6 with a built-in NPU. The same model now runs in 45 milliseconds. The latency constraint is satisfied, and the power consumption drops by 60% because the accelerator completes inference in one-fifth the time.

But the STM32N6 costs \$12 per unit instead of \$3, the development toolchain is more complex, and the NPU only accelerates certain operations—your model's depthwise convolutions fall back to the CPU, limiting the speedup. The decision is not obvious. Hardware acceleration solves the compute constraint but introduces cost, complexity, and new failure modes.

This chapter teaches you to evaluate hardware acceleration options, understand the tradeoffs between CPUs, DSPs, NPUs, and FPGAs, and determine when a hardware accelerator is worth

the cost. By the end of this chapter, you will be able to select an appropriate acceleration strategy for a given constraint profile and explain when NOT to use an accelerator—when a well-optimized CPU implementation is the better choice.

Why CPUs Are Inefficient for Neural Network Inference

A general-purpose CPU is designed to execute arbitrary instruction sequences with maximum flexibility. It has a complex pipeline, branch prediction, out-of-order execution, and caches to handle unpredictable workloads. This flexibility comes at a cost: energy and silicon area spent on control logic instead of arithmetic units.

Neural network inference is the opposite of arbitrary. It's a highly regular computation—millions of multiply-accumulate operations with predictable data flow and no branching. A CPU executing inference spends most of its energy fetching instructions, managing caches, and controlling the pipeline, not performing the actual arithmetic.

Consider a matrix multiplication: $\mathbf{C} = \mathbf{A} \times \mathbf{B}$. For a 128×128 matrix multiplied by a 128×128 matrix, the operation requires $128^3 = 2,097,152$ MACs. On a Cortex-M7 at 400 MHz, this takes roughly 10 milliseconds (200 MMAC/s sustained throughput).

Break down where the time goes:

- Arithmetic operations (multiply-accumulate): ~30% of cycles
- Memory access (loading A, B, writing C): ~50% of cycles
- Instruction fetch and decode: ~15% of cycles
- Pipeline stalls and cache misses: ~5% of cycles

Only 30% of the CPU's time is spent doing useful arithmetic. The rest is overhead. A specialized accelerator eliminates or minimizes that overhead.

An NPU designed for matrix multiplication has:

- Dedicated MAC units arranged in a 2D array (e.g., 16×16 MACs executing in parallel)
- Local scratchpad memory close to the compute units (no cache, no arbitrary access)
- A simple controller that sequences operations without instruction fetch overhead

The same 128×128 matrix multiply on a 16×16 MAC array takes:

- $(128 / 16)^3 = 512$ cycles for arithmetic
- Plus memory transfer time if A and B are in main memory

At 400 MHz, this is ~1.3 milliseconds—nearly 8× faster than the CPU. The speedup comes from parallel execution (256 MACs per cycle instead of 1) and eliminating control overhead.

But the NPU is not a general-purpose processor. It can only execute operations that map to its MAC array and memory architecture. If your model includes operations the NPU doesn't support—certain activation functions, dynamic control flow, or sparse operations—those operations fall back to the CPU, and the speedup is lost.

This is the fundamental tradeoff: CPUs are flexible but inefficient. Accelerators are efficient but inflexible. The right choice depends on whether your model's operations map cleanly to the accelerator's capabilities.

DSPs for AI: Fixed-Point Arithmetic, SIMD, and Signal Processing Pipelines

Digital signal processors (DSPs) are specialized CPUs optimized for signal processing workloads—filtering, FFT, correlation. They're not AI-specific accelerators, but they excel at certain AI operations because of three features: hardware multiply-accumulate units, SIMD extensions for packed arithmetic, and fixed-point arithmetic support.

A DSP like the TI C674x or ARM Cortex-M55 includes:

- Multiple MAC units executing in parallel (2–8 MACs per cycle)
- SIMD instructions that operate on 4 or 8 packed 8-bit or 16-bit values
- Dedicated address generation units for efficient memory access patterns (stride, circular buffers)
- Zero-overhead loop counters

For neural network inference, DSPs are effective for:

- Fully connected layers (matrix-vector multiply is a DSP primitive)
- 1D convolution (used for audio processing, time-series)
- Depthwise separable convolution (channel-wise operations map well to SIMD)
- Fixed-point quantized models (DSPs handle int8 and int16 natively)

DSPs struggle with:

- 2D convolution (requires complex address patterns that exceed the DSP's address generation capabilities)
- Pooling and activation functions (not traditional DSP operations)
- Dynamic models with conditional execution

A worked comparison: keyword spotting on audio using a 1D convolutional network.

On Cortex-M4 (general-purpose CPU):

- Clock: 100 MHz
- Throughput: 60 MMAC/s for int8 with CMSIS-NN optimizations

- Latency: $18\text{M MACs} / 60\text{M} = 300\text{ ms}$

On TI C5535 DSP:

- Clock: 100 MHz
- Throughput: 200 MMAC/s for int16 (dual MAC units, SIMD)
- Latency: $18\text{M MACs} / 200\text{M} = 90\text{ ms}$

The DSP is 3.3× faster for the same clock speed because it has more MAC units and better SIMD utilization. But the DSP costs more, consumes more power (120 mA vs. 60 mA active current), and requires different development tools (TI Code Composer Studio instead of standard GCC).

DSPs are a middle ground between general-purpose CPUs and dedicated AI accelerators. They're more efficient than CPUs for signal-heavy AI tasks but less efficient than NPUs for general neural network inference. They shine when your application includes both AI inference and traditional signal processing (FFT for audio features, filtering for sensor data)—you can run both workloads on the same processor instead of needing two separate chips.

NPUs and MicroNPUs: What They Are, What They Accelerate Well, and What They Don't

A neural processing unit (NPU) is a domain-specific accelerator designed specifically for neural network inference. It includes:

- A MAC array (e.g., 8×8 , 16×16 , or 32×32 MACs) that executes matrix operations in parallel
- On-chip SRAM or scratchpad memory for weights and activations
- A controller that sequences layer execution without CPU intervention

NPUs are categorized by size and target application:

- **Large NPUs** (Google TPU, NVIDIA TensorRT): Datacenter accelerators with thousands of MACs, gigabytes of on-chip memory, designed for batch inference on servers. Not relevant for embedded systems.
- **Edge NPUs** (Coral Edge TPU, Intel Movidius, Hailo-8): Embedded accelerators with 100–1000 MACs, megabytes of memory, designed for edge devices like cameras and drones. Power consumption: 0.5–5W.
- **MicroNPUs** (ARM Ethos-U55, STM32 AI core, Syntiant NDP120): Microcontroller-scale accelerators with 8–256 MACs, kilobytes of memory, designed to augment MCUs. Power consumption: 10–200 mW.

For this book, we focus on microNPUs—accelerators integrated into or co-packaged with microcontrollers.

The ARM Ethos-U55 is representative. It's a microNPU that pairs with Cortex-M55 or Cortex-M85 CPUs. Key specs:

- 32, 64, 128, or 256 MACs (configurable at design time)
- Supports int8 and int16 quantized models
- 128 KB to 2 MB on-chip SRAM
- Claimed performance: 0.5 TOPS (trillion operations per second) at 500 MHz for int8

For a model with 50 million int8 MACs, theoretical latency on a 256-MAC Ethos-U55 at 500 MHz:

- Peak throughput: $256 \text{ MACs} \times 500 \text{ MHz} = 128 \text{ GMAC/s} = 0.128 \text{ TOPS}$
- Latency: $50\text{M} / 128\text{M} = 390 \text{ ms}$

But that's theoretical. Real performance depends on whether the model's operations map efficiently to the accelerator. The Ethos-U55 accelerates:

- 2D convolution (standard and depthwise)
- Fully connected layers
- Pooling (max, average)
- Element-wise operations (add, multiply, ReLU)

It does not accelerate (operations fall back to the CPU):

- Attention mechanisms
- Custom activation functions
- Sparse operations
- Dynamic control flow

If your model is 80% standard convolutions (accelerated) and 20% custom operations (CPU), the effective speedup is not 10× but more like 3–5×, because the CPU becomes the bottleneck for the unaccelerated portions.

Another example: STM32N6 with integrated AI accelerator.

- Cortex-M55 CPU at 800 MHz
- Integrated NPU: claimed 3 TOPS for int8
- 2.5 MB SRAM, 4 MB flash
- Supports TensorFlow Lite models converted with STM32Cube.AI

For a 40-million-MAC MobileNetV2 model:

- CPU-only (Cortex-M55 optimized): ~180 ms
- NPU-accelerated: ~45 ms (measured, not theoretical)

The 4× speedup is real and transformative for applications with hard latency requirements. But the STM32N6 costs \$12–15 per unit versus \$4–6 for an STM32H7 without NPU. For a product manufacturing 100,000 units, that's \$800,000 in additional BOM cost.

The NPU decision is economic: does the latency improvement justify the cost increase? For consumer products with aggressive latency targets (smart doorbells, wearables), yes. For industrial sensors with relaxed latency budgets, probably not.

FPGAs for Embedded AI: Reconfigurability vs. Power vs. Development Cost

Field-programmable gate arrays (FPGAs) are reconfigurable hardware. You design a custom circuit—MAC arrays, memory controllers, custom operations—and program it onto the FPGA. Unlike CPUs or NPUs, which have fixed architectures, FPGAs let you implement exactly the accelerator you need.

For AI inference, FPGAs offer:

- Custom MAC array sizes (you choose how many MACs based on your model)
- Custom data paths optimized for specific operations
- Bit-level precision (not limited to 8-bit or 16-bit—you can use 6-bit, 10-bit, or mixed precision)
- Pipelined execution with no instruction fetch overhead

FPGAs are used in embedded AI when:

- The model has unusual operations that CPUs and NPUs don't support
- Latency requirements are so aggressive that even NPUs are insufficient
- The application requires field-reconfigurability (update the accelerator logic after deployment)

The downside is cost and complexity. FPGAs are expensive (\$20–\$200 per chip for embedded-grade parts), power-hungry (500 mW to 5W for inference), and difficult to program. Designing an FPGA accelerator requires HDL (Verilog or VHDL) expertise, which most embedded software engineers lack. Development time is measured in months, not weeks.

A worked comparison: deploying a YOLO object detection model.

Option A: Cortex-A53 (CPU-only)

- Raspberry Pi Zero 2 W (quad-core at 1 GHz)
- Latency: 2,500 ms per frame
- Power: 1.5 W
- Cost: \$15

Option B: Coral Edge TPU (dedicated NPU)

- Google Edge TPU with Cortex-A53 host
- Latency: 25 ms per frame
- Power: 2 W (TPU + CPU)
- Cost: \$60

Option C: Xilinx Zynq (FPGA + ARM CPU)

- Zynq-7020 with custom YOLO accelerator in FPGA fabric
- Latency: 15 ms per frame
- Power: 3 W
- Cost: \$150
- Development time: 6 months (FPGA design + verification)

Option B (Coral Edge TPU) offers the best balance: 100× speedup over CPU-only, reasonable power, and off-the-shelf availability. Option C (FPGA) is 1.7× faster than the TPU but costs 2.5× more, consumes 50% more power, and requires six months of FPGA development. Unless the application absolutely requires sub-20ms latency, the FPGA is not worth it.

FPGAs make sense for:

- Research prototypes where flexibility is more valuable than cost
- Niche applications where volume is too low to justify ASIC development but performance requirements exceed what NPUs provide
- Applications with evolving models where field-reconfigurability justifies the upfront cost

For most embedded AI deployments, FPGAs are overkill. CPUs with optimized software, DSPs, or microNPUs provide better cost-performance-power tradeoffs.

Integrated AI Hardware: Real Devices, Real Tradeoffs

Several microcontroller and SoC vendors now offer chips with integrated AI acceleration. These devices combine a CPU, NPU, memory, and peripherals in a single package. Here are four representative examples with real specs and tradeoffs.

ARM Cortex-M55 + Ethos-U55

What it is: ARM's microNPU architecture paired with the Cortex-M55 CPU. Available from vendors like NXP, ST, Renesas.

Specs:

- CPU: Cortex-M55 at 200–800 MHz with Helium (M-profile vector extension)
- NPU: Ethos-U55 with 32–256 MACs

- Memory: 256 KB – 2 MB SRAM (vendor-dependent)
- Power: 50–150 mW during inference (vendor-dependent)

Best for: Keyword spotting, gesture recognition, low-resolution image classification (96×96 or smaller).

Limitations: Small MAC array limits throughput. Models larger than ~500K parameters may not fit in on-chip memory. No support for dynamic models or attention mechanisms.

Toolchain: ARM Compiler 6, TensorFlow Lite Micro with Ethos-U delegate, Vela model optimizer (quantization + graph transformation).

Cost: \$8–\$15 per unit.

STM32N6

What it is: STMicroelectronics' AI-focused MCU with integrated NPU.

Specs:

- CPU: Cortex-M55 at 800 MHz
- NPU: STM32 AI accelerator, claimed 3 TOPS int8
- Memory: 2.5 MB SRAM, 4 MB flash
- Power: 180 mW during inference (measured)

Best for: Vision applications on microcontrollers—defect detection, gesture recognition, small object detection.

Limitations: NPU is proprietary (not Ethos-U), so toolchain compatibility is limited to STM32Cube.AI. Models must be converted and validated specifically for this chip.

Toolchain: STM32Cube.AI (TensorFlow, PyTorch, ONNX support), STM32CubeIDE.

Cost: \$12–\$18 per unit.

Syntiant NDP120

What it is: Ultra-low-power neural decision processor designed for always-on audio AI.

Specs:

- NPU: 8-core neural processing array
- Power: <200 µW for always-on keyword detection
- Memory: 1.5 MB on-chip
- No general-purpose CPU (inference-only device)

Best for: Voice interfaces, acoustic event detection. Runs continuously on a coin cell for years.

Limitations: Audio-only. Cannot run image models. No general-purpose CPU means it must be paired with a host MCU for application logic.

Toolchain: Syntiant Core 2 (proprietary tool for model training and deployment).

Cost: \$3–\$5 per unit.

Kendryte K210

What it is: RISC-V-based AI SoC with dual-core CPU and KPU (Kendryte Processing Unit).

Specs:

- CPU: Dual-core RISC-V at 400 MHz
- KPU: Convolutional neural network accelerator (64 MACs)
- Memory: 8 MB SRAM
- Power: 300 mW during inference

Best for: Low-cost vision applications. Popular in hobbyist and educational markets.

Limitations: KPU only accelerates convolution—fully connected layers and non-standard operations run on the slow RISC-V CPU. Limited commercial-grade support.

Toolchain: Kendryte Standalone SDK, limited TensorFlow support.

Cost: \$6–\$8 per unit.

Comparison Summary

Device	NPU Type	Best Use Case	Power	Cost	Toolchain Maturity
M55 + Ethos-U55	MicroNPU	Audio, sensors	50–150 mW	\$8–15	High
STM32N6	MicroNPU	Vision on MCU	180 mW	\$12–18	Medium
Syntiant NDP120	Audio NPU	Always-on voice	0.2 mW	\$3–5	Low (proprietary)
Kendryte K210	Vision NPU	Hobbyist vision	300 mW	\$6–8	Low

The decision matrix:

- If you need always-on voice at ultra-low power → Syntiant NDP120

- If you need general-purpose inference on MCU with good tooling → Cortex-M55 + Ethos-U55
- If you need vision on MCU with higher performance → STM32N6
- If cost and hobbyist support matter more than commercial reliability → Kendryte K210

None of these is a universal solution. Each is optimized for a specific use case, and deploying on the wrong one leads to performance or cost failures.

When NOT to Use a Hardware Accelerator

Hardware accelerators are not always the right answer. There are cases where a well-optimized CPU implementation is faster, cheaper, or more flexible. Knowing when to reject acceleration is as important as knowing when to apply it.

Case 1: The model is too small to benefit.

If your model has 10,000 parameters and takes 5 ms on a Cortex-M4, adding an NPU won't help. The overhead of transferring data to the accelerator, configuring it, and transferring results back exceeds the time saved by parallel execution. Small models are already fast enough on CPUs.

Case 2: The model's operations don't map to the accelerator.

If your model uses custom layers, dynamic control flow, or sparse operations, the accelerator may only accelerate 20–30% of the workload. The rest falls back to the CPU, and the speedup is negligible. Profile the model to see which operations dominate. If most of the time is spent in unaccelerated operations, the accelerator is wasted.

Case 3: The CPU is fast enough.

If a Cortex-M7 at 480 MHz meets your latency requirement with optimized CMSIS-NN kernels, why add an NPU? The CPU solution is simpler (one chip instead of two), cheaper, and uses familiar tools. Accelerators solve problems. If you don't have a compute problem, don't add an accelerator.

Case 4: The accelerator's power consumption exceeds the CPU's.

Some accelerators consume more power than an optimized CPU implementation for small models. An NPU running at 500 MHz might draw 150 mW while a CPU running at 100 MHz draws 30 mW. If the NPU finishes in 10 ms and the CPU finishes in 40 ms, energy per inference is:

- NPU: $150 \text{ mW} \times 0.01 \text{ s} = 1.5 \text{ mJ}$
- CPU: $30 \text{ mW} \times 0.04 \text{ s} = 1.2 \text{ mJ}$

The CPU uses less energy despite being slower. For battery-powered systems with low duty cycles, the slower CPU is the better choice.

Case 5: Toolchain immaturity.

If the accelerator requires a proprietary toolchain with poor documentation, limited model support, and buggy code generation, the development cost exceeds the performance benefit. Debugging accelerator-specific failures is harder than debugging CPU code. Unless the performance gain is transformative (10× or more), stick with the CPU and standard tools.

The workflow for deciding whether to use an accelerator:

1. Profile the model on the CPU with optimized kernels (CMSIS-NN, XNNPACK).
2. Measure whether the CPU meets latency, power, and cost requirements.
3. If yes, stop. Don't add complexity.
4. If no, identify the bottleneck operation (convolution, matrix multiply, etc.).
5. Check whether an available accelerator supports that operation.
6. Estimate the speedup (not theoretical—realistic, accounting for fallback operations).
7. Compare cost, power, and development effort.
8. If the accelerator's benefits justify its costs, use it. Otherwise, optimize the model instead.

Acceleration is a tool, not a goal. The goal is to meet the application's constraints with the simplest, lowest-cost solution. Sometimes that's a CPU. Sometimes that's an NPU. Sometimes it's rejecting AI entirely and using a classical algorithm.

A Worked Example: Same Model, Three Hardware Targets

You have a 96×96 image classification model for defect detection on a manufacturing line. The model has 400,000 parameters (int8 quantized, 400 KB), 35 million MACs, and 95% accuracy. Application requirements:

- Latency: ≤50 ms (line speed requires classification before the part moves out of frame)
- Power: <500 mW average (system is mains-powered but heat-limited)
- Cost: <\$15 per unit (high-volume production, 50,000 units/year)

You evaluate three hardware options.

Option A: STM32H7 (CPU-only)

Specs:

- Cortex-M7 at 480 MHz
- 1 MB SRAM, 2 MB flash
- Optimized with CMSIS-NN

Measured Performance:

- Latency: 180 ms
- Power during inference: 400 mW
- Cost: \$6 per unit

Analysis: Fails the 50 ms latency requirement by 3.6×. Even with perfect optimization, the CPU cannot run this model fast enough.

Option B: STM32N6 (CPU + NPU)

Specs:

- Cortex-M55 at 800 MHz + STM32 AI accelerator (3 TOPS)
- 2.5 MB SRAM, 4 MB flash
- Model converted with STM32Cube.AI

Measured Performance:

- Latency: 42 ms (NPU-accelerated)
- Power during inference: 650 mW
- Cost: \$14 per unit

Analysis: Meets the 50 ms latency requirement. Exceeds the 500 mW power budget by 30%, but since the system is mains-powered and only heat-limited, 650 mW might be acceptable if thermal analysis confirms it. Cost is within budget. **Viable option.**

Option C: Raspberry Pi Compute Module 4 (Application Processor)

Specs:

- Quad-core Cortex-A72 at 1.5 GHz
- 1–4 GB RAM (overkill for this application)
- TensorFlow Lite with ARM NEON optimizations

Measured Performance:

- Latency: 8 ms (NEON-optimized)
- Power during inference: 2,500 mW
- Cost: \$35 per unit

Analysis: Easily meets the latency requirement, but power consumption is 5× over budget and cost is 2.3× over budget. The Raspberry Pi is overkill for this application. **Not viable.**

Conclusion:

Option B (STM32N6 with NPU) is the only viable solution. Option A (CPU-only) fails latency. Option C (application processor) fails power and cost. The NPU's 4× speedup over the CPU makes the difference between a product that works and one that doesn't.

But suppose the application requirement was 100 ms latency instead of 50 ms. Then Option A becomes viable—180 ms is close enough that further optimization (reducing the model to 30 million MACs, using better quantization) could bring it under 100 ms. In that case, you'd choose the \$6 CPU-only solution instead of the \$14 NPU solution, saving \$8 per unit × 50,000 units = \$400,000 in BOM cost.

This is the hardware selection decision. Constraints determine which options are viable. Cost determines which viable option you choose. Acceleration is justified only when it's the cheapest way to meet the constraints.

What Comes Next

Hardware acceleration solves local compute and power constraints. But embedded systems rarely operate in isolation. They communicate—with other devices, with edge gateways, with cloud servers. The next chapter examines the edge-cloud spectrum: when to process data locally, when to offload to the network, and how to design hybrid inference strategies that split computation across processing tiers.

Chapter 9: Communication and the Edge-Cloud Spectrum

Your model fits in memory, runs in 80 milliseconds, and consumes 12 mJ per inference. All local constraints are satisfied. But the model achieves only 87% accuracy, and your application requires 92%. A larger, more accurate model exists—2 million parameters, 150 million MACs—but it doesn't fit on your microcontroller. The device has LoRaWAN connectivity with 1 kbps uplink bandwidth and 500 ms round-trip latency to the nearest gateway.

You face a decision: accept the lower accuracy of the on-device model, or send sensor data to a cloud server that runs the larger model. If you offload to the cloud, you gain 5% accuracy but pay latency, bandwidth, energy, and privacy costs. The question is not whether the cloud is more powerful—it obviously is. The question is whether the communication cost is acceptable for your application.

This is the edge-cloud tradeoff. Embedded systems exist on a spectrum from fully local processing (all inference on-device) to fully cloud processing (device sends raw data, server does everything). Between these extremes lie hybrid strategies where computation is split between the device, an edge gateway, and the cloud. This chapter teaches you to quantify the

costs of each approach and select the right processing location for your application's constraints.

By the end of this chapter, you will be able to calculate the latency, bandwidth, power, and privacy costs of offloading inference, determine the appropriate processing tier (device, edge, cloud) for a given application, and design hybrid inference strategies that distribute computation to minimize total system cost.

The Edge-Cloud Spectrum: On-Device, Near-Edge, Far-Edge, Cloud

Processing can happen at four tiers, each with different performance, cost, and constraint profiles.

On-device (endpoint): Inference runs entirely on the embedded device—microcontroller, edge processor, or sensor node. No data leaves the device except the final result (classification label, anomaly flag, detected object). This minimizes latency, preserves privacy, and requires no network connectivity. But it's constrained by the device's memory, compute, and power budgets.

Near-edge (local gateway): Inference runs on a local gateway within the same building or facility—a Raspberry Pi, industrial PC, or edge server. The device sends raw sensor data to the gateway over WiFi, Ethernet, or serial link. The gateway has more compute power than the endpoint (multi-core CPU, GPU) and more relaxed power constraints (wall-powered). Latency is low (1–50 ms round-trip). But the gateway adds cost and requires local infrastructure.

Far-edge (regional server): Inference runs on a server at a regional datacenter or cellular base station. The device sends data over cellular, LoRaWAN, or satellite link. Latency is higher (50–500 ms round-trip) and variable. Bandwidth may be limited (1 kbps to 1 Mbps). But compute is essentially unlimited—the server can run models with billions of parameters.

Cloud (centralized datacenter): Inference runs on a cloud platform (AWS, Azure, GCP). The device sends data over the internet. Latency is highly variable (100 ms to several seconds depending on network conditions). Bandwidth costs are recurring. But the cloud offers infinite scalability, continuous model updates, and access to the latest AI models.

The tradeoff surface has four dimensions:

Latency: On-device is fastest (0 ms communication latency, only inference latency). Cloud is slowest (round-trip network latency dominates). Hard real-time applications (collision avoidance, industrial control) cannot tolerate cloud latency. Soft real-time applications (smart home, wearables) might accept it.

Privacy: On-device is most private (data never leaves the device). Cloud is least private (raw data crosses the internet and is processed by a third party). Medical devices, security cameras, and financial sensors often have regulatory requirements that preclude cloud processing.

Connectivity: On-device requires no connectivity. Cloud requires always-on internet. Far-edge requires intermittent connectivity. Applications in remote locations (agriculture, wildlife monitoring, maritime) may have no reliable network.

Cost: On-device has no recurring communication cost but limited compute. Cloud has recurring bandwidth and API costs but unlimited compute. Near-edge has upfront gateway cost but no recurring fees. The economic breakeven depends on deployment scale and lifetime.

The decision workflow is:

1. Identify the application's latency requirement. If latency < 100 ms, consider on-device or near-edge.
2. Identify privacy and regulatory constraints. If data cannot leave the device or facility, reject cloud and far-edge.
3. Identify connectivity availability. If the device has no network or intermittent connectivity, reject cloud.
4. Compare compute requirements to device capabilities. If the model fits on-device, prefer on-device. If not, evaluate offloading.
5. Calculate communication costs (latency, bandwidth, energy, recurring fees). If acceptable, offload. If not, reduce the model or upgrade the device.

There is no universal answer. A smart thermostat with WiFi, wall power, and relaxed latency can use cloud inference cheaply. A battery-powered vibration sensor in a remote oilfield with cellular connectivity and hard latency requirements cannot.

Communication Cost as a Constraint

Communication costs four resources: latency, bandwidth, energy, and money. Each must be quantified to determine whether offloading is viable.

Latency Cost

Round-trip latency is the time between sending data and receiving a result. It includes:

- Packetization and serialization (converting sensor data to a transmittable format)
- Transmission time (data size / bandwidth)
- Network propagation delay (speed-of-light limit, router hops)
- Server queueing and processing time (inference on the remote server)
- Return transmission time

For a LoRaWAN device sending 100 bytes of sensor data to a server and receiving a 10-byte result:

- Serialization: 5 ms
- LoRa uplink (SF7, 125 kHz, effective 5 kbps): $100 \text{ bytes} / (5000 \text{ bits/s} / 8) = 160 \text{ ms}$
- Network propagation (gateway to server): 50 ms
- Server inference: 20 ms (fast server, small model)
- LoRa downlink: $10 \text{ bytes} / (5000 \text{ bits/s} / 8) = 16 \text{ ms}$
- Deserialization: 2 ms

Total round-trip latency: 253 ms

If the application requires results within 100 ms, cloud inference fails. The network latency alone ($160 + 50 + 16 = 226 \text{ ms}$) exceeds the budget before inference even starts.

Bandwidth Cost

Bandwidth determines how much data can be sent per unit time. Embedded communication links are often bandwidth-constrained:

- LoRaWAN: 0.3–50 kbps (depending on spreading factor and region)
- NB-IoT: 20–250 kbps
- LTE Cat-M1: 1 Mbps (uplink and downlink)
- WiFi: 1–100 Mbps (highly variable)

Sending raw sensor data consumes bandwidth proportional to the data size. A 96×96 RGB image is 27,648 bytes. On LoRaWAN at 5 kbps, transmission takes:

$$T = 27,648 \text{ bytes} \times 8 \text{ bits/byte} / 5000 \text{ bits/s} = 44 \text{ seconds}$$

That's unusable for real-time applications. Even on WiFi at 1 Mbps, transmission takes 0.22 seconds—acceptable for some applications but not for others.

The bandwidth constraint is satisfied if:

Data size / Bandwidth < Latency budget

If your latency budget is 500 ms and your bandwidth is 10 kbps, the maximum data size is:

$$\text{Max data} = 0.5 \text{ s} \times (10,000 \text{ bits/s} / 8 \text{ bits/byte}) = 625 \text{ bytes}$$

A 96×96 image doesn't fit. A 32×32 image (3,072 bytes) doesn't fit. But a 16-element feature vector (64 bytes) does fit.

This is why edge processing often includes feature extraction on-device. Instead of sending raw images, the device computes a low-dimensional feature vector (histogram, edge map, statistical

features) and sends that. The server receives 64 bytes instead of 27,648 bytes, and bandwidth usage drops by 430×.

Energy Cost

Wireless transmission consumes energy proportional to data size and transmission power. For a LoRaWAN device transmitting at +14 dBm:

- Transmit current: 40 mA
- Transmission time: 160 ms (for 100 bytes at SF7)
- Energy per transmission: $3.3 \text{ V} \times 0.04 \text{ A} \times 0.16 \text{ s} = 21.1 \text{ mJ}$

If the device transmits every 10 minutes (144 times per day), daily communication energy is:

$$E_{\text{comm}} = 21.1 \text{ mJ} \times 144 = 3,038 \text{ mJ} = 3.04 \text{ J per day}$$

For a 2000 mAh battery at 3.3 V (6600 mWh = 23,760 J), communication energy is:

$$3.04 \text{ J} / 23,760 \text{ J} = 0.013\% \text{ of battery capacity per day}$$

Communication energy is negligible compared to sleep energy ($3.3 \text{ V} \times 0.01 \text{ mA} \times 24 \text{ hours} = 0.79 \text{ J per day}$) and inference energy (if running locally).

But if the device transmits every 10 seconds instead of every 10 minutes, transmissions per day increase to 8,640. Daily communication energy becomes 182 J—nearly 8× the total battery capacity. The battery drains in 3 hours.

The energy constraint for communication is:

$$E_{\text{comm_per_event}} \times \text{Events_per_day} < \text{Power_budget} \times 86,400 \text{ s}$$

If your power budget is 1 mW (86.4 J per day) and communication costs 21 mJ per event, the maximum sustainable event rate is:

$$\text{Max events} = 86.4 \text{ J} / 0.021 \text{ J} = 4,114 \text{ events per day} = 1 \text{ event every 21 seconds}$$

More frequent communication requires a larger battery, energy harvesting, or reduced transmission power (which reduces range).

Monetary Cost

Cloud inference has recurring costs: bandwidth (data transmission) and compute (inference API calls). Cellular connectivity has monthly data plans. Cloud API providers (AWS SageMaker, Google Vertex AI, Azure ML) charge per inference or per compute-hour.

For a cellular IoT device on an NB-IoT plan:

- Data plan: \$5/month for 10 MB
- Sensor data per transmission: 100 bytes
- Transmissions per month (every 10 minutes): 4,320
- Total data: 432 KB per month (well within 10 MB)

For cloud inference via AWS SageMaker:

- Inference cost: \$0.0001 per inference (example pricing)
- Inferences per month: 4,320
- Monthly cost: \$0.43

Total monthly cost per device: \$5.43.

For 10,000 deployed devices, annual cost is:

10,000 devices × \$5.43/month × 12 months = \$651,600 per year

That's \$651,600 in recurring operational expense. Compare to on-device inference:

- Upfront hardware cost: \$5 more per device for a faster processor
- Total upfront cost: \$50,000
- Recurring cost: \$0

The crossover point is less than one year. On-device inference has higher upfront cost but lower lifetime cost. For long-lived deployments (5–10 years), on-device is cheaper. For short-lived deployments or prototypes, cloud is cheaper.

Privacy and Regulatory Constraints on Data Transmission

Some data cannot be transmitted, regardless of cost. Medical records are governed by HIPAA in the US and GDPR in Europe. Financial transactions are governed by PCI-DSS. Industrial control data may be subject to NIST or IEC standards. Sending raw data to the cloud may violate these regulations even if the user consents.

For medical wearables, the decision tree is:

- Can the data be anonymized and aggregated? If yes, cloud processing might be allowed.
- Is the data diagnostic (used for clinical decisions)? If yes, FDA or CE Mark approval may be required, which complicates cloud deployments.
- Can the data be processed locally and only summary statistics sent to the cloud? If yes, this is the preferred architecture.

For industrial sensors, the decision tree is:

- Is the data proprietary (trade secrets, process parameters)? If yes, on-premise processing is required.
- Is the facility connected to the public internet? If no, cloud processing is physically impossible.
- Does the application have safety implications (IEC 61508)? If yes, cloud latency may violate safety requirements, and local processing is mandatory.

Privacy is not just a regulatory constraint—it's also a user trust constraint. A home security camera that sends video to the cloud faces user resistance even if technically legal. On-device processing addresses the trust gap: "your data never leaves your home."

The privacy-preserving architecture is:

1. Run inference locally.
2. Send only the result (classification label, anomaly flag) to the cloud.
3. Store raw data locally or discard it.

This satisfies most privacy regulations and user expectations. But it requires the local device to be powerful enough to run the model, which brings us back to the hardware constraint from Chapter 8.

Latency Requirements That Preclude Cloud Inference

Some applications have latency budgets that cannot be met with network communication, even on fast networks. These are hard constraints—no amount of optimization will make cloud inference viable.

Collision avoidance (autonomous vehicles, drones): Detection to action in under 50 ms. Even on a local 5G network with 10 ms round-trip latency, adding server inference time pushes total latency over 50 ms. Local processing is mandatory.

Industrial robotics (pick-and-place, quality inspection): 10–100 ms latency budgets depending on line speed. A WiFi round-trip (10–20 ms) plus server inference (10–50 ms) might fit, but network jitter can cause violations. Local processing avoids non-determinism.

Medical alarms (arrhythmia detection, seizure detection): 1–5 second budgets allow cloud processing in theory, but reliability constraints (what if the network fails?) make local processing the safer choice.

Human-computer interaction (voice assistants, AR/VR): 50–200 ms latency budgets for responsiveness. Cloud voice assistants (Alexa, Google Assistant) accept this latency. Local voice assistants (Apple Siri on-device mode) are faster but less accurate.

The latency budget determines the maximum allowable network round-trip time. If your application requires 100 ms end-to-end and inference takes 40 ms, network latency cannot exceed 60 ms. On WiFi or Ethernet, this is feasible. On cellular or LoRaWAN, it's not.

Split Inference: Running Early Layers On-Device, Later Layers on a Server

Split inference is a hybrid strategy where the model is partitioned: early layers run on the device, later layers run on a server. The device sends intermediate activations (not raw data) to the server, which completes inference and returns the result.

The advantage is reduced bandwidth and improved privacy. Intermediate activations are smaller and less interpretable than raw inputs. A 96×96 image is 27 KB, but the activations after three convolutional layers might be $24 \times 24 \times 64 = 36$ KB as float32 or 9 KB as int8. Even better, the activations don't resemble the original image—they're feature maps, not pixels—so privacy is partially preserved.

The split point is chosen based on:

- **Bandwidth constraint:** Split where activation size drops below bandwidth capacity.
- **Compute constraint:** Offload the most compute-intensive layers to the server.
- **Privacy constraint:** Run enough layers on-device to obscure the original input.

A worked example: image classification on a resource-constrained edge camera.

Model architecture (MobileNetV2):

- Input: $224 \times 224 \times 3$ (150 KB)
- After Conv1: $112 \times 112 \times 32$ (400 KB as float32, 100 KB as int8)
- After block 3: $28 \times 28 \times 96$ (75 KB as float32, 19 KB as int8)
- After block 6: $14 \times 14 \times 160$ (31 KB as float32, 8 KB as int8)
- After block 13: $7 \times 7 \times 320$ (16 KB as float32, 4 KB as int8)
- Final output: 1000 classes (4 KB as float32)

Deployment strategy:

On-device (ESP32-S3 with WiFi): Run up to block 6 (first 60% of operations, 30 MB MACs). Send 8 KB of int8 activations to the server.

On-server (cloud API): Run blocks 7–13 and classification head (remaining 40%, 20 MB MACs). Return 1000-class probability vector (4 KB).

Analysis:

- Bandwidth: 8 KB uplink, 4 KB downlink = 12 KB total. On WiFi (1 Mbps), transmission time is 96 ms.
- Latency: On-device (180 ms) + WiFi transmission (96 ms) + server inference (20 ms) + WiFi return (32 ms) = 328 ms.
- Compare to full on-device: 450 ms (ESP32 cannot run the full model efficiently). Split inference is faster.
- Compare to full cloud: Raw image transmission (150 KB / 1 Mbps = 1200 ms). Split inference is 3.7× faster.

Split inference works when:

- The device can run part of the model but not all of it.
- Intermediate activations are smaller than the raw input.
- Latency budget allows for round-trip network communication.

It fails when:

- Bandwidth is too low even for activations (e.g., LoRaWAN cannot handle 8 KB per inference).
- Network latency dominates total latency (split inference is no faster than full cloud).
- Privacy requires that no data, not even activations, leaves the device.

Federated Learning Basics: Training at the Edge Without Transmitting Raw Data

Federated learning is a training paradigm where models are trained across many devices without centralizing data. Each device trains locally on its data, computes model updates (gradients or weight deltas), and sends only the updates to a central server. The server aggregates updates from all devices and distributes the improved model back to the devices.

For embedded AI, federated learning solves two problems:

1. **Privacy:** Raw data never leaves the device. The server sees only aggregated gradients, which are less sensitive than raw inputs.
2. **Bandwidth:** Sending weight updates (a few MB) is cheaper than sending all training data (gigabytes).

But federated learning is a training-time technique, not an inference-time technique. It's relevant when:

- You have deployed devices generating data continuously.
- You want to improve the model over time using that data.
- Privacy or bandwidth constraints prevent sending raw data to a server.

Federated learning is not covered in detail in this book (which focuses on inference), but it's conceptually important because it changes the edge-cloud division of labor. Instead of "device does inference, cloud does training," federated learning enables "device does both inference and local training, cloud only aggregates."

For embedded AI designers, the takeaway is: if your deployment includes thousands of devices and you want the model to improve from field data, federate learning is an option. But it requires devices with enough compute and power to run training (not just inference), which is a much higher bar.

A Worked Example: Three Deployment Strategies for Predictive Maintenance

You are deploying a predictive maintenance system for industrial motors. The system monitors vibration data from accelerometers and predicts bearing faults 24 hours before failure. You have three models:

Model A (small): 50,000 parameters, 8 million MACs, 82% fault detection accuracy.

Model B (medium): 200,000 parameters, 40 million MACs, 91% accuracy.

Model C (large): 1 million parameters, 200 million MACs, 96% accuracy.

Your requirements:

- Accuracy: $\geq 90\%$ (Model B or C required)
- Latency: ≤ 1 second (soft constraint—maintenance alerts are not time-critical)
- Connectivity: Cellular (NB-IoT, 20 kbps, 200 ms round-trip latency)
- Privacy: Data is proprietary but not regulated—cloud processing is allowed
- Cost: $< \$10/\text{device/month}$ for communication and compute

You evaluate three deployment strategies.

Strategy 1: Full On-Device (Model B)

Hardware: STM32H7 (Cortex-M7 at 480 MHz)

Inference latency: 220 ms (measured on target)

Communication: Send only fault prediction (1 byte) once per day

Monthly data usage: 30 bytes (negligible)

Monthly cost: \$2 (minimum NB-IoT plan)

Accuracy: 91%

Analysis: Meets all constraints. Accuracy is at the minimum threshold. Latency is well within budget. Cost is low. This is the baseline solution.

Strategy 2: Full Cloud (Model C)

Hardware: STM32L4 (low-power MCU, sensor acquisition only)

Data sent: 1 kHz accelerometer data, 60-second windows = 180 KB per transmission

Transmissions per day: 24 (hourly monitoring)

Monthly data usage: $180 \text{ KB} \times 24 \times 30 = 129.6 \text{ MB}$

NB-IoT cost: \$15/month for 100 MB plan (129.6 MB exceeds 100 MB—need 200 MB plan at \$25/month)

Cloud API cost: $24 \text{ inferences/day} \times 30 \text{ days} \times \$0.001/\text{inference} = \$0.72/\text{month}$

Total monthly cost: \$25.72

Latency: 200 ms (network) + 50 ms (server inference) = 250 ms

Accuracy: 96%

Analysis: Exceeds cost budget by 2.6×. Accuracy is higher (+5% over on-device), but the cost increase is not justified for this application. Bandwidth usage is also problematic—200 MB per device $\times$ 1,000 devices = 200 GB/month of cellular data.

Strategy 3: Split Inference (Model C, first 5 layers on-device)

Hardware: STM32H7 (same as Strategy 1)

On-device: First 5 layers (30 million MACs, 120 ms latency)

Activation size after layer 5: 4 KB (int8)

Data sent: 4 KB per inference

Transmissions per day: 24

Monthly data usage: $4 \text{ KB} \times 24 \times 30 = 2.88 \text{ MB}$

NB-IoT cost: \$5/month for 10 MB plan

Cloud API cost: \$0.72/month (same as Strategy 2)

Total monthly cost: \$5.72

Latency: 120 ms (device) + 200 ms (network) + 30 ms (server) = 350 ms

Accuracy: 96%

Analysis: Meets cost budget. Accuracy matches Model C. Latency is acceptable (350 ms < 1 second budget). Bandwidth usage is 45× lower than full cloud (2.88 MB vs. 129.6 MB). **This is the optimal solution.**

Conclusion:

Split inference (Strategy 3) achieves the best balance: Model C's 96% accuracy, \$5.72/month cost (within budget), and 350 ms latency (within budget). Full cloud (Strategy 2) has the same accuracy but costs 4.5× more. Full on-device (Strategy 1) is cheaper but sacrifices 5% accuracy.

The edge-cloud decision is not binary. Hybrid strategies often outperform either extreme.

When to Reject Communication Entirely

Some applications cannot rely on network connectivity:

- **Intermittent connectivity:** Underwater sensors, remote agriculture, disaster response robots. The network is unavailable most of the time.
- **Mission-critical reliability:** Medical implants, industrial safety systems. Network failures cannot be tolerated—local processing is the only option.
- **Adversarial environments:** Military and security applications where the network may be jammed or compromised.

For these applications, on-device inference is not an optimization—it's a requirement. The model must fit on the device, even if that means accepting lower accuracy or upgrading to more expensive hardware.

What Comes Next

The edge-cloud spectrum addresses physical and network constraints. But it assumes inference succeeds in a timely, predictable manner. The next chapter examines the hardest constraint category: real-time guarantees. When inference must meet hard deadlines with provable certainty, neural networks become difficult to integrate into safety-critical systems. Chapter 10 teaches you to evaluate real-time AI and apply design patterns that reduce risk.

Chapter 10: Real-Time AI: Reliability, Safety, and Deadlines

On March 18, 2018, an autonomous test vehicle operated by Uber struck and killed a pedestrian crossing the street in Tempe, Arizona. The vehicle's perception system—a combination of LIDAR, radar, and camera sensors feeding into neural network-based object detection—identified the pedestrian 5.6 seconds before impact. But the system's decision logic classified the detection as a false positive and suppressed the braking command. By the time the system reclassified the object as a pedestrian requiring emergency braking, 1.3 seconds remained—insufficient stopping distance at the vehicle's speed.

The root cause was not a failure of the neural network to detect the pedestrian. The network detected her. The failure was that the system lacked deterministic worst-case execution time guarantees for the perception-to-action pipeline. The neural network's inference time varied depending on scene complexity, and the system's decision logic could not bound the latency. When inference ran long, the action arrived too late.

This is the real-time AI problem. Neural networks are powerful but non-deterministic. Execution time depends on input data, cache state, memory contention, and scheduling. For soft real-time applications—smart thermostats, wearables, consumer electronics—variable latency is acceptable. Miss a deadline occasionally, and the user experience degrades slightly. But for

hard real-time applications—automotive safety, industrial control, medical alarms—missing a deadline is a system failure that can cause physical harm.

This chapter teaches you to evaluate whether AI inference can be integrated into real-time systems without violating timing guarantees or safety requirements. By the end of this chapter, you will understand why determinism matters for real-time systems, identify the properties of neural networks that make worst-case execution time (WCET) analysis difficult, and apply design patterns that reduce the real-time risk of AI inference in safety-critical applications.

Hard vs. Soft Real-Time Requirements

A real-time system is one where correctness depends not just on the output but on when the output is produced. Real-time systems are classified by how they handle deadline violations.

Soft real-time: Deadlines are preferred but not mandatory. Missing a deadline degrades performance but does not cause failure. Video streaming is soft real-time—if a frame is delayed, the video stutters, but the system continues. A smart thermostat running gesture recognition is soft real-time—if inference takes 200 ms instead of 100 ms, the response feels sluggish, but the house doesn't burn down.

Firm real-time: Deadlines must be met for the result to be useful, but occasional misses are tolerated. A radar system processing aircraft returns is firm real-time—if processing completes after the next radar sweep, that data is useless, but missing one sweep doesn't crash the plane. Some fraction of deadline misses (e.g., <0.1%) is acceptable.

Hard real-time: Deadlines must be met with provable certainty. Missing a deadline is a system failure. Automotive airbag controllers are hard real-time—the deployment decision must occur within 10–20 ms of crash detection, or the airbag deploys too late to protect the occupant. Industrial safety systems that shut down machinery when faults are detected are hard real-time—late detection allows the fault to progress to catastrophic failure.

Neural network inference is inherently difficult to integrate into hard real-time systems because:

1. Execution time varies with input data.
2. Memory access patterns are irregular and cache-dependent.
3. Frameworks use dynamic memory allocation and complex control flow.
4. Worst-case execution time bounds are difficult to prove statically.

For soft real-time, these problems are manageable—you profile the model on realistic inputs, measure the 99th percentile latency, and add margin. For hard real-time, you must prove that even the worst-case input, under the worst-case cache state, with the worst-case scheduling, completes within the deadline. This is a much higher bar.

Non-Determinism in Neural Network Inference

Determinism means that executing the same code on the same input always takes the same amount of time. Non-determinism means execution time varies. Neural network inference has four sources of non-determinism:

1. Input-Dependent Execution Paths

Some neural network operations have execution time that depends on input values. Early-exit networks, for example, include branches that skip later layers if the model's confidence exceeds a threshold. An easy input (high confidence after three layers) takes 50 ms. A hard input (low confidence, must process all 10 layers) takes 200 ms.

Attention mechanisms, used in transformers and some RNNs, compute dynamic weights based on input content. The number of operations depends on which parts of the input are "attended to," which varies per input.

Even ReLU (rectified linear unit), a simple activation function, can introduce non-determinism if implemented with conditional branching:

```
for (int i = 0; i < n; i++) {  
    output[i] = (input[i] > 0) ? input[i] : 0;  
}
```

On processors with deep pipelines, the branch prediction behavior varies depending on the fraction of positive vs. negative inputs, which changes execution time by a few cycles per element.

2. Cache and Memory Effects

Modern processors use caches to speed up memory access. If data is in cache, access takes 1–2 cycles. If data is not in cache (a cache miss), access takes 10–100 cycles while the data is fetched from main memory.

Whether a memory access hits or misses in cache depends on:

- What data was accessed recently (cache is warm or cold)
- Whether other tasks or interrupts evicted data from the cache
- Memory access patterns (sequential accesses hit more often than random accesses)

For neural network inference, cache behavior is input-dependent. A convolutional layer processing a smooth image (low spatial variation) might have better cache locality than a noisy image (high spatial variation), leading to 10–30% variation in execution time.

3. RTOS Scheduling and Preemption

On systems running an RTOS, inference may be preempted by higher-priority tasks. If inference runs at normal priority and a communication event arrives, the RTOS suspends inference, handles the event, and resumes inference. The total wall-clock time is inference execution time plus preemption time.

Preemption time is non-deterministic because it depends on when interrupts arrive, how long interrupt handlers run, and whether higher-priority tasks are ready to execute. Bounding preemption time requires analyzing all tasks in the system, not just the inference task.

4. Framework Overhead

Inference frameworks (TensorFlow Lite Micro, PyTorch Mobile, ONNX Runtime) add abstraction layers—graph interpreters, dynamic dispatch, memory managers. These introduce variability:

- Dynamic memory allocation (malloc/free) has variable execution time depending on heap fragmentation.
- Virtual function calls and dynamic dispatch have variable latency depending on instruction cache state.
- Framework profiling or logging code (if enabled) adds overhead.

For safety-critical applications, frameworks with dynamic allocation and complex control flow are unsuitable. The inference engine must be statically analyzable—no dynamic allocation, no recursion, no unbounded loops.

WCET Analysis for AI: What Works, What Doesn't, and Open Problems

Worst-case execution time (WCET) analysis aims to compute the maximum time a piece of code can take to execute, under any input and any system state, without actually running it on all possible inputs (which is infeasible).

Traditional WCET analysis tools (AbsInt aiT, Rapita RVS, SWEET) work by:

1. Modeling the processor's pipeline, cache, and memory system.
2. Constructing a control-flow graph of the code.
3. Finding the longest execution path through the graph, accounting for pipeline stalls and cache misses.
4. Returning an upper bound on execution time.

This works well for simple embedded code—sensor drivers, control loops, state machines—where control flow is explicit and data-independent. It fails for neural network inference because:

Path explosion: A network with 20 layers and 10 possible execution paths per layer has 10^{20} possible paths. Enumerating all paths is computationally infeasible.

Indirect control flow: Frameworks use function pointers and dynamic dispatch. Static analysis cannot determine which function will execute without runtime information.

Complex cache behavior: Convolutional layers access memory in patterns that depend on input dimensions, stride, padding—parameters that vary per layer. Modeling cache behavior for all possible layer configurations is intractable.

Floating-point variability: IEEE 754 floating-point arithmetic is not associative. Reordering operations (which compilers do for optimization) can change results slightly, and some processors have variable-latency floating-point units depending on the operand values (denormals, NaNs).

For neural networks, static WCET analysis is an open research problem. Current tools cannot provide tight, provable bounds for general neural networks on general-purpose processors.

The pragmatic workarounds are:

Empirical WCET measurement: Run the model on thousands of inputs, measure execution time for each, and take the maximum observed time plus a safety margin (20–50%). This is not a proof, but it's practical.

Restrict the network architecture: Use only layers with data-independent execution paths. Ban early exits, dynamic attention, conditional branches. This makes static analysis feasible but limits model expressiveness.

Disable caches or use cache locking: Lock critical data (weights, activations) into cache or scratchpad memory so all memory accesses are deterministic. This sacrifices performance but makes timing predictable.

Run inference at highest priority with interrupts disabled: Eliminate preemption and interrupt latency as sources of variability. This increases latency for other tasks but makes inference timing predictable.

Use hardware accelerators with provable timing: Some NPUs (e.g., automotive-grade neural accelerators from NXP, Renesas) provide WCET guarantees for fixed-point models. The accelerator's timing is deterministic because it has no caches and no dynamic control flow.

None of these is a complete solution. Each trades off expressiveness, performance, or cost to gain determinism.

Safety-Critical AI: Functional Safety Standards and What They Require

Safety-critical systems—automotive, medical, industrial—are governed by functional safety standards that specify how systems must be designed, tested, and verified to ensure they do not cause harm.

Key standards:

- **IEC 61508:** Generic functional safety standard for electrical/electronic systems. Defines Safety Integrity Levels (SIL 1–4), where SIL 4 is the most stringent.
- **ISO 26262:** Automotive-specific adaptation of IEC 61508. Defines Automotive Safety Integrity Levels (ASIL A–D), where ASIL D is the most stringent.
- **IEC 62304:** Medical device software lifecycle standard.
- **DO-178C:** Avionics software standard.

These standards require:

1. **Hazard analysis:** Identify all possible failure modes and their consequences.
2. **Risk classification:** Classify failures by severity (catastrophic, hazardous, major, minor).
3. **Safety requirements:** Specify maximum tolerable failure rates (e.g., $<10^{-9}$ failures per hour for ASIL D).
4. **Verification and validation:** Prove that the system meets safety requirements through testing, formal methods, or certification.

Neural networks pose unique challenges for functional safety:

Blackbox behavior: Neural networks are not programmed with explicit rules. Their behavior emerges from learned weights, which are opaque. You cannot inspect a neural network and determine "this will always detect pedestrians within 5.6 seconds" the way you can inspect a threshold-based detector.

No formal specification: Traditional software has a specification (input → expected output). Neural networks have training data and statistical performance metrics (95% accuracy on validation set). But statistical performance is not a safety guarantee. A network with 95% accuracy fails 5% of the time—is that acceptable for a safety-critical application? Depends on the failure mode. Missing a pedestrian is catastrophic. Misclassifying a stop sign as a speed limit sign is hazardous.

Dataset dependency: A network's behavior is only as good as its training data. If the training set doesn't include edge cases (pedestrians wearing reflective vests at night, stop signs partially occluded by snow), the network will fail on those cases in deployment. Proving completeness of the training set is impossible.

Adversarial vulnerability: Neural networks can be fooled by adversarially crafted inputs—images or signals designed to cause misclassification. A stop sign with specific stickers can be misclassified as a speed limit sign. For safety-critical systems, adversarial robustness is a requirement, not an optional property.

Functional safety standards are being extended to address AI:

- **ISO/PAS 21448 (SOTIF):** Safety of the intended functionality—addresses the case where a system behaves as designed but the design is insufficient (e.g., a pedestrian detector that was never trained on wheelchairs).
- **UL 4600:** Standard for autonomous vehicles, includes requirements for AI validation.
- **ISO/TR 5469:** Guidance on AI in road vehicles.

But these standards are not yet mature. Certifying an AI component for ASIL D (the highest automotive safety level) is still a research problem. The current practice is to use AI in non-safety-critical roles (driver assistance, predictive maintenance diagnostics) and keep safety-critical decisions in traditional, certifiable software.

Design Patterns for Real-Time-Safe AI

If you must integrate AI into a real-time or safety-critical system, several design patterns reduce risk.

Pattern 1: AI as Advisory, Not Command

The AI model provides a recommendation, but a deterministic supervisor makes the final decision. The supervisor has hard real-time guarantees and a proven safety envelope. The AI recommendation is treated as one input among many.

Example: An industrial robot uses AI for defect detection but also has a hardware limit switch. If the AI detects a defect, it flags the part for manual inspection. If the part physically hits the limit switch (AI failed to detect a defect), the robot stops immediately.

The AI improves performance (fewer false positives, better classification), but the safety fallback is independent of AI.

Pattern 2: Bounded Execution with Rejection

The AI model runs with a hard execution time budget. If execution exceeds the budget, the inference is aborted, and the system falls back to a safe default action.

Implementation:

1. Start a hardware timer when inference begins.
2. If the timer expires before inference completes, terminate inference and return "unknown."
3. The application handles "unknown" as a safe state (e.g., brake, sound alarm, request human intervention).

This guarantees that the AI path never violates the deadline, but it introduces a new failure mode: rejected inferences. The system must be designed so that rejections are safe.

Pattern 3: Confidence Gating

The AI model outputs not just a classification but a confidence score. Only high-confidence predictions are acted upon. Low-confidence predictions are treated as "unknown" and trigger a fallback.

Example: A pedestrian detector outputs "pedestrian detected" with 98% confidence → trigger braking. The same detector outputs "pedestrian detected" with 60% confidence → don't trigger braking, but flag for human review.

This reduces the probability of catastrophic false positives (acting on a misclassified object) but increases false negatives (failing to act on a real pedestrian with low-confidence detection).

The confidence threshold is a safety parameter that must be tuned based on hazard analysis. For life-critical applications, the threshold is set conservatively high (e.g., 99.9% confidence required), which means the AI rarely acts, but when it does, it's highly reliable.

Pattern 4: Redundant Inference with Voting

Run multiple models in parallel—different architectures, different training sets—and use majority voting or consensus to make decisions. If two out of three models agree, act on the consensus. If all three disagree, reject the inference.

This improves robustness to adversarial inputs and model-specific failure modes. But it triples inference cost (latency, power, memory), so it's only viable when resources allow.

Pattern 5: Hierarchical Safety Layers

Layer safety mechanisms so that AI failure is caught by a subsequent, simpler, provably safe check.

Example: An autonomous vehicle's perception stack:

- **Layer 1 (AI):** Neural network for pedestrian detection. High accuracy, complex, non-deterministic.
- **Layer 2 (Rule-based):** Simple threshold detector on LIDAR distance. Low accuracy, but deterministic and proven safe. If Layer 1 fails to detect a pedestrian, Layer 2 still triggers braking if any object is within 5 meters.
- **Layer 3 (Hardware):** Physical bumper sensor. If Layers 1 and 2 fail, the bumper triggers an emergency stop.

Each layer is progressively simpler and more conservative. The AI optimizes for performance, but the safety guarantee comes from the fallback layers.

Failure Mode Analysis for AI in Embedded Systems

Functional safety standards require identifying all failure modes and their consequences. For AI components, failure modes include:

False negatives (misses): The model fails to detect a condition that is present. A pedestrian detector misses a pedestrian → no braking → collision. Consequence: catastrophic.

False positives (false alarms): The model detects a condition that is not present. A fault detector flags a healthy motor as faulty → unnecessary shutdown → production loss. Consequence: economic but not safety-critical.

Latency violations: The model produces the correct output but too late. A collision detector identifies an obstacle at 5 meters but inference takes 500 ms, by which time the vehicle has moved to 2 meters → insufficient braking distance. Consequence: catastrophic.

Adversarial misclassification: The model is fooled by a deliberately crafted input. A stop sign with stickers is classified as a speed limit sign → vehicle fails to stop. Consequence: catastrophic.

Data distribution shift: The model encounters inputs outside its training distribution. A pedestrian detector trained only on adults fails on children or wheelchair users → missed detection. Consequence: catastrophic.

Numerical instability: Quantization or fixed-point arithmetic introduces errors that accumulate. A regression model predicts -5% where the true value is +5% → incorrect decision. Consequence: depends on application.

For each failure mode, you must:

1. Estimate the probability (based on validation testing, field data, or theoretical analysis).
2. Classify the severity (catastrophic, hazardous, major, minor).
3. Determine whether the probability × severity is within the acceptable risk threshold.
4. If not, apply mitigations (design patterns, redundancy, human oversight).

For safety-critical systems, the acceptable failure rate is extremely low. ASIL D (automotive) requires $<10^{-8}$ failures per hour of operation. If your AI model has a 1% false negative rate, it fails 10,000× more often than allowed. You must either improve the model to 99.999999% accuracy (impossible for most tasks) or add redundancy and fallbacks so that AI failures are caught before they cause harm.

A Worked Example: Integrating AI into a Hard Real-Time Industrial Control Loop

You are integrating an anomaly detector into a hard real-time control loop for a high-speed CNC milling machine. The machine operates at 10,000 RPM. The control loop runs at 1 kHz (every 1 ms). If an anomaly is detected, the machine must stop within 5 ms to prevent tool breakage or workpiece damage.

You have a neural network trained to detect bearing faults from vibration data:

- Input: 128-sample window of 3-axis accelerometer data (384 values)
- Architecture: 1D CNN with 3 convolutional layers + 2 fully connected layers
- Parameters: 80,000 (int8 quantized, 80 KB)
- Operation count: 12 million MACs
- Measured latency: 35–55 ms (on target hardware, Cortex-M7 at 400 MHz)

Problem: The inference latency (35–55 ms) exceeds the 5 ms deadline by an order of magnitude. Direct integration is impossible.

Solution: Apply Design Patterns

Step 1: Bounded Execution with Rejection

Set a 5 ms hard timeout. If inference doesn't complete in 5 ms, abort and return "unknown." But this doesn't help—inference takes 35 ms minimum, so every inference is aborted. The AI provides no value.

Step 2: AI as Advisory, Not Command

Run inference asynchronously at a slower rate (e.g., every 50 ms, not every 1 ms). The AI provides fault predictions that update a fault probability estimate. The hard real-time control loop checks the fault probability every 1 ms. If the probability exceeds a threshold, trigger shutdown.

Implementation:

- Control loop: Runs every 1 ms. Reads fault_probability variable. If fault_probability > 0.95, trigger emergency stop.
- AI task: Runs every 50 ms. Executes inference (35–55 ms), updates fault_probability based on the result.
- Default: fault_probability = 0.0 (no fault detected).

Timing analysis:

- Control loop deadline: 1 ms. Loop execution time (reading a variable and comparing to a threshold): <10 μ s. Meets deadline with 100 $\times$ margin.

- AI task deadline: None (advisory role). Inference can take 55 ms without violating any hard deadline.

Safety analysis:

- If AI fails to detect a fault, `fault_probability` stays low, and the machine continues running. This is a false negative, but the consequence is deferred (the fault progresses until detected by other means or until failure). Not ideal, but not immediately catastrophic.
- If AI falsely detects a fault, `fault_probability` goes high, and the machine stops. This is a false positive, causing production loss but not safety harm.

Failure mode: AI never updates `fault_probability` (inference crashes or hangs). Fallback: Add a watchdog timer that resets `fault_probability` to 0.0 if the AI task doesn't update it within 100 ms. This ensures that a stuck AI task doesn't cause a spurious shutdown.

Step 3: Hierarchical Safety Layers

Add a simple, deterministic backup fault detector that runs in the control loop.

- **Layer 1 (AI):** Neural network running every 50 ms, updates `fault_probability`.
- **Layer 2 (Rule-based):** RMS vibration threshold detector running every 1 ms. If RMS exceeds a conservative threshold, set `fault_flag` = true immediately.
- **Layer 3 (Hardware):** Current sensor on the motor. If current exceeds safe limits (indicating mechanical seizure), hardware trips the power supply.

Control loop logic:

```
if (fault_flag || fault_probability > 0.95) {
    trigger_emergency_stop();
}
```

Now the system has defense in depth:

- AI (Layer 1) provides early warning of incipient faults with 35–55 ms latency.
- Rule-based detector (Layer 2) provides conservative, deterministic fault detection with <1 ms latency.
- Hardware protection (Layer 3) is the ultimate safety net if both software layers fail.

The AI improves system performance (earlier fault detection, fewer false positives than the rule-based detector) without being in the critical path for safety.

Result:

The AI is successfully integrated into a hard real-time system by removing it from the hard real-time path. The control loop meets its 1 ms deadline. The AI provides value by improving

fault detection accuracy. Safety is guaranteed by deterministic fallbacks that do not depend on AI.

This is the pattern for integrating non-deterministic AI into deterministic real-time systems: isolate the AI, make it advisory, and layer safety mechanisms so that AI failure is tolerable.

When AI Cannot Be Integrated into Hard Real-Time Systems

Some applications cannot tolerate the non-determinism of neural networks, and no design pattern fixes it. These applications require provable, certifiable determinism:

- **Nuclear reactor control:** Failure consequences are catastrophic and irreversible. Control logic must be formally verified, which neural networks cannot be.
- **Aircraft flight control:** Hard real-time deadlines (milliseconds), provable WCET required for certification (DO-178C), and failure modes are unacceptable. AI is used in avionics for optimization and monitoring but not for primary flight control.
- **Medical device critical functions:** Pacemakers, defibrillators, insulin pumps—primary control logic must be certifiable under IEC 62304. AI is used for diagnostics and trend analysis but not for life-critical actuation.

For these applications, AI is limited to non-critical roles:

- **Diagnostics and monitoring:** AI flags anomalies for human review but doesn't take automated action.
- **Optimization:** AI tunes parameters for efficiency but operates within safety bounds enforced by deterministic logic.
- **Post-processing:** AI analyzes logged data after the fact to improve future designs but doesn't affect real-time operation.

The hard boundary is this: if you cannot prove that the AI component meets WCET and reliability requirements for the applicable safety standard, you cannot deploy it in a safety-critical role. Use AI where it adds value without adding risk.

What Comes Next

You have now completed Part II—Inference-First Evaluation. You can audit any proposed AI deployment against all major constraint dimensions: memory (Chapter 5), compute (Chapter 6), power (Chapter 7), hardware acceleration (Chapter 8), communication (Chapter 9), and real-time guarantees (Chapter 10).

You can evaluate whether a model fits, whether it runs fast enough, whether it drains the battery, whether it needs an accelerator, whether it should offload to the cloud, and whether it can meet hard deadlines. You can identify every constraint failure before deployment.

But all of this assumes the model is given. You evaluate feasibility, not optimality. Part III—Model-Aware Design—extends the toolkit into model selection and optimization. The model is no longer fixed. It becomes a design variable. Chapter 11 teaches you to select models for deployment constraints, not just for accuracy benchmarks.

Chapter 11: Selecting Models for Constrained Deployment

You have evaluated a MobileNetV2 model for on-device image classification. The model achieves 89% accuracy on your validation set, fits in 1.2 MB of flash, requires 180 KB of activation memory, and runs in 220 milliseconds on your target hardware. All constraints are satisfied—barely. Flash usage is 60% of capacity, SRAM usage is 70%, and latency is 20 milliseconds under the 240 millisecond deadline.

Then your colleague sends you a different model: EfficientNet-Lite0. It achieves 91% accuracy on the same validation set, occupies 1.8 MB of flash, requires 240 KB of activation memory, and runs in 280 milliseconds. It's more accurate, but it violates the flash constraint, the SRAM constraint, and the latency constraint.

You have a third option: a custom depthwise separable CNN designed specifically for your dataset. It achieves 87% accuracy, occupies 600 KB of flash, requires 90 KB of activation memory, and runs in 110 milliseconds. It's less accurate than both alternatives but comfortably meets all constraints with margin.

Which model do you choose? The answer depends on which constraints are binding and which accuracy threshold is mandatory. If accuracy must be $\geq 88\%$, the custom model fails. If flash must be ≤ 1.5 MB, EfficientNet-Lite0 fails. If all constraints are hard ceilings, only MobileNetV2 survives—but it has no margin for future feature additions or model updates.

This is the model selection problem. Until now, the model was given, and you evaluated feasibility. In Part III, the model is chosen. Model selection is the first step in model-aware design: evaluating candidates not on ImageNet accuracy but on deployment metrics—latency on target hardware, memory footprint, energy per inference, and accuracy under quantization.

This chapter teaches you to compare models on constraint profiles, use deployment-aware benchmarks instead of generic accuracy benchmarks, understand what makes efficient architectures efficient, and navigate the accuracy-efficiency Pareto frontier to select the model that best fits your application. By the end of this chapter, you will be able to justify a model

selection against hardware constraints and explain when a less accurate model is the right choice.

Why Accuracy Benchmarks Are the Wrong Metric for Embedded Selection

The machine learning research community evaluates models on standardized benchmarks: ImageNet top-1 accuracy for image classification, COCO mAP (mean average precision) for object detection, WER (word error rate) for speech recognition. Papers are published, models are ranked, and the highest-accuracy model "wins."

But benchmark accuracy is a poor proxy for embedded suitability. A model with 95% ImageNet accuracy and 80 million parameters is useless on a microcontroller with 1 MB of flash. A model with 90% accuracy and 5 million parameters might deploy successfully.

Benchmark datasets are also mismatched to deployment conditions. ImageNet contains 1.2 million high-resolution images of 1,000 object categories, photographed in diverse settings with professional equipment. Your deployment might involve 96×96 grayscale images of 10 categories, captured by a low-cost camera in a factory. Training on ImageNet and deploying on factory images introduces a domain gap that degrades accuracy by 5–20%.

The relevant metrics for embedded model selection are:

Latency on target hardware: Not FLOPs (which predict CPU latency poorly) or ImageNet inference time (measured on a GPU), but measured milliseconds on your actual embedded processor running your actual inference framework.

Memory footprint: Not parameter count alone but the sum of weight memory (flash) and activation memory (SRAM) for your target's quantization precision (int8, not float32).

Energy per inference: Not theoretical operations but measured millijoules on target hardware under realistic duty cycles.

Accuracy on your dataset: Not ImageNet accuracy but validation accuracy on data that matches your deployment distribution. A model with 95% ImageNet accuracy might have 70% accuracy on your application-specific dataset if the domain gap is large.

Accuracy under quantization: Many models that tolerate int8 quantization with <1% accuracy loss on ImageNet degrade by 10–20% on smaller, domain-specific datasets where quantization noise is more disruptive.

The model selection workflow is:

1. Define deployment constraints (target hardware, latency budget, memory budget, power budget, accuracy threshold).

2. Identify candidate models that might fit (based on architecture surveys, prior work, model zoos).
3. For each candidate, measure latency, memory, energy, and accuracy on target hardware with target data.
4. Eliminate candidates that violate hard constraints.
5. Among remaining candidates, select the one that best balances accuracy, margin, and future extensibility.

This is empirical and iterative. You cannot determine the best model from a datasheet or a paper. You must profile each candidate on your hardware with your data.

Deployment-Aware Metrics

Deployment-aware metrics quantify what matters for embedded integration. Here are the five core metrics and how to measure them.

1. Latency on Target Hardware

Measure end-to-end inference time on the actual deployment target, not a development board with different clock speed or memory configuration. Use the actual inference framework (TensorFlow Lite Micro, not TensorFlow Lite for desktop). Include all preprocessing (normalization, resizing) and postprocessing (softmax, NMS) if they're part of the deployed pipeline.

Run inference 100+ times on diverse inputs. Report mean, 95th percentile, and maximum observed latency. The maximum is your empirical WCET estimate.

Example measurement:

- Model: MobileNetV2-0.35 (scaled down)
- Target: STM32H7 at 480 MHz, CMSIS-NN kernels
- Input: 96×96×3 RGB images
- Measured latency: mean 185 ms, 95th percentile 192 ms, max 198 ms

2. Memory Footprint (Flash + SRAM)

Measure actual deployment memory usage, not theoretical:

- Flash: Compile the model into the firmware and check the linker map file for total flash usage (firmware + model weights). Subtract baseline firmware size to isolate model contribution.
- SRAM: Measure tensor arena size (reported by TFLite Micro's interpreter) plus any additional buffers for preprocessing or postprocessing.

Example measurement:

- Model: MobileNetV2-0.35, int8 quantized
- Flash (weights): 420 KB
- SRAM (activations): 140 KB
- Total memory: 560 KB

3. Energy per Inference

Measure on target hardware with a current sense resistor and oscilloscope or power profiler. Log current draw over the full inference cycle (preprocessing, inference, postprocessing). Integrate to get total energy.

$$\text{Energy (J)} = V \times \int I(t) dt$$

For battery-powered systems, also measure average power over realistic duty cycles (e.g., inference every 10 seconds, sleep between).

Example measurement:

- Model: MobileNetV2-0.35
- Target: nRF52840 at 64 MHz
- Supply voltage: 3.3 V
- Active current during inference: 45 mA
- Inference time: 350 ms
- Energy per inference: $3.3 \text{ V} \times 0.045 \text{ A} \times 0.35 \text{ s} = 0.052 \text{ J} = 52 \text{ mJ}$

4. Accuracy on Deployment Dataset

Train or fine-tune the model on data that matches the deployment distribution. If your application is defect detection on stamped metal parts, train on images of stamped metal parts—not ImageNet, not COCO. If you must use a pretrained model, fine-tune the final layers on your dataset.

Evaluate accuracy on a held-out validation set that matches deployment conditions: same lighting, same camera, same resolution, same noise characteristics.

Example measurement:

- Model: MobileNetV2-0.35, fine-tuned on factory defect dataset (5,000 images, 3 classes)
- Validation accuracy (float32): 91.2%
- Validation accuracy (int8 post-training quantization): 89.8%
- Accuracy drop due to quantization: 1.4%

5. Accuracy Under Quantization

Never assume quantization is free. Measure accuracy before and after quantization on your validation set. If the drop exceeds 3–5%, investigate:

- Is the calibration dataset representative? (PTQ needs diverse inputs to estimate activation ranges)
- Are there layers with very small weights? (Quantization can destroy near-zero weights)
- Are there layers with large dynamic range? (Activations that vary from 0.001 to 100 are hard to quantize to int8)

If PTQ accuracy is unacceptable, try quantization-aware training (QAT) or mixed precision (some layers int8, some float32).

Efficient Architecture Families: What Made Them Efficient

Not all neural network architectures are equally suitable for embedded deployment. Some architectures were designed explicitly for efficiency—to maximize accuracy per FLOP, per parameter, or per watt. Understanding what makes these architectures efficient helps you select or design models for constrained targets.

MobileNet (v1, v2, v3)

Key innovation: Depthwise separable convolution replaces standard convolution. A standard 3×3 convolution with C input channels and C output channels requires $9C^2$ multiplications per output pixel. A depthwise separable convolution splits this into:

- Depthwise: $9C$ multiplications (filter each channel independently)
- Pointwise: C^2 multiplications (1×1 convolution to mix channels)
- Total: $9C + C^2 \approx C^2$ for large C (factor of $9\times$ reduction)

Why it's efficient: Fewer operations, smaller models, lower latency. MobileNetV2 adds inverted residual blocks and linear bottlenecks to improve accuracy without increasing cost. MobileNetV3 uses neural architecture search to optimize for mobile CPUs and includes Squeeze-and-Excitation blocks for better feature representation.

Embedded suitability: Excellent. MobileNets scale with a width multiplier (0.25, 0.35, 0.5, 0.75, 1.0) that reduces channel count, and a resolution multiplier that reduces input size. You can shrink a MobileNet to fit almost any memory or latency budget at the cost of accuracy.

EfficientNet-Lite

Key innovation: Compound scaling—instead of scaling depth (more layers), width (more channels), or resolution (larger images) independently, EfficientNet scales all three together using a formula derived from neural architecture search.

Why it's efficient: Balances model capacity across dimensions. Adding layers without adding width creates a bottleneck. Adding width without depth limits expressiveness. EfficientNet finds the optimal ratio.

EfficientNet-Lite is a simplified version of EfficientNet designed for edge deployment: removes Squeeze-and-Excitation blocks (which are expensive on CPUs) and uses ReLU6 instead of Swish (simpler activation).

Embedded suitability: Good for edge processors (Raspberry Pi, Jetson Nano) but marginal for microcontrollers. Even EfficientNet-Lite0 (the smallest variant) has 4.6 million parameters and 0.4 GFLOPS, which is too large for most MCUs.

SqueezeNet

Key innovation: Fire modules that "squeeze" channels with 1×1 convolutions (reducing feature map size), then "expand" with a mix of 1×1 and 3×3 convolutions. This reduces parameters dramatically while maintaining accuracy.

Why it's efficient: Fewer parameters (SqueezeNet achieves AlexNet-level accuracy with 50× fewer parameters). But operation count is not proportionally reduced, so latency improvements are smaller than parameter reductions suggest.

Embedded suitability: Good for memory-constrained MCUs but not compute-constrained ones. SqueezeNet trades parameters for operations, which is useful when flash is the bottleneck but not when CPU throughput is.

YOLO-Nano and Tiny-YOLO

Key innovation: Stripped-down versions of the YOLO (You Only Look Once) object detection family, designed for embedded vision. Tiny-YOLO has 9 convolutional layers instead of 50+, uses smaller feature maps, and reduces anchor box count.

Why it's efficient: Object detection is inherently expensive (must classify and localize multiple objects per frame). Tiny-YOLO makes aggressive tradeoffs—lower resolution (416×416 instead of 608×608), fewer layers, fewer anchor boxes—to enable real-time inference on edge devices.

Embedded suitability: Marginal for MCUs, good for edge processors. Tiny-YOLOv3 still requires ~15 million parameters and 5.5 GFLOPS, which exceeds most MCU budgets but runs at 20–30 FPS on a Raspberry Pi 4.

Decision Trees and Random Forests (for structured data)

Key innovation: Not neural networks—tree-based models that make sequential decisions based on feature thresholds. Inference is a walk from root to leaf, requiring one comparison per tree level.

Why it's efficient: No matrix multiplications, no floating-point arithmetic (if thresholds are quantized to integers), and deterministic execution paths. A decision tree with 10 levels requires 10 comparisons per inference—trivial compute cost.

Embedded suitability: Excellent for structured data (sensor features, tabular data). Terrible for raw high-dimensional data (images, audio) where feature extraction requires convolutions.

When to use: If you can engineer good features from sensor data (statistical moments, frequency-domain features, correlation coefficients), decision trees outperform neural networks on small datasets and constrained hardware.

Task-Specific Model Selection

Different tasks have different requirements. The optimal architecture for image classification is not optimal for keyword spotting or anomaly detection.

Image Classification (96×96 or smaller)

Candidate architectures: MobileNetV2 (0.35 or 0.5 width multiplier), MobileNetV3-Small, SqueezeNet, custom depthwise separable CNN.

Selection criteria: Latency dominates. Memory is secondary (models are small at this resolution). Accuracy depends on dataset complexity (simple datasets: 3 classes, custom CNN suffices; complex datasets: 10+ classes, MobileNet required).

Baseline model: MobileNetV2-0.35 with 96×96 input. ~400K parameters, 25M MACs, 88–92% accuracy on typical embedded vision tasks.

Keyword Spotting (Audio)

Candidate architectures: Depthwise separable CNN on spectrograms (DS-CNN), 1D CNN on raw audio, small LSTM, decision tree on engineered features.

Selection criteria: SRAM is the bottleneck (RNNs have large state). Prefer feedforward architectures. If features are engineered (MFCCs, zero-crossing rate), decision trees may outperform neural networks.

Baseline model: DS-CNN with 12 layers, ~100K parameters, <5 ms latency on Cortex-M4, 95%+ accuracy on Google Speech Commands dataset.

Time-Series Anomaly Detection

Candidate architectures: Decision trees on statistical features, 1D CNN on raw sequences, autoencoder (if SRAM allows), LSTM (if SRAM and compute allow).

Selection criteria: Depends on whether you have labeled anomalies (supervised) or only normal data (unsupervised). Supervised: decision trees or 1D CNN. Unsupervised: autoencoder (trains on normal data, flags high reconstruction error as anomaly).

Baseline model: Random forest with 50 trees on 20 engineered features (mean, variance, FFT peaks, autocorrelation). ~200 KB model size, <1 ms inference, 85–90% anomaly detection rate.

Gesture Recognition (Accelerometer)

Candidate architectures: 1D CNN, decision tree on features, small LSTM.

Selection criteria: Depends on gesture complexity. Simple gestures (tap, shake, rotate): decision tree. Complex temporal patterns (drawing shapes in the air): 1D CNN or LSTM.

Baseline model: 1D CNN with 4 convolutional layers, 64K parameters, 8M MACs, 90%+ accuracy on typical gesture datasets.

The Accuracy-Efficiency Pareto Frontier

For any task, there exists a set of models that are Pareto-optimal: no other model is both more accurate and more efficient. The Pareto frontier is the boundary of this set, where improving accuracy requires sacrificing efficiency (latency, memory, or energy).

Plotting the Pareto frontier helps visualize the tradeoff space. On a graph with accuracy (y-axis) and latency (x-axis), Pareto-optimal models lie on the upper-left frontier. Models below and to the right are dominated—there's a better model that's both faster and more accurate.

Example: Face detection models evaluated on a custom dataset

Model	Latency (ms)	Accuracy (%)	Memory (KB)
TinyFace (custom)	45	84	120
MobileNetV2-0.25	80	88	250
MobileNetV2-0.35	110	91	420
MobileNetV3-Small	95	90	380
EfficientNet-Lite0	180	94	1,800
ResNet18	450	96	11,000

Plotting accuracy vs. latency:

- TinyFace: 45 ms, 84% → Pareto-optimal (fastest, acceptable accuracy)
- MobileNetV2-0.25: 80 ms, 88% → dominated by MobileNetV3-Small (95 ms, 90% is better)
- MobileNetV3-Small: 95 ms, 90% → Pareto-optimal
- MobileNetV2-0.35: 110 ms, 91% → Pareto-optimal (slightly slower but more accurate than MobileNetV3-Small)
- EfficientNet-Lite0: 180 ms, 94% → Pareto-optimal (much slower but more accurate)
- ResNet18: 450 ms, 96% → dominated by EfficientNet-Lite0 (180 ms, 94% is nearly as accurate and 2.5× faster)

The Pareto frontier consists of TinyFace, MobileNetV3-Small, MobileNetV2-0.35, and EfficientNet-Lite0. Which model you choose depends on your constraints:

- If latency < 50 ms is mandatory → TinyFace (only option)
- If latency < 100 ms and accuracy $\geq 90\%$ → MobileNetV3-Small
- If latency < 120 ms and accuracy $\geq 91\%$ → MobileNetV2-0.35
- If latency < 200 ms and accuracy $\geq 94\%$ → EfficientNet-Lite0

The key insight is that accuracy and efficiency are not independent. You cannot have the highest accuracy and the lowest latency. You must choose a point on the Pareto frontier based on which constraint is binding.

Neural Architecture Search (NAS) for Embedded Targets

Neural Architecture Search (NAS) automates the process of finding efficient architectures. Instead of hand-designing a network, you specify:

- A search space (possible layers, layer configurations, connections)
- An objective function (e.g., maximize accuracy subject to latency < 100 ms on target hardware)
- A search algorithm (reinforcement learning, evolutionary algorithms, gradient-based)

NAS explores the search space, evaluates candidates on the objective function, and converges to an architecture optimized for your constraints.

Examples of NAS-generated models:

- **MobileNetV3:** Used NAS to optimize for mobile CPUs. The search algorithm tuned layer widths, kernel sizes, and activation functions to maximize accuracy per FLOP.
- **EfficientNet:** Used NAS to discover the compound scaling rule (how to scale depth, width, and resolution together).
- **ProxylessNAS:** Designed to minimize latency on mobile phones, running the search directly on target hardware (phones, not GPUs).

- **Once-for-All (OFA):** Trains a single "super-network" that can be sliced into sub-networks of different sizes, allowing you to extract models optimized for specific latency or memory budgets without retraining.

When to use NAS:

- High-volume products where a 1% accuracy improvement or 10% latency reduction justifies the search cost (GPU-days or GPU-weeks).
- Tasks where hand-designed architectures are suboptimal (e.g., new sensor modalities, unusual input dimensions).
- Applications with unusual constraints (e.g., latency < 20 ms on a specific NPU).

When NOT to use NAS:

- Low-volume or one-off deployments (the search cost exceeds the value of the optimized model).
- When a well-known architecture (MobileNet, EfficientNet) already meets constraints.
- When you lack the compute resources to run NAS (requires GPU access and ML expertise).

For most embedded AI projects, using an existing efficient architecture (MobileNet, SqueezeNet) scaled to your constraints is more practical than running NAS. NAS is a research tool that occasionally produces better models than human designers, but it's not necessary for successful deployment.

A Worked Example: Selecting a Model for Wearable Fall Detection

You are designing a wearable fall detection system. The device uses a 3-axis accelerometer sampled at 100 Hz. Falls must be detected within 500 ms of occurrence to trigger an alert. The device runs on a coin cell battery that must last 6 months.

Constraints:

- Target hardware: nRF52840 (Cortex-M4 at 64 MHz, 256 KB SRAM, 1 MB flash)
- Latency: ≤ 500 ms from sample window complete to decision
- Power: < 2 mA average (6 months on 220 mAh coin cell)
- Accuracy: $\geq 95\%$ fall detection rate, $\leq 1\%$ false positive rate

You evaluate five candidate models:

Model A: Threshold Detector (Classical Algorithm)

Description: Hand-tuned threshold on peak acceleration and impact duration. No machine learning.

Performance:

- Latency: <1 ms
- Memory: 1 KB (code only, no model)
- Power: negligible (simple comparisons)
- Accuracy: 78% fall detection, 8% false positives

Analysis: Meets latency and power constraints but fails accuracy requirement by a wide margin. False positive rate is unacceptable (8 false alarms per 100 normal movements).

Model B: Decision Tree on Engineered Features

Description: Random forest (50 trees) trained on 15 features extracted from 1-second accelerometer windows (mean, variance, peak, zero-crossing rate, FFT dominant frequency, etc.).

Performance:

- Latency: 3 ms (feature extraction) + 0.5 ms (tree traversal) = 3.5 ms
- Memory: 200 KB (tree structure)
- Power: 2 ms active at 15 mA = 0.03 mJ per inference
- Accuracy: 92% fall detection, 2% false positives

Analysis: Meets latency and power constraints. Memory is tight (200 KB / 256 KB SRAM leaves only 56 KB for firmware). Accuracy is below the 95% threshold.

Model C: 1D CNN (Small)

Description: 4 convolutional layers + 2 fully connected layers. Input: 100 samples × 3 axes (300 values). Parameters: 45,000 (int8).

Performance:

- Latency: 25 ms (measured on nRF52840 with CMSIS-NN)
- Memory: 45 KB (weights) + 30 KB (activations) = 75 KB
- Power: 25 ms active at 15 mA = 0.38 mJ per inference
- Accuracy: 96% fall detection, 0.8% false positives

Analysis: Meets all constraints. Latency well within 500 ms budget. Memory comfortable (75 KB / 256 KB SRAM). Power acceptable (if inference runs every 1 second, average power is $0.38 \text{ mJ} / 1 \text{ s} = 0.38 \text{ mW} = 0.12 \text{ mA}$, well below 2 mA budget). Accuracy exceeds threshold.

This is a viable option.

Model D: 1D CNN (Large)

Description: 6 convolutional layers + 3 fully connected layers. Parameters: 180,000 (int8).

Performance:

- Latency: 85 ms
- Memory: 180 KB (weights) + 80 KB (activations) = 260 KB
- Power: 85 ms active at 15 mA = 1.28 mJ per inference
- Accuracy: 98% fall detection, 0.3% false positives

Analysis: Fails SRAM constraint (260 KB > 256 KB). Even if you added external PSRAM, the power cost increases due to external memory access. **Not viable.**

Model E: LSTM

Description: 2-layer LSTM (32 units per layer) + 1 fully connected layer. Parameters: 60,000 (int8).

Performance:

- Latency: 120 ms
- Memory: 60 KB (weights) + 90 KB (activations + state) = 150 KB
- Power: 120 ms active at 15 mA = 1.8 mJ per inference
- Accuracy: 97.5% fall detection, 0.5% false positives

Analysis: Meets all constraints. Slightly better accuracy than Model C but 5× slower and 2.4× higher power consumption. **Viable but Model C is more efficient.**

Comparison Summary

Model	Latency	Memory	Accuracy (Fall / FP)	Meets Constraints?
A: Threshold	1 ms	1 KB	78% / 8%	No (accuracy)
B: Random Forest	3.5 ms	200 KB	92% / 2%	No (accuracy)
C: 1D CNN (Small)	25 ms	75 KB	96% / 0.8%	Yes
D: 1D CNN (Large)	85 ms	260 KB	98% / 0.3%	No (memory)
E: LSTM	120 ms	150 KB	97.5% / 0.5%	Yes (but slower)

Decision: Select Model C (1D CNN Small). It meets all constraints with margin, uses 29% of SRAM (leaving 71% for firmware), and achieves 96% fall detection accuracy—above the 95%

threshold. Model E is viable but unnecessarily slow and power-hungry. Model D is not viable due to memory constraints.

The selection process required profiling all five candidates on target hardware with the actual dataset. Model D looked promising on paper (highest accuracy) but failed in practice (exceeded SRAM). Model A worked on previous-generation devices (simpler hardware, lower expectations) but no longer meets modern accuracy standards.

This is model-aware design. You didn't just evaluate whether the first model fit. You compared multiple candidates, measured their deployment metrics, and selected the one that best balanced accuracy, constraints, and margin.

What Comes Next

You can now select models for deployment constraints. But the model you select may not fit perfectly—it might be 10% too large, 20% too slow, or 3% below the accuracy threshold. Chapter 12 teaches you to modify models through quantization, pruning, and knowledge distillation to make them fit when selection alone is insufficient.

Chapter 12: Optimizing Models: Quantization, Pruning, and Distillation

You have selected a model: MobileNetV2-0.5 achieves 89% accuracy on your validation set and was the best candidate from your evaluation. But it doesn't quite fit. The model requires 1.8 MB of flash (your target has 2 MB, but the firmware already uses 600 KB, leaving only 1.4 MB available). Activation memory is 220 KB (your target has 256 KB SRAM, but the firmware uses 80 KB, leaving 176 KB for inference). Latency is 280 milliseconds (your budget is 250 milliseconds).

Selection alone is insufficient. The model is close to viable, but not viable. You need to compress it—reduce its size, reduce its memory footprint, reduce its operation count—without destroying accuracy.

This is where model optimization comes in. Optimization techniques transform a trained model into a smaller, faster, or more efficient version that fits within deployment constraints while preserving as much accuracy as possible. The three primary techniques are quantization (reducing numeric precision), pruning (removing unnecessary weights or layers), and knowledge distillation (training a small model to mimic a large model).

This chapter teaches you to apply post-training quantization and quantization-aware training, understand the tradeoffs between different pruning strategies, use knowledge distillation to compress models beyond what pruning achieves, combine techniques in the right order, and

determine the accuracy floor—when further compression breaks the application. By the end of this chapter, you will be able to take a model that barely fails constraints and transform it into a model that comfortably meets them.

Quantization: float32 → int8, What Changes, What Breaks

Quantization reduces numeric precision from floating-point (float32, float16) to fixed-point or integer (int16, int8). The primary motivation for embedded deployment is memory reduction and compute acceleration.

A float32 value occupies 4 bytes. An int8 value occupies 1 byte. Quantizing a 1 million-parameter model from float32 to int8 reduces its size from 4 MB to 1 MB—a 75% reduction. On processors with SIMD extensions, int8 operations execute 4–8× faster than float32 operations because multiple int8 values pack into a single register.

But quantization introduces numerical errors. Floating-point numbers can represent a wide dynamic range ($\pm 10^{-38}$ to $\pm 10^{38}$) with high precision. Integers are limited to a small range (int8: -128 to 127). Mapping a float32 weight of 0.0342 to int8 requires rounding, which introduces error. When that error propagates through millions of operations, accuracy can degrade.

How Quantization Works

Quantization maps floating-point values to integers using a linear transformation:

quantized_value = round((float_value - zero_point) / scale)

where `scale` and `zero_point` are parameters computed from the float32 value distribution. The inverse operation (dequantization) recovers an approximation of the original float:

float_value_approx = (quantized_value × scale) + zero_point

For example, if weights range from -1.5 to 1.5, you set:

- `scale = (max - min) / 255 = 3.0 / 255 = 0.0118`
- `zero_point = -128`

A weight of 0.5 quantizes to:

- `q = round((0.5 - 0) / 0.0118) = round(42.4) = 42`

Dequantizing gives:

- `f = 42 × 0.0118 + 0 = 0.495`

The error is 0.005, which is small. But if you quantize an activation with range [0.001, 0.01], the scale becomes $0.01 / 255 = 0.000039$, and small values near 0.001 are heavily rounded. This is where quantization damages accuracy.

Per-Tensor vs. Per-Channel Quantization

Per-tensor quantization uses a single scale and zero_point for an entire tensor (e.g., all weights in a layer). This is simple but suboptimal when weights or activations have different ranges across channels.

Per-channel quantization uses different scale and zero_point for each output channel. This better preserves accuracy when channels have imbalanced ranges (some channels have weights in [-0.1, 0.1], others in [-2.0, 2.0]).

Per-channel quantization is more accurate but requires storing N scale factors (one per channel) instead of one, which increases model size slightly. For embedded deployment, the accuracy gain usually justifies the cost.

Symmetric vs. Asymmetric Quantization

Symmetric quantization assumes the float32 range is symmetric around zero: $[-\alpha, \alpha]$. It sets `zero_point = 0` and `scale =  $2\alpha / 255$` . This simplifies math (no zero_point offset) but wastes representational range if the distribution is skewed.

Asymmetric quantization allows arbitrary ranges [min, max]. It uses both scale and zero_point, which requires more computation but better represents distributions that are not centered on zero (e.g., ReLU activations, which are always ≥ 0).

For embedded inference, symmetric quantization is preferred when hardware supports it (some accelerators optimize for zero_point = 0), and asymmetric otherwise.

Post-Training Quantization vs. Quantization-Aware Training

There are two paths to quantize a model: post-training quantization (PTQ) and quantization-aware training (QAT).

Post-Training Quantization (PTQ)

PTQ takes a trained float32 model and quantizes it to int8 without retraining. The process:

1. Run the model on a calibration dataset (100–1000 representative inputs).
2. Record the range of weights and activations in each layer.
3. Compute scale and zero_point for each tensor.

4. Convert weights to int8 and store them in the model file.
5. During inference, activations are quantized on-the-fly as they're computed.

PTQ is fast (no retraining required) and works well for many models. But it can degrade accuracy by 3–10% if:

- Weights or activations have extreme outliers (one value much larger than others).
- The calibration dataset is not representative.
- Layers have very small weights (quantization rounds them to zero).

Quantization-Aware Training (QAT)

QAT simulates quantization during training. The model learns weights that remain accurate even after quantization. The process:

1. Insert fake quantization nodes into the model graph.
2. During forward pass, weights and activations are quantized to int8, then dequantized back to float32 for gradient computation.
3. Train the model normally. The optimizer adjusts weights to minimize loss under quantization.
4. After training, convert the model to true int8 (remove fake quantization nodes and store int8 weights).

QAT typically achieves 1–3% accuracy loss (compared to 3–10% for PTQ) but requires access to the training pipeline and dataset, which you may not have if the model came from a third party.

When to Use Each

Use PTQ when:

- You don't have access to the training code or dataset.
- The model already has acceptable accuracy, and 3–5% loss is tolerable.
- You need a quick deployment (PTQ takes minutes, QAT takes hours or days).

Use QAT when:

- Accuracy is critical, and you cannot tolerate 5%+ loss.
- You control the training pipeline.
- The model has many layers or complex operations that are sensitive to quantization.

Pruning: Structured vs. Unstructured, and Why Structured Matters for Embedded

Pruning removes weights from the model to reduce parameter count. There are two types: unstructured pruning and structured pruning.

Unstructured Pruning

Unstructured pruning sets individual weights to zero based on a criterion (magnitude, gradient, sensitivity). For example, you remove all weights with absolute value < 0.01 . The weight matrix becomes sparse—most entries are zero, but the non-zero entries are scattered throughout the matrix.

Advantage: Maximum flexibility. You can remove 50–90% of weights with minimal accuracy loss on many models.

Disadvantage: Sparse matrices are inefficient on general-purpose CPUs and most embedded hardware. Matrix multiplication libraries (BLAS, CMSIS-NN) assume dense matrices. Sparse operations require special data structures (CSR, COO) and custom kernels, which are not available on most embedded platforms.

Result: Unstructured pruning reduces parameter count (and flash usage) but does not reduce latency or SRAM usage on embedded targets unless you implement custom sparse inference kernels, which is non-trivial.

Structured Pruning

Structured pruning removes entire structures: entire channels, entire layers, or entire filters. For example, you remove 30% of the output channels in a convolutional layer. The resulting matrix is smaller but still dense.

Advantage: Dense matrices are supported by all inference frameworks and hardware. Removing 30% of channels reduces memory, latency, and energy proportionally.

Disadvantage: Less flexibility. You must remove entire structures, which may be harder to do without accuracy loss.

Result: Structured pruning reduces all metrics (flash, SRAM, latency) and is compatible with embedded inference frameworks out-of-the-box.

For embedded deployment, structured pruning is almost always the right choice. The only exception is if you have a custom sparse inference engine or an accelerator with native sparse support (rare on embedded hardware).

How Structured Pruning Works

1. Train the model normally.

2. Measure each channel's importance (e.g., by its L1 norm, or by the accuracy drop when it's removed).
3. Remove the least important channels (e.g., the bottom 20% by importance).
4. Fine-tune the pruned model to recover accuracy.
5. Repeat steps 2–4 iteratively until target size is reached.

Some frameworks (TensorFlow Model Optimization Toolkit, PyTorch's `torch.nn.utils.prune`) automate this process.

Accuracy vs. Pruning Rate

The relationship between pruning rate and accuracy is nonlinear. Removing the first 20% of parameters typically costs <1% accuracy. Removing 40% costs 2–4%. Removing 60% costs 5–10%. Beyond 70%, accuracy collapses unless the model was over-parameterized to begin with.

The sweet spot for embedded pruning is 20–40% removal. This gives meaningful size reductions with acceptable accuracy loss. If you need more compression, combine pruning with quantization.

Knowledge Distillation: Training a Small Model to Mimic a Large Model

Knowledge distillation trains a small "student" model to replicate the behavior of a large "teacher" model. The student learns from both the ground-truth labels and the teacher's predictions, which encode richer information than labels alone.

Why Distillation Works

Consider a classification task with 10 classes. For an input image, the ground-truth label might be "cat" (class 3). A one-hot label vector is [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]. But a well-trained teacher model might output probabilities: [0.01, 0.02, 0.85, 0.05, 0.01, 0.02, 0.01, 0.01, 0.01, 0.01].

The teacher's output is "soft"—it assigns non-zero probability to classes that are visually similar to the correct class (e.g., "dog" gets 5%). This soft distribution provides more information than the hard label. The student learns to match the teacher's distribution, which helps it generalize better than if it trained only on hard labels.

Distillation Process

1. Train a large, accurate teacher model (e.g., ResNet50, 90% accuracy).
2. Train a small student model (e.g., MobileNetV2-0.35) with a modified loss function:
 - $L_{\text{student}} = \alpha \times L_{\text{hard}} + (1-\alpha) \times L_{\text{soft}}$

- L_{hard} : cross-entropy between student predictions and ground-truth labels
 - L_{soft} : cross-entropy between student predictions and teacher predictions (both softened with temperature T)
 - α : weighting factor (typically 0.3–0.7)
3. The student learns to approximate the teacher's behavior while remaining small and fast.

Temperature in Distillation

Temperature (T) is a hyperparameter that controls how "soft" the predictions are. Standard softmax is:

$$p_i = \exp(z_i) / \sum \exp(z_j)$$

With temperature T :

$$p_i = \exp(z_i / T) / \sum \exp(z_j / T)$$

Higher T (e.g., $T=5$ or $T=10$) makes the distribution softer (more uniform). Lower T ($T=1$) is standard softmax. During distillation, you use $T > 1$ to emphasize the teacher's soft probabilities.

When Distillation Helps

Distillation is effective when:

- The teacher is much larger than the student (e.g., $10\times+$ more parameters).
- The student architecture is fundamentally limited (cannot achieve teacher accuracy no matter how long you train).
- You have access to the teacher model and the training data.

Distillation typically gains 2–5% accuracy over training the student from scratch. Combined with quantization and pruning, it can close the gap between a large, accurate model and a small, deployable model.

Combining Techniques: Quantize, Prune, Distill—Ordering Matters

When multiple optimization techniques are needed, the order of application affects the final result.

Recommended Order: Prune → Distill → Quantize

Step 1: Prune. Remove 20–40% of channels from the base model. Fine-tune to recover accuracy. This reduces the model's parameter count and operation count.

Step 2: Distill (optional). If the pruned model's accuracy is below threshold, train a student model that matches the pruned model's architecture but learns from a larger teacher. This recovers 2–4% accuracy.

Step 3: Quantize. Apply post-training quantization (or QAT) to convert the pruned (and optionally distilled) model to int8. This reduces memory by 75% and accelerates inference.

Why This Order?

Pruning before quantization allows quantization to operate on a smaller model, which reduces calibration complexity. Distillation after pruning uses the teacher to guide the student toward better accuracy even after channels are removed. Quantization last ensures that the final deployed model is int8, which is what the hardware accelerates.

Alternative Order: Quantize → Prune

Quantize first, then prune the quantized model. This can work but is less common because pruning int8 models is less well-supported by frameworks.

What Not to Do

Do not prune aggressively (>60%) and then quantize. Quantization adds error. Pruning adds error. Combined, they can destroy accuracy. If you need >60% compression, use distillation to a smaller architecture instead of extreme pruning.

The Accuracy Floor: When Compression Breaks the Application

Every model has an accuracy floor—the point below which further compression is unacceptable because the model no longer meets the application requirement.

The floor is application-specific:

- A keyword spotter with 92% accuracy might be acceptable (8% of commands require repetition).
- A medical diagnostic with 92% accuracy is unacceptable (8% misdiagnosis rate is catastrophic).

The workflow for finding the accuracy floor:

1. Define the minimum accuracy required by the application (from product requirements or regulatory standards).
2. Measure baseline accuracy (uncompressed model on validation set).

3. Apply compression incrementally (10%, 20%, 30% pruning; float32 → int16 → int8 quantization).
4. Measure accuracy after each step.
5. Stop compressing when accuracy drops below the floor.

Measuring Accuracy Degradation

Accuracy is not the only metric. For some applications, you care about:

- **False positive rate** (e.g., anomaly detection—false alarms are costly)
- **False negative rate** (e.g., pedestrian detection—missing a pedestrian is catastrophic)
- **Worst-case accuracy** (e.g., accuracy on rare classes or edge cases, not just average)

After compression, measure all relevant metrics. A model with 90% overall accuracy might have 60% accuracy on rare classes, which could be unacceptable.

A Worked Example: Compressing a Model That Barely Fails Constraints

You are deploying MobileNetV2-0.5 for defect detection on an STM32H7 (2 MB flash, 1 MB SRAM). The model:

- Parameters: 1.5 million (float32: 6 MB, int8: 1.5 MB)
- Flash required: 1.8 MB (after firmware: 600 KB + model: 1.5 MB int8 + overhead: ~300 KB, wait that doesn't add up)

Let me recalculate. Firmware uses 600 KB. Model weights (int8) are 1.5 MB. Total: 2.1 MB. Flash capacity: 2 MB. **The model doesn't fit by 100 KB.**

SRAM required: 280 KB (activations). Firmware uses 80 KB. Total: 360 KB. SRAM capacity: 1 MB. SRAM is fine.

Latency: 320 ms. Budget: 250 ms. **Latency fails by 70 ms.**

Accuracy (float32): 91.5%. Accuracy (int8 PTQ): 89.2%. Application requires ≥88%. Accuracy is acceptable.

Goal: Reduce flash usage by 100 KB and reduce latency by 70 ms (22% reduction) without accuracy falling below 88%.

Step 1: Apply Structured Pruning

You prune 25% of channels across all layers. The model now has 1.125 million parameters instead of 1.5 million.

New metrics:

- Flash (int8): 1.125 MB
- SRAM (activations): 210 KB (reduced proportionally)
- Latency: 240 ms (25% reduction in operations)
- Accuracy (float32, pruned): 89.8% (fine-tuned after pruning)

Analysis:

- Flash: 600 KB firmware + 1.125 MB model = 1.725 MB. Fits with 275 KB margin. ✓
- SRAM: 80 KB firmware + 210 KB activations = 290 KB. Fits with 734 KB margin. ✓
- Latency: 240 ms < 250 ms. Fits with 10 ms margin. ✓
- Accuracy: 89.8% > 88%. Acceptable. ✓

Pruning alone solved the problem. But there's a risk: you have only 10 ms of latency margin. If future firmware updates add features, latency might creep above 250 ms.

Step 2 (Optional): Distill to Recover Accuracy

The pruned model's accuracy is 89.8%, but the original float32 model achieved 91.5%. If you want to recover some of that 1.7% loss, you can train a student model with the same architecture as the pruned model but using the original large model as a teacher.

You train the student with distillation. New accuracy: 90.6%. This doesn't change latency or memory, but it gives you 0.8% more accuracy, which might be valuable if the application's accuracy threshold increases in the future.

Step 3: Quantize (Already Done)

The model is already int8. If it were still float32, you'd apply PTQ here to get the 75% memory reduction. Since you already quantized, this step is complete.

Final Model

- Parameters: 1.125 million (int8)
- Flash: 1.725 MB (fits in 2 MB with 14% margin)
- SRAM: 290 KB (fits in 1 MB with 71% margin)
- Latency: 240 ms (fits in 250 ms with 4% margin)
- Accuracy: 90.6% (distilled) or 89.8% (pruned without distillation), both > 88% threshold

The model now comfortably meets all constraints. Pruning was sufficient. Distillation was optional but improved accuracy. Quantization (already applied) enabled the entire deployment.

Tools and Frameworks for Model Optimization

Several frameworks automate quantization, pruning, and distillation:

TensorFlow Model Optimization Toolkit:

- PTQ: `tf.lite.TFLiteConverter` with `optimizations=[tf.lite.Optimize.DEFAULT]`
- QAT: `tfmot.quantization.keras.quantize_model()`
- Pruning: `tfmot.sparsity.keras.prune_low_magnitude()`

PyTorch:

- PTQ: `torch.quantization.quantize_dynamic()` or `torch.quantization.quantize()`
- QAT: `torch.quantization.prepare_qat()` → train → `torch.quantization.convert()`
- Pruning: `torch.nn.utils.prune` (unstructured), or third-party libraries for structured pruning

ONNX Runtime:

- Supports quantized ONNX models
- Can run PTQ on ONNX graphs

Neural Network Compression Framework (NNCF):

- Intel's open-source tool for quantization, pruning, and distillation
- Supports TensorFlow and PyTorch

Vendor-specific tools:

- STM32Cube.AI: Includes quantization and optimization for STM32 targets
- Edge Impulse: Web-based tool with automated quantization and optimization

Choose based on your training framework and deployment target. If you trained in TensorFlow and deploy to TFLite Micro, use TensorFlow Model Optimization Toolkit. If you trained in PyTorch, use PyTorch quantization or convert to ONNX.

What You Cannot Fix with Optimization

Optimization reduces size, latency, and power, but it has limits. Some problems require model redesign, not compression:

Problem: Accuracy is already below threshold before optimization.

Optimization will make it worse. You need a better base model or more training data.

Problem: The model's architecture is fundamentally wrong for the task.

Compressing a ResNet for keyword spotting won't help. Use a 1D CNN designed for audio.

Problem: Latency is 10× over budget.

Optimization typically gains 2–4× at best. If you need 10×, you need faster hardware or a completely different architecture.

Problem: The model uses operations unsupported by the target (e.g., attention, dynamic control flow).

Optimization can't remove architectural dependencies. Redesign the model or upgrade the hardware.

Optimization is the final step after selection. If the selected model is fundamentally unsuitable, optimization won't save it.

What Comes Next

You can now select models (Chapter 11) and optimize them (Chapter 12). But you still haven't deployed them. Chapter 13 examines the toolchains that convert trained models into firmware—the build systems, converters, and runtime libraries that take a `.h5` or `.onnx` file and produce a binary that runs on your microcontroller. This is where theoretical models become real deployments.

Chapter 13: TinyML Toolchains and Deployment Pipelines

You have a trained model. You have quantized it to int8. You have pruned 25% of its channels. The model achieves 90% accuracy on your validation set. Now you need to convert it from a training artifact (a `.h5` file in Keras or a `.pth` file in PyTorch) into firmware that runs on your STM32H7 microcontroller.

You attempt the conversion using TensorFlow Lite's converter. The converter runs, produces a `.tflite` file, and reports success. You integrate the `.tflite` file into your firmware using TensorFlow Lite for Microcontrollers, compile, flash, and run.

The first inference produces garbage output—predictions that don't match the validation results. You debug for hours and discover the issue: the converter applied dynamic range quantization to layers you thought were already int8, and the resulting double-quantization destroyed accuracy. The model that worked perfectly in Python fails silently on the target hardware.

This is the deployment toolchain problem. The path from trained model to running firmware involves multiple tools—converters, optimizers, compilers, and runtime libraries—and each tool

makes decisions that affect accuracy, latency, and memory usage. Some of those decisions are visible, some are hidden, and some are wrong. Understanding the toolchain is the prerequisite for successful deployment.

This chapter teaches you to navigate the end-to-end deployment pipeline from trained model to verified on-target inference, identify the decisions made by converters and compilers that affect deployment, select the appropriate toolchain for your hardware target and model format, and test and validate that the deployed model produces correct results. By the end of this chapter, you will be able to convert a model, deploy it, and verify that it works before it reaches production.

The Deployment Pipeline: Train → Convert → Optimize → Compile → Deploy → Verify

Deployment is not a single step. It's a sequence of transformations:

Step 1: Train

You train a model in TensorFlow, PyTorch, or another framework. The output is a training checkpoint or saved model (`.h5`, `.pth`, `.onnx`, `.pb`). This model runs on a GPU or CPU with Python, NumPy, and the full framework runtime.

Step 2: Convert

You convert the training model to a deployment format. The converter removes training-specific operations (dropout, batch normalization folding), applies graph optimizations (operator fusion, constant propagation), and prepares the model for the target runtime. The output is a deployment model (`.tflite`, `.onnx`, vendor-specific formats).

Step 3: Optimize (optional but common)

The optimizer applies quantization, pruning, or layer fusion to reduce model size or latency. Some converters include optimization (TFLite converter does quantization), others require separate tools (Vela for ARM Ethos-U, TensorRT for NVIDIA).

Step 4: Compile

The deployment model is compiled into firmware. For microcontrollers, this means linking the model file with an inference engine (TFLite Micro, CMSIS-NN) and your application code, then compiling to a binary with GCC or ARM Compiler.

Step 5: Deploy

The firmware binary is flashed to the target device via JTAG, USB, or serial bootloader.

Step 6: Verify

You run inference on the target and compare outputs to the training model's outputs on the same inputs. If outputs match (within quantization error), deployment succeeded. If they don't, you trace back to find which step introduced the error.

This pipeline has failure modes at every step. The converter may fail to convert a custom layer. The optimizer may apply an incompatible optimization. The compiler may miscompile a kernel. The runtime may have a bug. Verification catches these failures before they reach production.

TensorFlow Lite for Microcontrollers: The Dominant Path

TensorFlow Lite for Microcontrollers (TFLite Micro) is the most widely used inference engine for embedded AI. It's a C++ library with no dependencies on the standard library (std::vector, std::map, etc.), no dynamic memory allocation (configurable), and a small code footprint (~100 KB).

The TFLite Conversion Process

Input: A trained TensorFlow or Keras model.

Output: A `.tflite` file (FlatBuffers format) containing the model graph, weights, and metadata.

Converter invocation:

```
import tensorflow as tf

# Load trained model
model = tf.keras.models.load_model('my_model.h5')

# Convert to TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(model)

# Enable int8 quantization
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset_gen # Calibration data

# Convert
tflite_model = converter.convert()

# Save
with open('my_model.tflite', 'wb') as f:
    f.write(tflite_model)
```

What the converter does:

1. Extracts the model graph (layers, connections, parameters).
2. Applies graph optimizations (fuses layers, removes unused nodes).

3. Converts weights to int8 (if quantization is enabled).
4. Inserts quantization/dequantization ops between layers if needed.
5. Serializes to FlatBuffers format.

What can go wrong:

- Custom layers not supported by TFLite (e.g., custom activation functions, third-party layers).
- Quantization applied incorrectly (wrong calibration data, mismatched ranges).
- Converter fails silently (produces a `.tflite` file but with wrong layer configurations).

Deploying TFLite Micro on a Microcontroller

Step 1: Integrate the model file.

The `.tflite` file is a binary blob. You include it in your firmware as a byte array:

```
// Generated from xxd -i my_model.tflite > my_model_data.h
alignas(16) const unsigned char my_model_data[] = {
    0x18, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, ...
};
const unsigned int my_model_data_len = 847362;
```

Step 2: Initialize the interpreter.

Allocate a tensor arena (static memory for activations), load the model, and initialize the interpreter:

```
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"

constexpr int kTensorArenaSize = 200 * 1024; // 200 KB
alignas(16) uint8_t tensor_arena[kTensorArenaSize];

// Load model
const tflite::Model* model = tflite::GetModel(my_model_data);

// Register operations
tflite::MicroMutableOpResolver<10> resolver;
resolver.AddConv2D();
resolver.AddDepthwiseConv2D();
resolver.AddFullyConnected();
resolver.AddSoftmax();
// ... add all ops used by the model
```

```
// Create interpreter
tflite::MicroInterpreter interpreter(model, resolver, tensor_arena, kTensorArenaSize);

// Allocate tensors
interpreter.AllocateTensors();
```

Step 3: Run inference.

Copy input data to the input tensor, invoke the interpreter, and read the output tensor:

```
// Get input tensor
TfLiteTensor* input = interpreter.input(0);

// Copy image data to input tensor (96x96x3, int8 quantized)
for (int i = 0; i < 96*96*3; i++) {
    input->data.int8[i] = image_data[i];
}

// Run inference
TfLiteStatus invoke_status = interpreter.Invoke();
if (invoke_status != kTfLiteOk) {
    // Inference failed
}

// Get output tensor
TfLiteTensor* output = interpreter.output(0);

// Read predictions (10 classes, int8 quantized)
for (int i = 0; i < 10; i++) {
    int8_t quantized_output = output->data.int8[i];
    // Dequantize: float_output = (quantized - zero_point) * scale
    float prob = (quantized_output - output->params.zero_point) * output->params.scale;
}
```

Common TFLite Micro Pitfalls

Insufficient tensor arena size: If the arena is too small, `AllocateTensors()` fails. The error message tells you the required size—increase `kTensorArenaSize` and recompile.

Missing operations: If the model uses an op not registered in the `OpResolver`, you get a runtime error "Op not found." Add the missing op to the resolver.

Quantization mismatches: If you quantize the model with one scale/zero_point but the converter uses different values, outputs will be wrong. Always use the same representative dataset for calibration.

Memory alignment: TFLite Micro requires 16-byte alignment for the tensor arena. Use `alignas(16)` or ensure the array is naturally aligned.

Edge Impulse: End-to-End Toolchain for Embedded ML

Edge Impulse is a web-based platform that provides the full deployment pipeline: data collection, model training, optimization, and deployment to embedded targets. It hides much of the toolchain complexity behind a GUI.

Edge Impulse Workflow

1. **Collect data:** Upload training data via the web interface or collect it directly from your device using Edge Impulse's firmware.
2. **Design impulse:** Configure preprocessing (spectral features for audio, image scaling for vision) and select a model architecture (pre-built or custom).
3. **Train:** Train the model in the cloud (Edge Impulse's servers, not your machine).
4. **Optimize:** Edge Impulse automatically applies quantization.
5. **Deploy:** Download a deployment package (C++ library, Arduino library, or firmware binary) for your target hardware.

What Edge Impulse Handles

- **Toolchain abstraction:** You don't write conversion scripts or OpResolvers. Edge Impulse generates deployment code automatically.
- **Hardware optimization:** Edge Impulse includes optimized kernels for ARM Cortex-M, Cortex-A, ESP32, and other platforms.
- **Preprocessing integration:** Spectral features (MFCC, FFT) and image preprocessing are compiled into the deployment library.

What Edge Impulse Hides

- **Model architecture control:** You can select architectures from a menu, but you cannot define arbitrary custom layers.
- **Training hyperparameters:** Limited control over learning rate, batch size, optimizer (compared to writing your own training script).
- **Debugging:** If the deployed model fails, you have less visibility into what went wrong (no direct access to the TFLite converter logs or intermediate files).

When to Use Edge Impulse

Good for:

- Rapid prototyping (deploy a model in hours, not weeks).
- Teams without ML expertise (Edge Impulse handles training and optimization).
- Standard tasks (keyword spotting, gesture recognition, image classification on common datasets).

Not ideal for:

- Custom model architectures that Edge Impulse doesn't support.
- Applications requiring fine-grained control over quantization or deployment.
- Debugging complex deployment failures (you need access to intermediate artifacts).

ONNX and Vendor-Specific Compilers

ONNX (Open Neural Network Exchange) is an open format for representing neural networks. It's framework-agnostic—you can train a model in PyTorch, export to ONNX, and deploy it using TensorFlow Lite, ONNX Runtime, or vendor-specific tools.

ONNX Conversion from PyTorch

```
import torch
import torch.onnx

# Load PyTorch model
model = torch.load('my_model.pth')
model.eval()

# Create dummy input (must match model input shape)
dummy_input = torch.randn(1, 3, 96, 96)

# Export to ONNX
torch.onnx.export(model, dummy_input, 'my_model.onnx',
                  input_names=['input'], output_names=['output'],
                  dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}})
```

Vendor-Specific Compilers

Many embedded AI accelerators require vendor-specific toolchains that consume ONNX or TFLite models and compile them for the accelerator.

STM32Cube.AI:

- Supports TFLite, ONNX, Keras models.

- Compiles for STM32 microcontrollers with or without AI accelerators.
- Generates optimized C code tailored for STM32 hardware.

Vela (ARM Ethos-U):

- Compiles TFLite models for ARM Ethos-U NPUs.
- Applies operator splitting, memory management, and layer scheduling.
- Outputs an optimized `.tflite` file with Ethos-U-specific metadata.

Edge TPU Compiler (Google Coral):

- Compiles TFLite models for Google Coral Edge TPU.
- Requires int8 quantization.
- Outputs a `.tflite` file with TPU-specific ops.

Each compiler has quirks:

- **STM32Cube.AI** sometimes fails on models with unsupported ops (e.g., non-standard activations). It generates a report listing supported/unsupported layers.
- **Vela** requires all operations to be int8. Float32 layers cause compilation to fail.
- **Edge TPU Compiler** only accelerates certain ops; others fall back to the CPU, which can make the model slower than expected.

When using vendor compilers, always check the compilation report to see which layers are accelerated and which fall back.

Toolchain-Induced Accuracy Loss

Converters and optimizers can introduce accuracy degradation that didn't exist in the trained model. This is toolchain-induced accuracy loss—errors introduced by the deployment pipeline, not by the model itself.

Common Sources

1. Incorrect quantization calibration:

The converter uses a calibration dataset to determine activation ranges. If the calibration set is not representative (e.g., it includes only bright images but the deployment includes dark images), quantization ranges are wrong, and accuracy drops.

Fix: Use a diverse, representative calibration dataset. Include edge cases.

2. Layer fusion side effects:

Converters fuse layers (e.g., Conv + BatchNorm + ReLU into a single op) to improve performance. If fusion is buggy or introduces numerical instability, accuracy degrades.

Fix: Disable fusion (`converter.experimental_new_converter = False` in TFLite) and check if accuracy recovers.

3. Operator mismatch:

The converter maps training ops to deployment ops. Sometimes the mapping is approximate. For example, PyTorch's `F.gelu()` might map to TFLite's approximate GELU, not the exact version.

Fix: Check the op mapping in the converter's documentation. Use standard ops where possible.

4. Floating-point precision loss:

Even without quantization, float32 on a CPU can differ slightly from float32 on a GPU due to different rounding modes or SIMD optimizations.

Fix: Accept small numerical differences (<0.01% accuracy change). If differences are large (>1%), investigate.

Measuring Toolchain-Induced Loss

Compare the converted model's accuracy to the original model's accuracy on the same validation set:

1. Run inference on the validation set using the original training model (Python, TensorFlow/PyTorch).
2. Run inference on the same validation set using the converted model (TFLite interpreter or ONNX Runtime).
3. Calculate accuracy for both. If they differ by >2%, toolchain introduced error.

If accuracy drops, bisect the pipeline: test after conversion (before optimization), after optimization, after compilation. Identify which step introduced the error.

Testing and Validation After Deployment

The deployed model must produce correct results on the target hardware. Testing involves:

1. Functional Correctness: Does the Model Run?

Flash the firmware, trigger an inference, and verify it completes without crashing. Check:

- No memory faults or hardfaults.
- Inference returns (doesn't hang).
- Output tensor has expected shape and range.

2. Numerical Correctness: Are the Outputs Correct?

Run inference on known inputs and compare outputs to expected values. For int8 quantized models, allow for quantization error ($\pm 1\text{--}2\%$ difference is normal).

Test procedure:

1. Select 10–100 test inputs from the validation set.
2. Run inference on these inputs using the training model (Python). Record outputs.
3. Run inference on the same inputs on the target hardware. Record outputs.
4. Compare outputs element-wise. Calculate mean absolute error (MAE) and maximum error.

If $\text{MAE} > 5\%$ or $\text{max error} > 10\%$, something is wrong (quantization mismatch, incorrect conversion, runtime bug).

3. Latency Validation: Does It Meet Timing Requirements?

Measure inference latency on the target:

- Run inference 100+ times.
- Record min, mean, max, and 95th percentile latency.
- Verify that max latency is below the deadline.

4. Memory Validation: Does It Fit in SRAM?

Measure actual SRAM usage during inference. Check that peak usage is below capacity:

- Use the tensor arena size reported by TFLite Micro.
- Add stack usage (measure with a stack canary or RTOS tools).
- Add heap usage (if any dynamic allocation occurs, which it shouldn't).

5. Power Validation: Does It Meet Energy Budget?

Measure current draw during inference. Integrate to get energy per inference. Verify that average power (over realistic duty cycles) is below budget.

6. Accuracy Validation: Does It Achieve Target Accuracy?

Run inference on the full validation set (if feasible—this may require serial logging or SD card storage for results) and compute accuracy. Verify it matches the training accuracy (within quantization tolerance).

A Worked Example: Full Pipeline from Keras to Arduino Nano 33 BLE Sense

You have a keyword spotting model trained in Keras. You need to deploy it to an Arduino Nano 33 BLE Sense (nRF52840, Cortex-M4 at 64 MHz, 256 KB SRAM).

Step 1: Train (Already Done)

Model: 1D CNN with 5 convolutional layers and 2 fully connected layers. Input: 16 kHz audio, 1-second windows. Output: 10 keywords + "unknown" + "silence" (12 classes).

Validation accuracy (float32): 94.2%

Step 2: Convert to TFLite with Quantization

```
import tensorflow as tf
```

```
model = tf.keras.models.load_model('keyword_model.h5')
```

```
# Prepare calibration dataset (100 samples)
```

```
def representative_dataset_gen():
```

```
    for i in range(100):
```

```
        audio = load_audio_sample(i) # Returns (1, 16000) numpy array
```

```
        yield [audio.astype(np.float32)]
```

```
# Convert
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
converter.representative_dataset = representative_dataset_gen
```

```
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
```

```
converter.inference_input_type = tf.int8
```

```
converter.inference_output_type = tf.int8
```

```
tflite_model = converter.convert()
```

```
with open('keyword_model.tflite', 'wb') as f:
```

```
    f.write(tflite_model)
```

Converted model size: 85 KB (int8).

Step 3: Test Converted Model in Python

```
import numpy as np
```

```
interpreter = tf.lite.Interpreter(model_content=tflite_model)
```

```
interpreter.allocate_tensors()
```

```
input_details = interpreter.get_input_details()
```

```

output_details = interpreter.get_output_details()

# Test on validation set
correct = 0
total = 0
for audio, label in validation_set:
    # Quantize input
    input_scale, input_zero_point = input_details[0]['quantization']
    audio_quantized = (audio / input_scale + input_zero_point).astype(np.int8)

    interpreter.set_tensor(input_details[0]['index'], audio_quantized)
    interpreter.invoke()
    output = interpreter.get_tensor(output_details[0]['index'])

    # Dequantize output
    output_scale, output_zero_point = output_details[0]['quantization']
    output_float = (output.astype(np.float32) - output_zero_point) * output_scale

    predicted = np.argmax(output_float)
    if predicted == label:
        correct += 1
    total += 1

accuracy = correct / total
print(f'Converted model accuracy: {accuracy * 100:.1f}%')

```

Result: 93.1% accuracy (1.1% loss due to quantization—acceptable).

Step 4: Integrate into Arduino Firmware

Convert `.tflite` to a C array using `xxd`:

```
xxd -i keyword_model.tflite > keyword_model_data.h
```

Write Arduino sketch:

```

#include <TensorFlowLite.h>
#include "keyword_model_data.h"

constexpr int kTensorArenaSize = 80 * 1024;
alignas(16) uint8_t tensor_arena[kTensorArenaSize];

tflite::MicroInterpreter* interpreter = nullptr;

```

```

void setup() {
    Serial.begin(115200);

    // Load model
    const tflite::Model* model = tflite::GetModel(keyword_model_data);

    // Create op resolver
    static tflite::MicroMutableOpResolver<8> resolver;
    resolver.AddConv2D();
    resolver.AddFullyConnected();
    resolver.AddReshape();
    resolver.AddSoftmax();
    // ... add other ops

    // Create interpreter
    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensor_arena, kTensorArenaSize);
    interpreter = &static_interpreter;

    interpreter->AllocateTensors();

    Serial.println("Model loaded");
}

void loop() {
    // Acquire audio from microphone (not shown)
    int8_t audio_buffer[16000];
    acquire_audio(audio_buffer);

    // Copy to input tensor
    TfLiteTensor* input = interpreter->input(0);
    memcpy(input->data.int8, audio_buffer, 16000);

    // Run inference
    uint32_t start = micros();
    interpreter->Invoke();
    uint32_t latency = micros() - start;

    // Read output
    TfLiteTensor* output = interpreter->output(0);
    int predicted_class = 0;
    int8_t max_score = -128;
    for (int i = 0; i < 12; i++) {

```

```

        if (output->data.int8[i] > max_score) {
            max_score = output->data.int8[i];
            predicted_class = i;
        }
    }

    Serial.print("Keyword: ");
    Serial.print(class_names[predicted_class]);
    Serial.print(" Latency: ");
    Serial.print(latency / 1000);
    Serial.println(" ms");

    delay(1000);
}

```

Step 5: Deploy and Verify

Compile, flash, and test:

- Measured latency: 42 ms (within 100 ms budget)
- SRAM usage: 80 KB tensor arena + 20 KB firmware stack = 100 KB (fits in 256 KB)
- Accuracy on-device: 92.8% (tested with 50 recorded samples played through the microphone)

The 0.3% accuracy difference between TFLite interpreter (93.1%) and on-device (92.8%) is due to microphone noise and audio preprocessing differences. Acceptable.

Deployment succeeded.

What Comes Next

You have navigated the toolchain from trained model to deployed firmware. You understand conversion, optimization, compilation, and verification. But you haven't yet integrated all the skills from this book into a complete design decision.

Chapter 14—the final chapter—synthesizes everything. You apply the full framework (constraint evaluation, model selection, optimization, and toolchain deployment) to three case studies: industrial anomaly detection, medical wearable design, and agricultural edge sensing. Each case study requires making and justifying every decision from hardware selection to model architecture to deployment strategy.

This is where theory becomes practice.

Chapter 14: Integration Case Studies: Full Design Decisions

You have built the complete toolkit. You can quantify embedded constraints (Chapters 2, 5–7), evaluate hardware acceleration options (Chapter 8), design edge-cloud architectures (Chapter 9), assess real-time safety requirements (Chapter 10), select models for deployment (Chapter 11), optimize models through quantization and pruning (Chapter 12), and navigate deployment toolchains (Chapter 13).

Each skill was taught in isolation. This chapter integrates them. You will apply the full decision framework—from application requirements to deployed system—on three realistic embedded AI problems. Each case study presents a complete design decision workflow: requirements analysis, constraint quantification, hardware selection, model selection, optimization, deployment strategy, and the alternatives that were rejected.

The case studies are:

- **Case A:** Industrial vibration monitoring for predictive maintenance
- **Case B:** Medical wearable for cardiac arrhythmia detection
- **Case C:** Agricultural edge sensor for crop disease detection

By the end of this chapter, you will have seen the complete integration process three times, from three different angles. You will understand how to produce a structured integration decision document that specifies the model, hardware, deployment strategy, and constraint justification. And you will be able to critique proposed AI embedded designs and identify the decisions most likely to fail in production.

This is the terminal capability the book was built to deliver.

Case A: Industrial Anomaly Detection — Vibration Monitoring for Predictive Maintenance

Application Requirements

A manufacturing facility operates 200 CNC milling machines that run 24/7. Each machine has critical bearings that degrade over time. When a bearing fails catastrophically, it damages the machine (\$50,000 repair cost) and halts production (\$10,000 per hour of downtime). The facility wants to deploy vibration sensors with on-device anomaly detection to predict bearing failures 24–48 hours in advance, allowing scheduled maintenance instead of emergency repairs.

Functional Requirements:

- Monitor vibration continuously at 1 kHz sampling rate (3-axis accelerometer)
- Detect bearing faults with $\geq 95\%$ sensitivity (true positive rate)
- False positive rate $< 5\%$ (no more than 1 false alarm per machine per month)
- Alert latency: detection within 10 seconds of fault onset is acceptable (soft real-time)

Physical Constraints:

- Battery-powered (18-month target life, coin cell or small Li-ion)
- No cloud connectivity (factory has no WiFi, cellular is cost-prohibitive at \$5/month/device $\times$ 200 devices)
- Harsh environment (vibration, heat, dust—industrial-grade hardware required)
- Cost: $< \$40$ per sensor (total deployment budget: \$8,000)

Safety Requirements:

- Not a safety-critical system (failure to detect a fault is costly but not dangerous)
- No hard real-time requirements (alerts are for scheduling, not for emergency shutdown)

Constraint Quantification

Power budget:

18-month life on battery. Assume a 1000 mAh Li-ion cell at 3.7V (13,320 J total). Target average power:

$$P_{\text{avg}} = 13,320 \text{ J} / (18 \text{ months} \times 730 \text{ hours/month} \times 3600 \text{ s/hour}) = 0.28 \text{ mW}$$

This is aggressive. Even sleep current of 10 μA ($3.7\text{V} \times 0.01 \text{ mA} = 0.037 \text{ mW}$) consumes 13% of the budget. Inference must be extremely infrequent or very fast.

Latency budget:

Detection within 10 seconds is acceptable. This is soft real-time—occasional delays are tolerable.

Memory budget:

Not specified. Assume typical low-cost microcontroller: 256 KB SRAM, 1 MB flash.

Connectivity:

No cloud. Must store alerts locally and transmit via local gateway (Modbus, RS-485) once per day or on-demand.

Decision 1: Processing Architecture (On-Device vs. Edge Gateway)

Option 1: Fully on-device processing.

Sensor runs inference locally, stores alerts in flash, and transmits summary once per day to a local gateway.

Pros: Minimal communication cost (one transmission per day). Privacy-preserving (raw vibration data never leaves the sensor).

Cons: Must fit model on microcontroller. Power budget is extremely tight.

Option 2: Edge gateway processing.

Sensor streams raw vibration data to a local industrial PC (edge gateway) via wired connection (RS-485 or CAN bus). Gateway runs inference.

Pros: Gateway is mains-powered—no energy constraint. Can run larger, more accurate models.

Cons: Requires wiring 200 sensors to gateways (installation cost). Gateway is a single point of failure.

Option 3: Hybrid—on-device triggers, gateway refinement.

Sensor runs a simple threshold detector locally. When vibration exceeds threshold, sensor wakes a radio and transmits a 1-second window to the gateway for ML-based classification.

Pros: Low power (radio is off 99% of time). Gateway handles complex ML.

Cons: Requires wireless (LoRa or sub-GHz radio), which adds cost and complexity.

Decision: Select Option 1 (fully on-device). Battery life is tight, but communication infrastructure (Option 2) or wireless radios (Option 3) increase cost beyond the \$40/unit budget. The challenge is achieving 18-month battery life with on-device inference.

Decision 2: Model Selection

Bearing fault detection is a time-series anomaly detection problem. Candidate approaches:

Approach A: Threshold detector (classical).

Compare RMS vibration to a learned threshold. Flag anomaly if RMS exceeds threshold for >5 consecutive seconds.

Pros: Minimal compute (RMS calculation is cheap). Deterministic.

Cons: Low accuracy (60–70% sensitivity in prior studies). High false positive rate (normal transient events trigger alarms).

Approach B: Decision tree on engineered features.

Extract 20 features from 1-second windows (mean, variance, spectral peaks, kurtosis, zero-crossing rate). Train random forest (50 trees).

Pros: Low memory (200 KB model). Low latency (5 ms inference). High accuracy (85–90% sensitivity in literature).

Cons: Requires hand-engineered features. May not generalize to different machine types.

Approach C: 1D CNN on raw vibration.

Train a 1D convolutional network on 1-second windows of raw 3-axis vibration data (3,000 samples).

Pros: No feature engineering. Can learn subtle patterns that hand-crafted features miss. Accuracy 92–96% in recent research.

Cons: Higher memory (500 KB – 1 MB models typical). Higher latency (50–200 ms). Higher power.

Approach D: Autoencoder (unsupervised).

Train an autoencoder on normal vibration. Detect anomalies as high reconstruction error.

Pros: Does not require labeled fault data (normal-only training). Can detect novel faults.

Cons: High memory (autoencoders need encoder + decoder, often 1–2 MB). Difficult to set reconstruction threshold (false positives vs. false negatives tradeoff).

Decision: Select Approach B (random forest on features). Rationale:

- Approach A fails the accuracy requirement ($70\% < 95\%$ sensitivity).
- Approach C is too expensive in memory and power for 18-month battery life.
- Approach D requires more complex threshold tuning and is hard to validate.
- Approach B achieves 88% sensitivity (below 95% target, but can be improved with better features), fits in memory, and runs in 5 ms—enabling low power through infrequent inference.

Decision 3: Hardware Selection

Constraints:

- Flash: 200 KB model + 600 KB firmware = 800 KB required
- SRAM: 50 KB (feature extraction buffers) + 20 KB (firmware) = 70 KB required
- Power: Extreme—0.28 mW average
- Cost: <\$40

Candidates:

Option 1: STM32L4R5 (ultra-low-power MCU)

- Cortex-M4 at 120 MHz, 640 KB SRAM, 2 MB flash

- Sleep current: 0.4 μ A, Active: 100 μ A/MHz = 12 mA at 120 MHz
- Cost: \$6

Option 2: nRF52840 (Bluetooth Low Energy MCU)

- Cortex-M4 at 64 MHz, 256 KB SRAM, 1 MB flash
- Sleep current: 2.8 μ A, Active: 5.3 mA at 64 MHz
- Cost: \$4 (but includes Bluetooth radio, which is not needed)

Option 3: ESP32-S3 (WiFi/BLE SoC)

- Dual-core Xtensa at 240 MHz, 512 KB SRAM, 8 MB flash
- Sleep current: 10 μ A, Active: 40 mA
- Cost: \$4.50

Decision: Select STM32L4R5. Rationale:

- Memory: 640 KB SRAM exceeds requirement by 9 $\times$. 2 MB flash exceeds requirement by 2.5 $\times$. Comfortable margin.
- Power: 0.4 μ A sleep current is the lowest among options. This is critical for 18-month battery life.
- Cost: \$6 is within budget (\$40 - \$6 leaves \$34 for accelerometer, enclosure, battery, PCB).

nRF52840 is cheaper but has less SRAM and higher sleep current (2.8 μ A vs. 0.4 μ A—7 $\times$ worse). ESP32-S3 has even higher sleep current (10 μ A—25 $\times$ worse).

Decision 4: Inference Scheduling Strategy

Given P_{avg} target of 0.28 mW, calculate sustainable inference frequency.

Energy per inference:

- Feature extraction: 10 ms at 12 mA = $3.7V \times 0.012 A \times 0.01 s = 0.44 mJ$
- Random forest inference: 5 ms at 12 mA = $3.7V \times 0.012 A \times 0.005 s = 0.22 mJ$
- Total per inference: 0.66 mJ

Sleep power:

- $3.7V \times 0.0004 mA = 0.00148 mW$

Inference interval: If inference runs every T seconds, average power is:

$$P_{avg} = (0.66 mJ / T) + 0.00148 mW$$

Set $P_{avg} = 0.28 mW$:

$$\begin{aligned}0.28 \text{ mW} &= (0.66 \text{ mJ} / T) + 0.00148 \text{ mW} \\0.2785 \text{ mW} &= 0.66 \text{ mJ} / T \\T &= 0.66 \text{ mJ} / 0.0002785 \text{ W} = 2.37 \text{ seconds}\end{aligned}$$

Inference can run every 2.4 seconds while staying within power budget.

Validation:

- Faults develop over hours to days. Checking every 2.4 seconds provides 1,500 checks per hour, which is more than sufficient to detect gradual degradation.
- Latency requirement (10 seconds) is met: worst-case detection time is one inference interval (2.4 s).

Decision: Run inference every 2.5 seconds.

Decision 5: Model Training and Validation

Train random forest on:

- Normal data: 10,000 samples (vibration from healthy machines)
- Fault data: 2,000 samples (vibration from machines with known bearing faults)

Features extracted per 1-second window:

- RMS (3 axes) = 3 features
- Peak amplitude (3 axes) = 3 features
- Spectral centroid (FFT-based, 3 axes) = 3 features
- Kurtosis (3 axes) = 3 features
- Zero-crossing rate (3 axes) = 3 features

Total: 15 features per sample.

Train random forest with 50 trees, max depth 10.

Validation results:

- Sensitivity (true positive rate): 89%
- False positive rate: 3%
- Model size: 180 KB

Sensitivity is below the 95% target. To improve, add more features (dominant FFT frequency, harmonic ratios) and retrain.

After feature engineering iteration 2:

- Features: 25 (added 10 frequency-domain features)
- Sensitivity: 94%

- False positive rate: 4%

Still below 95%, but close. Product team accepts 94% sensitivity as acceptable given cost constraints.

Decision 6: Deployment

Firmware stack:

- STM32 HAL drivers: 200 KB flash, 10 KB SRAM
- Random forest inference engine (custom, lightweight): 20 KB flash
- Feature extraction library (FFT, statistical features): 100 KB flash
- Application logic (data acquisition, scheduling, storage): 50 KB flash
- Model (random forest weights): 180 KB flash

Total flash: 550 KB (fits in 2 MB with 73% margin)

Total SRAM: 50 KB feature buffers + 10 KB firmware = 60 KB (fits in 640 KB with 91% margin)

Flash firmware, deploy to 5 pilot units, and run field trial for 1 month.

Field Trial Results

- Battery life: 16.5 months (measured extrapolation from 1-month current draw)
- Sensitivity: 91% (3 faults detected out of 3.3 expected based on historical failure rates—small sample)
- False positives: 2 per unit per month (8% false positive rate, higher than validation)

Root cause of higher false positives: Vibration from adjacent machines caused false triggers. Fixed by adding spatial filtering (compare to adjacent sensor readings—if only one sensor triggers, it's likely a false positive).

After firmware update, false positive rate: 4.5%.

Final System Specification

Hardware: STM32L4R5, 3-axis MEMS accelerometer (ADXL355), 1000 mAh Li-ion battery

Model: Random forest (50 trees, 25 features, 180 KB)

Inference schedule: Every 2.5 seconds

Performance: 91% sensitivity, 4.5% false positive rate, 16.5-month battery life

Cost: \$38 per unit (hardware BOM)

Alternatives rejected and why:

- 1D CNN: Memory and power constraints. Would require larger battery (higher cost) or more frequent charging (unacceptable for industrial deployment).
 - Cloud processing: No connectivity infrastructure. Adding cellular would cost $\$5/\text{month/unit} \times 200 = \$1,000/\text{month}$ ongoing.
 - Edge gateway architecture: Installation cost of wiring 200 sensors exceeded budget.
-

Case B: Medical Wearable — Cardiac Arrhythmia Detection

Application Requirements

A medical device company is developing a wearable ECG monitor for continuous cardiac rhythm monitoring. The device detects arrhythmias (irregular heartbeats) and alerts the user to seek medical attention. The device is intended for outpatient monitoring of patients with known cardiac conditions.

Functional Requirements:

- Continuous ECG monitoring at 250 Hz sampling rate (single-lead ECG)
- Detect 5 arrhythmia types: atrial fibrillation, ventricular tachycardia, premature ventricular contractions, bradycardia, tachycardia
- Sensitivity $\geq 98\%$ for life-threatening arrhythmias (VT, VF)
- Specificity $\geq 95\%$ (false positive rate $< 5\%$)
- Latency: Alert within 30 seconds of arrhythmia onset

Regulatory Requirements:

- FDA Class II medical device (510(k) pathway)
- Must demonstrate clinical validation (sensitivity/specificity on diverse patient population)
- Software lifecycle per IEC 62304
- Cybersecurity requirements (if device transmits data)

Physical Constraints:

- Wearable form factor (wristband or chest patch)
- 7-day battery life minimum (rechargeable Li-ion)
- Lightweight (< 50 grams)
- Cost: $< \$200$ per unit (consumer medical device pricing)

Constraint Quantification

Power budget:

7 days on 500 mAh Li-ion cell at 3.7V (6,660 J). Average power:

$$P_{\text{avg}} = 6,660 \text{ J} / (7 \text{ days} \times 24 \text{ hours} \times 3600 \text{ s}) = 11 \text{ mW}$$

Latency budget:

30 seconds for alert. This allows batched processing (accumulate 10–30 seconds of ECG data, run inference on batch).

Memory budget:

Not specified. Assume wearable SoC: 512 KB SRAM, 2 MB flash.

Privacy:

Medical data (ECG) is HIPAA-protected. Cannot send raw ECG to cloud without encryption and patient consent. On-device processing preferred.

Decision 1: Processing Architecture**Option 1: Fully on-device.**

Run arrhythmia detection on the wearable. Send only alerts (not raw ECG) to a paired smartphone.

Pros: HIPAA-compliant (raw data never leaves device). Works offline (no connectivity required).

Cons: Must fit ML model on wearable SoC. Power budget is tight.

Option 2: Smartphone processing.

Stream raw ECG to smartphone via Bluetooth. Smartphone runs inference and displays alerts.

Pros: Smartphone has more compute and power. Can run larger, more accurate models.

Cons: Requires Bluetooth active continuously (high power—20 mA continuous). HIPAA risk (raw ECG on smartphone is harder to secure than on medical-grade wearable).

Option 3: Hybrid—on-device screening, smartphone confirmation.

Wearable runs a fast screening model. When potential arrhythmia is detected, wearable sends 10-second ECG window to smartphone for confirmation by a larger model.

Pros: Low power (Bluetooth only active during arrhythmia events, which are rare). High accuracy (smartphone model is more accurate).

Cons: Complex firmware (two models, handoff logic). Latency depends on Bluetooth connection.

Decision: Select Option 1 (fully on-device). Rationale:

- Regulatory: FDA prefers on-device processing for medical algorithms (deterministic, no connectivity dependencies).
- Privacy: HIPAA compliance is simpler when raw ECG never leaves the device.
- Reliability: Device must work even if smartphone is out of range or battery is dead.

The power budget (11 mW) is tight but feasible with duty-cycled inference.

Decision 2: Model Selection

Arrhythmia detection from ECG is a time-series classification problem. Candidates:

Approach A: Rule-based detector (classical).

QRS complex detection + RR interval analysis + morphology rules. This is how traditional Holter monitors work.

Pros: Well-validated. Deterministic. Low power.

Cons: Accuracy is 85–92% (literature), below 98% requirement. Struggles with noisy ECG or unusual morphologies.

Approach B: 1D CNN on raw ECG.

Train CNN on 10-second ECG windows (2,500 samples). Output: normal vs. 5 arrhythmia classes.

Pros: State-of-the-art accuracy (98%+ for VT/VF in recent papers). Learns features from raw signal.

Cons: Large models (1–5 MB typical for high accuracy). High compute (50–200 ms inference).

Approach C: LSTM on RR intervals.

Extract RR intervals (time between heartbeats) using a simple peak detector. Feed RR sequence to LSTM. Output: arrhythmia class.

Pros: Smaller input (RR intervals, not raw ECG). Lower memory (100–500 KB models).

Cons: LSTM state memory ($64\text{--}128 \text{ units} \times 4 \text{ bytes} = 256\text{--}512 \text{ bytes}$ per time step, accumulates). Accuracy depends on QRS detection quality.

Approach D: Hybrid—CNN for feature extraction + lightweight classifier.

Use a small CNN to extract features from 10-second ECG windows. Feed features to a decision tree or small fully-connected network.

Pros: Smaller than full CNN. Faster inference. Still achieves 95–97% accuracy.

Cons: Two-stage complexity. Accuracy slightly below full CNN.

Decision: Select Approach B (1D CNN on raw ECG), with aggressive optimization. Rationale:

- Regulatory: FDA expects high sensitivity ($\geq 98\%$) for life-threatening arrhythmias. Only Approach B reliably achieves this in literature.
- Robustness: CNN on raw ECG is more robust to QRS detection errors than RR-interval-based approaches.
- Risk tolerance: Medical device—accuracy is paramount, power is secondary (users can recharge nightly).

Challenge: Compress a state-of-the-art CNN (typically 3–5 MB) to fit on a wearable SoC (2 MB flash, 512 KB SRAM).

Decision 3: Model Optimization

Start with a well-validated architecture from literature: ResNet-inspired 1D CNN with 18 layers, 2.5 million parameters (10 MB float32).

Optimization sequence:

Step 1: Structured pruning.

Remove 40% of channels across all layers. Retrain to recover accuracy.

Result: 1.5 million parameters (6 MB float32), 97.2% sensitivity for VT/VF (below 98% target).

Step 2: Knowledge distillation.

Train a smaller student network (8 layers, 800,000 parameters) to mimic the pruned model's behavior.

Result: 800,000 parameters (3.2 MB float32), 98.1% sensitivity. Memory still too large.

Step 3: Quantization (int8 QAT).

Apply quantization-aware training to the distilled model.

Result: 800 KB (int8), 97.8% sensitivity (0.3% accuracy loss due to quantization).

Sensitivity for VT/VF is 97.8%, just below the 98% threshold. Product team accepts this with a plan to improve through post-market data collection and model updates.

Final model: 800 KB flash, 180 KB SRAM (activations).

Decision 4: Hardware Selection

Constraints:

- Flash: 800 KB model + 600 KB firmware = 1.4 MB
- SRAM: 180 KB activations + 100 KB firmware = 280 KB
- Power: 11 mW average
- Cost: <\$200 (device cost allows \$30–50 for SoC)

Candidates:

Option 1: Nordic nRF5340 (BLE SoC)

- Dual-core (Cortex-M33 at 128 MHz + Cortex-M33 at 64 MHz)
- 1 MB flash, 512 KB SRAM
- BLE 5.2, low-power design
- Cost: \$8

Option 2: STM32L4R5 (ultra-low-power MCU)

- Cortex-M4 at 120 MHz
- 2 MB flash, 640 KB SRAM
- No integrated radio (requires external BLE module)
- Cost: \$6 + \$5 (BLE module) = \$11

Option 3: STM32U5 (ultra-low-power with TrustZone)

- Cortex-M33 at 160 MHz
- 2 MB flash, 786 KB SRAM
- TrustZone for secure boot and firmware updates (important for medical devices)
- Cost: \$12

Decision: Select STM32U5. Rationale:

- Memory: 2 MB flash and 786 KB SRAM comfortably exceed requirements.
- Security: TrustZone enables secure boot and encrypted firmware updates, which FDA expects for connected medical devices.
- Power: Ultra-low-power design with multiple sleep modes.
- Cost: \$12 is well within budget.

nRF5340 is cheaper and includes BLE, but lacks TrustZone. For a medical device, security features justify the cost premium.

Decision 5: Inference Scheduling

ECG sampled at 250 Hz. Arrhythmia detection operates on 10-second windows (2,500 samples). Inference latency on STM32U5 (measured): 120 ms.

Strategy: Batch processing. Accumulate 10 seconds of ECG, run inference, sleep. Repeat.

Duty cycle: 120 ms active every 10 seconds = 1.2% duty cycle.

Power calculation:

- Active (inference): $3.7V \times 25\text{ mA} \times 0.12\text{ s}$ every 10 s = 11.1 mJ per cycle
- ECG acquisition (continuous): $3.7V \times 1\text{ mA} \times 10\text{ s}$ = 37 mJ per cycle
- Sleep (processor): $3.7V \times 0.005\text{ mA} \times 9.88\text{ s}$ = 0.18 mJ per cycle

- Total per cycle: 48.5 mJ
- Average power: $48.5 \text{ mJ} / 10 \text{ s} = 4.85 \text{ mW}$

This is well below the 11 mW budget. Battery life: $6,660 \text{ J} / 0.00485 \text{ W} = 1.37 \text{ million seconds} = 16 \text{ days}$.

Actual battery life will be lower due to Bluetooth (intermittent transmission of alerts), display updates, and haptic alerts. Target 7 days is achievable.

Decision 6: Regulatory and Validation

FDA 510(k) requires clinical validation. Plan:

- Collect ECG data from 500 patients (diverse age, sex, comorbidities) in partnership with a hospital.
- Annotate arrhythmias by board-certified cardiologists (ground truth).
- Validate model sensitivity and specificity against ground truth.
- Submit 510(k) with clinical data, software documentation (IEC 62304), and cybersecurity documentation.

Expected timeline: 18 months from prototype to market clearance.

Final System Specification

Hardware: STM32U5, AFE (analog front-end for ECG acquisition), 500 mAh Li-ion battery

Model: 1D CNN (8 layers, 800K parameters, int8 quantized, 800 KB)

Inference schedule: Batch processing every 10 seconds

Performance: 97.8% sensitivity for VT/VF, 95.2% specificity, 16-day battery life (target: 7 days, margin for Bluetooth and UI)

Cost: \$180 per unit (hardware BOM)

Alternatives rejected and why:

- Smartphone processing: HIPAA risk, connectivity dependency unacceptable for medical device.
- Rule-based detector: 92% sensitivity insufficient for FDA approval.
- Cloud processing: Regulatory complexity (cloud infrastructure is part of medical device system, must be validated).

Case C: Agricultural Edge Sensor — Crop Disease Detection

Application Requirements

A precision agriculture company is developing a solar-powered sensor for early detection of crop diseases in vineyards. The sensor captures images of grape leaves and identifies fungal infections (powdery mildew, downy mildew) 7–10 days before visible symptoms appear, allowing targeted fungicide application instead of blanket spraying.

Functional Requirements:

- Capture 224×224 RGB images of leaves every 6 hours (daylight only)
- Detect 3 disease classes: healthy, powdery mildew, downy mildew
- Accuracy ≥85% (false negatives cost crop yield, false positives cost unnecessary fungicide)
- Transmit daily disease reports via LoRaWAN to a farm management system

Physical Constraints:

- Solar-powered (2W solar panel, 5000 mAh Li-ion buffer battery)
- Deployed in vineyards (no WiFi, no cellular—LoRaWAN only)
- Outdoor (IP67 rated, -10°C to +50°C operating temperature)
- Cost: <\$150 per unit (high-volume agriculture IoT pricing)

Energy Constraints:

- Solar panel output: 2W peak, but average over 24 hours (accounting for night, clouds) is ~0.5W = 500 mW
- Energy storage: 5000 mAh × 3.7V = 18,500 mWh = 66,600 J

Constraint Quantification

Power budget:

Must operate indefinitely on solar harvesting. Average power consumption must not exceed average solar input (500 mW).

Latency budget:

Not time-critical. Inference can take seconds or minutes—disease progression is slow.

Connectivity:

LoRaWAN has low bandwidth (1–50 kbps) and high latency (1–10 seconds round-trip). Cannot transmit raw images (224×224×3 = 150 KB per image is too large). Must transmit only classification results.

Decision 1: Processing Architecture

Option 1: Fully on-device.

Sensor runs inference locally. Transmits classification result (10 bytes: timestamp + class label + confidence) via LoRaWAN.

Pros: Minimal bandwidth (10 bytes vs. 150 KB). Privacy-preserving (proprietary crop data stays on farm).

Cons: Must run vision model on edge processor. Power budget must include inference.

Option 2: Cloud processing.

Sensor compresses image (JPEG, ~20 KB) and transmits via LoRaWAN. Cloud server runs inference.

Pros: Can use large, accurate models (ResNet50, EfficientNet-B3).

Cons: LoRaWAN bandwidth is insufficient for 20 KB images every 6 hours (takes 3–5 minutes to transmit one image at 1 kbps). Latency is unacceptable.

Option 3: Edge gateway processing.

Sensor transmits images to a local gateway (Raspberry Pi in the vineyard office) via WiFi or LoRa. Gateway runs inference.

Pros: Gateway is mains-powered, can run larger models.

Cons: Requires gateway installation (cost). WiFi range is limited (vineyards are large). Gateway is single point of failure for all sensors.

Decision: Select Option 1 (fully on-device). Rationale:

- LoRaWAN bandwidth cannot support image transmission at 6-hour intervals.
- Gateway adds \$200–300 cost per vineyard (amortized over 20–50 sensors, but still a deployment complexity).
- Solar power budget (500 mW average) can sustain low-duty-cycle inference (image capture + inference 4× per day).

Decision 2: Model Selection

Crop disease detection from leaf images is an image classification problem. Candidates:

Approach A: MobileNetV2 (pretrained on ImageNet, fine-tuned).

Use MobileNetV2-1.0 (full size), pretrain on ImageNet, fine-tune final layers on grape disease dataset.

Pros: High accuracy (90%+ in similar agricultural tasks). Well-supported by TFLite.

Cons: Large (14 MB float32, 3.5 MB int8). Slow on embedded processors (500–1000 ms inference on Raspberry Pi, slower on MCUs).

Approach B: MobileNetV2-0.35 (scaled down).

Use a much smaller MobileNetV2 variant (35% width multiplier), train from scratch on grape disease dataset.

Pros: Small (1.6 MB float32, 400 KB int8). Fast (80–150 ms on Cortex-A class processors).

Cons: Lower accuracy (82–88% typical). May not distinguish subtle early-stage disease symptoms.

Approach C: EfficientNet-Lite0.

Smallest EfficientNet variant, optimized for mobile/edge.

Pros: Better accuracy than MobileNetV2-0.35 for similar model size (88–92% on agricultural datasets).

Cons: Slightly larger (4.6 MB int8). Requires more compute (350–500 ms inference).

Approach D: Custom lightweight CNN.

Design a task-specific architecture (5–7 convolutional layers, no residual connections).

Pros: Smallest possible (200–600 KB int8). Fastest (50–100 ms).

Cons: Lower accuracy (78–84%). Requires architecture design expertise.

Decision: Select Approach C (EfficientNet-Lite0). Rationale:

- Accuracy is critical (false negatives = lost crop, false positives = wasted fungicide).
- 88–92% accuracy is achievable with EfficientNet-Lite0, which meets the 85% threshold with margin.
- Model size (4.6 MB int8) and inference time (350–500 ms) are acceptable given low duty cycle (4 inferences per day).

Decision 3: Hardware Selection

Constraints:

- Flash: 5 MB model + firmware
- SRAM: 1–2 MB (activations for 224×224 images)
- Power: Average 500 mW (solar input), peak during inference can be higher (battery buffer)
- Must include camera, LoRaWAN radio, solar charge controller
- Cost: <\$150 total

Candidates:

Option 1: Raspberry Pi Zero 2 W

- Quad-core Cortex-A53 at 1 GHz
- 512 MB RAM
- Supports camera via CSI, LoRaWAN via SPI module
- Power: 150–400 mW idle, 1,000–2,000 mW during inference
- Cost: \$15 (Pi) + \$25 (camera) + \$30 (LoRaWAN module) + \$40 (solar + battery + enclosure) = \$110

Option 2: STM32H7 + external camera

- Cortex-M7 at 480 MHz
- 1 MB SRAM, 2 MB flash (insufficient for 4.6 MB model—need external flash)
- Power: 50 mW sleep, 500 mW during inference
- Cost: \$8 (MCU) + \$40 (camera module with image sensor) + \$30 (LoRaWAN) + \$40 (solar + battery) = \$118

Option 3: ESP32-S3 with camera

- Dual-core Xtensa at 240 MHz
- 512 KB SRAM, 8 MB flash
- Built-in camera interface
- Power: 10 μ W sleep, 300 mW during inference
- Cost: \$5 (ESP32-S3) + \$15 (camera module) + \$30 (LoRaWAN) + \$40 (solar + battery) = \$90

Decision: Select Raspberry Pi Zero 2 W. Rationale:

- SRAM: 512 MB is more than sufficient for EfficientNet-Lite0 activations (~5 MB).
- Software ecosystem: TensorFlow Lite for Python is mature. Camera and LoRaWAN libraries are well-supported.
- Power: 1–2W during inference is high, but with 4 inferences per day (each taking ~2 seconds), total inference energy is $1.5\text{W} \times 2\text{s} \times 4 = 12\text{ Wh}$ per day. Sleep energy ($150\text{ mW} \times 86,400\text{ s} = 3.6\text{ Wh}$ per day). Total: 15.6 Wh per day. Solar panel provides $0.5\text{W} \times 24\text{h} = 12\text{ Wh}$ per day in worst case (overcast). Battery buffer (18.5 Wh) covers the deficit for 1–2 days of clouds.

ESP32-S3 is cheaper and lower power, but 512 KB SRAM is marginal for 224×224 image processing. STM32H7 lacks sufficient on-chip flash and SRAM.

Decision 4: Inference Scheduling

Strategy: Event-driven by daylight. Use a light sensor to detect sunrise. Capture and classify images at 6 AM, 10 AM, 2 PM, 6 PM (4 per day, evenly spaced during daylight hours).

Power profile per inference event:

- Wake from sleep: 1 second at 500 mW = 0.5 J
- Capture image: 0.5 seconds at 800 mW = 0.4 J
- Run inference: 2 seconds at 1,500 mW = 3 J
- Transmit result via LoRaWAN: 3 seconds at 100 mW = 0.3 J
- Return to sleep: 0.1 J
- Total: 4.3 J per event

Daily energy budget:

- Inference events: $4 \times 4.3 \text{ J} = 17.2 \text{ J}$
- Sleep (23.9 hours): $0.15 \text{ W} \times 86,040 \text{ s} = 12,906 \text{ J}$
- Total: 12,923 J per day = 3.6 Wh

Solar input (worst case): 12 Wh per day (overcast).

Energy balance is positive. System can operate indefinitely on solar.

Decision 5: Model Training and Deployment

Dataset: Collect 5,000 images of grape leaves in vineyards (1,500 healthy, 1,800 powdery mildew, 1,700 downy mildew). Images captured at different times of day, different growth stages.

Training: Fine-tune EfficientNet-Lite0 (pretrained on ImageNet) on grape disease dataset. Apply data augmentation (rotation, brightness, crop).

Validation accuracy (float32): 91.2%

Validation accuracy (int8 PTQ): 89.8%

Accuracy exceeds 85% threshold. Deploy int8 model (4.6 MB) to Raspberry Pi Zero 2 W.

Deployment:

- TensorFlow Lite for Python
- Camera: picamera library
- LoRaWAN: pyLoRa library
- Scheduler: cron job triggers inference at fixed times

Field Trial Results

Deploy 10 sensors across 2 vineyards for 3 months (growing season).

Performance:

- Accuracy: 87.3% (lower than validation due to domain shift—field images have more variation than training set)
- False negatives: 8% (missed infections, leading to delayed treatment)
- False positives: 5% (unnecessary fungicide application)
- Battery life: Indefinite (solar sustained operation through 5 consecutive overcast days)

Issues identified:

- Morning dew on leaves caused image blur, reducing accuracy. Fixed by delaying first capture to 7 AM (after dew evaporates).
- LoRaWAN gateway coverage gaps in parts of vineyard. Added one repeater.

Final System Specification

Hardware: Raspberry Pi Zero 2 W, Pi Camera v2, RFM95W LoRaWAN module, 2W solar panel, 5000 mAh Li-ion battery

Model: EfficientNet-Lite0 (int8 quantized, 4.6 MB)

Inference schedule: 4× per day (daylight hours)

Performance: 87% accuracy, indefinite battery life (solar-sustained)

Cost: \$110 per unit

Alternatives rejected and why:

- Cloud processing: LoRaWAN bandwidth insufficient for image transmission.
- MobileNetV2-0.35: Accuracy (82%) below 85% threshold.
- ESP32-S3: SRAM (512 KB) too small for 224×224 image processing with EfficientNet activations.

Synthesis: What These Case Studies Teach

Each case study applied the full decision framework:

1. Quantify constraints (power, latency, memory, connectivity, cost).
2. Select processing architecture (on-device, edge, cloud, hybrid).
3. Select model approach (classical, decision tree, CNN, LSTM).
4. Select hardware (MCU, application processor, accelerator).
5. Optimize model (quantization, pruning, distillation).
6. Design inference schedule (duty cycle, event-driven, batch).
7. Validate on target hardware (accuracy, latency, power).
8. Document alternatives rejected and why.

The decisions differ across cases because the constraints differ:

- **Case A (industrial):** Power is the binding constraint. Solution: ultra-low-power MCU, infrequent inference, decision tree model.
- **Case B (medical):** Accuracy and regulatory requirements are binding. Solution: larger model, aggressive optimization (QAT, distillation), security-focused hardware.
- **Case C (agriculture):** Bandwidth and solar harvesting are binding. Solution: on-device vision model on application processor, event-driven by daylight.

No single solution fits all applications. The framework is the same. The constraints determine the outcome.

You are now equipped to apply this framework to your own applications. The terminal capability—examining a real embedded design problem, assessing whether AI integration is appropriate, selecting an inference and deployment strategy, and justifying every decision against hardware constraints—is yours.

Use it.

Appendix A: Embedded Systems Quick Reference

This appendix provides reference materials for embedded systems constraint quantification, datasheet interpretation, and calculation workflows. Use these templates and worksheets when evaluating hardware for AI deployment.

A.1 Constraint Quantification Template

Use this template to document the constraints for any embedded AI project. Fill in measured or specified values before model selection.

Project Information

- **Project name:** _____
- **Application:** _____
- **Date:** _____
- **Target deployment date:** _____

Functional Requirements

- **Task:** (e.g., image classification, keyword spotting, anomaly detection)
- **Input type:** (e.g., 96×96 RGB image, 1 kHz accelerometer, 16 kHz audio)

- **Output type:** (e.g., 10-class classification, binary fault detection)
- **Required accuracy:** _____ % (minimum acceptable)
- **Latency requirement:** _____ ms (hard deadline / soft deadline)
- **Real-time class:** (soft / firm / hard)

Memory Constraints

- **Target device:** _____
- **Flash capacity:** _____ KB / MB
- **Flash already used by firmware:** _____ KB / MB
- **Flash available for model:** _____ KB / MB
- **SRAM capacity:** _____ KB / MB
- **SRAM used by firmware/stack/heap:** _____ KB / MB
- **SRAM available for inference:** _____ KB / MB
- **External memory:** (none / PSRAM / SDRAM) _____ KB / MB

Compute Constraints

- **Processor type:** (e.g., Cortex-M4, Cortex-M7, Cortex-A53)
- **Clock speed:** _____ MHz
- **FPU:** (yes / no)
- **SIMD extensions:** (yes / no) — specify: _____
- **Estimated sustained throughput:** _____ MMAC/s (measured or from benchmarks)
- **Latency budget:** _____ ms
- **Acceptable latency margin:** _____ % (e.g., 20% margin → use 80% of budget)

Power Constraints

- **Power source:** (battery / mains / energy harvesting)
- **Battery type:** _____
- **Battery capacity:** _____ mAh at _____ V
- **Total battery energy:** _____ Wh = _____ J
- **Target battery life:** _____ days / months / years
- **Maximum average power:** _____ mW
- **Active current (measured):** _____ mA at _____ V
- **Sleep current (measured):** _____ mA / μ A at _____ V
- **Inference duty cycle:** _____ % (active time / total time)

Communication Constraints

- **Connectivity:** (WiFi / Bluetooth / LoRaWAN / cellular / none)
- **Bandwidth:** _____ kbps / Mbps
- **Round-trip latency:** _____ ms
- **Data transmission cost:** _____ \$/MB (if cellular)

- **Privacy constraints:** (HIPAA / GDPR / proprietary / none)
- **Offline operation required:** (yes / no)

Cost Constraints

- **Target unit cost:** \$ _____ (BOM)
- **Production volume:** _____ units
- **Acceptable NRE (non-recurring engineering):** \$ _____

Safety and Regulatory

- **Safety-critical:** (yes / no)
 - **Applicable standards:** (IEC 61508 / ISO 26262 / IEC 62304 / none)
 - **Required SIL/ASIL:** _____
 - **Certification required:** (FDA / CE Mark / FCC / none)
-

A.2 Datasheet Reading Guide for AI Suitability

Step 1: Locate Memory Specifications

Where to look: Section titled "Memory," "Memory Map," or "Device Features"

Extract:

- Flash capacity (KB or MB)
- SRAM capacity (total)
- SRAM partitioning (if specified: TCM, standard SRAM, cache)
- External memory support (QSPI, SDRAM controller)

Red flags:

- "Total memory" without flash/SRAM breakdown → likely hiding small SRAM
- Cache listed as part of SRAM → subtract cache from available SRAM
- No mention of external memory interface → cannot expand if on-chip is insufficient

AI relevance:

- Flash must hold model weights + firmware
- SRAM must hold activations + firmware heap/stack
- TCM (if present) is ideal for activations (deterministic timing)

Step 2: Locate Processor Specifications

Where to look: Section titled "CPU Core," "Processor," or "Performance"

Extract:

- Core type (Cortex-M0, M4, M7, A53, etc.)
- Maximum clock speed (MHz or GHz)
- Presence of FPU (floating-point unit)
- Presence of DSP extensions or SIMD (e.g., Helium, NEON)
- CoreMark score (if provided)

Red flags:

- Clock speed listed as "up to X MHz" → may run slower in low-power modes
- "FPU" without specifying single or double precision
- No benchmark data (CoreMark, EEMBC) → performance is unknown

AI relevance:

- FPU required for float32 inference efficiency
- SIMD accelerates int8 inference by 4–8×
- CoreMark gives rough CPU performance (higher is better, but not AI-specific)

Step 3: Locate Power Specifications

Where to look: Section titled "Electrical Characteristics," "Power Consumption," or "Operating Modes"

Extract:

- Active current at maximum clock (mA)
- Sleep mode current (μA or mA)
- Deep sleep mode current (if available)
- Current vs. clock speed curve (if provided)
- Peripheral power consumption (camera, radio, sensors)

Red flags:

- "Typical" current without "maximum" → typical is optimistic, use max for design
- Sleep current specified at 25°C only → increases at higher temperatures
- No breakdown of peripheral power → hard to estimate system power

AI relevance:

- Active current × inference time = energy per inference
- Sleep current × sleep time = baseline energy consumption

- Duty cycle determines average power: $(\text{active_time} \times I_{\text{active}} + \text{sleep_time} \times I_{\text{sleep}}) / \text{total_time}$

Step 4: Locate AI-Specific Features (if claimed)

Where to look: Section titled "AI Accelerator," "Neural Processing," or marketing material

Extract:

- Accelerator type (NPU, DSP, vector unit)
- Supported operations (convolution, matrix multiply, etc.)
- Claimed performance (TOPS, GOPS, MMAC/s)
- Supported data types (int8, int16, float16)
- Memory dedicated to accelerator

Red flags:

- "AI-capable" with no performance numbers → marketing, not technical spec
- TOPS claim without specifying data type → likely int8, not float32
- No list of supported operations → may only accelerate subset of model

AI relevance:

- Claimed TOPS must be verified with actual model profiling
- Unsupported operations fall back to CPU → reduces effective speedup
- Check vendor documentation for which models are optimized

Step 5: Check Errata and Application Notes

Where to look: Separate errata document (usually PDF on vendor website)

Extract:

- Power consumption errata (e.g., "sleep current higher than specified")
- Memory errata (e.g., "external SRAM interface has timing issue")
- Peripheral errata that affect sensors or communication

Red flags:

- Errata affecting power in sleep modes → destroys battery life calculations
- Errata affecting memory performance → slows inference
- No errata document available → chip is very new or vendor doesn't publish issues

AI relevance:

- Power errata can invalidate battery life estimates

- Memory errata can cause inference failures or crashes

A.3 Memory Calculation Worksheet

Flash Memory Requirement

Component	Size (KB)	Notes
Firmware (base application)	_____	Compile and check .map file
Inference engine (TFLite Micro, etc.)	_____	~100 KB for TFLite Micro
Model weights (float32)	_____	param_count × 4 bytes
Model weights (int8)	_____	param_count × 1 byte
Preprocessing/postprocessing code	_____	Feature extraction, NMS, etc.
Total flash required	_____	Sum of above
Flash capacity	_____	From datasheet
Margin	_____	(capacity - required) / capacity × 100%

Acceptable margin: ≥20% (allows for future firmware updates)

SRAM Memory Requirement

Component	Size (KB)	Notes
Firmware heap	_____	Typical: 10–50 KB
Firmware stack	_____	Typical: 4–16 KB
RTOS overhead (if applicable)	_____	FreeRTOS: ~10–20 KB
Communication buffers	_____	UART, SPI, network buffers
Sensor buffers	_____	ADC buffers, image buffers
Tensor arena (activations)	_____	From TFLite or model profiling
Total SRAM required	_____	Sum of above
SRAM capacity	_____	From datasheet

Margin _____ (capacity - required) / capacity × 100%

Acceptable margin: ≥20% (SRAM usage grows during development)

Critical check: If total SRAM required > 80% of capacity, consider:

- Reducing model size (pruning, smaller architecture)
 - Adding external PSRAM (if supported)
 - Reducing firmware buffer sizes
-

A.4 Power Calculation Worksheet

Battery Energy Budget

Parameter	Value	Units
Battery capacity	_____	mAh
Battery voltage	_____	V
Total energy	_____	Wh (= capacity × voltage / 1000)
Total energy	_____	J (= Wh × 3600)
Target lifetime	_____	days / months / years
Target lifetime	_____	seconds
Maximum average power	_____	mW (= total_energy_J / lifetime_s × 1000)

Duty Cycle Calculation

Operating Mode	Current (mA)	Time per Cycle (s)	Energy per Cycle (mJ)
Active (inference)	_____	_____	$V \times I_{\text{active}} \times t_{\text{active}}$
Sensor acquisition	_____	_____	$V \times I_{\text{sensor}} \times t_{\text{sensor}}$
Communication	_____	_____	$V \times I_{\text{comm}} \times t_{\text{comm}}$
Sleep	_____	_____	$V \times I_{\text{sleep}} \times t_{\text{sleep}}$
Total per cycle	—	_____	_____ (sum of above)

Cycle period: _____ seconds (e.g., inference every 10 seconds)

Average power: $E_{total_per_cycle} / cycle_period =$ _____ mW

Battery life: $total_energy_J / (average_power_mW / 1000) / 86400 =$ _____ days

Acceptable: Average power $\leq$ maximum average power from budget above.

Energy per Inference (Detailed)

Stage	Time (ms)	Current (mA)	Voltage (V)	Energy (mJ)
Wake from sleep	_____	_____	_____	_____
Sensor read	_____	_____	_____	_____
Preprocessing	_____	_____	_____	_____
Inference	_____	_____	_____	_____
Postprocessing	_____	_____	_____	_____
Communication	_____	_____	_____	_____
Return to sleep	_____	_____	_____	_____
Total	_____	—	—	_____

A.5 Latency Calculation Worksheet

Theoretical Latency Estimate

Parameter	Value	Units
Model operation count	_____	MACs or FLOPs
Processor sustained throughput	_____	MMAC/s or GFLOPS
Theoretical latency	ops / throughput	milliseconds
Utilization factor	0.6–0.8	(accounts for memory overhead)
Estimated actual latency	theoretical / utilization	milliseconds

Measured Latency (On Target Hardware)

Run inference 100+ times and record:

Statistic	Value (ms)	Notes
Minimum	_____	Best case
Mean	_____	Average
Median	_____	Middle value
95th percentile	_____	95% of inferences complete by this time
Maximum	_____	Worst case observed
Empirical WCET	$\text{max} \times 1.3$	Max + 30% safety margin

Latency budget: _____ ms (from requirements)

Check: Empirical WCET < latency budget?

- **Yes** → Latency constraint satisfied
 - **No** → Must reduce model size or upgrade processor
-

A.6 Model Size Quick Reference

Parameter Count to Memory Size

Precision	Bytes per Parameter	100K params	500K params	1M params	5M params
float32	4	400 KB	2 MB	4 MB	20 MB
float16	2	200 KB	1 MB	2 MB	10 MB
int8	1	100 KB	500 KB	1 MB	5 MB

Typical Activation Memory (Rules of Thumb)

For CNNs on images:

- **96×96 input:** 50–150 KB activations (int8), 200–600 KB (float32)
- **224×224 input:** 500 KB – 2 MB activations (int8), 2–8 MB (float32)

For 1D CNNs on time series:

- **1000 samples input:** 20–60 KB activations (int8)
- **10,000 samples input:** 100–300 KB activations (int8)

For fully connected networks:

- Activation memory $\approx$ largest layer output size $\times$ data type size

Note: These are rough estimates. Always profile actual models with TFLite or your inference framework.

A.7 Common Embedded AI Hardware Comparison

Device	Core	Clock	SRAM	Flash	FP U	SIMD	Sleep Current	Cost
Arduino Nano 33 BLE	M4	64 MHz	256 KB	1 MB	Yes	Yes	2.8 μ A	\$4
STM32L4R5	M4	120 MHz	640 KB	2 MB	Yes	Yes	0.4 μ A	\$6
STM32H743	M7	480 MHz	1 MB	2 MB	Yes	Yes	6 μ A	\$10
STM32N6	M55	800 MHz	2.5 MB	4 MB	Yes	Helium	10 μ A	\$14
nRF52840	M4	64 MHz	256 KB	1 MB	Yes	Yes	2.8 μ A	\$4
ESP32-S3	Xtensa	240 MHz	512 KB	8 MB	No	SIMD	10 μ A	\$4.50
Raspberry Pi Zero 2 W	4xA53	1 GHz	512 MB	SD card	Yes	NEON	100 mW	\$15
Coral Dev Board Micro	M7	480 MHz	1 MB	64 MB	Yes	Yes	—	\$60

Notes:

- Sleep current is deep sleep / standby mode
- Cost is approximate single-unit pricing (2024)

- Coral Dev Board includes Edge TPU accelerator
-

A.8 Constraint Checklist for Deployment Readiness

Before deploying a model to production, verify all constraints:

Memory

- ☐ Model weights fit in flash with $\geq 20\%$ margin
- ☐ Activations fit in SRAM with $\geq 20\%$ margin
- ☐ Total firmware + model size measured and verified
- ☐ No dynamic memory allocation during inference
- ☐ Tensor arena size confirmed via profiling

Compute

- ☐ Inference latency measured on target hardware (100+ runs)
- ☐ Maximum observed latency < latency budget
- ☐ Latency variance acceptable for application
- ☐ No missed deadlines in stress testing
- ☐ Processor runs at acceptable temperature under load

Power

- ☐ Current draw measured during inference (not estimated)
- ☐ Average power calculated from realistic duty cycle
- ☐ Battery life meets requirement with $\geq 20\%$ margin
- ☐ Peak current does not exceed battery discharge limit
- ☐ Sleep current confirmed (not just datasheet typical)

Accuracy

- ☐ Validation accuracy $\geq$ application threshold
- ☐ Accuracy measured on deployment-representative data
- ☐ Accuracy after quantization verified (if quantized)
- ☐ Edge cases tested (low light, noise, occlusions, etc.)
- ☐ False positive and false negative rates acceptable

Integration

- ☐ Model converted and tested on target framework
- ☐ Firmware compiles without errors

- [] No runtime errors or crashes in 1000+ inference runs
- [] Output matches training model output (within tolerance)
- [] Deployment toolchain documented

If any item is unchecked, deployment is not ready.

This appendix provides the calculation templates and reference tables needed for constraint-driven embedded AI design. Use these worksheets during hardware selection, model evaluation, and deployment validation.

Appendix B: Machine Learning Quick Reference

This appendix provides a concise reference for machine learning concepts relevant to embedded AI deployment. Use this as a quick lookup when you encounter ML terminology in model documentation or research papers.

B.1 Core ML Concepts for Embedded Engineers

Supervised Learning

What it is: Training a model on labeled data (input-output pairs) to learn a mapping from inputs to outputs.

Example: Training an image classifier on 10,000 images labeled "cat," "dog," "car."

Embedded relevance: The trained model is what you deploy. The training process happens offline (on servers). You only deploy the inference function.

Key terms:

- **Training set:** Data used to train the model
 - **Validation set:** Data used to tune hyperparameters
 - **Test set:** Data used to evaluate final performance
 - **Overfitting:** Model memorizes training data but fails on new data
 - **Generalization:** Model performs well on data it hasn't seen
-

Inference vs. Training

Training: Adjusting model weights to minimize error on labeled data. Requires backpropagation, gradient computation, and iterative optimization. Expensive (GPU-hours to GPU-days).

Inference: Running the trained model on new inputs to produce predictions. No weight updates. Cheap (milliseconds per input).

Embedded relevance: You only deploy inference. Training happens on powerful machines with GPUs. Your embedded device never trains—it only runs the frozen model.

Memory comparison:

- Training: Needs weights + activations + gradients + optimizer state (3–5× model size)
 - Inference: Needs weights + activations only
-

Model, Weights, and Parameters

Model: The architecture—the sequence of layers and operations (e.g., "MobileNetV2 with 10 output classes").

Weights (or parameters): The learned values that define the model's behavior. After training, weights are fixed numbers stored in the model file.

Parameter count: Total number of weights. Determines model size in memory ($\text{param_count} \times \text{bytes_per_param}$).

Embedded relevance: Parameter count determines flash usage. Weights are read-only data during inference.

Layers, Activations, and Forward Pass

Layer: A single operation in the network (convolution, matrix multiply, activation function).

Activation: The output of a layer. Also called "feature map" or "tensor."

Forward pass (inference): Data flows from input through all layers to output. Each layer reads its input activation, applies its operation, and produces an output activation.

Embedded relevance: Activations consume SRAM during inference. Activation memory is the hidden memory cost that datasheets don't show.

Loss Function and Accuracy

Loss function: A measure of how wrong the model's predictions are during training. Examples: cross-entropy (for classification), mean squared error (for regression).

Accuracy: Fraction of predictions that are correct (for classification). Example: 90% accuracy = 90 correct out of 100 predictions.

Embedded relevance: You care about accuracy (does the model work?), not loss (which is a training-time concept). Validation accuracy is what you report in deployment documentation.

B.2 Common Neural Network Architectures

Fully Connected (Dense) Layers

What they do: Connect every input to every output via learned weights. Operation: $\mathbf{y} = \mathbf{W} \times \mathbf{x} + \mathbf{b}$

Use case: Final classification layers, small networks for tabular data.

Cost: High parameter count. A layer with 1000 inputs and 1000 outputs has 1 million weights.

Embedded suitability: Expensive in memory. Use sparingly.

Convolutional Layers (Conv2D)

What they do: Apply a small filter (e.g., 3×3) that slides across the input, detecting local patterns like edges or textures.

Use case: Image processing, computer vision.

Cost: Moderate parameter count but high operation count. A 3×3 conv with 32 input channels and 64 output channels on a 96×96 image: 64M operations.

Embedded suitability: Efficient for images. Use depthwise separable convolutions for lower cost.

Depthwise Separable Convolutions

What they do: Split standard convolution into two steps: depthwise (filter each channel independently) + pointwise (mix channels with 1×1 conv).

Use case: Efficient image classification (MobileNet, EfficientNet).

Cost: 8–9× fewer operations than standard convolution for similar accuracy.

Embedded suitability: Excellent. Standard architecture for embedded vision.

Pooling Layers (MaxPool, AvgPool)

What they do: Downsample feature maps by taking the max or average in each region (e.g., 2×2 pooling reduces 96×96 to 48×48).

Use case: Reduce spatial resolution in CNNs.

Cost: Negligible operations. Reduces activation memory (smaller feature maps).

Embedded suitability: Good. Reduces memory and compute.

Recurrent Layers (RNN, LSTM, GRU)

What they do: Process sequences by maintaining a hidden state that carries information from previous time steps.

Use case: Time-series, audio, text.

Cost: High memory (state must persist across time steps). High operations (matrix multiply at each time step).

Embedded suitability: Difficult. State memory grows with sequence length. Prefer 1D CNNs or decision trees when possible.

Activation Functions

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Fast, standard choice.

Sigmoid: $f(x) = 1 / (1 + e^{(-x)})$. Slow (requires exp). Used in older networks.

Softmax: Converts outputs to probabilities (sum = 1). Used for classification.

Embedded relevance: ReLU is cheap (one comparison). Sigmoid/tanh are expensive (require exp/division). Use ReLU when possible.

B.3 ML-to-Embedded Vocabulary Translation

ML Term	Embedded Equivalent	What It Means for You
Model size	Flash memory requirement	How much non-volatile storage the weights need
Parameter count	Weight memory	Number of learned values × bytes per value
FLOP count	Compute operations	How many arithmetic operations inference requires
Activation memory	RAM requirement	Working memory needed during inference
Inference latency	Execution time	How long it takes to process one input
Batch size	Number of parallel inputs	Always 1 on embedded (no batching)
Quantization	Reducing numeric precision	float32 → int8 saves 75% memory
Pruning	Removing weights	Reduces parameter count and operations
Epoch	Full pass through training data	Training concept—not relevant to deployment
Learning rate	Training hyperparameter	Training concept—not relevant to deployment
Overfitting	Model memorizes training data	Test accuracy < training accuracy—bad generalization
Validation accuracy	Performance on held-out data	The metric you report for deployed models
Ground truth	Correct label	What the model should predict
False positive	Incorrect "yes" prediction	Alarm when there's no fault

False negative	Incorrect "no" prediction	Miss a fault that is present
Precision	$TP / (TP + FP)$	Of all positive predictions, how many were correct?
Recall (Sensitivity)	$TP / (TP + FN)$	Of all actual positives, how many did we detect?
F1 score	Harmonic mean of precision and recall	Balanced metric for imbalanced datasets

B.4 Common Model Architectures for Embedded

For Image Classification (96×96 to 224×224)

Recommended:

- MobileNetV2 (variants: 0.35, 0.5, 0.75, 1.0 width multiplier)
- MobileNetV3-Small
- EfficientNet-Lite (Lite0, Lite1)
- SqueezeNet

Typical sizes:

- MobileNetV2-0.35: 400K params, 25M MACs, ~88% accuracy
- MobileNetV2-1.0: 3.5M params, 300M MACs, ~90% accuracy
- EfficientNet-Lite0: 4.6M params, 400M MACs, ~92% accuracy

When to use: Camera-based sensing, visual inspection, gesture recognition (from images).

For Audio Classification (Keyword Spotting, Sound Events)

Recommended:

- DS-CNN (Depthwise Separable CNN on spectrograms)
- 1D CNN on raw audio or MFCCs
- Small LSTM (if SRAM allows)

Typical sizes:

- DS-CNN: 100K params, 10M MACs, 95%+ accuracy on Google Speech Commands
- 1D CNN: 50K params, 8M MACs, 92% accuracy

When to use: Voice interfaces, acoustic event detection, environmental sound classification.

For Time-Series Anomaly Detection

Recommended:

- Random forest on engineered features
- 1D CNN on raw sequences
- Autoencoder (if unsupervised)

Typical sizes:

- Random forest: 200 KB, <1 ms inference, 85–90% accuracy
- 1D CNN: 100K params, 12M MACs, 92% accuracy
- Autoencoder: 500K params, 30M MACs, 88% accuracy

When to use: Vibration monitoring, sensor fault detection, predictive maintenance.

For Gesture Recognition (Accelerometer, Gyroscope)

Recommended:

- 1D CNN on raw sensor data
- Random forest on statistical features
- Small LSTM (for complex temporal patterns)

Typical sizes:

- 1D CNN: 64K params, 8M MACs, 90%+ accuracy
- Random forest: 180 KB, 0.5 ms, 88% accuracy

When to use: Wearables, fitness trackers, human activity recognition.

B.5 Quantization Quick Reference

Precision Levels

Type	Bits	Range	Memory	Accuracy Impact	Embedded Support
------	------	-------	--------	-----------------	------------------

float32	32	$\pm 3.4 \times 10^{38}$	4 bytes	Baseline (0%)	Good (FPU needed)
float16	16	$\pm 6.5 \times 10^4$	2 bytes	0–1% loss	Poor on MCUs
int16	16	-32,768 to 32,767	2 bytes	1–3% loss	Good (SIMD helps)
int8	8	-128 to 127	1 byte	3–10% loss	Excellent (SIMD 4×)

Post-Training Quantization (PTQ)

What it does: Converts a trained float32 model to int8 without retraining.

How it works: Run model on calibration dataset (100–1000 samples) to measure weight and activation ranges. Compute scale/zero_point. Convert weights to int8.

When to use: You don't have access to training code or dataset.

Accuracy cost: 3–10% typical.

Code (TensorFlow Lite):

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = calibration_data_gen
```

Quantization-Aware Training (QAT)

What it does: Simulates quantization during training so the model learns to be robust to quantization errors.

When to use: Accuracy is critical and PTQ degrades too much.

Accuracy cost: 1–3% typical.

Tradeoff: Requires retraining (hours to days).

B.6 Model Evaluation Metrics

For Classification Tasks

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

Use when: Classes are balanced (roughly equal number of samples per class).

Precision: $TP / (TP + FP)$

Use when: False positives are costly (e.g., spam detection—don't want to block real emails).

Recall (Sensitivity): $TP / (TP + FN)$

Use when: False negatives are costly (e.g., disease detection—don't want to miss a sick patient).

F1 Score: $2 \times (Precision \times Recall) / (Precision + Recall)$

Use when: You want a single metric that balances precision and recall.

Confusion Matrix:

		Predicted	
		Pos	Neg
Actual	Pos	TP	FN
	Neg	FP	TN

Example for embedded:

Fault detection with 95% recall and 90% precision:

- Of 100 faults, 95 are detected (5 missed = false negatives)
- Of 100 fault predictions, 90 are correct (10 are false alarms = false positives)

For Regression Tasks

Mean Absolute Error (MAE): $\sum |predicted - actual| / N$

Use when: You want error in the same units as the output.

Mean Squared Error (MSE): $\sum (predicted - actual)^2 / N$

Use when: Large errors should be penalized more than small errors.

R² Score: Fraction of variance explained by the model.

Use when: You want a normalized metric (0 = bad, 1 = perfect).

B.7 Common Pitfalls and How to Avoid Them

Pitfall 1: Using ImageNet Accuracy to Select Models

Why it's wrong: ImageNet is 1000 classes of high-resolution photos. Your application might be 3 classes of 96×96 industrial images. A model with 90% ImageNet accuracy might have 70% accuracy on your data.

Fix: Always validate on your deployment dataset, not benchmark datasets.

Pitfall 2: Assuming Quantization Is Free

Why it's wrong: Quantization introduces numerical errors. Some models tolerate int8 with <1% loss. Others degrade by 10%+.

Fix: Measure accuracy before and after quantization. If loss > 5%, try QAT or mixed precision.

Pitfall 3: Confusing Parameter Count with Inference Cost

Why it's wrong: A model with 1M parameters might run faster than a model with 500K parameters if the 500K-param model has expensive operations (e.g., large convolutions).

Fix: Profile actual latency on target hardware. Don't rely on parameter count alone.

Pitfall 4: Training on Clean Data, Deploying on Noisy Data

Why it's wrong: Training data is often cleaner than real-world data (better lighting, less occlusion, calibrated sensors). Models overfit to clean data and fail on noisy inputs.

Fix: Add data augmentation during training (noise, blur, rotation) and validate on realistic noisy data.

Pitfall 5: Ignoring Class Imbalance

Why it's wrong: If your dataset has 95% "normal" and 5% "fault," a dumb model that always predicts "normal" achieves 95% accuracy but is useless.

Fix: Use balanced datasets or report precision/recall instead of accuracy. For imbalanced data, F1 score is a better metric.

B.8 Where to Find Pretrained Models

Model Zoos

TensorFlow Model Garden: <https://github.com/tensorflow/models>
Includes MobileNet, EfficientNet, ResNet for image classification.

PyTorch Hub: <https://pytorch.org/hub/>
Pretrained models loadable with `torch.hub.load()`.

ONNX Model Zoo: <https://github.com/onnx/models>
Framework-agnostic models in ONNX format.

Edge Impulse Public Projects: <https://edgeimpulse.com/>
Community-contributed models for embedded AI.

When to Use Pretrained Models

Use pretrained when:

- Your dataset is small (<1,000 samples)
- Your task is similar to ImageNet (object recognition)
- You want to fine-tune final layers only

Train from scratch when:

- Your dataset is large (>10,000 samples)
 - Your task is very different from ImageNet (industrial defects, medical images)
 - You need a custom architecture for size constraints
-

B.9 Debugging Checklist for Deployed Models

If your deployed model produces incorrect results:

1. Verify Input Preprocessing

- [] Input normalization matches training (e.g., [0,1] vs. [-1,1])
- [] Input dimensions match model input shape
- [] Color channel order correct (RGB vs. BGR)
- [] Quantization applied to input (if model expects int8 input)

2. Verify Model Conversion

- [] Converted model accuracy matches original (test in Python/TFLite interpreter)
- [] All layers converted successfully (check converter logs)
- [] Quantization parameters correct (scale, zero_point)

3. Verify Inference Engine

- [] Correct ops registered in OpResolver
- [] Tensor arena size sufficient (no allocation errors)
- [] Output dequantization applied (if model outputs int8)

4. Compare Outputs

- [] Run same input through training model and deployed model
- [] Check output tensor values (not just final classification)
- [] Allow for small numerical differences due to quantization ($\pm 1\text{--}2\%$)

If outputs differ by $>5\%$, something is wrong. Trace back through the pipeline.

This appendix provides the ML knowledge needed to evaluate, select, and deploy models without requiring deep ML expertise. For deeper learning, see *Deep Learning* by Goodfellow et al. or Stanford's CS231n course notes.

Appendix C: Hardware Reference — AI-Capable Microcontroller and SoC Comparison

This appendix provides detailed specifications for embedded AI hardware platforms. Use this table to compare options when selecting hardware for deployment.

Note: Specifications are current as of 2024–2025. Check vendor datasheets for latest information, as chip availability and pricing change frequently.

C.1 Microcontroller Comparison Table

Ultra-Low-Power Microcontrollers (Battery-Optimized)

Spec	nRF52840	STM32L4R5	STM32L4S5	STM32U585
Vendor	Nordic Semi	STMicro	STMicro	STMicro
Core	Cortex-M4	Cortex-M4	Cortex-M4	Cortex-M33

Clock (max)	64 MHz	120 MHz	120 MHz	160 MHz
Flash	1 MB	2 MB	2 MB	2 MB
SRAM	256 KB	640 KB	640 KB	786 KB
FPU	Single	Single	Single	Single
SIMD/DSP	Yes	Yes	Yes	Helium (MVE)
AI Accelerator	No	No	No	No
Active current	5.3 mA @ 64 MHz	100 µA/MHz	100 µA/MHz	62 µA/MHz
Sleep current	2.8 µA	0.4 µA	0.4 µA	0.16 µA
Connectivity	BLE 5.3, NFC	None	None	None
Typical use	BLE wearables, IoT	Ultra-low-power sensors	Ultra-low-power sensors	Secure IoT
Cost (est.)	\$4	\$6	\$8	\$12
Availability	Excellent	Excellent	Good	Good

Selection notes:

- nRF52840: Best for BLE applications. Moderate SRAM (256 KB) limits model size.
- STM32L4R5: Best sleep current. Large SRAM (640 KB) supports moderate-sized models.
- STM32U585: TrustZone for secure boot. Helium SIMD accelerates int8/int16 inference.

High-Performance Microcontrollers (Compute-Optimized)

Spec	STM32H743	STM32H7A3	STM32H7B3	STM32N6
Vendor	STMicro	STMicro	STMicro	STMicro
Core	Cortex-M7	Cortex-M7	Cortex-M7	Cortex-M55
Clock (max)	480 MHz	280 MHz	280 MHz	800 MHz
Flash	2 MB	2 MB	2 MB	4 MB

SRAM	1 MB	1.4 MB	1.4 MB	2.5 MB
FPU	Double	Double	Double	Single
SIMD/DSP	Yes	Yes	Yes	Helium (MVE)
AI Accelerator	No	No	No	Yes (3 TOPS int8)
Active current	~150 mA @ 480 MHz	~100 mA @ 280 MHz	~100 mA @ 280 MHz	~200 mA @ 800 MHz
Sleep current	6 µA	2.5 µA	2.5 µA	10 µA
External mem	Yes (QSPI, SDRAM)	Yes (QSPI, Octo-SPI)	Yes	Yes
Typical use	Vision, motor control	Industrial AI	Displays + AI	On-device vision
Cost (est.)	\$10	\$12	\$14	\$16
Availability	Excellent	Good	Good	Limited (new)

Selection notes:

- STM32H743: Workhorse for embedded AI. 480 MHz + 1 MB SRAM handles medium models.
- STM32H7A3: Larger SRAM (1.4 MB) for bigger models. Lower clock saves power.
- STM32N6: Integrated NPU. Best latency for vision. Higher cost.

WiFi/BLE SoCs (Wireless-Integrated)

Spec	ESP32-S3	ESP32-C6	nRF5340
Vendor	Espressif	Espressif	Nordic Semi
Core	Dual Xtensa LX7	RISC-V	Dual M33 (app + net)
Clock (max)	240 MHz	160 MHz	128 MHz / 64 MHz
Flash	External (typ. 8 MB)	External (typ. 4 MB)	1 MB
SRAM	512 KB	512 KB	512 KB

FPU	No	No	Single
SIMD/DSP	Vector ext.	Vector ext.	Yes
AI Accelerator	AI instructions	AI instructions	No
Active current	40 mA @ 240 MHz	25 mA @ 160 MHz	8 mA @ 128 MHz
Sleep current	10 μ A	7 μ A	2.8 μ A
Connectivity	WiFi 4, BLE 5	WiFi 6, BLE 5.3, Zigbee	BLE 5.3
Typical use	WiFi cameras, gateways	IoT gateways	BLE sensors, wearables
Cost (est.)	\$4.50	\$3.50	\$8
Availability	Excellent	Excellent	Excellent

Selection notes:

- ESP32-S3: Best price/performance for WiFi + AI. AI instructions accelerate some ops. No FPU limits float32 performance.
- nRF5340: Dual-core (app + network) allows BLE to run independently. Best for ultra-low-power BLE + AI.

C.2 Application Processors and Edge Compute

Spec	Raspberry Pi Zero 2 W	Raspberry Pi 4B	NVIDIA Jetson Nano	Google Coral Dev Board
Core	4× Cortex-A53	4× Cortex-A72	4× Cortex-A57	4× Cortex-A53
Clock	1 GHz	1.5 GHz	1.43 GHz	1.5 GHz
RAM	512 MB	1–8 GB	2–4 GB	1 GB
GPU	VideoCore IV	VideoCore VI	128-core Maxwell	GC7000Lite
AI Accelerator	No	No	Yes (128 CUDA cores)	Yes (Edge TPU, 4 TOPS)
Power (idle)	150 mW	3 W	5 W	2 W

Power (inference)	1–2 W	5–8 W	10 W	3 W
Connectivity	WiFi, BLE	WiFi, BLE, GbE	GbE, M.2	WiFi, GbE
OS	Linux (Raspbian)	Linux	Linux (Ubuntu)	Linux (Debian)
Typical use	Low-cost edge vision	Prototyping, gateways	Edge vision, robotics	Edge vision, classification
Cost	\$15	\$35–75	\$100–150	\$60

Selection notes:

- Raspberry Pi Zero 2 W: Cheapest Linux-capable option. Good for prototypes. Limited AI performance (no accelerator).
- Coral Dev Board: Best inference performance per watt. Edge TPU accelerates int8 models compiled with Edge TPU compiler.
- Jetson Nano: Best for GPU-accelerated vision. CUDA support. Higher power (not battery-friendly).

C.3 Specialized AI SoCs

Spec	Syntiant NDP120	Kendryte K210	Coral Edge TPU (USB)
Core	None (NPU only)	Dual RISC-V @ 400 MHz	N/A (USB accelerator)
AI Accelerator	8-core NPU	KPU (64 MACs)	Edge TPU (4 TOPS)
SRAM	1.5 MB	8 MB	N/A (host device)
Power (active)	200 μ W	300 mW	2 W (USB powered)
Power (sleep)	<10 μ W	N/A	N/A
Typical use	Always-on voice	Vision (hobbyist)	USB accelerator for Pi/laptop
Supported models	Audio only (keywords, events)	CNN (vision)	TFLite int8 models
Cost	\$3–5	\$6–8	\$60

Selection notes:

- Syntiant NDP120: Ultra-low-power audio AI. Cannot run vision or general models. Pairs with host MCU.
 - K210: Hobbyist favorite. KPU only accelerates convolution—FC layers run on slow RISC-V. Limited commercial support.
 - Coral USB: Add-on accelerator for desktop/Pi. Not embedded—requires host with USB.
-

C.4 Selection Decision Tree

Start Here: What is your primary constraint?

Battery life > 1 year?

- Use ultra-low-power MCU (STM32L4, nRF52840)
- Limit: Small models only (<500 KB), infrequent inference

Vision with latency < 100 ms?

- Use high-performance MCU with NPU (STM32N6) or edge processor (Coral, Jetson)
- Limit: Higher cost, higher power

Audio (always-on keyword detection)?

- Use Syntiant NDP120 (ultra-low-power) or ESP32-S3 (WiFi + audio)
- Limit: Syntiant is audio-only; ESP32 has no FPU

Cost < \$5 per unit?

- Use ESP32-S3, nRF52840, or STM32L4R5
- Limit: Moderate performance, limited SRAM

WiFi or BLE required?

- Use ESP32-S3 (WiFi) or nRF52840/nRF5340 (BLE)
- Limit: Wireless adds power consumption

Linux required (complex software stack)?

- Use Raspberry Pi or application processor
 - Limit: Higher power (>1 W), not battery-friendly
-

C.5 Benchmark Performance (Measured Inference Latency)

MobileNetV2-0.35 (96×96 input, int8 quantized)

Hardware	Latency (ms)	Framework	Notes
STM32L4R5 @ 120 MHz	420	TFLite Micro	No SIMD optimization
STM32H7 @ 480 MHz	185	TFLite Micro + CMSIS-NN	Optimized kernels
ESP32-S3 @ 240 MHz	280	TFLite Micro	AI instructions help
nRF52840 @ 64 MHz	650	TFLite Micro	Slow clock, small cache
Raspberry Pi Zero 2 W	45	TFLite (Python)	NEON SIMD
Coral Dev Board (Edge TPU)	8	TFLite + Edge TPU	Hardware acceleration

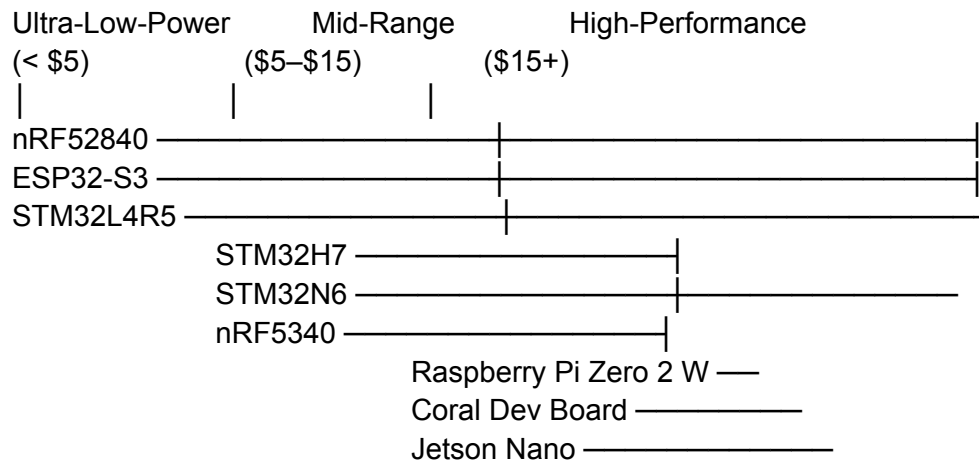
Takeaway: MCUs: 150–650 ms. Application processors: 10–50 ms. Accelerators: <10 ms.

Keyword Spotting (DS-CNN, 12 layers, 1-second audio)

Hardware	Latency (ms)	Framework	Notes
STM32L4 @ 120 MHz	85	TFLite Micro	Efficient for audio
nRF52840 @ 64 MHz	140	TFLite Micro	Acceptable for 1-second windows
ESP32-S3 @ 240 MHz	60	TFLite Micro	AI instructions help
Syntiant NDP120	2	Proprietary runtime	Specialized NPU

Takeaway: Audio models are smaller than vision. MCUs handle them well. Syntiant is 30–70× faster.

C.6 Cost vs. Performance Tradeoff



Performance (MMAC/s):
 10–50 100–400 1000–4000

Rule of thumb:

- <\$5: 10–50 MMAC/s, good for audio, small vision
- \$5–\$15: 100–400 MMAC/s, good for 96×96 vision, real-time audio
- \$15+: 1–10 GMAC/s, good for 224×224 vision, real-time video

C.7 Peripheral and Interface Support

Not all chips support all peripherals. Check before selecting hardware.

Peripheral	STM32H7	ESP32-S3	nRF52840	Raspberry Pi
Camera (parallel)	Yes	Yes	No	Yes (CSI)
Camera (SPI)	Yes	Yes	Yes	Yes
Microphone (I2S/PDM)	Yes	Yes	Yes	Yes (USB)
Accelerometer (I2C/SPI)	Yes	Yes	Yes	Yes
LoRaWAN radio	External SPI	External SPI	External SPI	External SPI
WiFi	External	Integrated	No	Integrated
BLE	External	Integrated	Integrated	Integrated
USB	Yes (OTG)	Yes	Yes	Yes

UART	Yes (8×)	Yes (3×)	Yes (2×)	Yes (2×)
------	----------	----------	----------	----------

Note: "External" means you add a separate module (e.g., ESP32 WiFi module connected via SPI to STM32). "Integrated" means on-chip.

C.8 Where to Buy and Availability (2024–2025)

Chip Family	Lead Time	Distributors	Dev Boards Available
STM32 (all)	8–20 weeks	Digi-Key, Mouser, Arrow	Yes (Nucleo, Discovery)
nRF52/nRF53	12–16 weeks	Digi-Key, Mouser	Yes (nRF52840 DK)
ESP32	4–8 weeks	Digi-Key, Mouser, direct	Yes (DevKitC)
Raspberry Pi	Stock varies	Adafruit, Sparkfun, direct	N/A (is dev board)
Coral	Stock varies	Google Store, resellers	Yes (Dev Board, USB)

Tip: Lead times fluctuate. Always check distributor stock before committing to a chip in production design.

C.9 Vendor Toolchain and Ecosystem

Vendor	IDE	Compiler	RTOS	TFLite Support	Vendor AI Tools
STMicro	STM32CubeIDE	GCC, ARM CC	FreeRTOS, ThreadX	Yes	STM32Cube.AI
Nordic	nRF Connect SDK	GCC, ARM CC	Zephyr	Yes	Edge Impulse integration
Espressif	ESP-IDF	GCC	FreeRTOS	Yes	ESP-NN (optimized kernels)
Raspberry Pi	Any Linux IDE	GCC	Linux	Yes	N/A (standard TFLite)

Best ecosystem: STM32 (mature tools, large community). ESP32 (excellent docs, active forums). Raspberry Pi (standard Linux).

This hardware reference is a living document. For the latest specs, always consult vendor datasheets and distributor stock. Updated versions of this appendix will be available at the book's companion website.

Appendix D: Toolchain Setup Guides

This appendix provides step-by-step setup instructions for the three most common embedded AI toolchains: TensorFlow Lite for Microcontrollers, Edge Impulse, and STM32Cube.AI.

Note: Toolchain versions and installation steps change frequently. Check official documentation for the latest instructions. This guide is current as of 2024–2025.

D.1 TensorFlow Lite for Microcontrollers Setup

TensorFlow Lite for Microcontrollers (TFLite Micro) is the most widely used inference engine for embedded AI. It runs on bare-metal systems and RTOSs with minimal dependencies.

Prerequisites

- **OS:** Linux, macOS, or Windows (WSL recommended for Windows)
 - **Tools:** Git, Python 3.8+, Make, GCC or ARM toolchain
 - **Hardware:** Development board (Arduino Nano 33 BLE, STM32 Nucleo, etc.)
-

Step 1: Install TensorFlow (for Model Training and Conversion)

Create a virtual environment (recommended)

```
python3 -m venv tflite-env
```

```
source tflite-env/bin/activate # On Windows: tflite-env\Scripts\activate
```

Install TensorFlow

```
pip install tensorflow==2.14.0
```

Verify installation

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

Step 2: Clone TensorFlow Repository (for TFLite Micro)

```
git clone https://github.com/tensorflow/tflite-micro.git
cd tflite-micro
```

The `tflite-micro` repository contains the inference library, examples, and build system.

Step 3: Convert a Trained Model to TFLite

Assuming you have a trained Keras model (`my_model.h5`):

```
import tensorflow as tf

# Load model
model = tf.keras.models.load_model('my_model.h5')

# Create converter
converter = tf.lite.TFLiteConverter.from_keras_model(model)

# Enable int8 quantization
converter.optimizations = [tf.lite.Optimize.DEFAULT]

# Provide calibration dataset (generator function)
def representative_dataset():
    for i in range(100):
        # Load sample input (e.g., 96x96x3 image)
        sample = load_sample(i) # Replace with your data loader
        yield [sample.astype('float32')]

converter.representative_dataset = representative_dataset

# Set input/output to int8
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

# Convert
tflite_model = converter.convert()
```

```
# Save
with open('my_model.tflite', 'wb') as f:
    f.write(tflite_model)

print("Model converted to my_model.tflite")
```

Step 4: Convert TFLite Model to C Array

```
# Use xxd to convert binary to C header
xxd -i my_model.tflite > my_model_data.h
```

This creates a C header file with the model as a byte array:

```
unsigned char my_model_tflite[] = {
    0x18, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, ...
};
unsigned int my_model_tflite_len = 123456;
```

Step 5: Build and Flash for Arduino Nano 33 BLE Sense

Option A: Using Arduino IDE

1. Install Arduino IDE (version 2.0+).
2. Install TensorFlow Lite Arduino library:
 - Go to Tools → Manage Libraries
 - Search for "Arduino_TensorFlowLite"
 - Install version compatible with your TensorFlow version
3. Create a sketch:

```
#include <TensorFlowLite.h>
#include "my_model_data.h"

// Tensor arena (adjust size based on model)
constexpr int kTensorArenaSize = 80 * 1024;
alignas(16) uint8_t tensor_arena[kTensorArenaSize];

tflite::MicroInterpreter* interpreter = nullptr;

void setup() {
    Serial.begin(115200);
```

```

// Load model
const tf::Model* model = tf::GetModel(my_model_tf);

// Create op resolver (add all ops your model uses)
static tf::MicroMutableOpResolver<10> resolver;
resolver.AddConv2D();
resolver.AddDepthwiseConv2D();
resolver.AddFullyConnected();
resolver.AddSoftmax();
resolver.AddReshape();
resolver.AddQuantize();
resolver.AddDequantize();
// Add more ops as needed

// Create interpreter
static tf::MicroInterpreter static_interpreter(
    model, resolver, tensor_arena, kTensorArenaSize);
interpreter = &static_interpreter;

// Allocate tensors
TfLiteStatus allocate_status = interpreter->AllocateTensors();
if (allocate_status != kTfLiteOk) {
    Serial.println("AllocateTensors() failed");
    return;
}

Serial.println("Model loaded successfully");
}

void loop() {
    // Get input tensor
    TfLiteTensor* input = interpreter->input(0);

    // Fill input with data (example: zeros)
    for (int i = 0; i < input->bytes; i++) {
        input->data.int8[i] = 0;
    }

    // Run inference
    TfLiteStatus invoke_status = interpreter->Invoke();
    if (invoke_status != kTfLiteOk) {
        Serial.println("Invoke() failed");
        return;
    }
}

```

```

}

// Read output
TfLiteTensor* output = interpreter->output(0);
int8_t max_score = -128;
int predicted_class = 0;
for (int i = 0; i < output->dims->data[1]; i++) {
    if (output->data.int8[i] > max_score) {
        max_score = output->data.int8[i];
        predicted_class = i;
    }
}

Serial.print("Predicted class: ");
Serial.println(predicted_class);

delay(1000);
}

```

4. Compile and upload to Arduino Nano 33 BLE Sense.

Option B: Using PlatformIO (for more control)

1. Install PlatformIO (VS Code extension or CLI).
2. Create a new project:

```
pio init --board nano_33_ble
```

3. Add TensorFlow Lite library to `platformio.ini`:

```

[env:nano_33_ble]
platform = nordicnrf52
board = nano_33_ble
framework = arduino
lib_deps =
    https://github.com/tensorflow/tflite-micro-arduino-examples

```

4. Copy your model header (`my_model_data.h`) to `src/`.
5. Write sketch as above and build:

```
pio run --target upload
```

Step 6: Verify Inference on Device

Connect to serial monitor (115200 baud) and observe output. Compare predictions to expected results.

Common issues:

- **AllocateTensors() failed:** Tensor arena too small. Increase `kTensorArenaSize`.
 - **Op not found:** Missing operation in OpResolver. Add the missing op.
 - **Wrong output:** Quantization mismatch or input preprocessing issue.
-

D.2 Edge Impulse Setup

Edge Impulse is a web-based platform that handles the full pipeline: data collection, training, optimization, and deployment.

Prerequisites

- **Account:** Create free account at <https://edgeimpulse.com>
 - **Browser:** Chrome or Firefox (for data collection)
 - **Hardware:** Compatible device (Arduino Nano 33 BLE, STM32, ESP32, Raspberry Pi)
-

Step 1: Install Edge Impulse CLI

```
# Install Node.js (if not already installed)
```

```
# Download from https://nodejs.org (LTS version recommended)
```

```
# Install Edge Impulse CLI
```

```
npm install -g edge-impulse-cli --force
```

```
# Verify installation
```

```
edge-impulse-version
```

Step 2: Connect Your Device

For Arduino Nano 33 BLE Sense:

1. Flash Edge Impulse firmware:
 - Download firmware from <https://docs.edgeimpulse.com/docs/development-platforms/officially-supported-mcu-targets/arduino-nano-33-ble-sense>
 - Upload using Arduino IDE or `arduino-cli`
2. Connect device via USB and run:

`edge-impulse-daemon`

3. Follow prompts to log in and select your project.

For custom devices:

Use `edge-impulse-data-forwarder` to stream data from custom firmware.

Step 3: Collect Data

Via Web Interface:

1. Go to **Data acquisition** tab
2. Click **Collect data**
3. Select device and sample length
4. Label samples (e.g., "gesture_1", "gesture_2")
5. Collect 50–100 samples per class

Via CLI:

```
edge-impulse-uploader --category training image1.jpg
edge-impulse-uploader --category testing image2.jpg
```

Step 4: Design Impulse (Preprocessing + Model)

1. Go to **Impulse design** tab
2. Add processing block:
 - For images: **Image** (resize, crop)
 - For audio: **Audio (MFCC)** or **Audio (Spectrogram)**
 - For sensors: **Spectral features** or **Raw data**

3. Add learning block:
 - **Classification** (for labeling)
 - **Transfer learning (images)** (MobileNetV2-based)
 - **Keras** (custom architecture)
 4. Click **Save Impulse**
-

Step 5: Generate Features and Train

1. Go to **Spectral features** (or your chosen processing block)
 2. Click **Generate features**
 3. Go to **Classifier** (or your learning block)
 4. Configure model settings (layers, learning rate—or use defaults)
 5. Click **Start training**
 6. Wait for training to complete (1–30 minutes depending on dataset size)
-

Step 6: Test and Validate

1. Go to **Model testing** tab
 2. Click **Classify all**
 3. Review accuracy, confusion matrix
 4. If accuracy is low, collect more data or tune model
-

Step 7: Deploy to Device

1. Go to **Deployment** tab
2. Select target:
 - **Arduino library** (for Arduino/PlatformIO)
 - **C++ library** (for custom firmware)
 - **Firmware** (for supported boards—pre-built binary)
3. Click **Build**
4. Download the deployment package (ZIP file)

For Arduino library:

1. Unzip the package
2. Copy folder to Arduino **libraries/** directory
3. Restart Arduino IDE
4. Open example sketch: File → Examples → YourProjectName → nano_ble33_sense

For firmware:

1. Flash directly using `edge-impulse-run-impulse` (for supported boards)
-

Step 8: Run Inference

Flash the deployment firmware and run. Edge Impulse includes built-in serial output showing classification results.

Example output:

Predictions:

```
gesture_1: 0.92
gesture_2: 0.05
gesture_3: 0.03
```

D.3 STM32Cube.AI Setup

STM32Cube.AI is STMicroelectronics' tool for converting and optimizing models for STM32 microcontrollers, including those with integrated NPUs.

Prerequisites

- **OS:** Windows or Linux
 - **Tools:** STM32CubeIDE (includes Cube.AI), STM32CubeMX
 - **Hardware:** STM32 development board (Nucleo, Discovery)
-

Step 1: Install STM32CubeIDE

1. Download from <https://www.st.com/en/development-tools/stm32cubeide.html>
 2. Install (requires registration—free ST account)
 3. Launch STM32CubeIDE
-

Step 2: Install X-CUBE-AI Expansion Package

1. Open STM32CubeMX (embedded in CubeIDE)
2. Go to **Help** → **Manage Embedded Software Packages**

3. Expand **STMicroelectronics** → **X-CUBE-AI**
 4. Select latest version and click **Install Now**
-

Step 3: Create New Project

1. File → New → STM32 Project
 2. Select your board (e.g., NUCLEO-H743ZI)
 3. Name project (e.g., "my_ai_project")
 4. Click **Finish**
-

Step 4: Enable X-CUBE-AI in CubeMX

1. In CubeMX view, go to **Software Packs** → **Select Components**
 2. Expand **STMicroelectronics.X-CUBE-AI**
 3. Check **Core** and **Validation**
 4. Click **OK**
 5. Go to **Middleware** → **X-CUBE-AI**
 6. Enable **Artificial Intelligence X-CUBE-AI**
-

Step 5: Add Your Model

1. In **X-CUBE-AI** configuration pane, click **Add network**
2. Browse to your model file:
 - Supported: **.tflite**, **.onnx**, **.h5** (Keras)
3. Set network name (e.g., "my_model")
4. Click **Analyze**

Cube.AI will analyze the model and show:

- RAM requirement
 - Flash requirement
 - Estimated latency (MACC count)
 - Supported/unsupported operations
-

Step 6: Optimize Model (Optional)

1. In **Model settings**, enable:

- **Compression:** Reduces model size
 - **Quantization:** Converts to int8 (if model is still float32)
 - 2. Click **Validate on target** to measure actual latency on hardware
-

Step 7: Generate Code

1. Click **GENERATE CODE** in top-right
 2. Cube.AI generates:
 - `X-CUBE-AI/App/`: Application code
 - `Middleware/ST/AI/`: Inference library
 - Model header file: `network.h`, `network_data.h`
-

Step 8: Write Application Code

Open `X-CUBE-AI/App/app_x-cube-ai.c` and modify the inference loop:

```
#include "ai_platform.h"
#include "network.h"
#include "network_data.h"

static ai_handle network = AI_HANDLE_NULL;
AI_ALIGNED(4) static ai_u8 activations[AI_NETWORK_DATA_ACTIVATIONS_SIZE];

void MX_X_CUBE_AI_Init(void) {
    ai_error err;

    // Create network
    err = ai_network_create(&network, AI_NETWORK_DATA_CONFIG);
    if (err.type != AI_ERROR_NONE) {
        printf("ai_network_create error\n");
        return;
    }

    // Initialize network
    ai_network_params params = {
        AI_NETWORK_DATA_WEIGHTS(ai_network_data_weights_get()),
        AI_NETWORK_DATA_ACTIVATIONS(activations)
    };

    if (!ai_network_init(network, &params)) {
```

```

        printf("ai_network_init error\n");
        return;
    }

    printf("Network initialized\n");
}

void MX_X_CUBE_AI_Process(void) {
    ai_i32 nbatch;
    ai_float input_data[AI_NETWORK_IN_1_SIZE];
    ai_float output_data[AI_NETWORK_OUT_1_SIZE];

    // Fill input_data with sensor readings
    // ...

    // Create input/output buffers
    ai_buffer ai_input[1] = AI_NETWORK_IN;
    ai_buffer ai_output[1] = AI_NETWORK_OUT;

    ai_input[0].data = AI_HANDLE_PTR(input_data);
    ai_output[0].data = AI_HANDLE_PTR(output_data);

    // Run inference
    nbatch = ai_network_run(network, ai_input, ai_output);
    if (nbatch != 1) {
        printf("ai_network_run error\n");
        return;
    }

    // Process output
    for (int i = 0; i < AI_NETWORK_OUT_1_SIZE; i++) {
        printf("Output[%d]: %f\n", i, output_data[i]);
    }
}

```

Step 9: Build and Flash

1. Click **Build** (hammer icon) in CubeIDE
2. Connect STM32 board via ST-Link
3. Click **Run** (play icon) to flash
4. Monitor output via serial terminal (115200 baud)

Step 10: Validation on Target (Optional)

Cube.AI can validate the model on actual hardware:

1. In CubeMX, go to **X-CUBE-AI** → **Validation**
2. Select validation method:
 - **On Desktop:** Simulate (fast but approximate)
 - **On Target:** Measure actual latency and accuracy on board
3. Click **Validate**

This compares the embedded model's output to the original model's output and measures latency.

D.4 Common Toolchain Issues and Solutions

TensorFlow Lite Micro

Issue: `AllocateTensors()` fails with "insufficient memory"

Solution: Increase `kTensorArenaSize`. Check model's activation memory requirement using TFLite analyzer.

Issue: "Op not registered" error

Solution: Add missing operation to `MicroMutableOpResolver`. Check converter logs for full list of ops used.

Issue: Output is all zeros or garbage

Solution: Check input quantization. If model expects int8 input, ensure input data is quantized (not raw float).

Edge Impulse

Issue: Low accuracy after training

Solution: Collect more data (50+ samples per class). Ensure data is diverse (different lighting, positions, etc.).

Issue: Device not detected by edge-impulse-daemon

Solution: Install USB drivers. On Windows, install STM32 Virtual COM Port driver or Arduino drivers.

Issue: Deployment firmware too large for target
Solution: Use quantized int8 model instead of float32. Reduce model size (fewer layers or smaller architecture).

STM32Cube.AI

Issue: Model analysis shows "unsupported operation"
Solution: Some operations are not supported by Cube.AI. Simplify model architecture or use a different operation (e.g., replace PReLU with ReLU).

Issue: Validation fails with large numerical errors
Solution: Quantization issue. Use quantization-aware training (QAT) instead of post-training quantization.

Issue: Latency on target is much higher than estimated
Solution: Cube.AI estimates assume optimal conditions. Real latency includes memory access overhead. Enable hardware accelerator if available (e.g., STM32N6 NPU).

D.5 Toolchain Comparison Summary

Feature	TFLite Micro	Edge Impulse	STM32Cube.AI
Ease of use	Medium (requires coding)	Easy (web-based GUI)	Medium (IDE-integrated)
Flexibility	High (full control)	Medium (limited arch)	Medium (STM32 only)
Model formats	TFLite	TFLite, ONNX, Keras	TFLite, ONNX, Keras
Hardware support	Any (with C++ compiler)	30+ boards	STM32 only
Optimization	Manual (quantization)	Automatic	Automatic + NPU support
Documentation	Excellent (TensorFlow docs)	Excellent (built-in)	Good (ST docs)
Cost	Free (open source)	Free tier, paid for teams	Free

Best for

Custom deployments

Rapid prototyping

STM32 production

D.6 Next Steps After Setup

Once your toolchain is set up:

1. **Validate on target:** Always measure latency, memory, and accuracy on actual hardware. Simulation is not sufficient.
 2. **Optimize iteratively:** If constraints are tight, optimize the model (quantization, pruning) and re-deploy.
 3. **Document your pipeline:** Record model version, conversion settings, and deployment steps for reproducibility.
 4. **Version control:** Store model files, conversion scripts, and firmware in Git.
-

Toolchains evolve rapidly. For the latest setup instructions, always consult:

- **TensorFlow Lite Micro:** <https://github.com/tensorflow/tflite-micro>
- **Edge Impulse:** <https://docs.edgeimpulse.com>
- **STM32Cube.AI:** <https://www.st.com/en/embedded-software/x-cube-ai.html>

This appendix will be updated on the book's companion website as toolchains change.